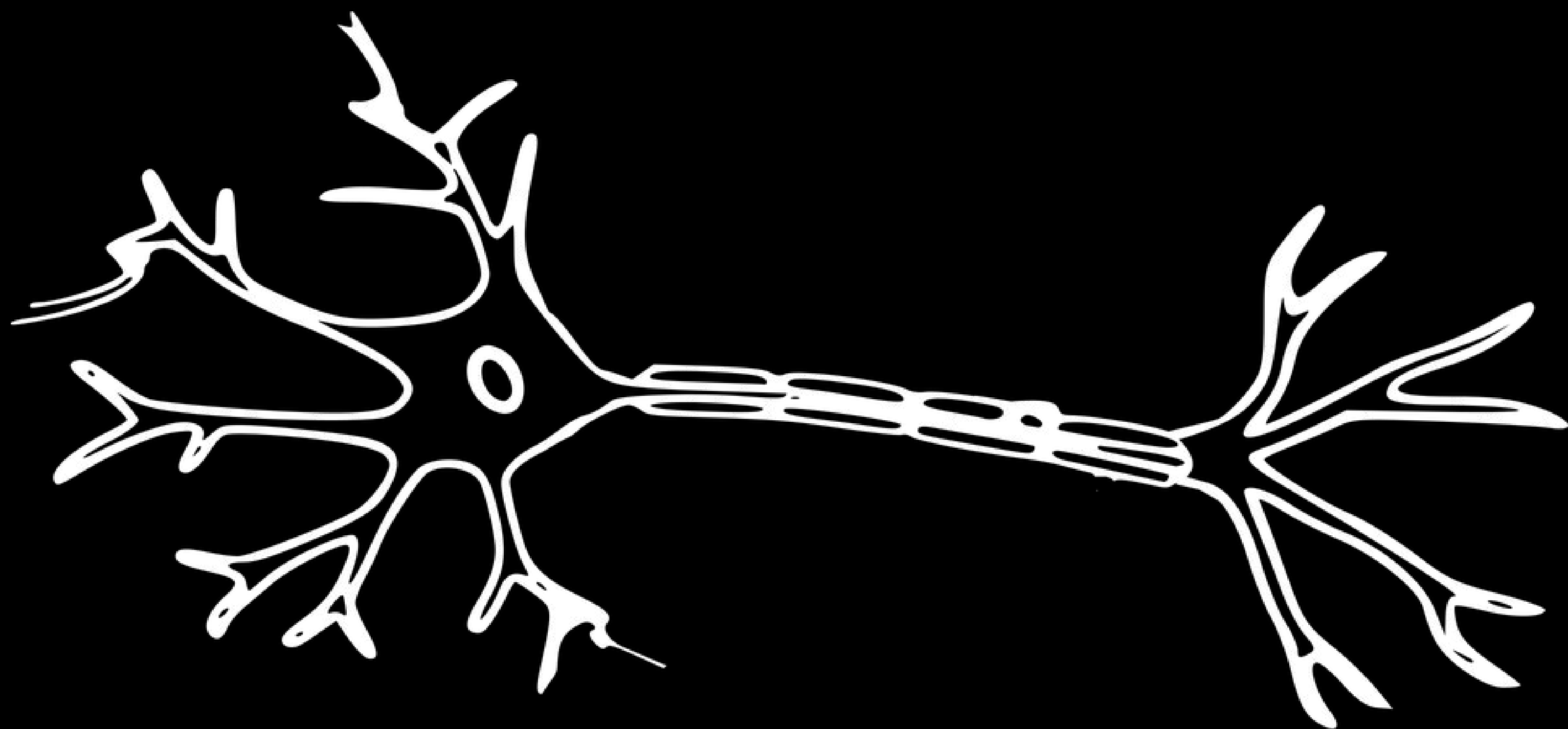


# Redes Neurais





# Redes Neurais

# Redes Neurais

- Os neurônios se conectam e recebem sinais elétricos de outros neurônios.

# Redes Neurais

- Os neurônios se conectam e recebem sinais elétricos de outros neurônios.
- Os neurônios processam sinais de entrada e podem ser ativados.

# rede neural artificial

modelo matemático de aprendizado inspirado  
em redes neurais biológicas

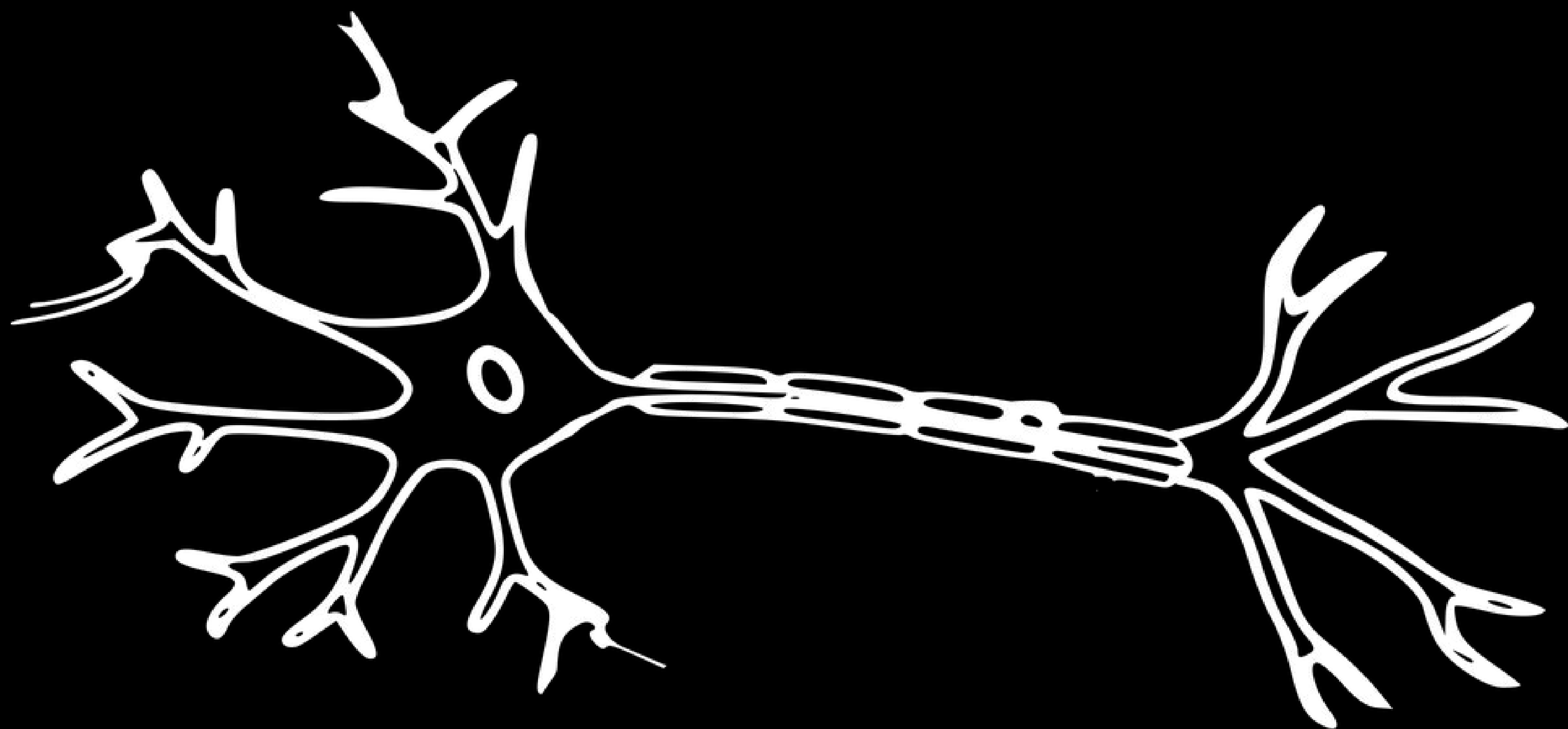
# Redes Neurais Artificiais

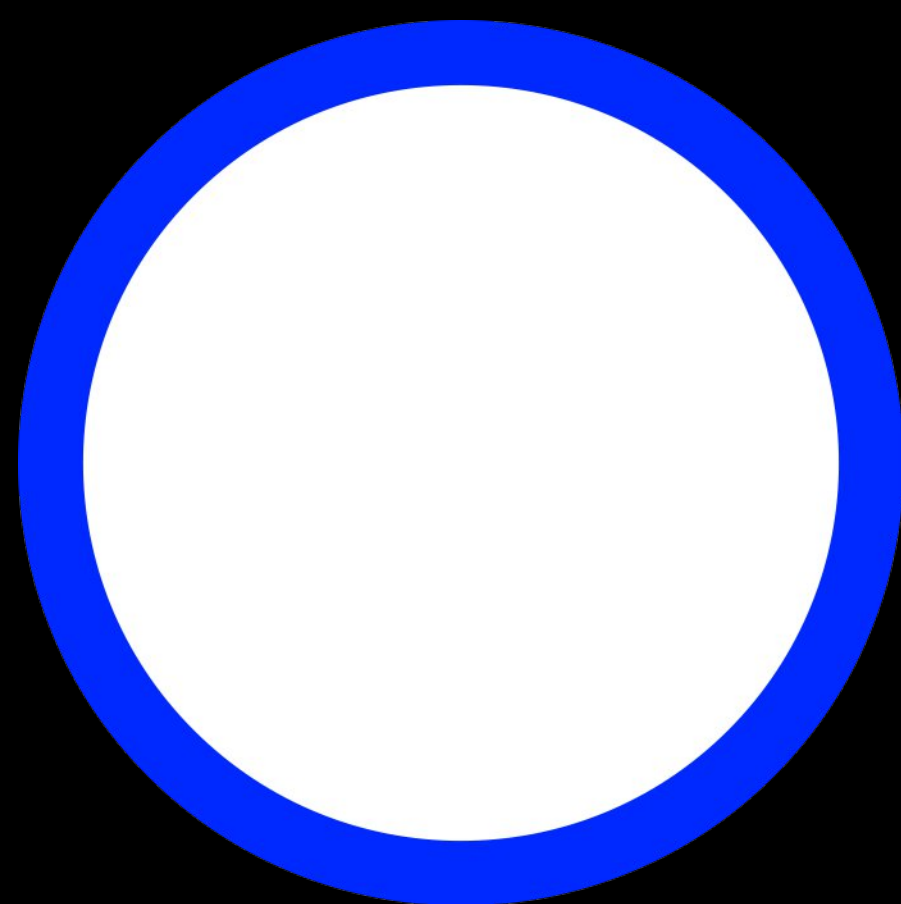
# Redes Neurais Artificiais

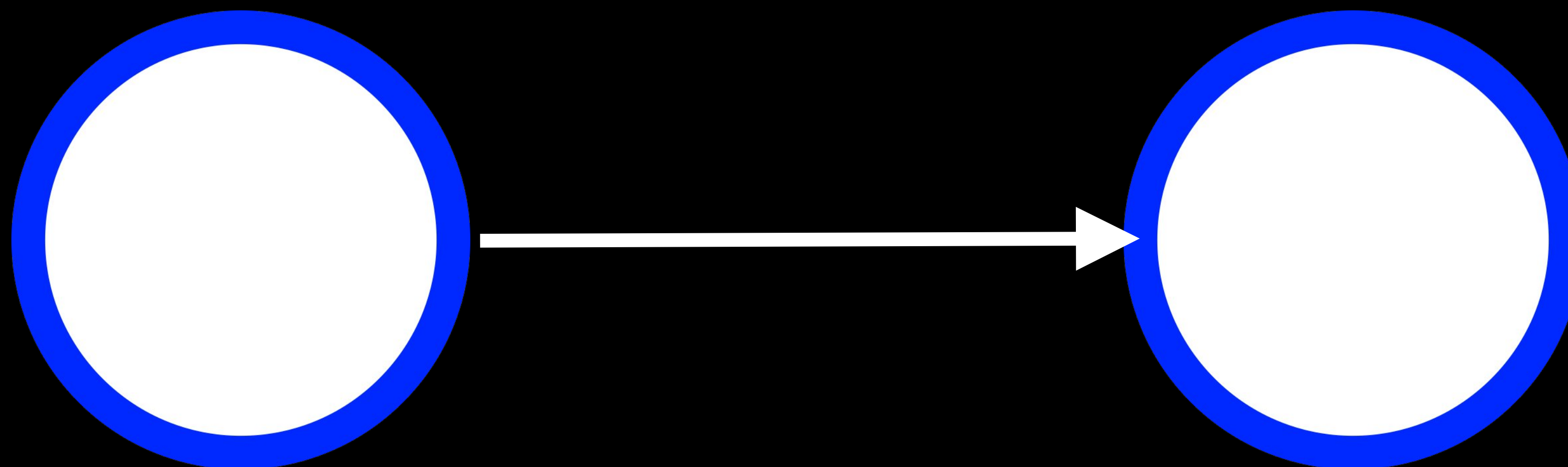
- Modela uma função matemática das entradas para as saídas com base na estrutura e nos parâmetros da rede.

# Redes Neurais Artificiais

- Modela uma função matemática das entradas para as saídas com base na estrutura e nos parâmetros da rede.
- Permite aprender os parâmetros da rede a partir dos dados.







$(x_1, x_2)$

$(x_1, x_2) \longrightarrow$

$(x_1, x_2)$



$$h(x_1, x_2) =$$

$$h(x_1, x_2) = w_0 + w_1x_1 + w_2x_2$$

$$h(x_1, x_2) = w_0 + w_1 x_1 + w_2 x_2$$

$$h(x_1, x_2) = w_0 + w_1x_1 + w_2x_2$$

função degrau

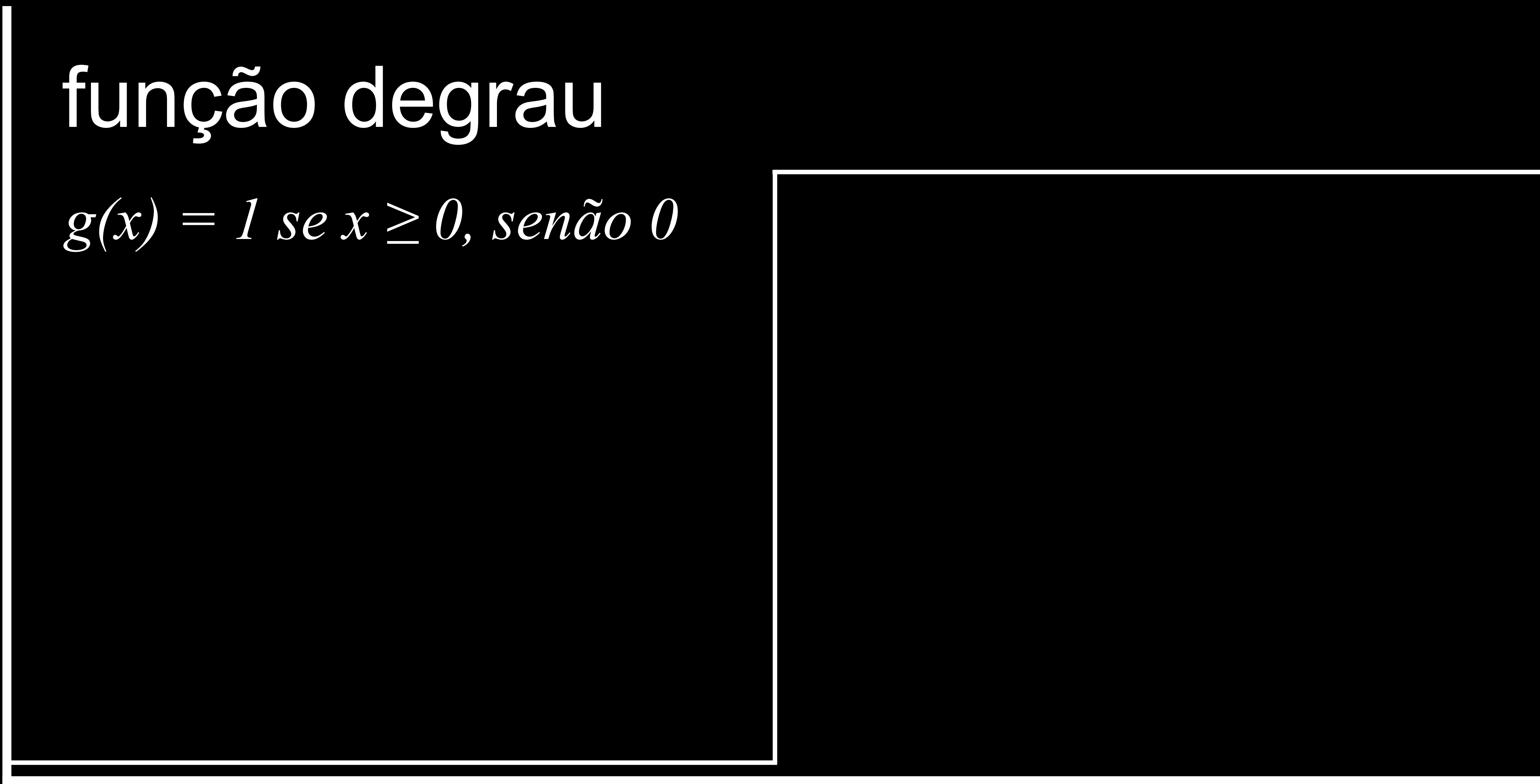
$$g(x) = 1 \text{ se } x \geq 0, \text{ senão } 0$$

saída

1

0

$\mathbf{w} \cdot \mathbf{x}$



# sigmoide logística

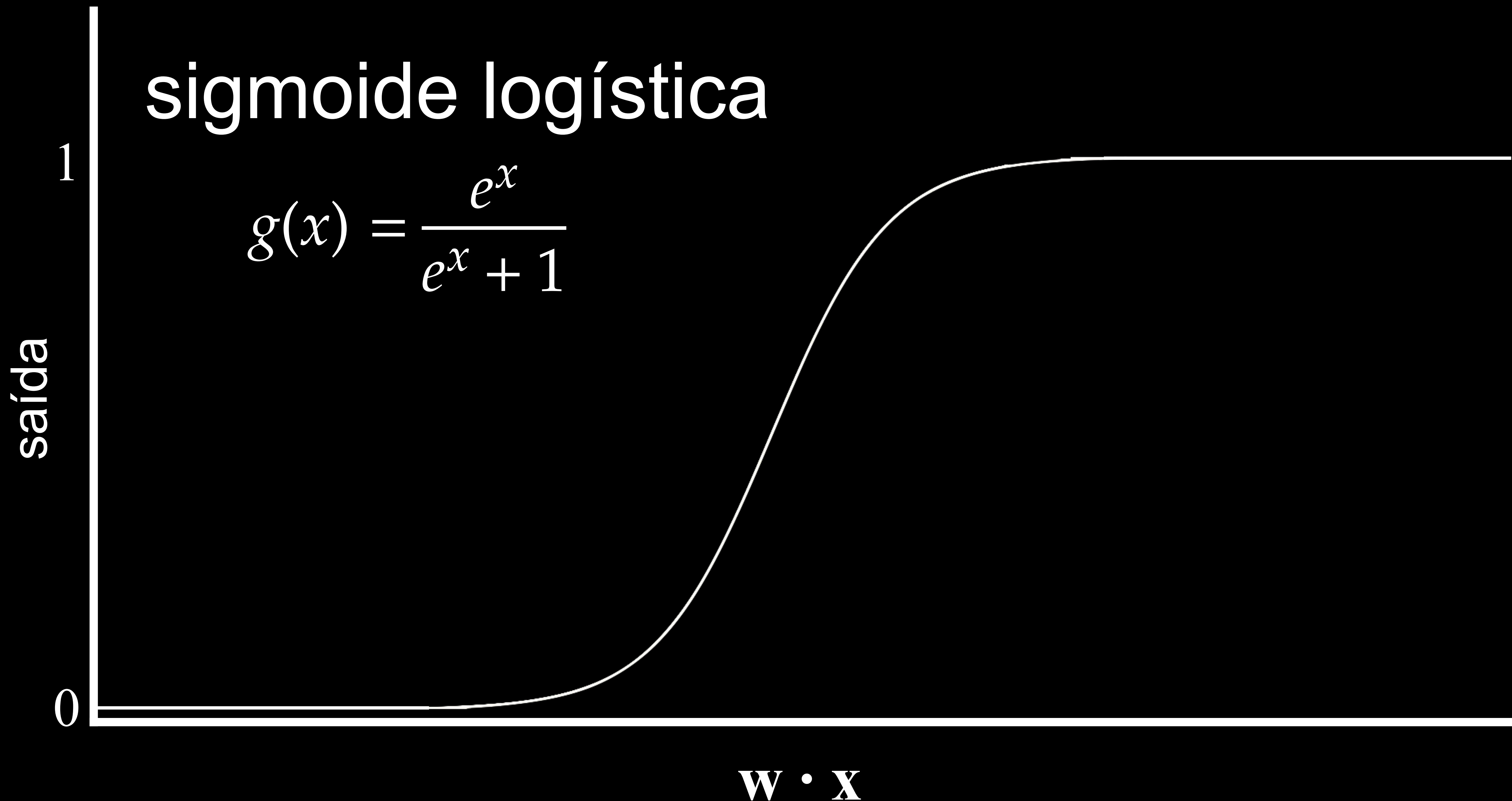
$$g(x) = \frac{e^x}{e^x + 1}$$

saída

1

0

$\mathbf{w} \cdot \mathbf{x}$



sigmoide logística

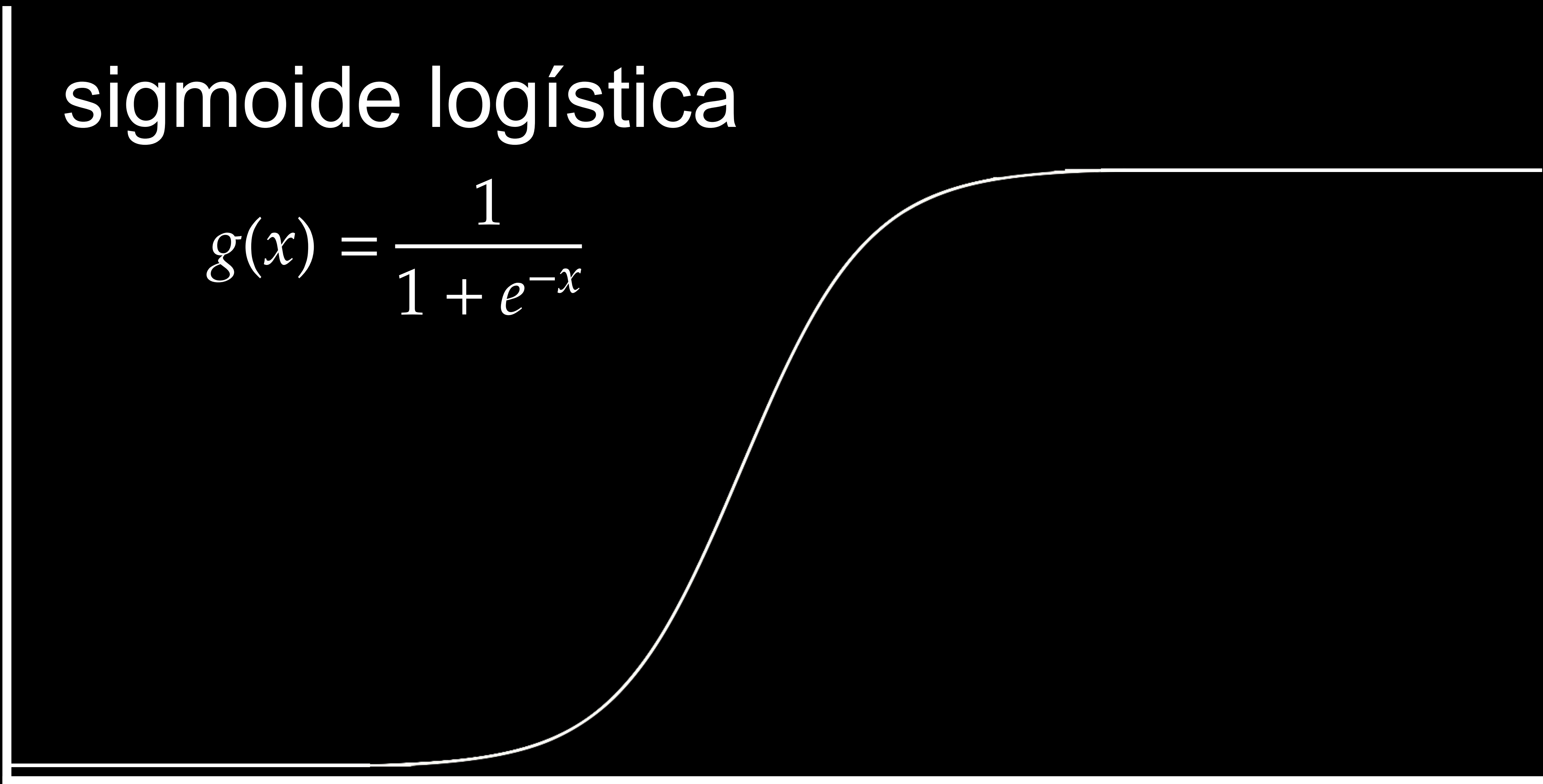
$$g(x) = \frac{1}{1 + e^{-x}}$$

saída

0

1

$\mathbf{w} \cdot \mathbf{x}$



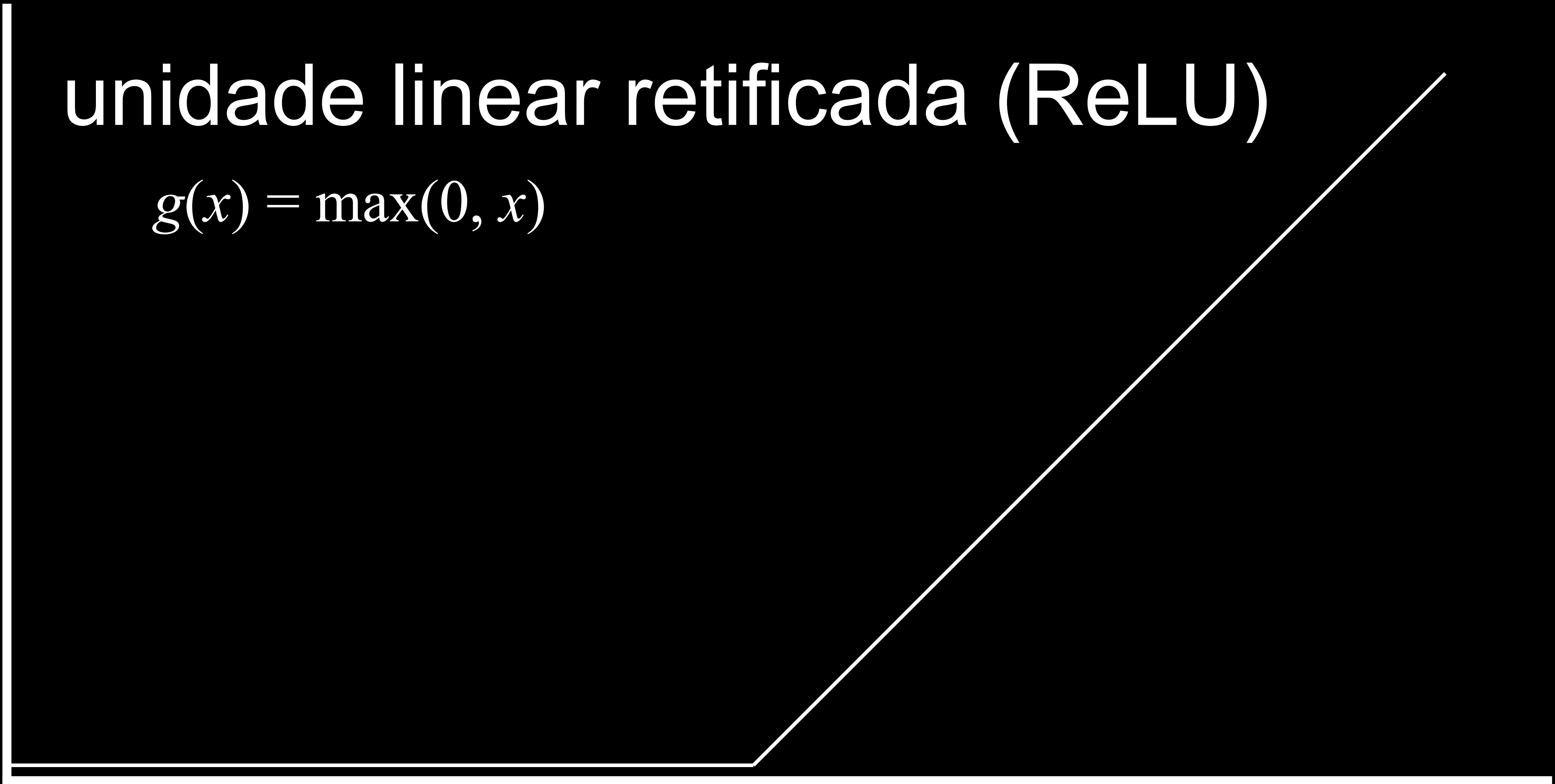
# unidade linear retificada (ReLU)

$$g(x) = \max(0, x)$$

saída

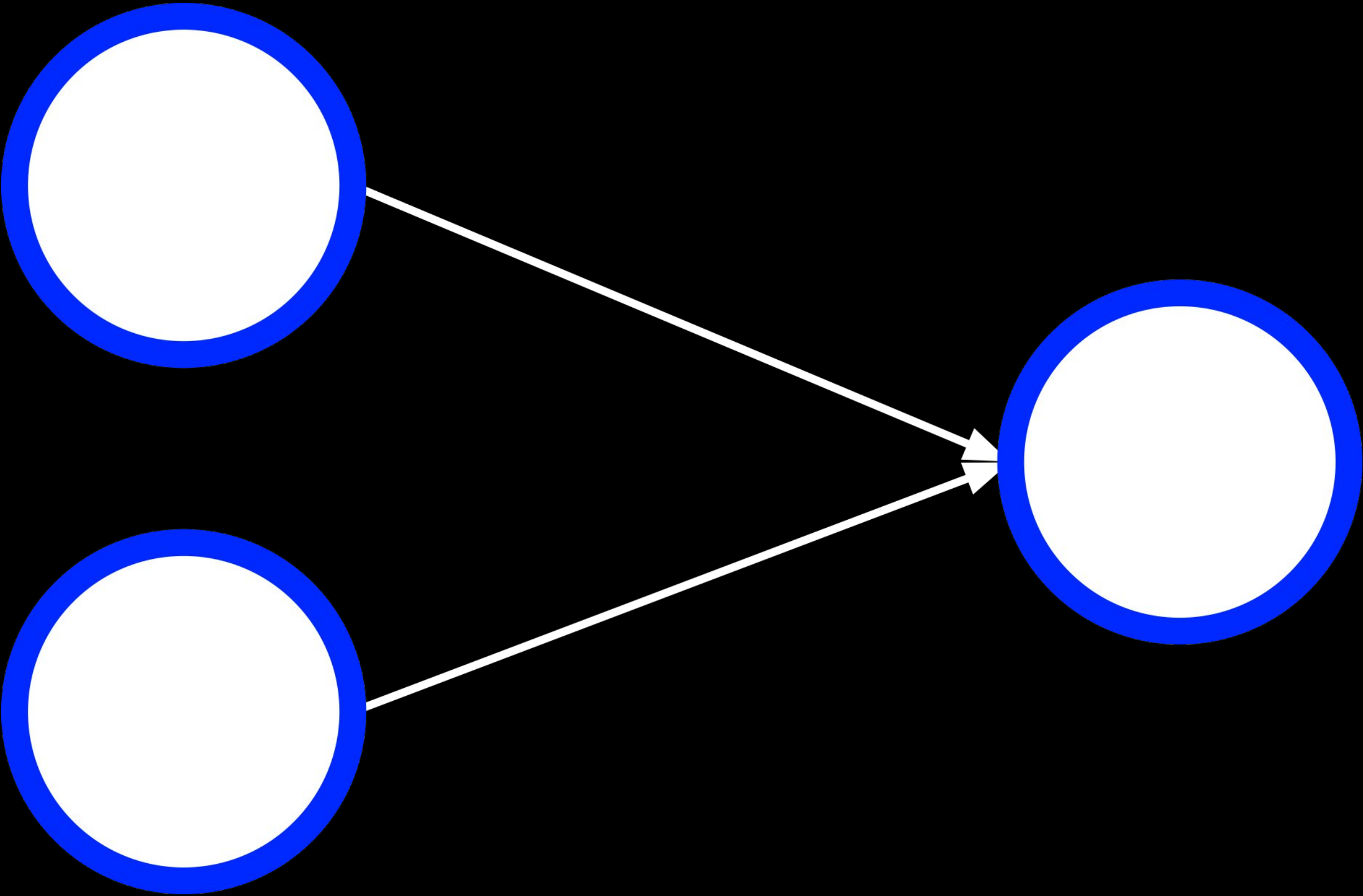
0

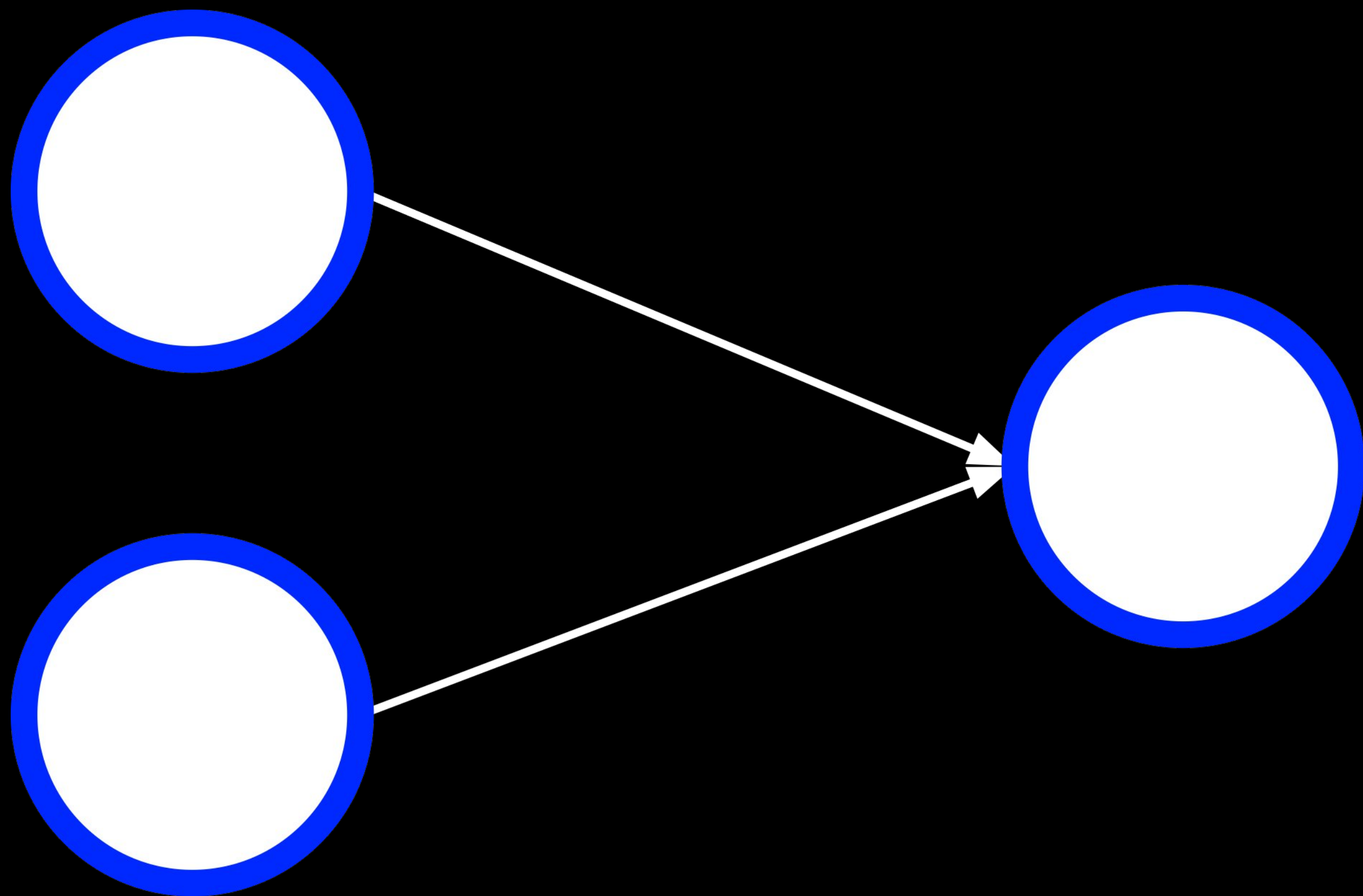
$\mathbf{w} \cdot \mathbf{x}$

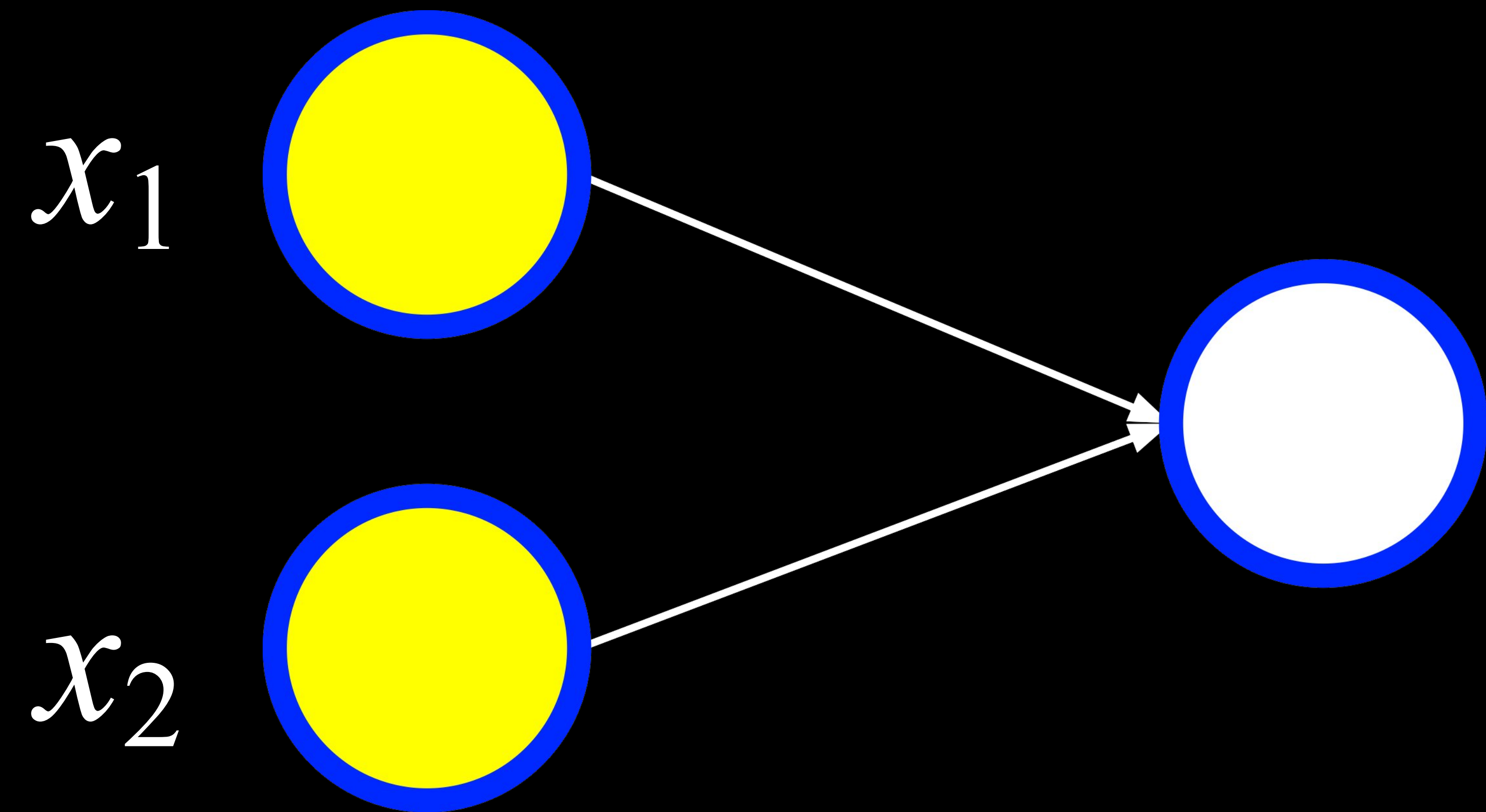


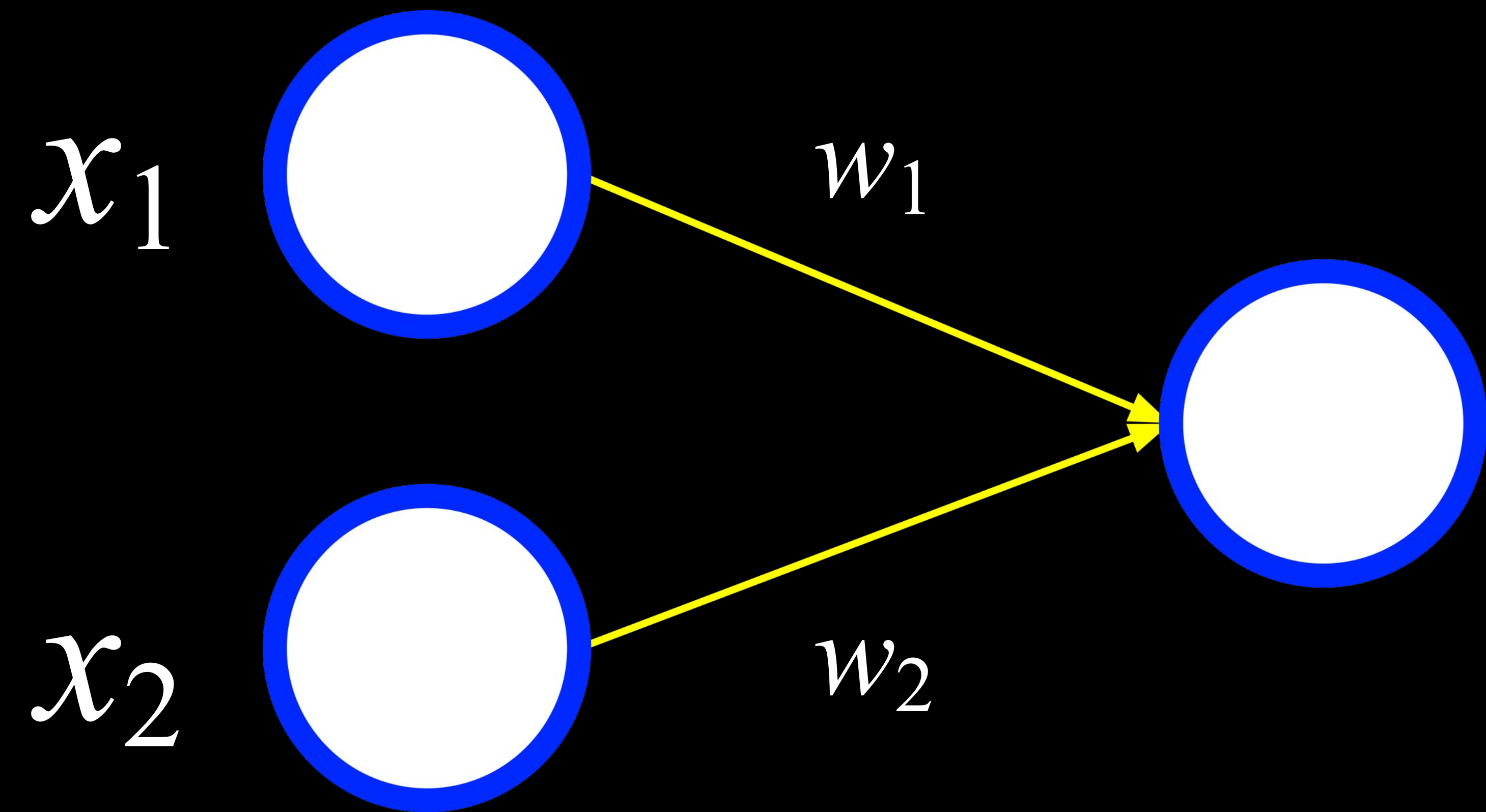
$$h(x_1, x_2) = w_0 + w_1x_1 + w_2x_2$$

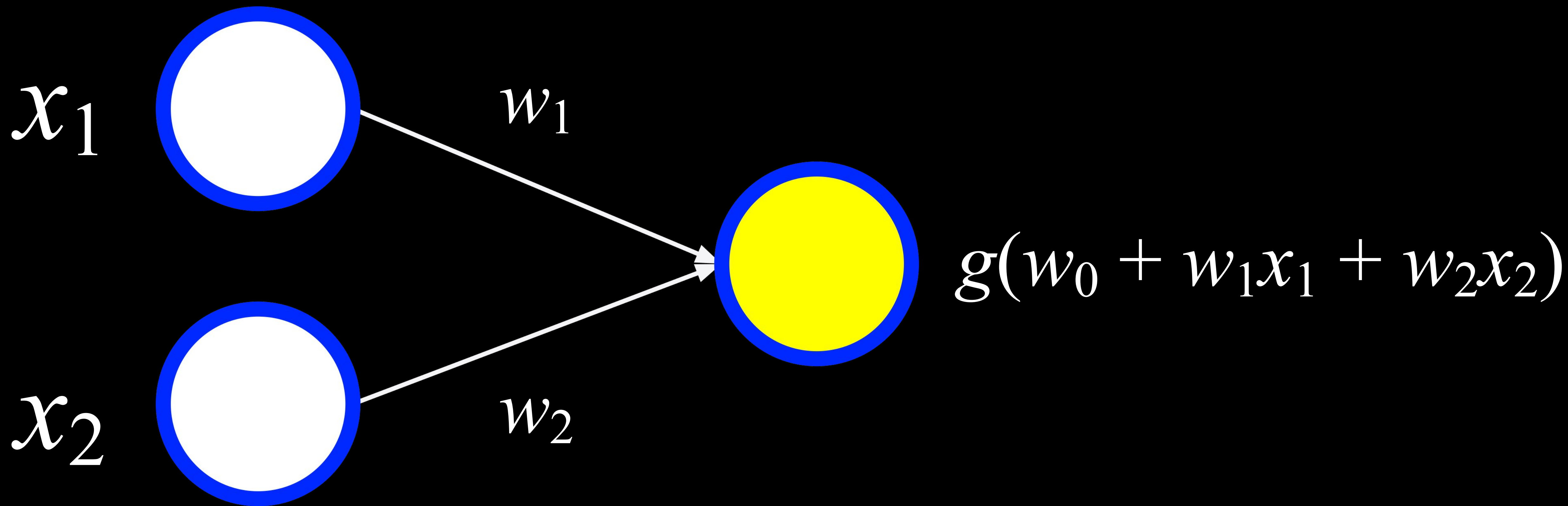
$$h(x_1, x_2) = g(w_0 + w_1x_1 + w_2x_2)$$

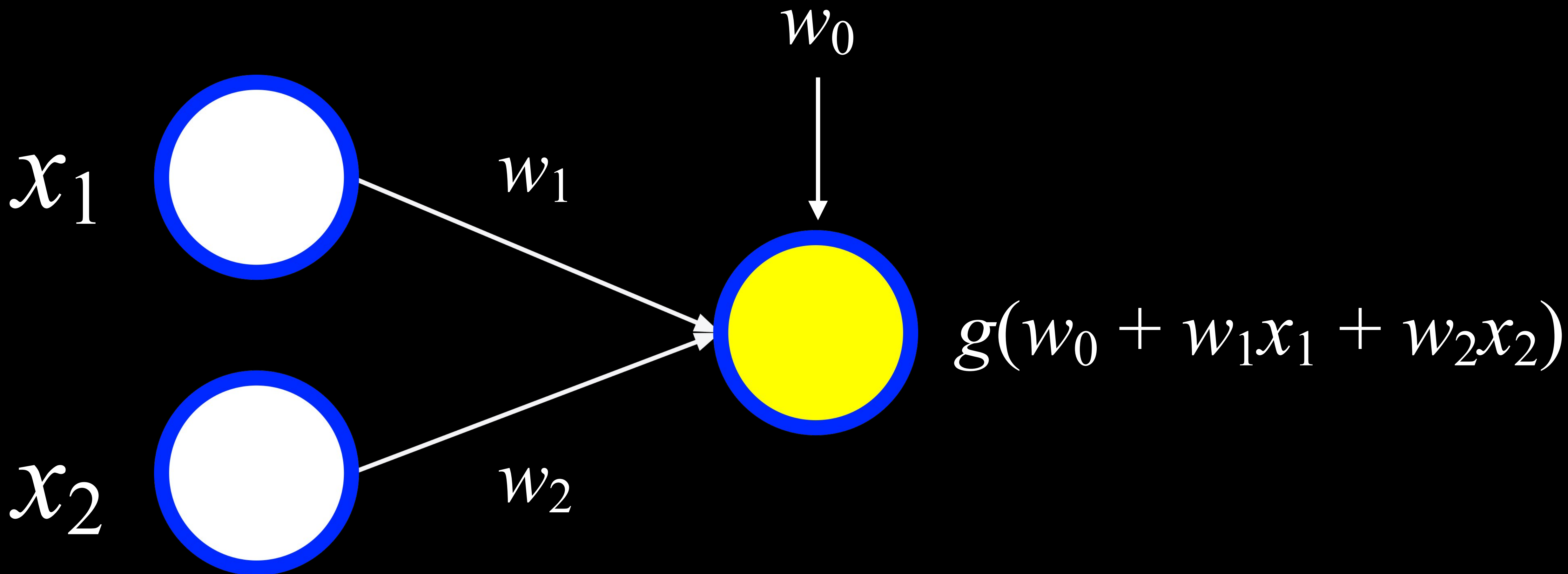






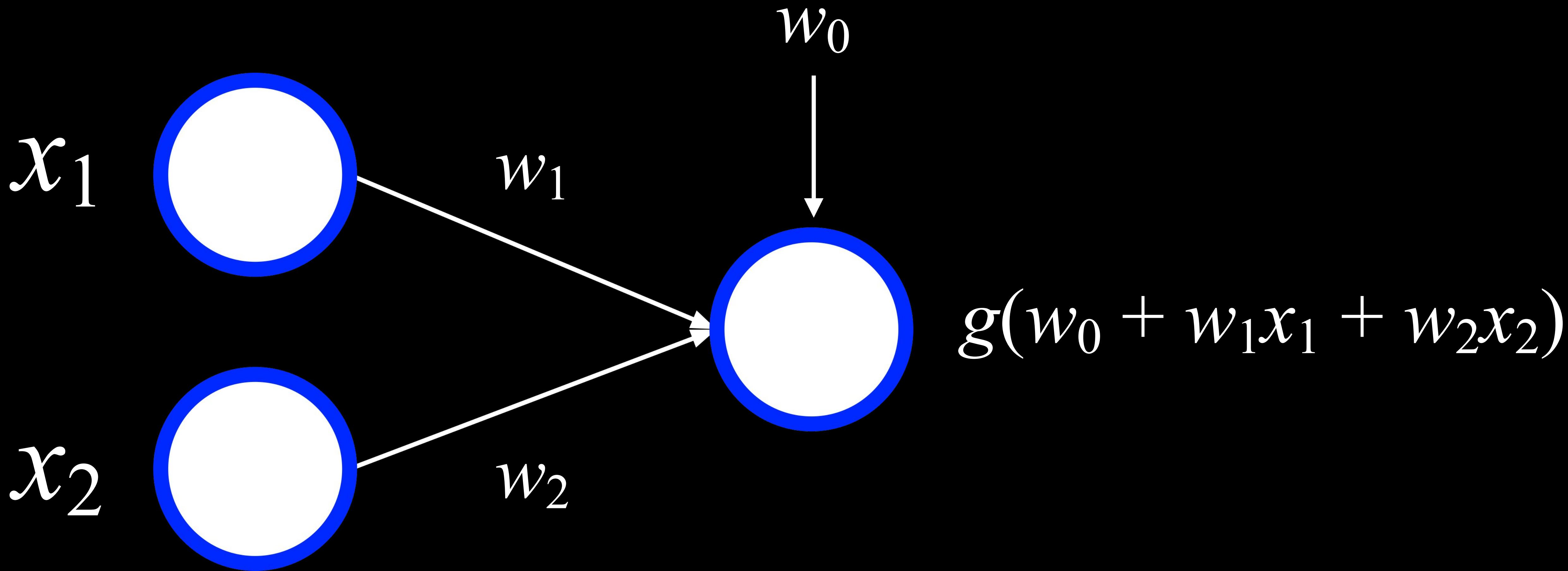


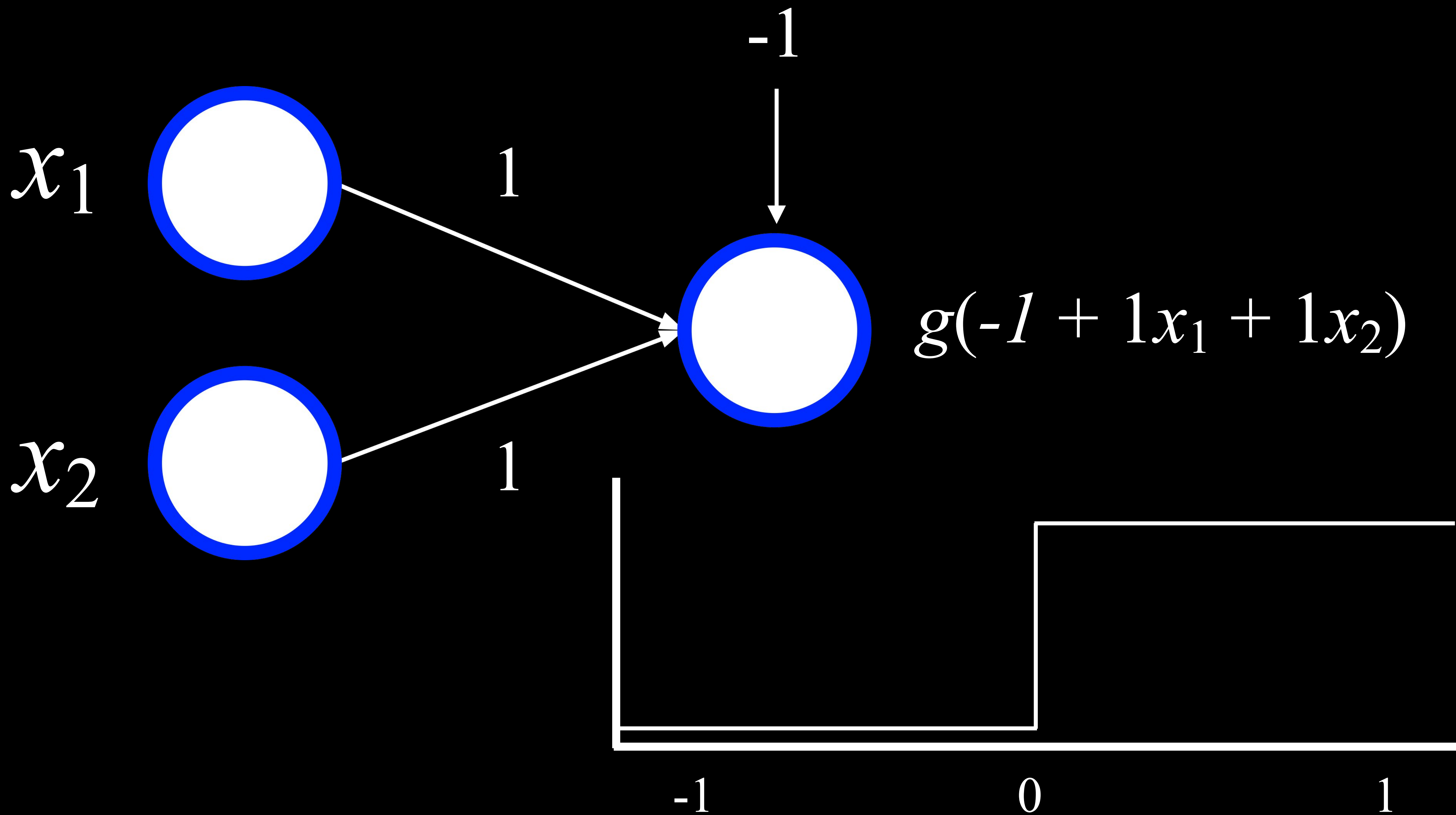


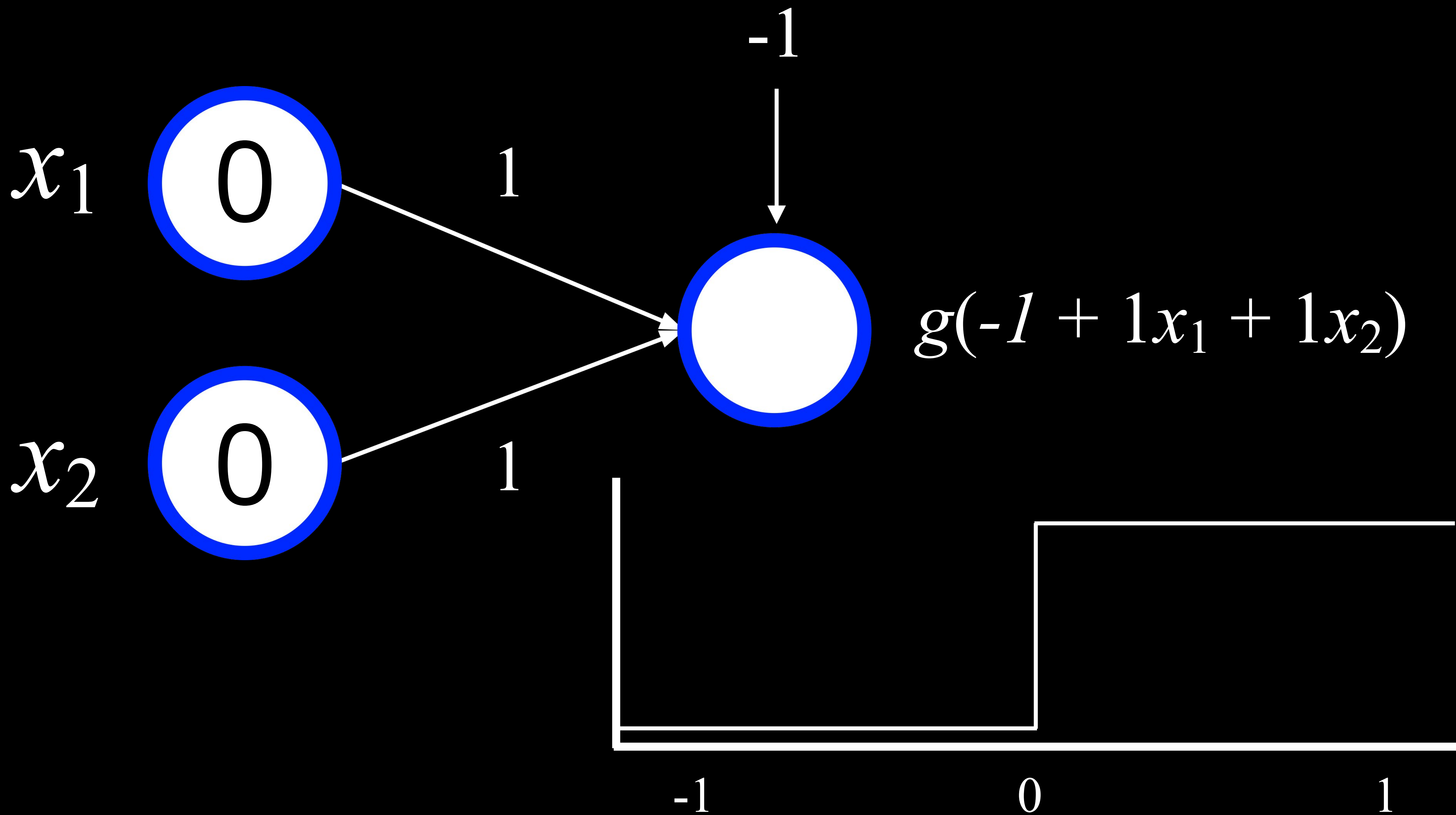


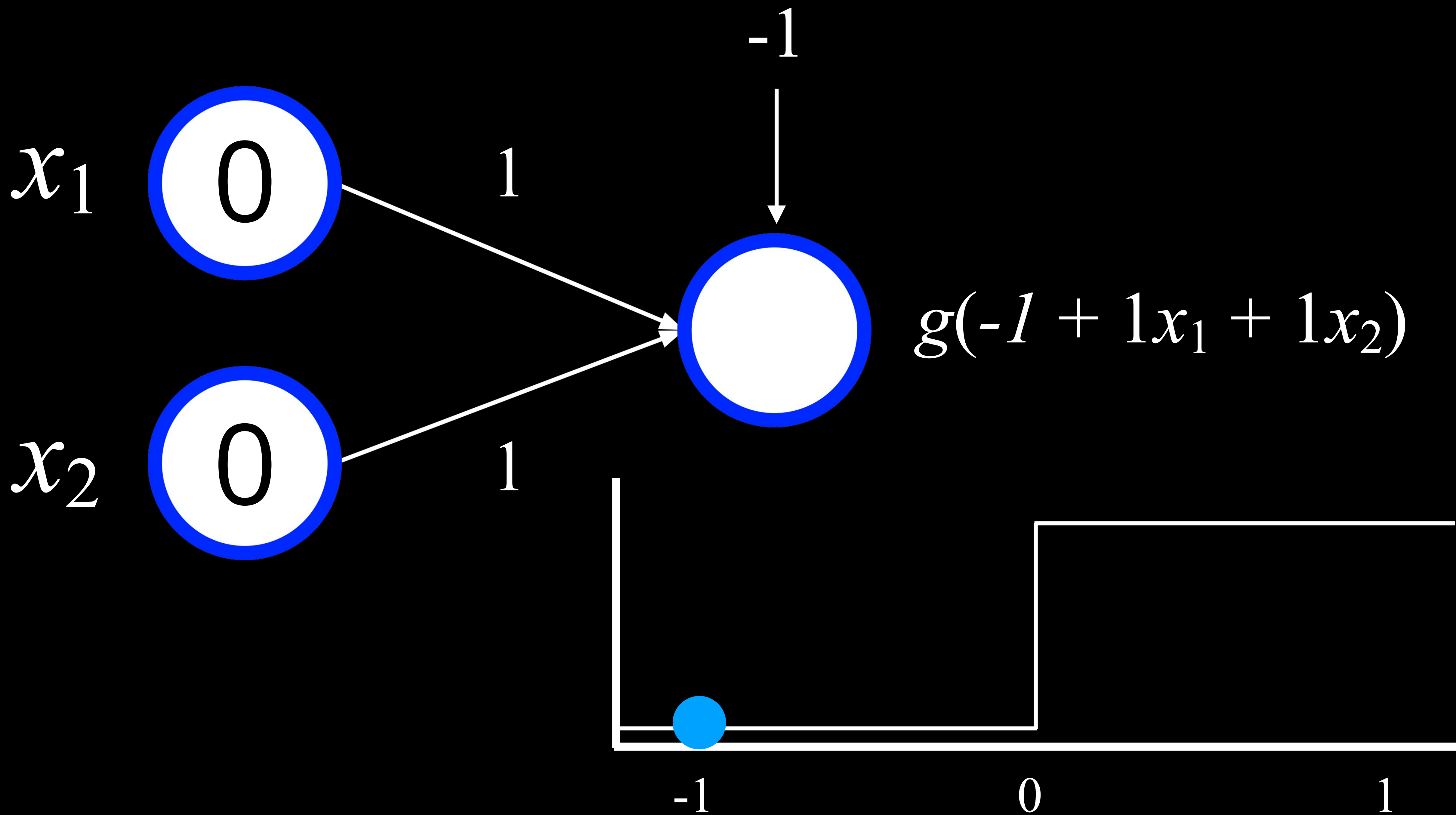
# Ou

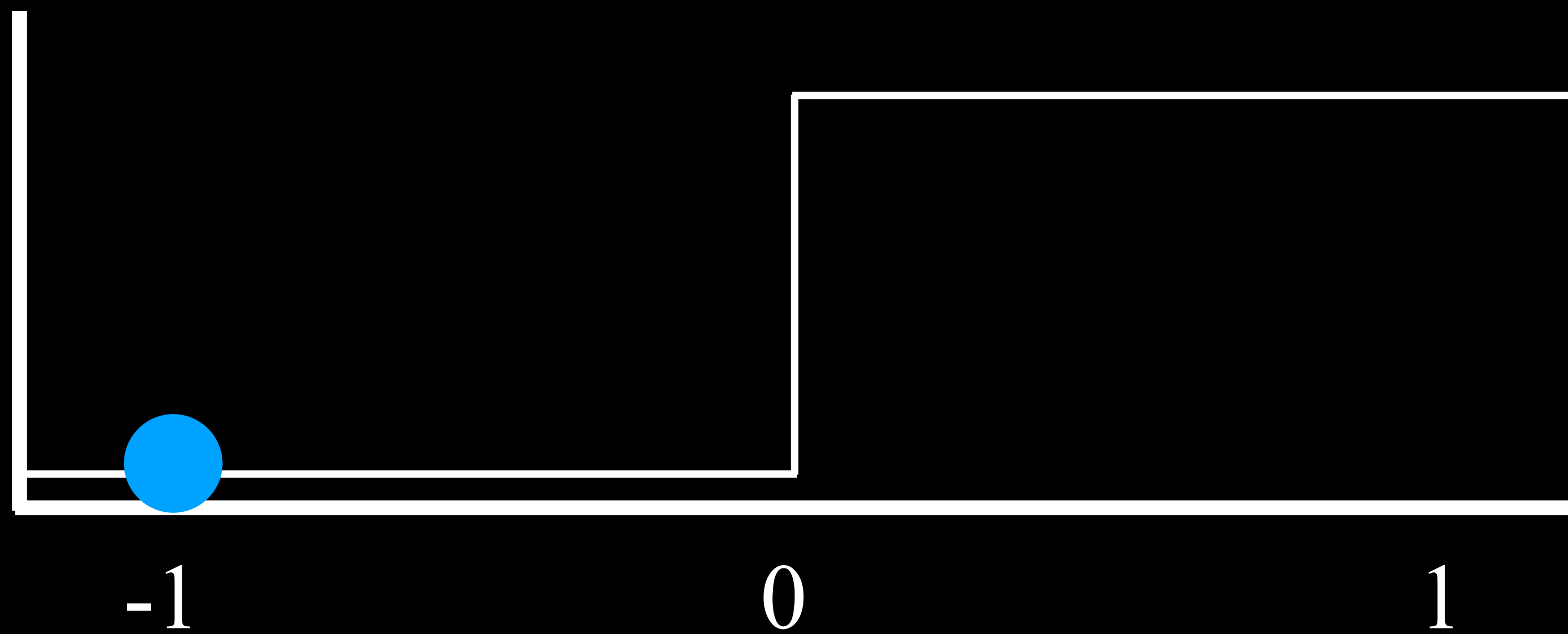
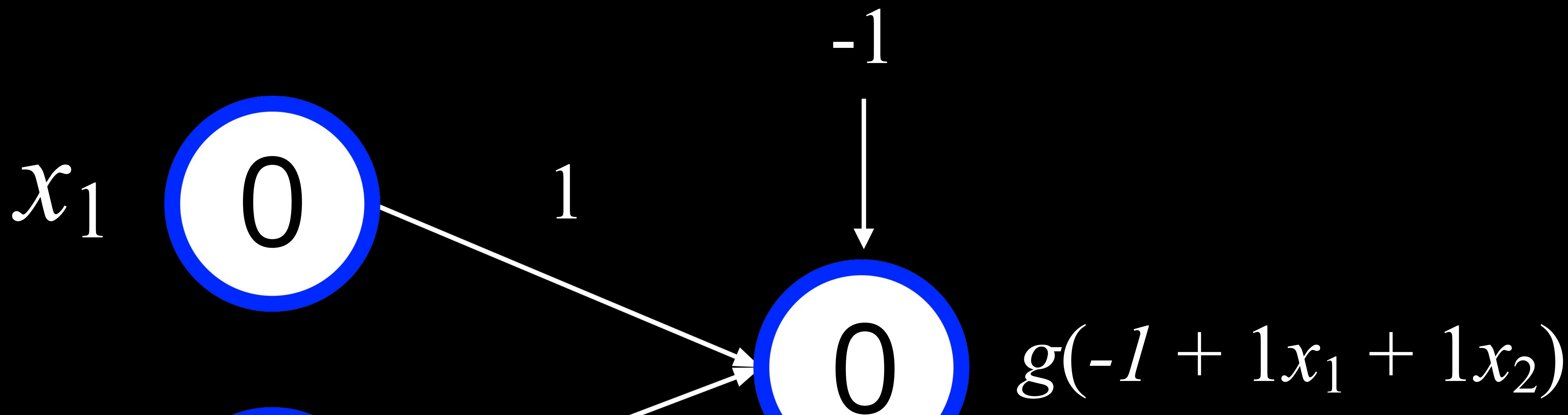
| $x$ | $y$ | $f(x, y)$ |
|-----|-----|-----------|
| 0   | 0   | 0         |
| 0   | 1   | 1         |
| 1   | 0   | 1         |
| 1   | 1   | 1         |

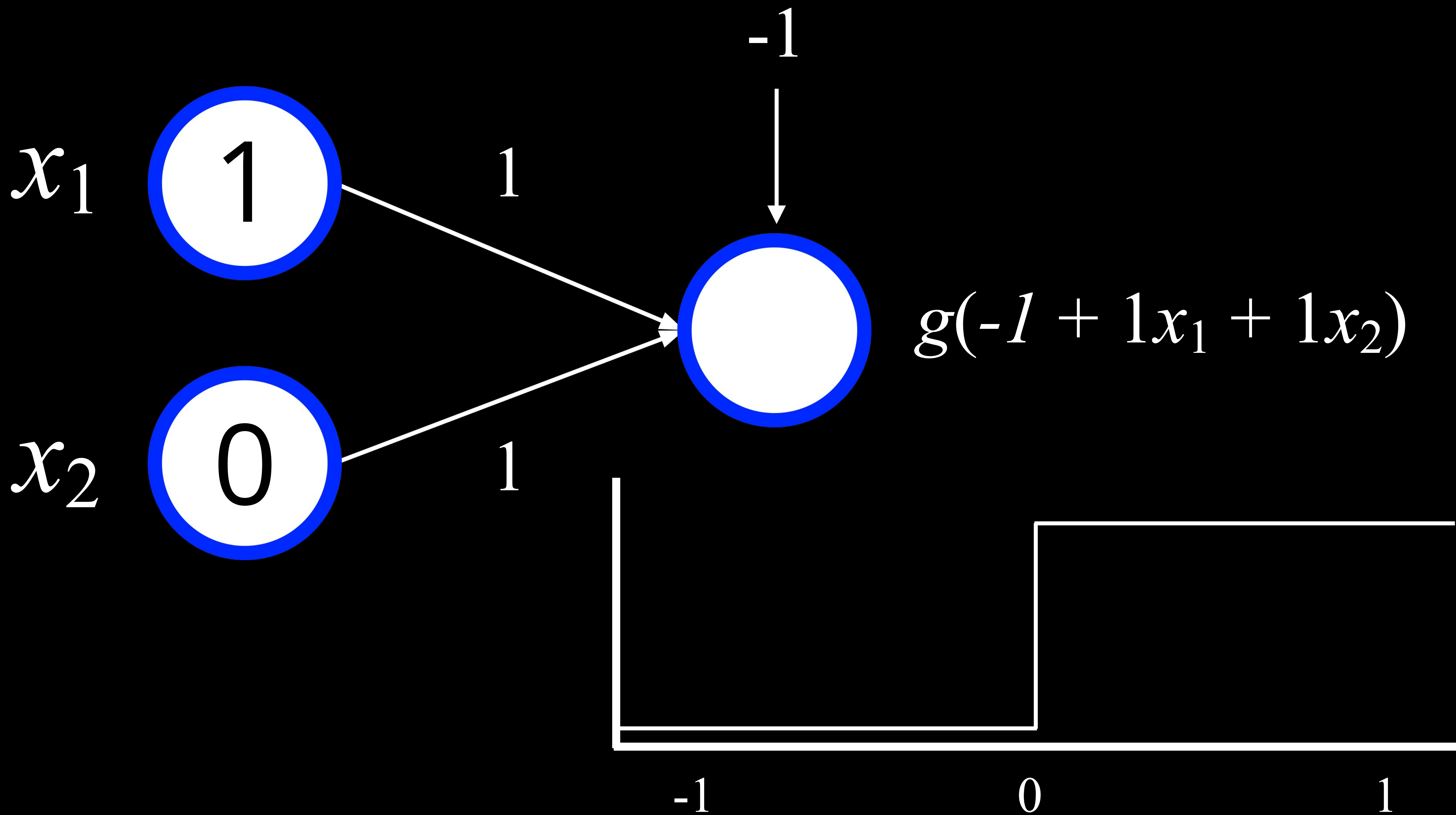


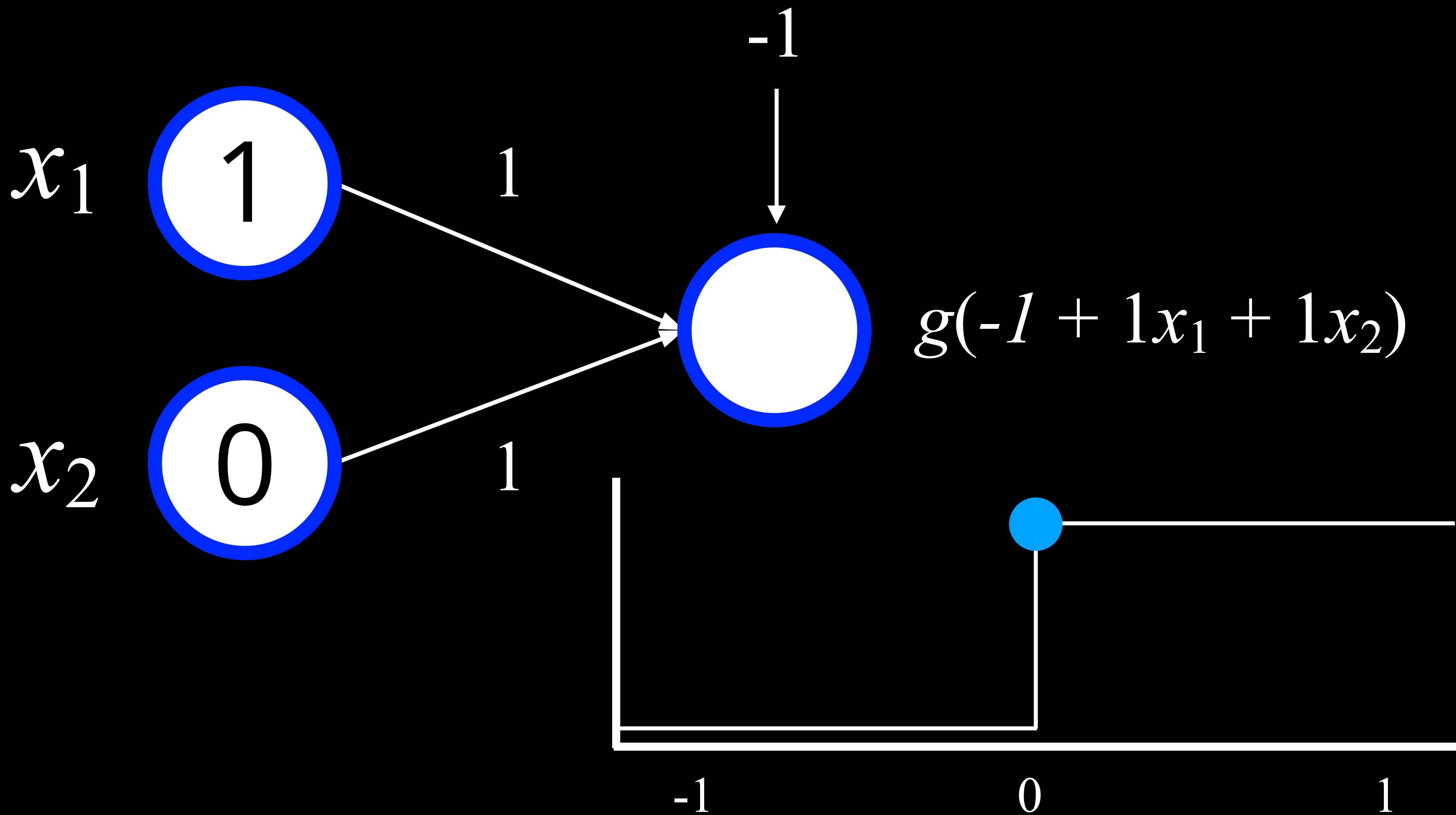


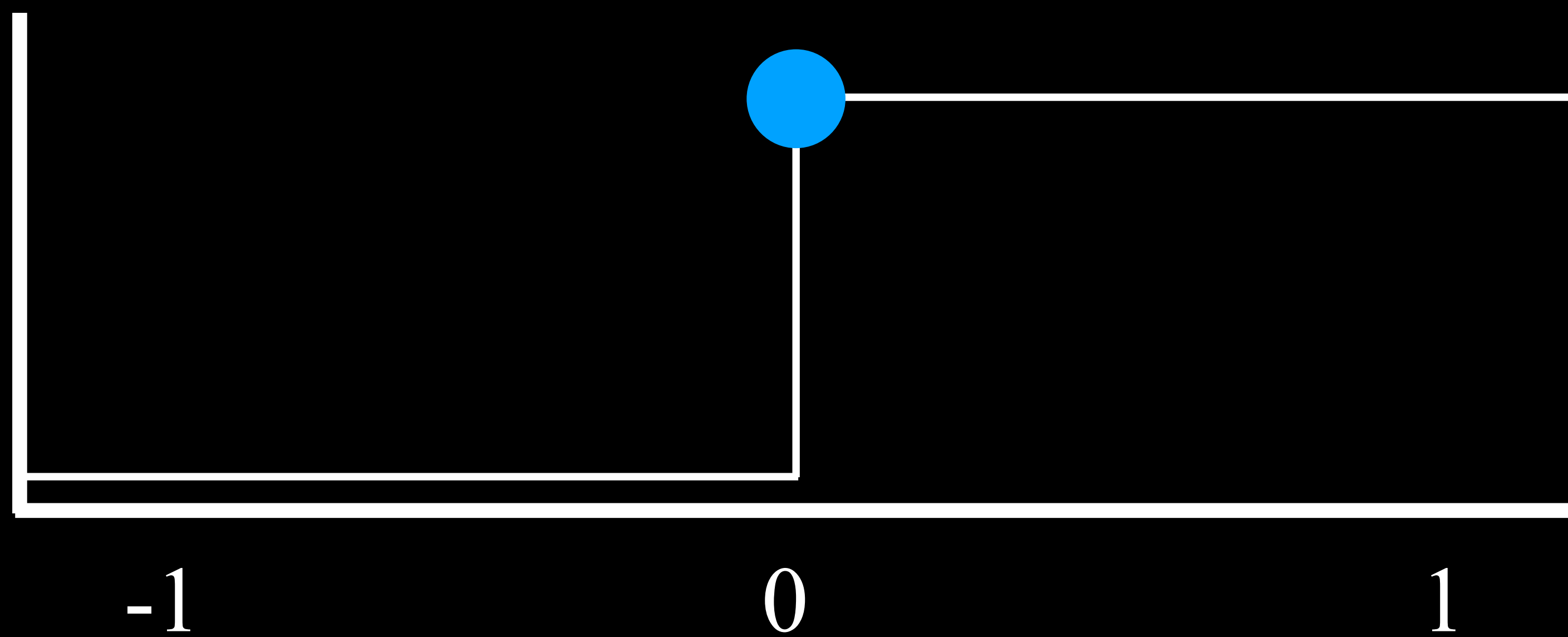
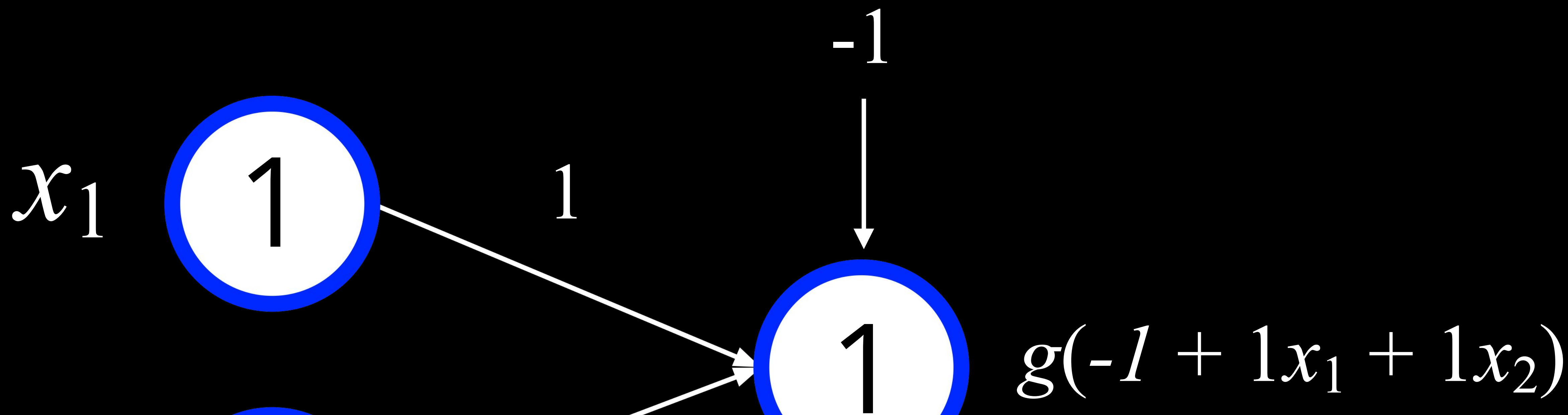


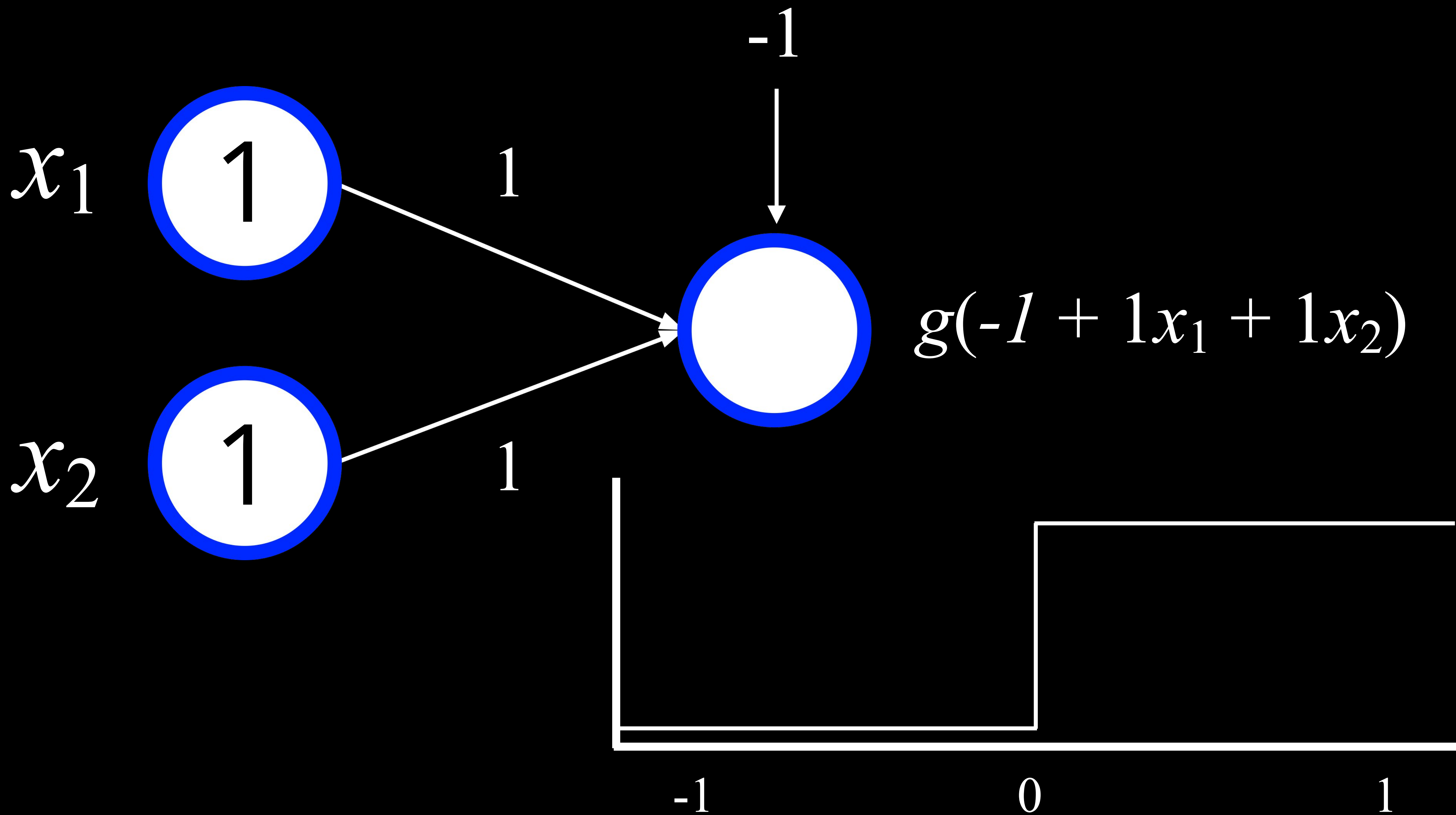


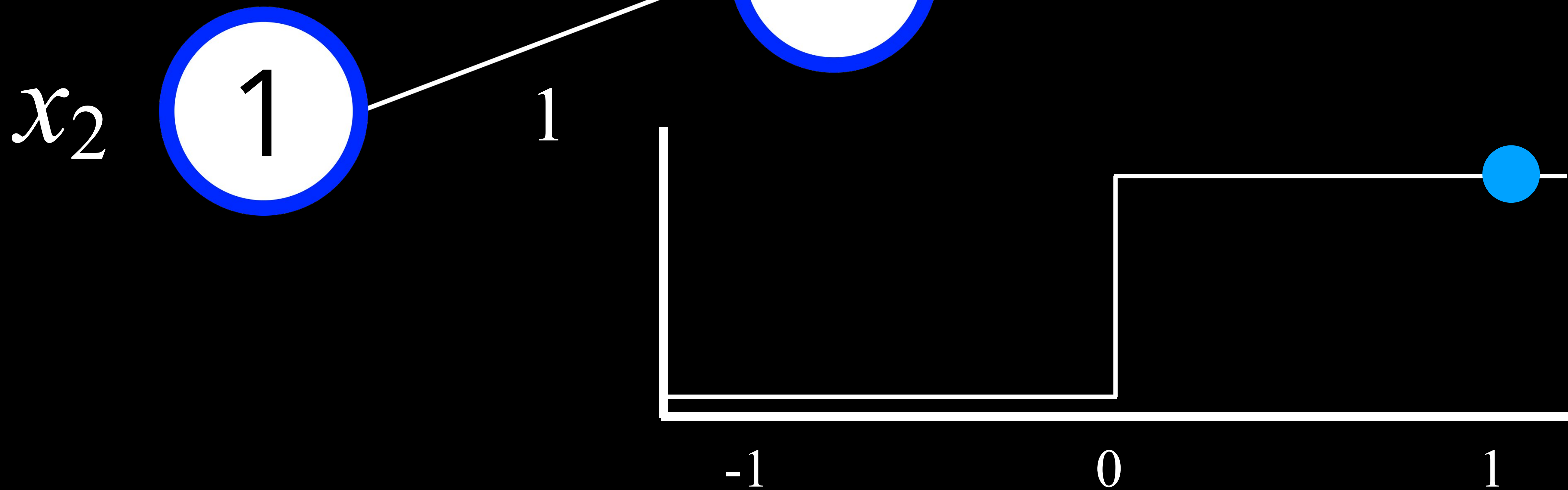
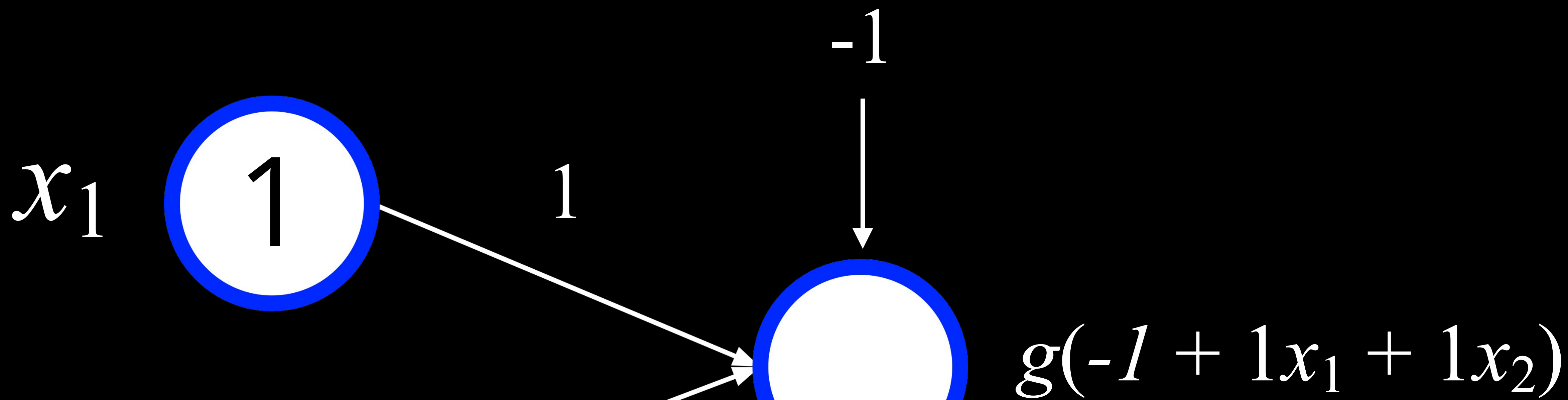


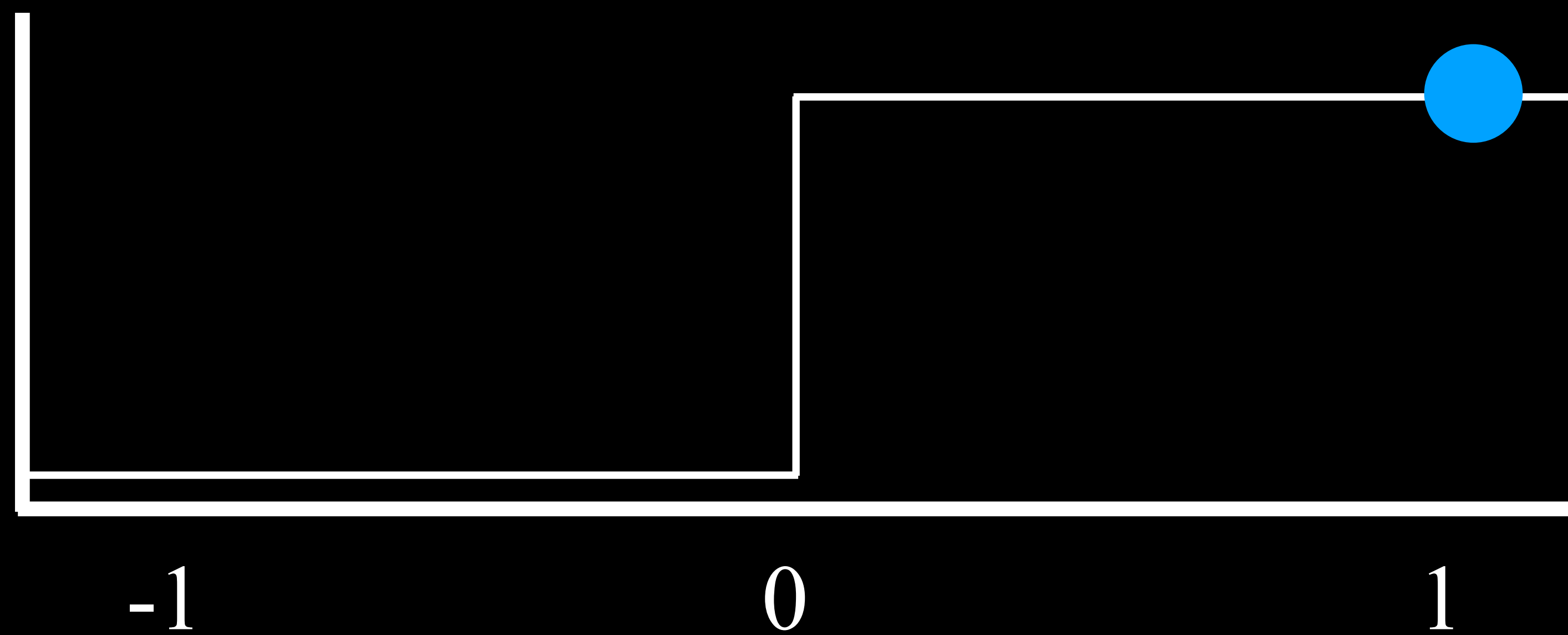
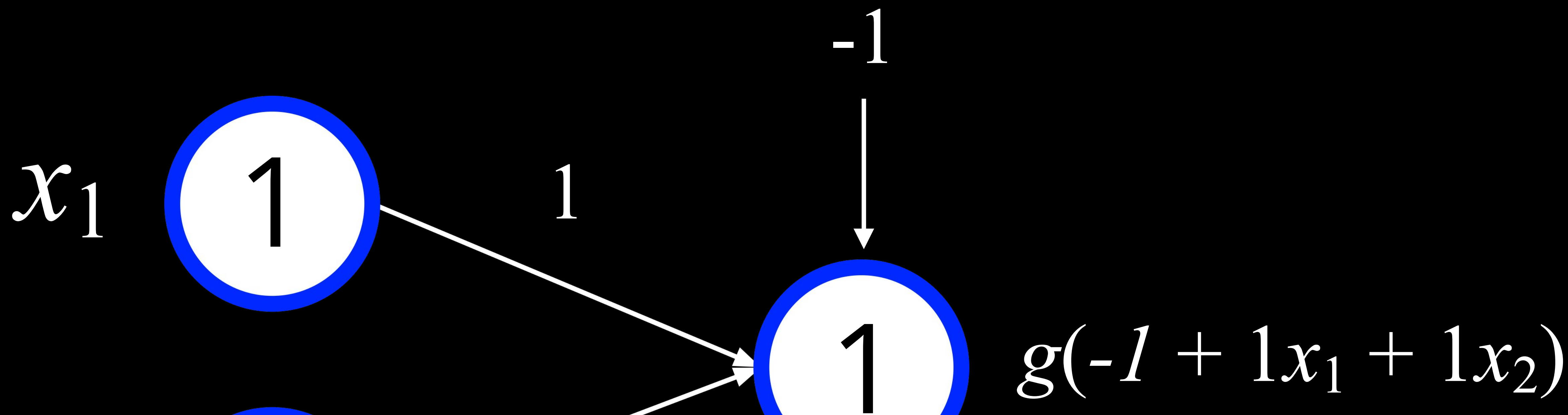






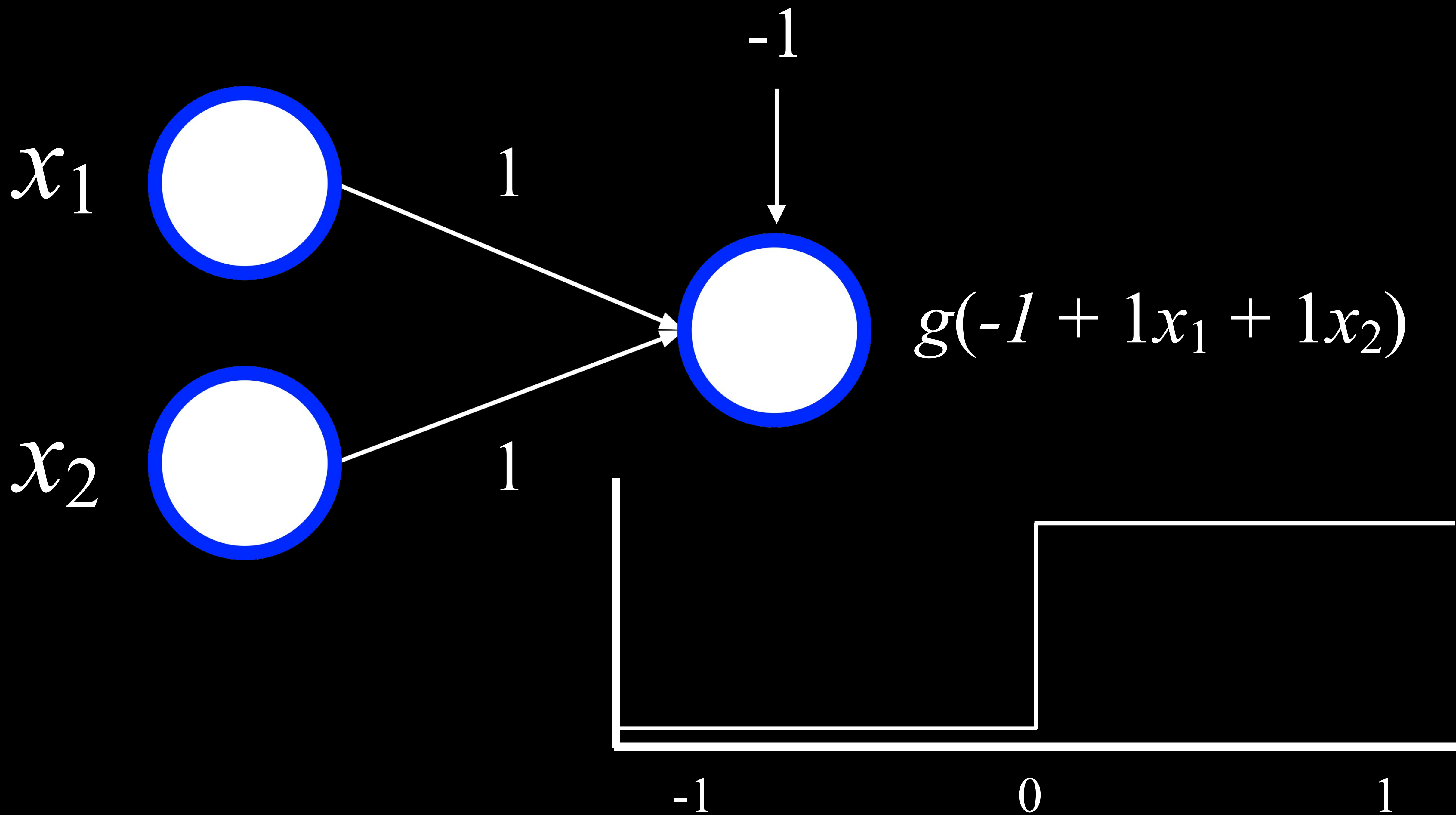


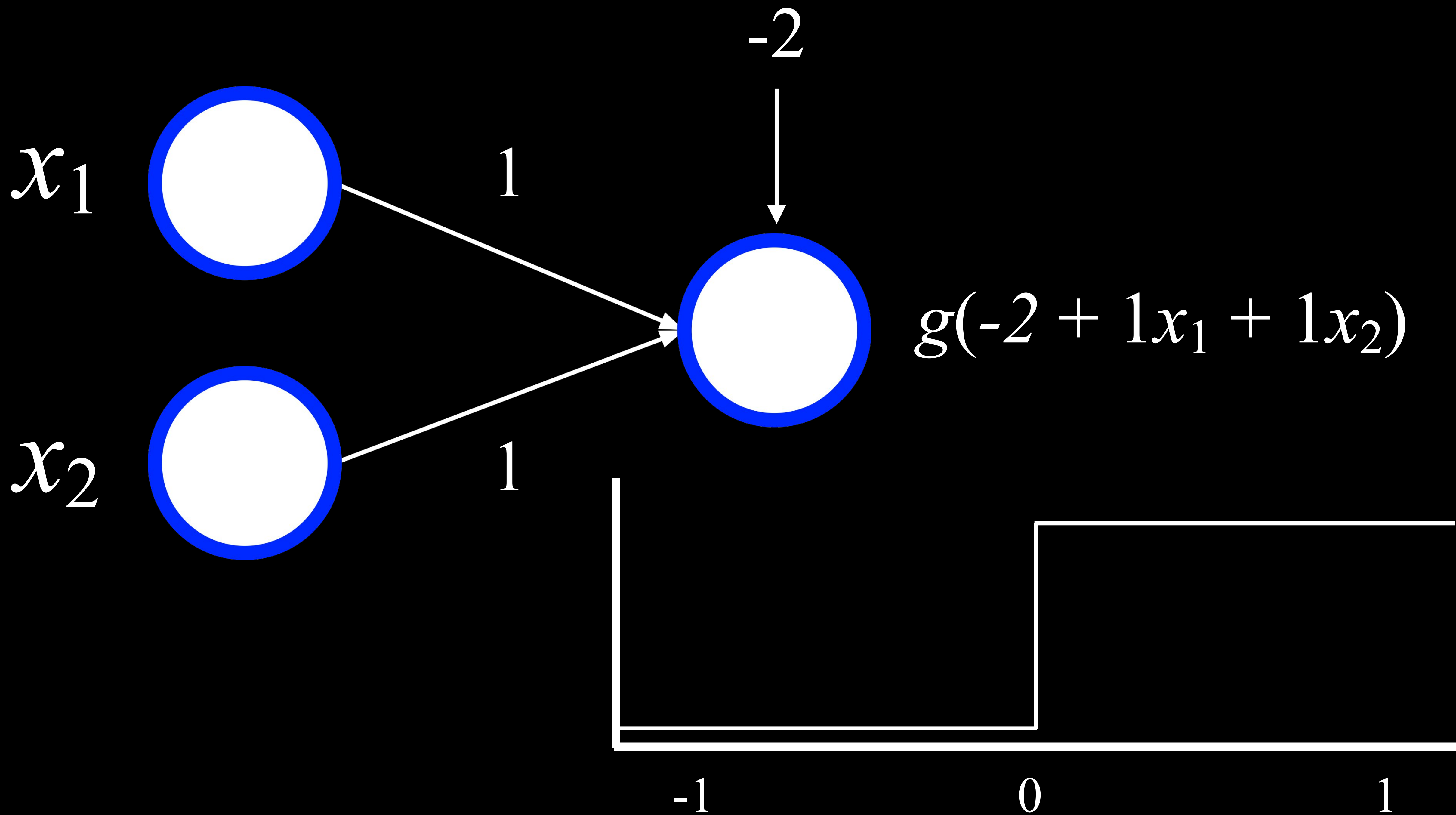


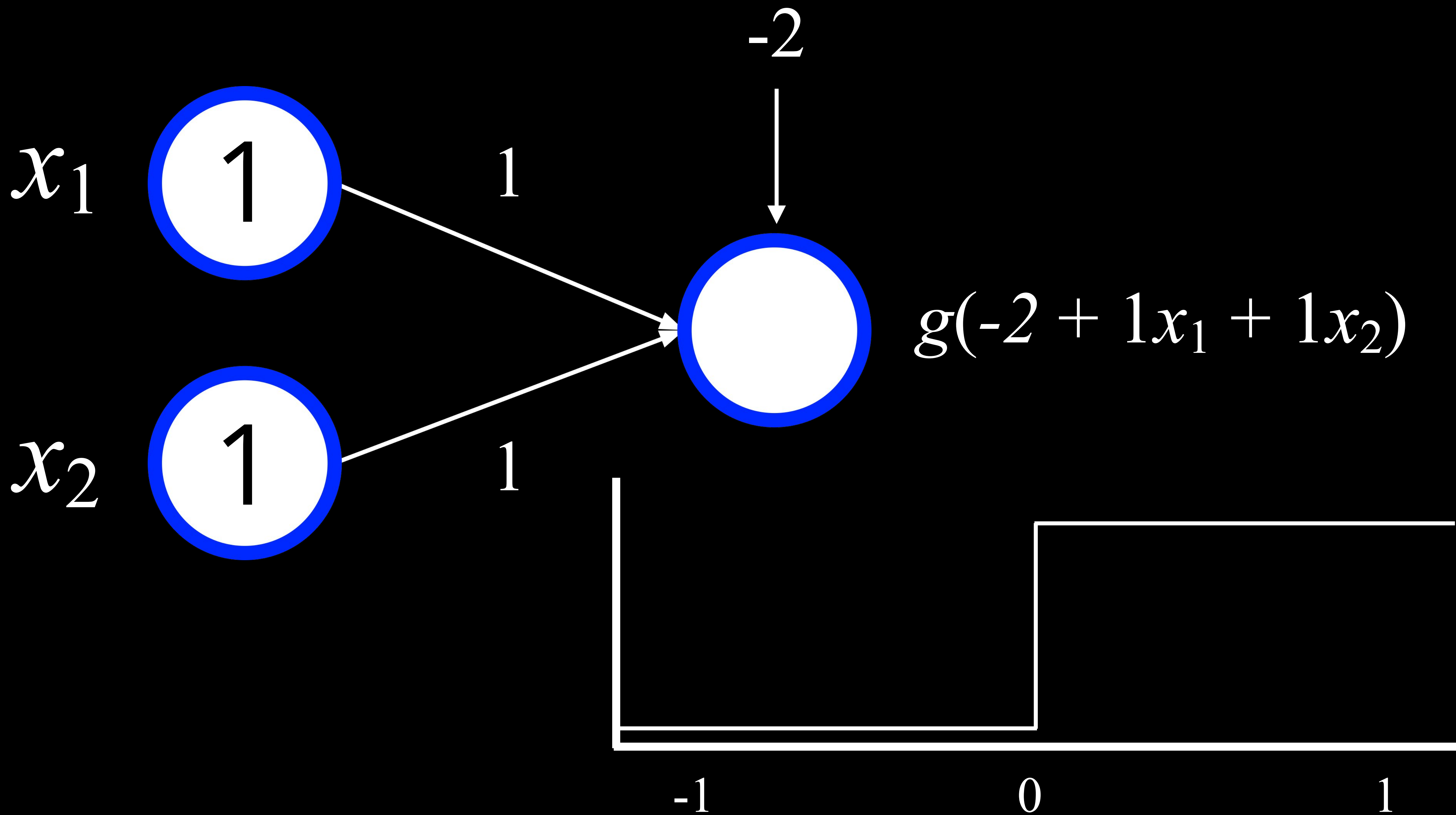


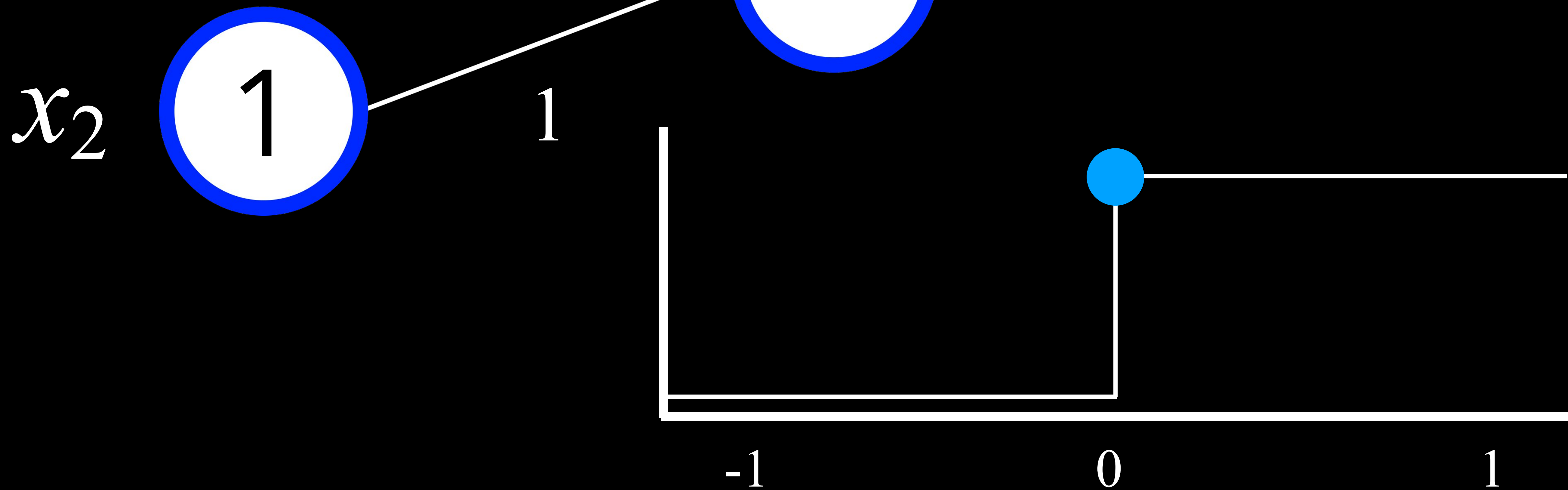
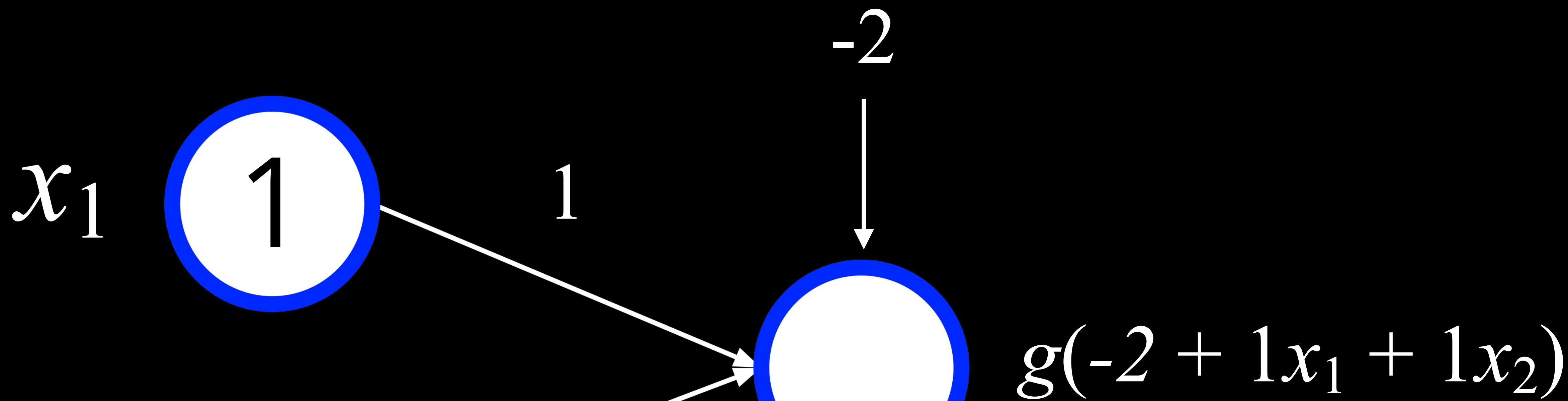
E

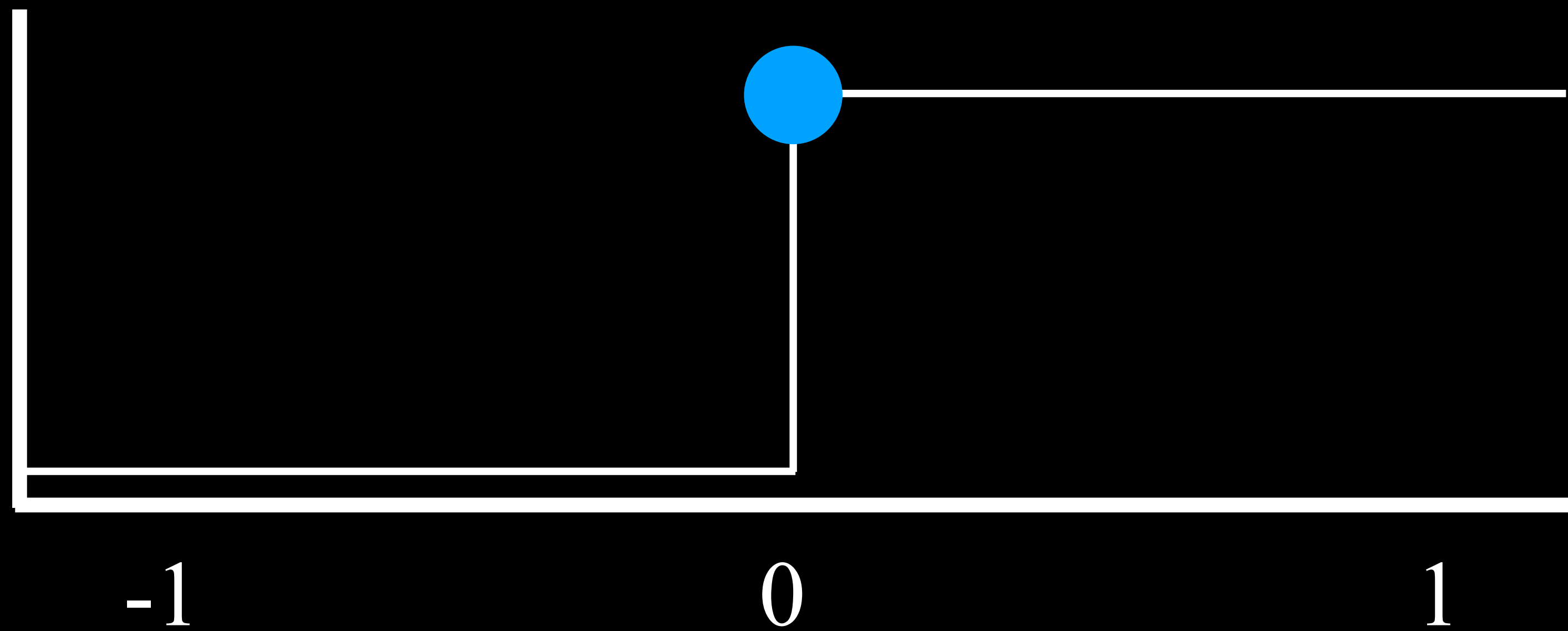
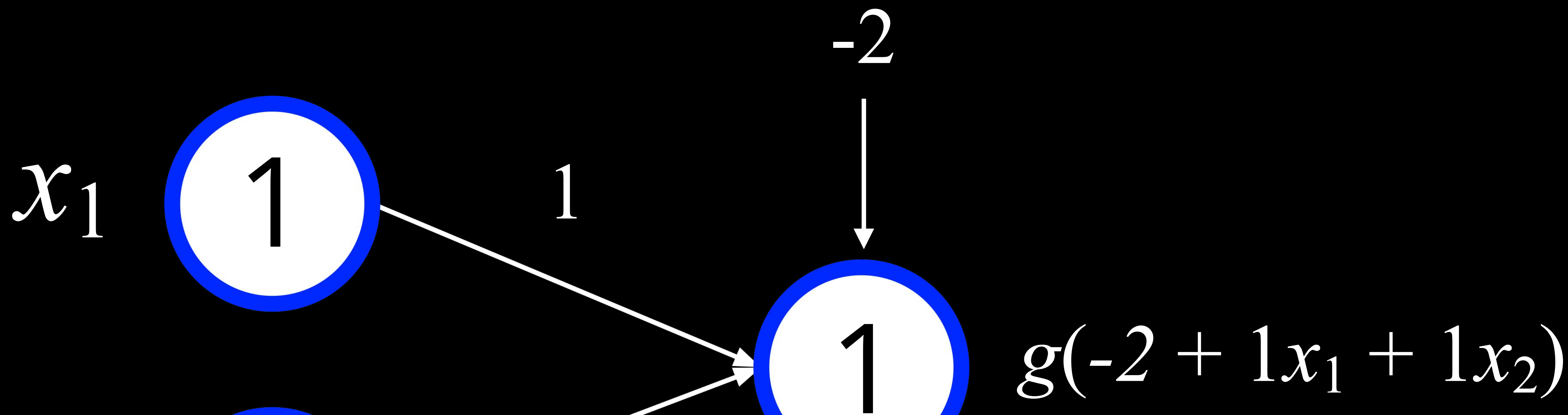
| $x$ | $y$ | $f(x, y)$ |
|-----|-----|-----------|
| 0   | 0   | 0         |
| 0   | 1   | 0         |
| 1   | 0   | 0         |
| 1   | 1   | 1         |

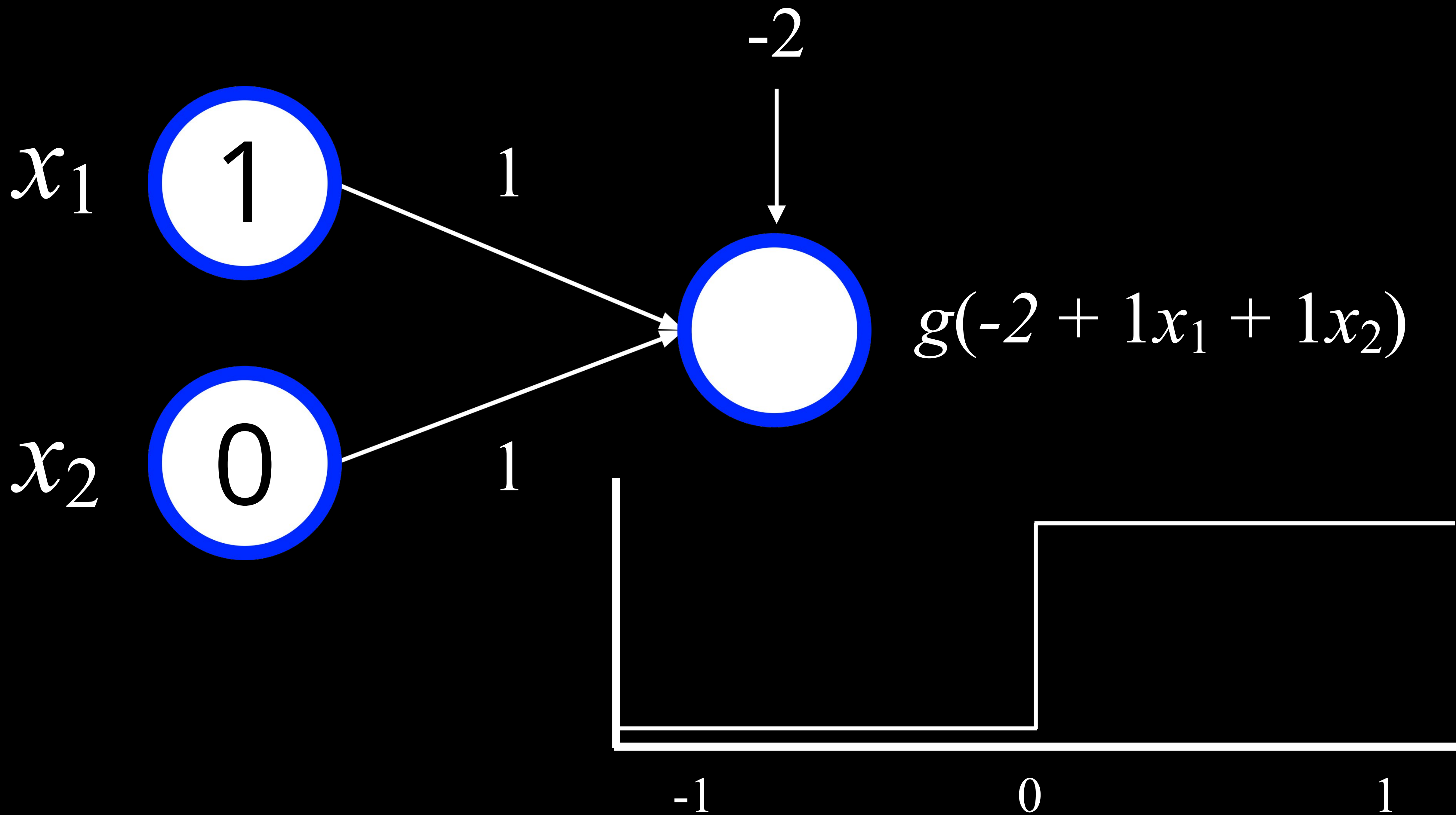


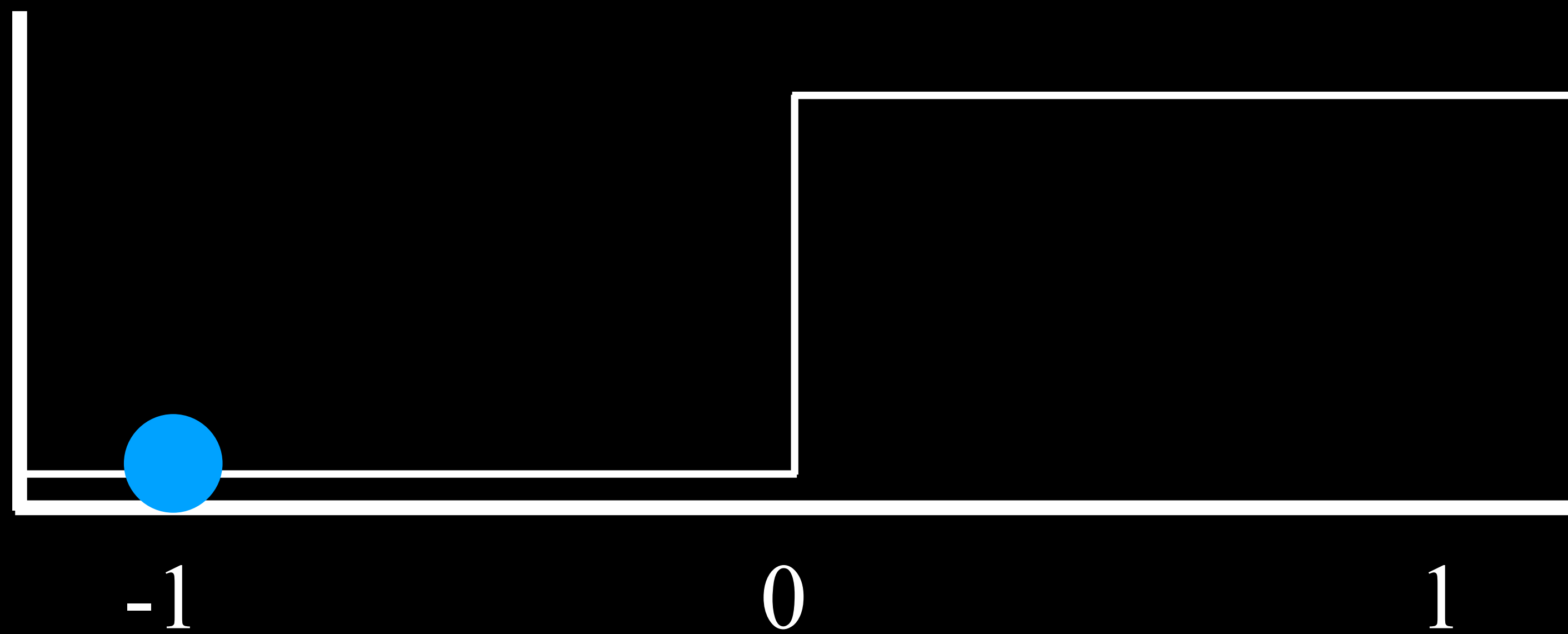
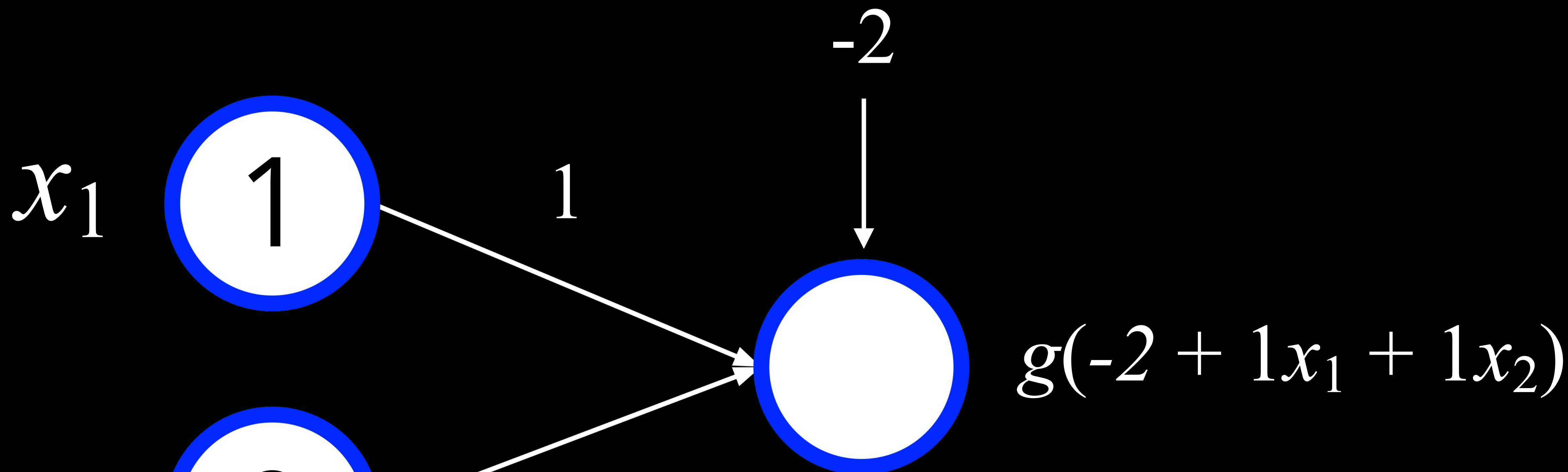


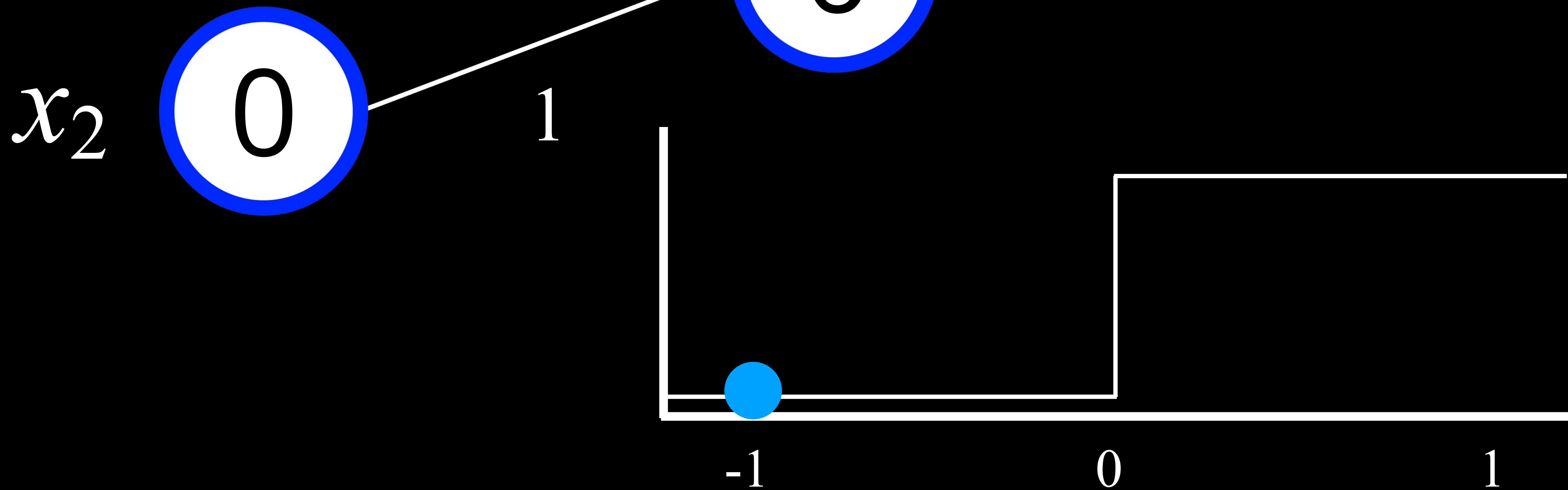
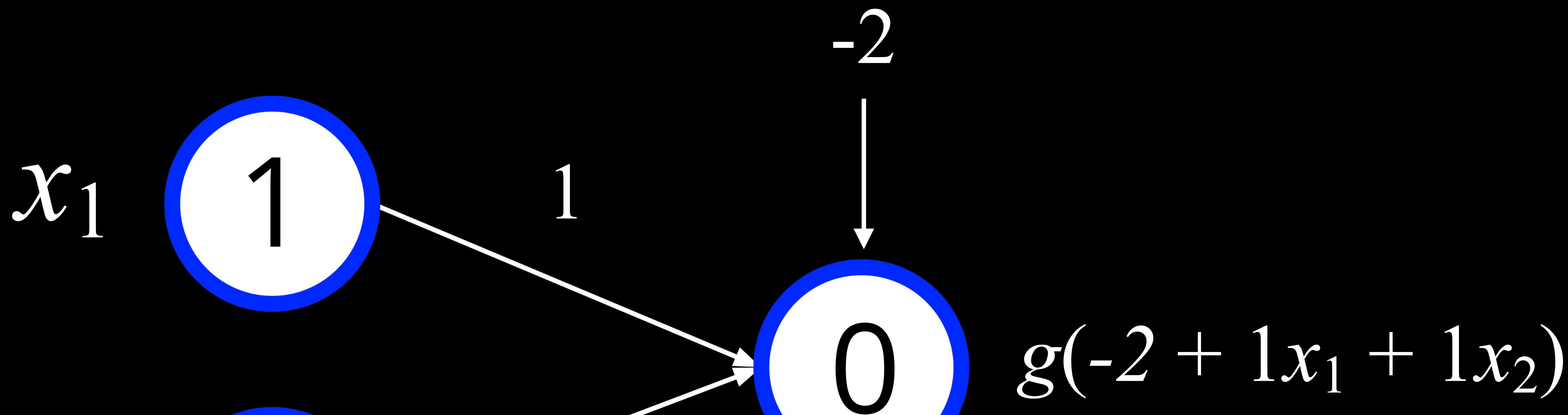


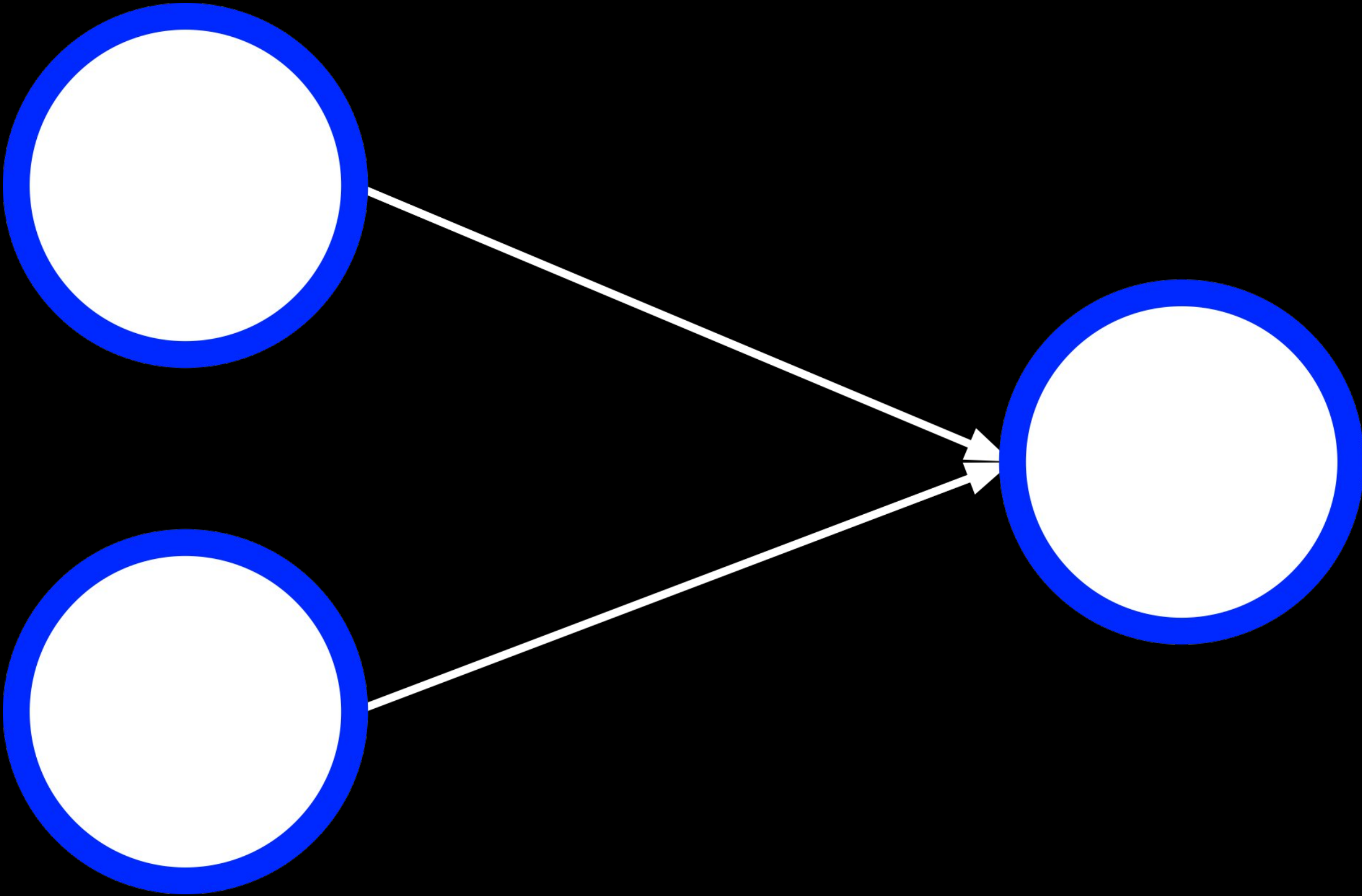


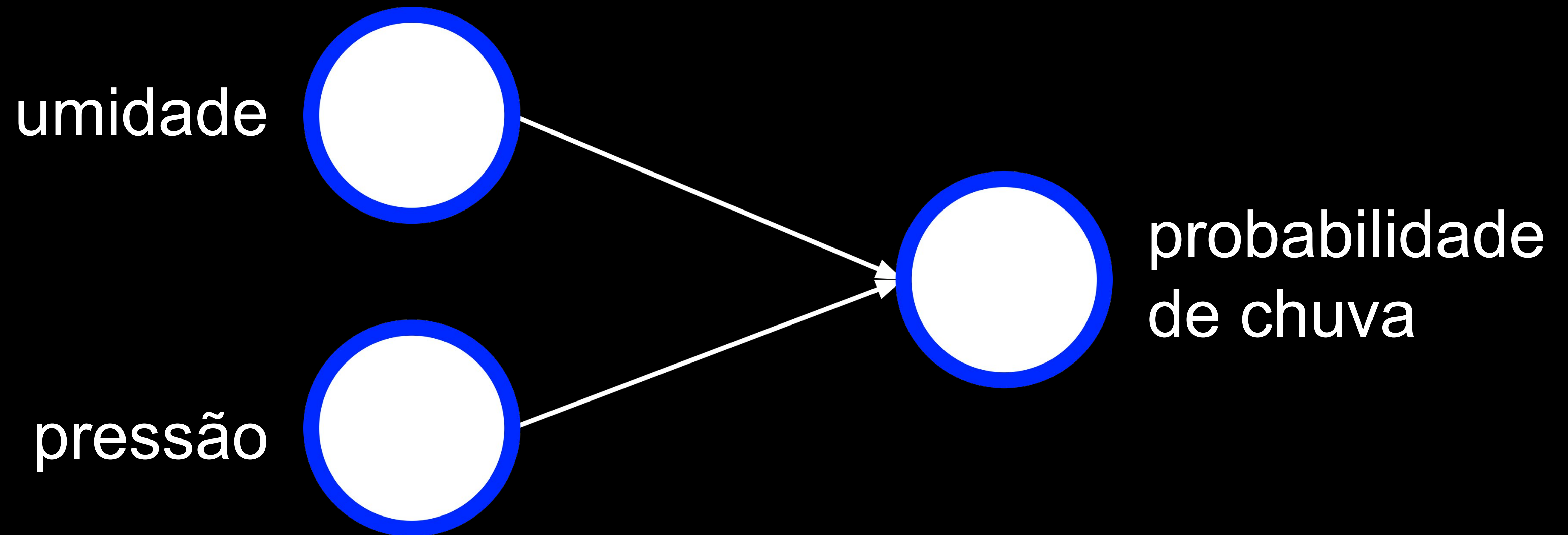


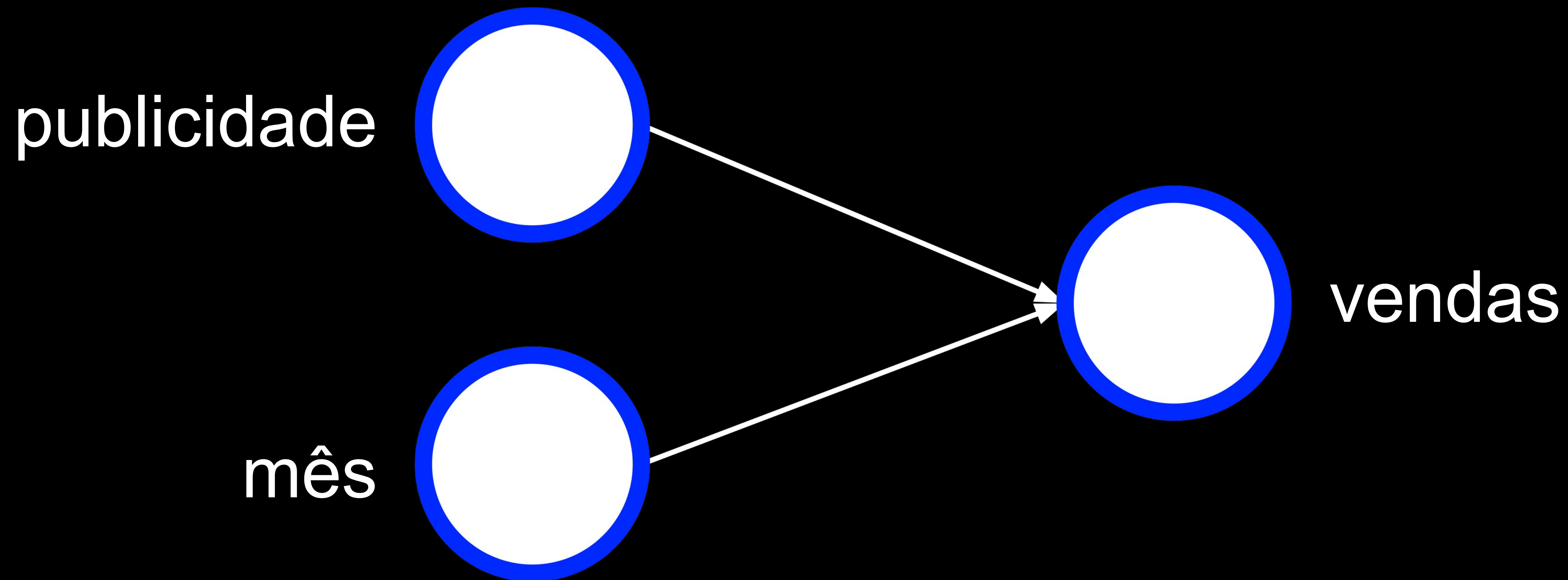


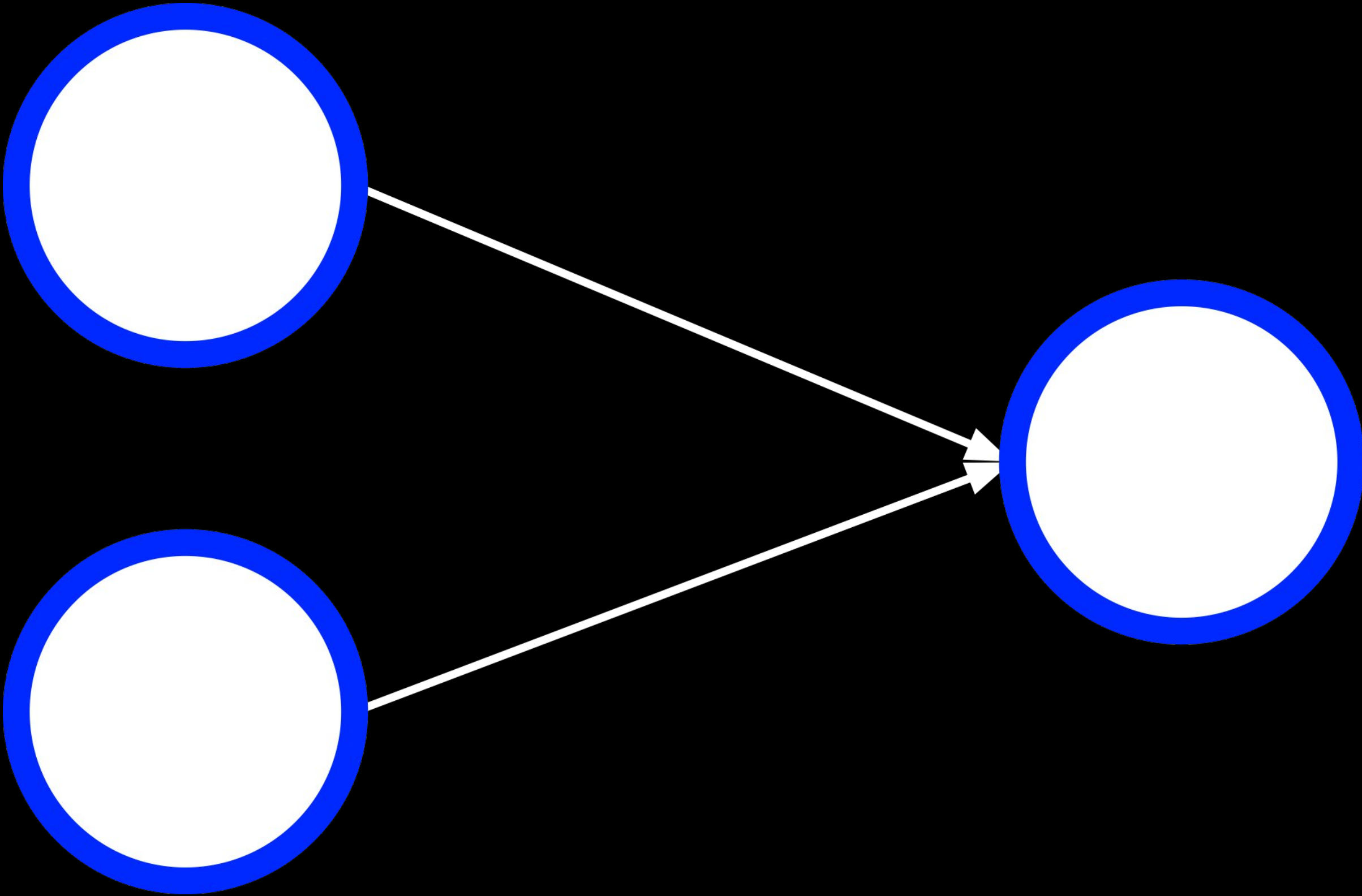


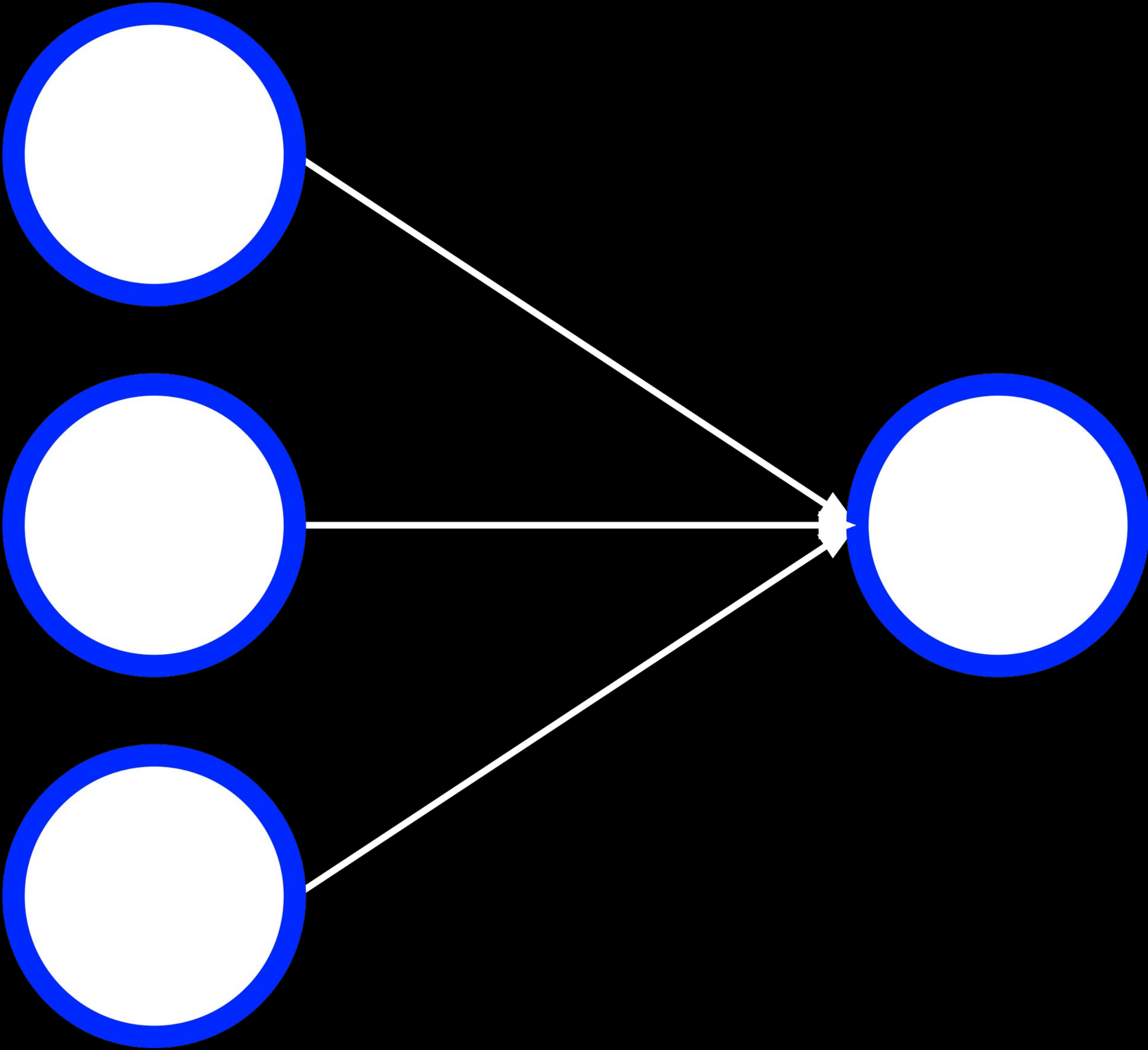


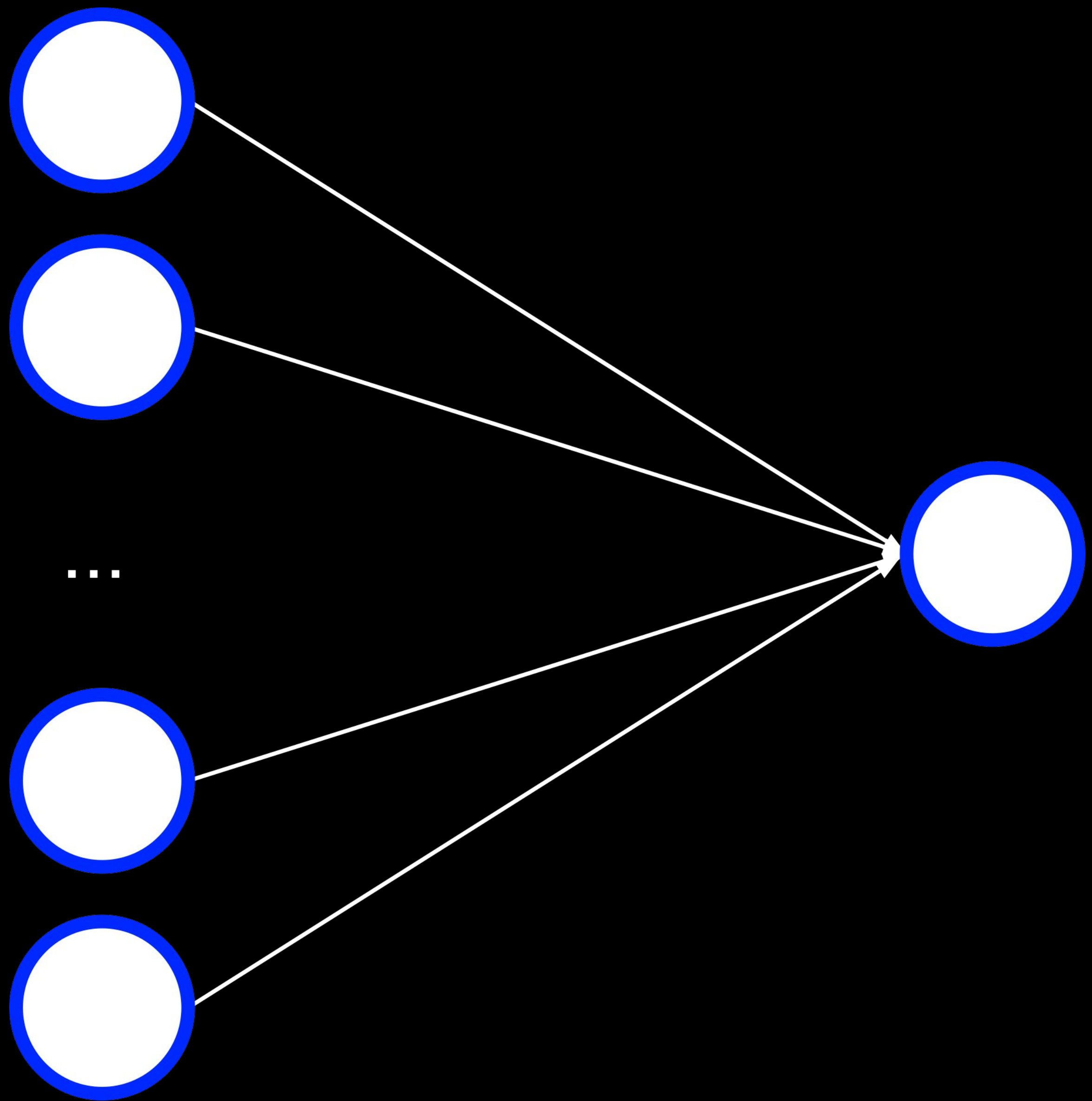


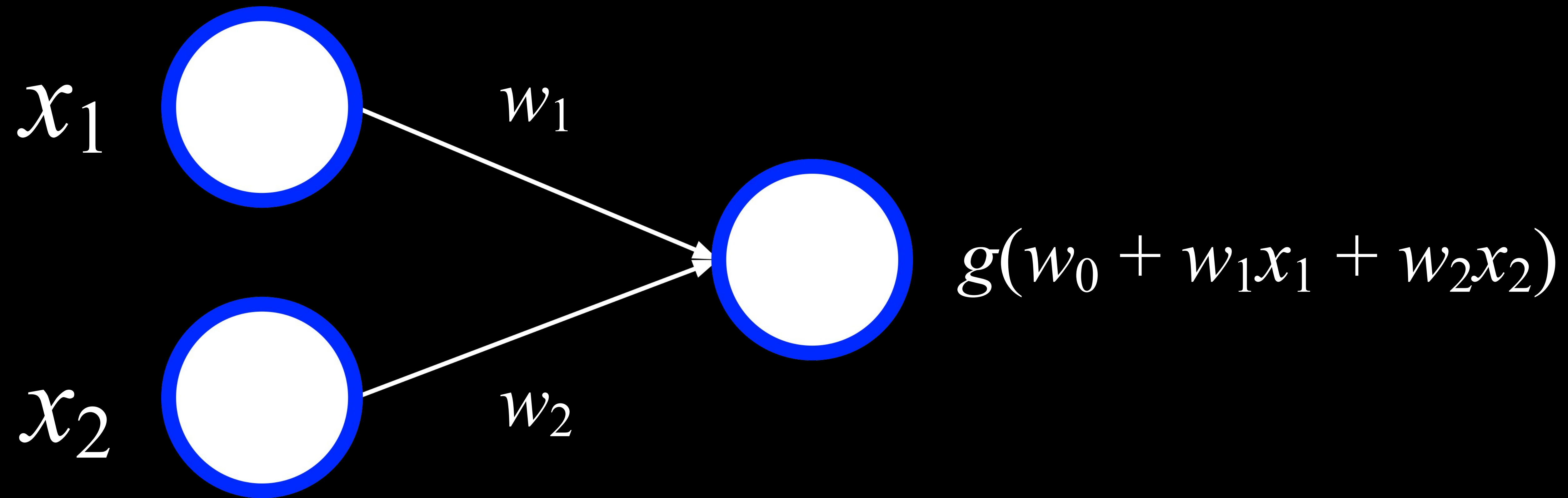


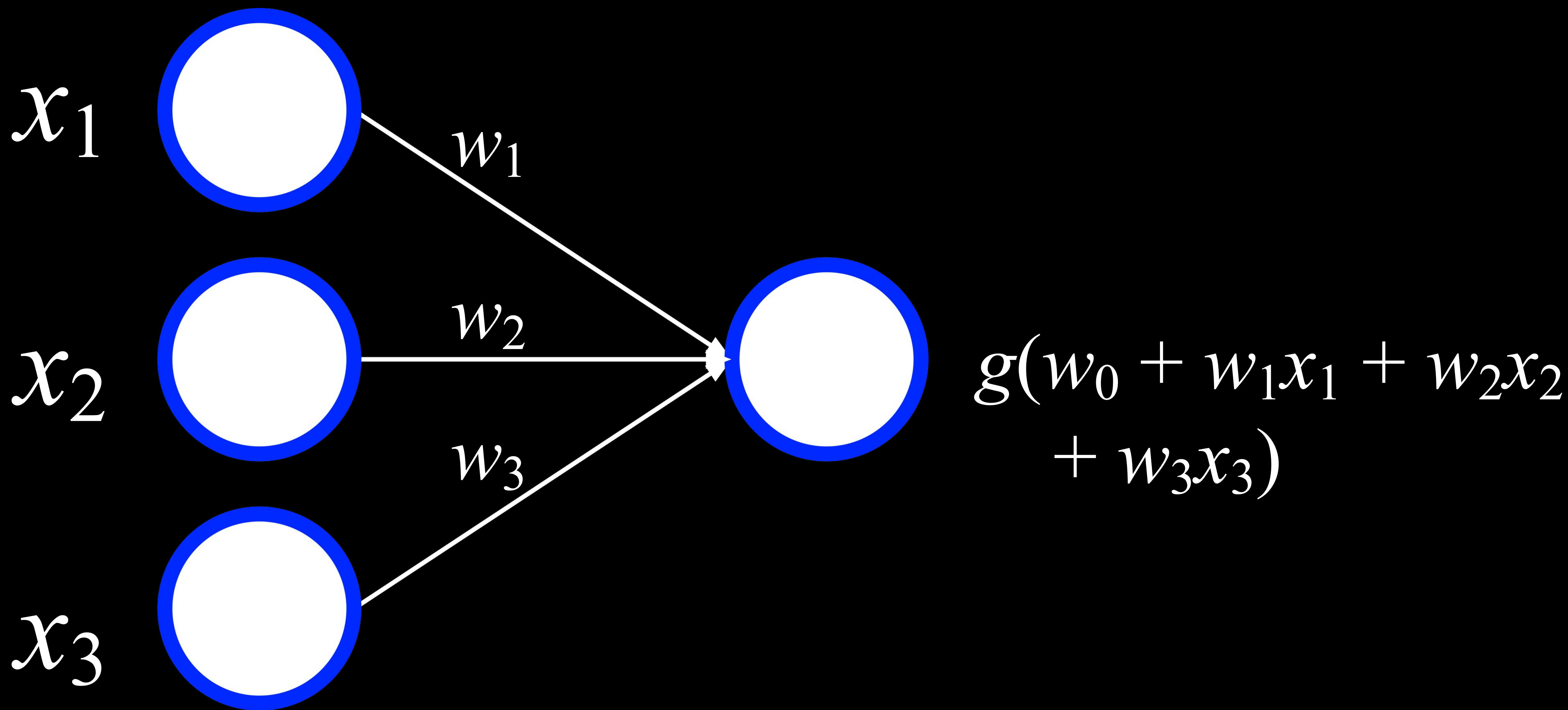


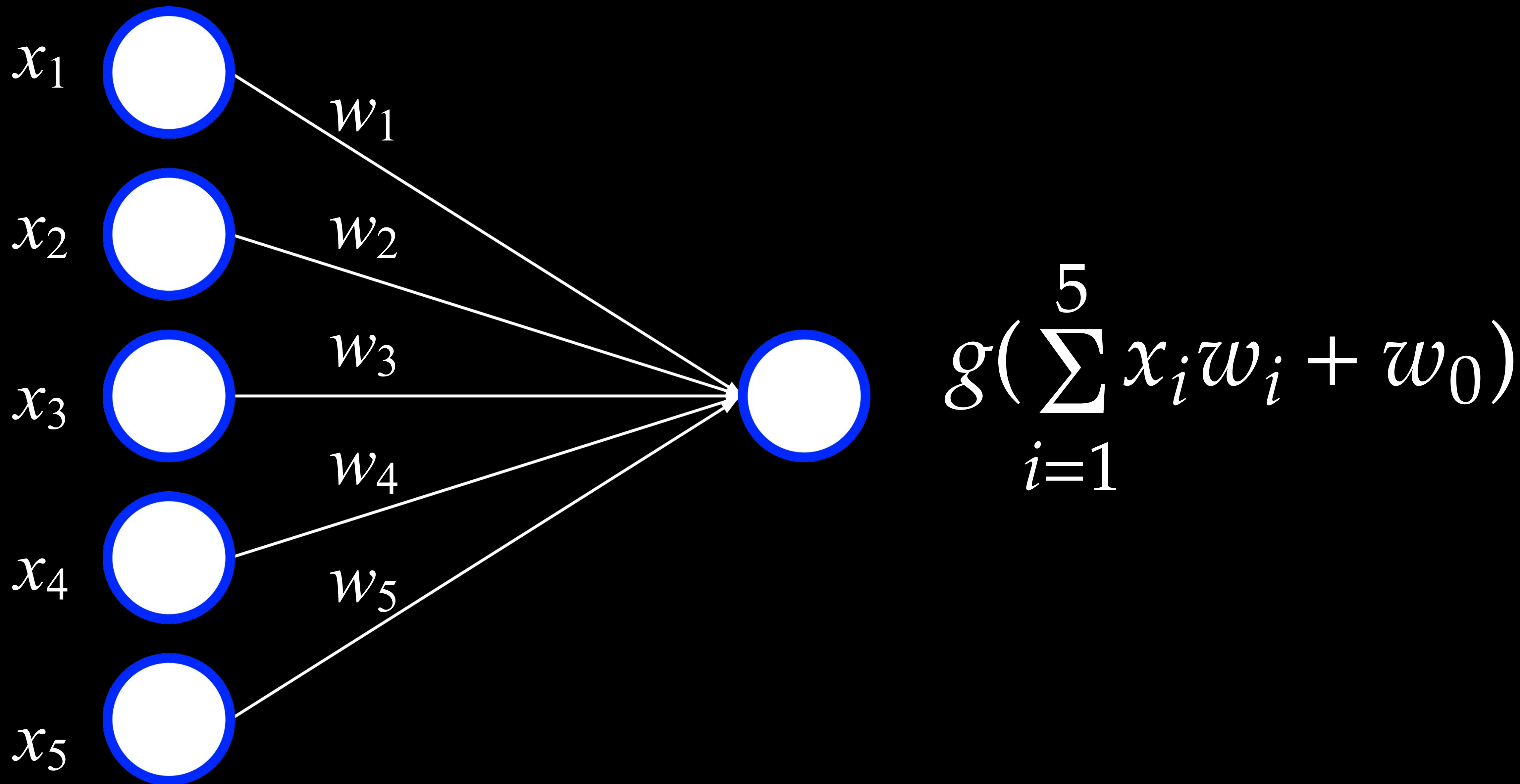


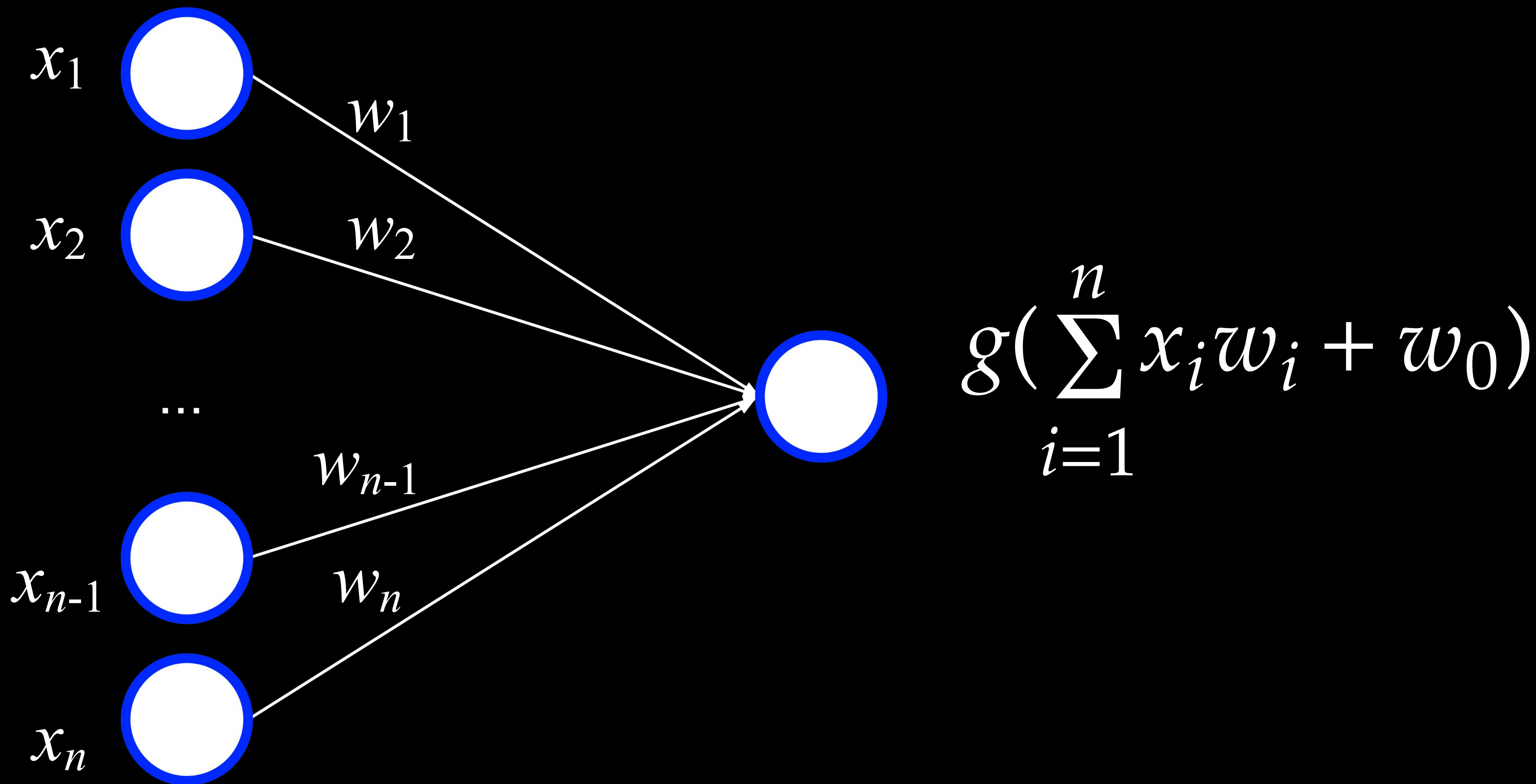












descida do gradiente  
(*gradient descent*)

algoritmo para minimizar a perda durante o  
treinamento de uma rede neural

# Gradient Descent

# Gradient Descent

- Comece com pesos escolhidos aleatoriamente.

# Gradient Descent

- Comece com pesos escolhidos aleatoriamente.
- Repita:

# Gradient Descent

- Comece com pesos escolhidos aleatoriamente.
- Repita:
  - Calcule o gradiente com todos os dados a direção que reduz a perda.

# Gradient Descent

- Comece com pesos escolhidos aleatoriamente.
- Repita:
  - Calcule o gradiente com todos os dados a direção que reduz a perda.
  - Atualize os pesos de acordo com o gradiente.

# Gradient Descent

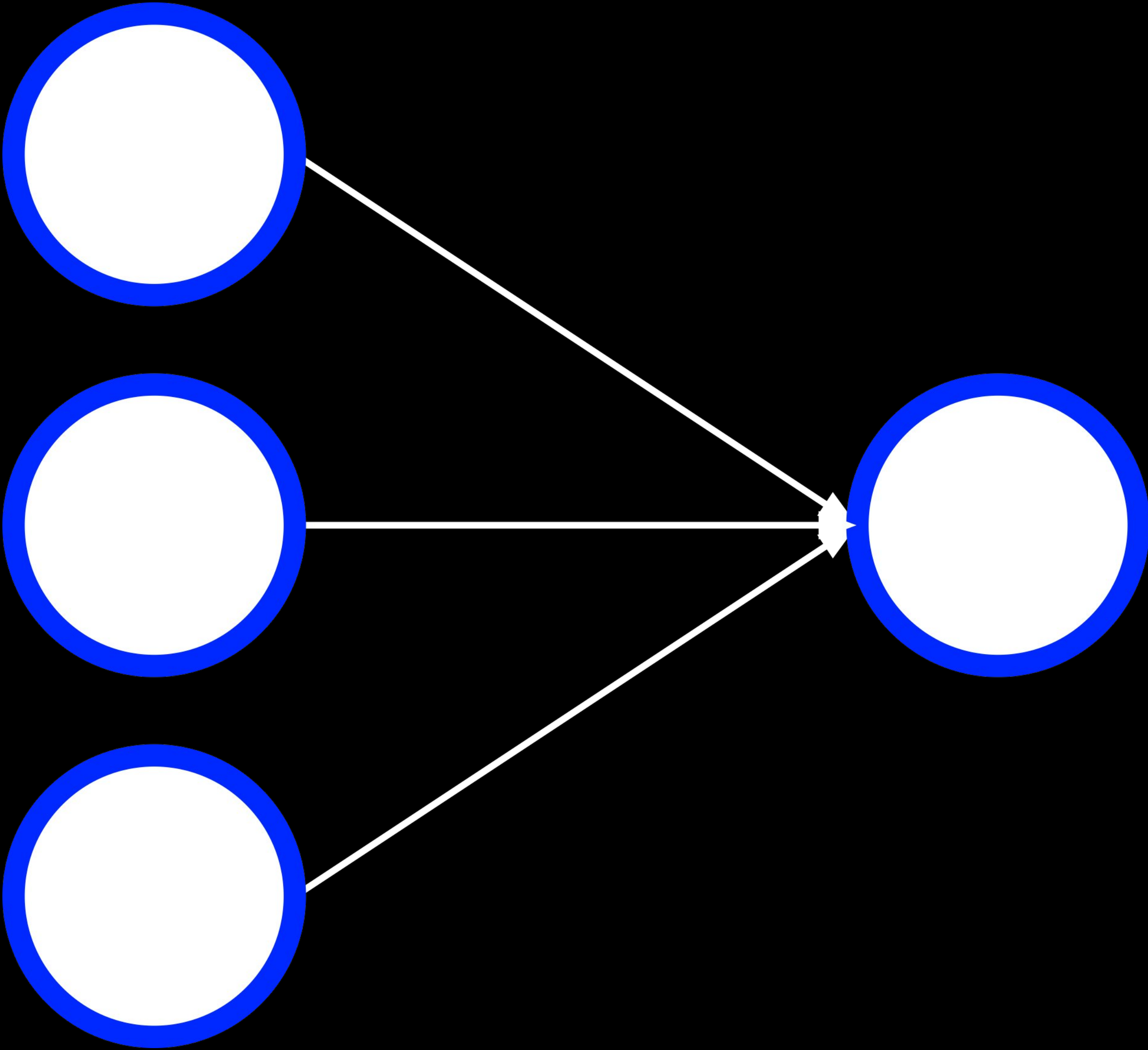
- Comece com pesos escolhidos aleatoriamente.
- Repita:
  - Calcule o gradiente com **todos os dados** a direção que reduz a perda.
  - Atualize os pesos de acordo com o gradiente.

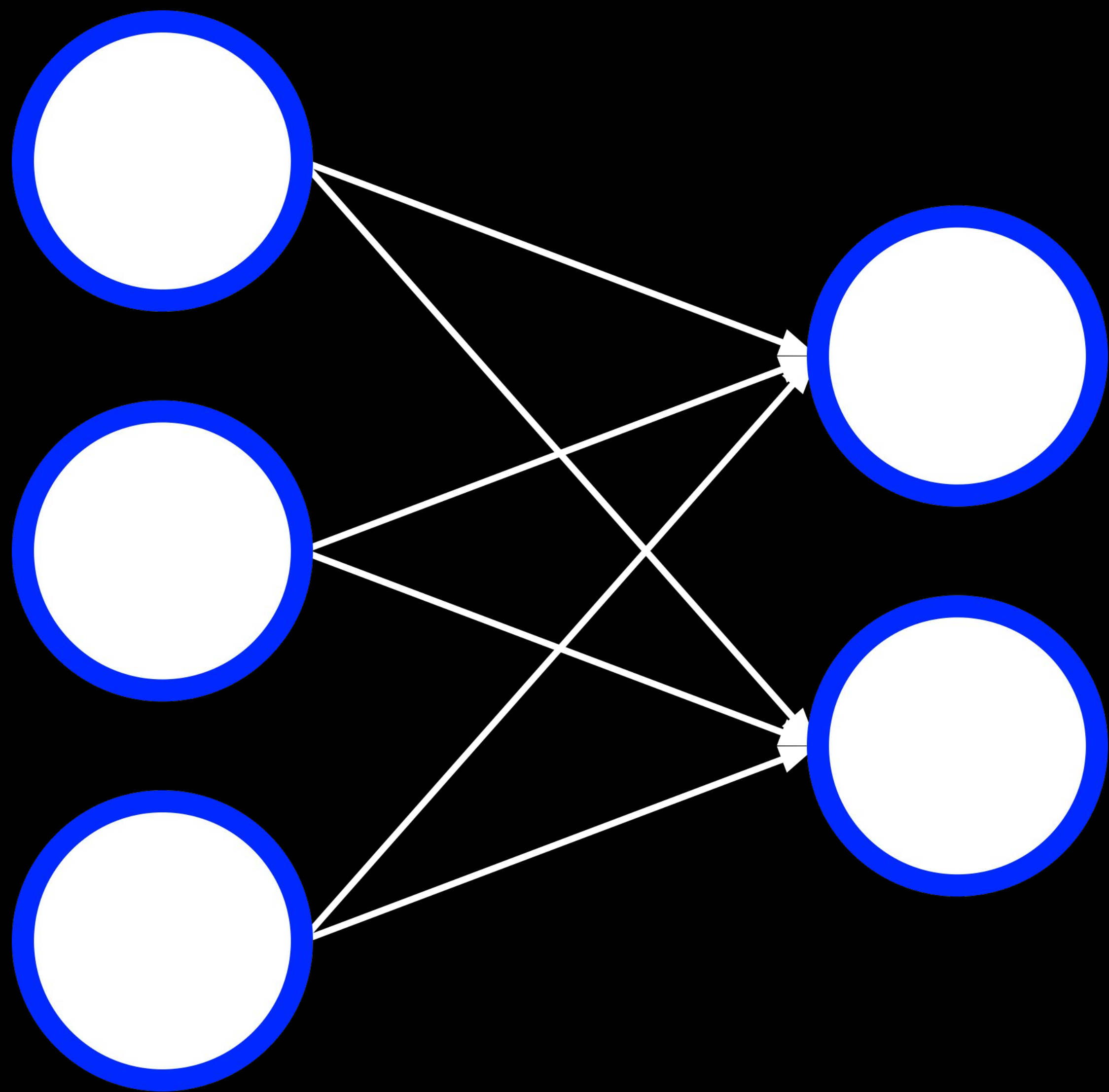
# Stochastic Gradient Descent

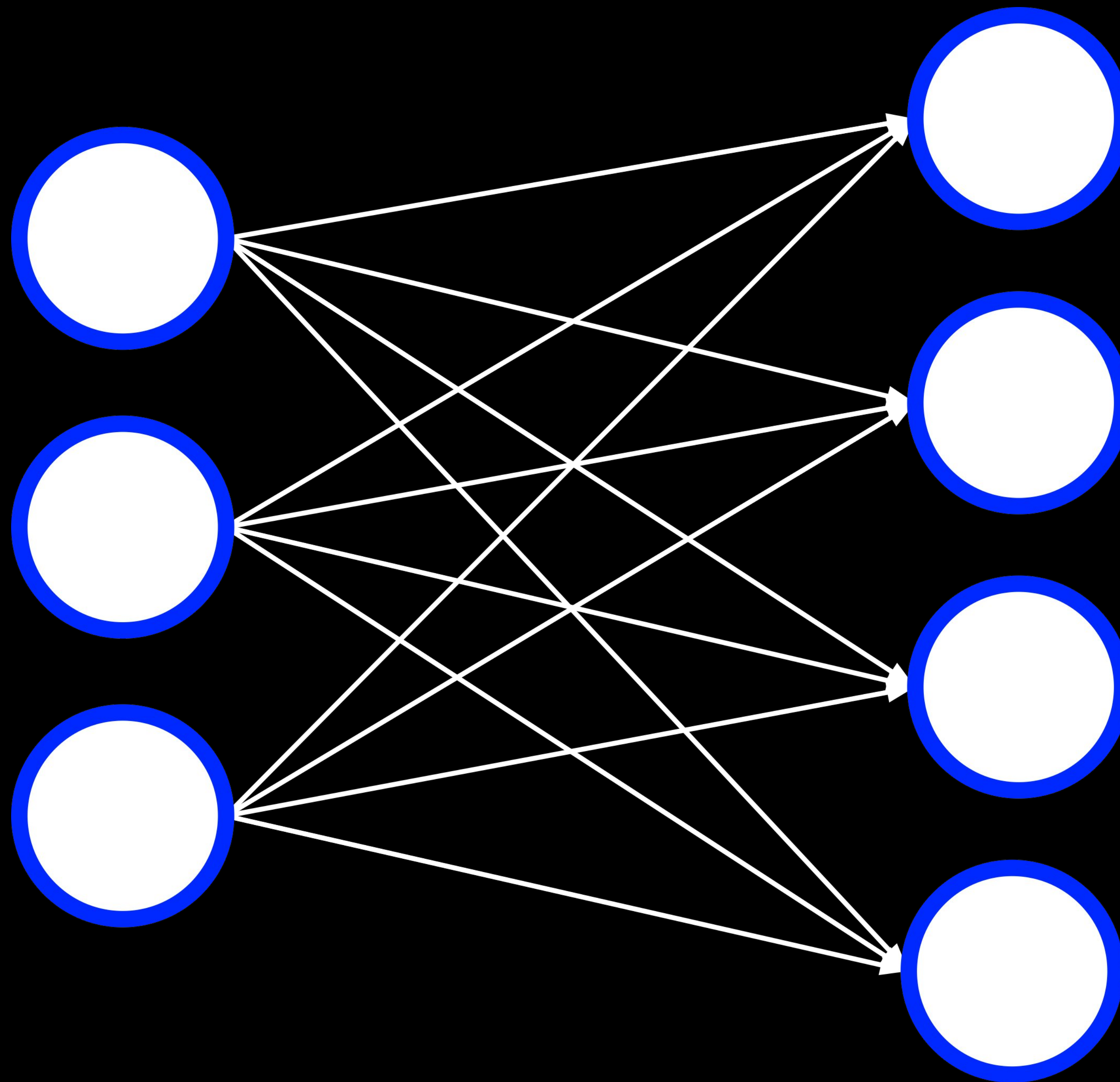
- Comece com pesos escolhidos aleatoriamente.
- Repita:
  - Calcule o gradiente com **um ponto de dados** a direção que reduz a perda.
  - Atualize os pesos de acordo com o gradiente.

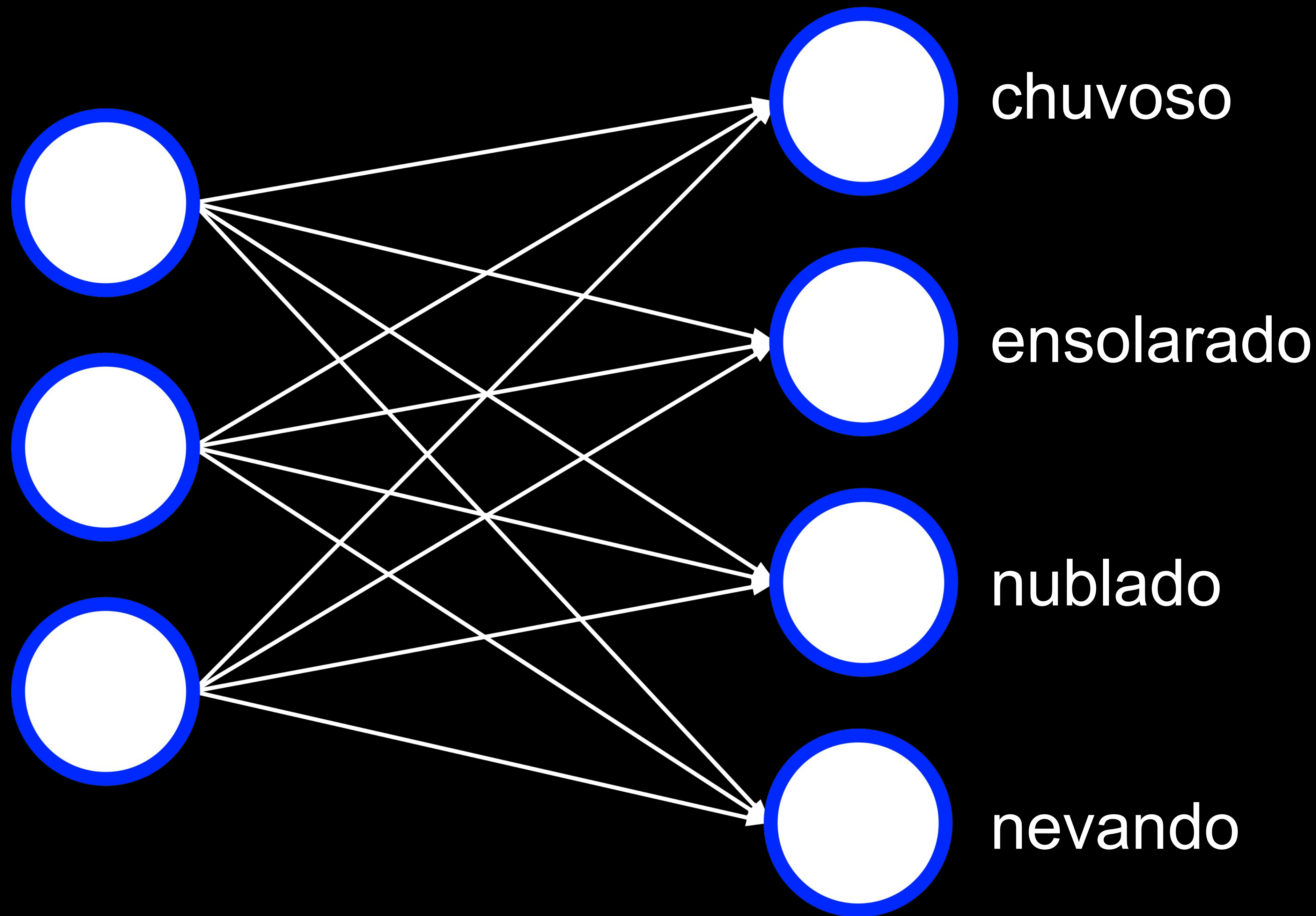
# Mini-Batch Gradient Descent

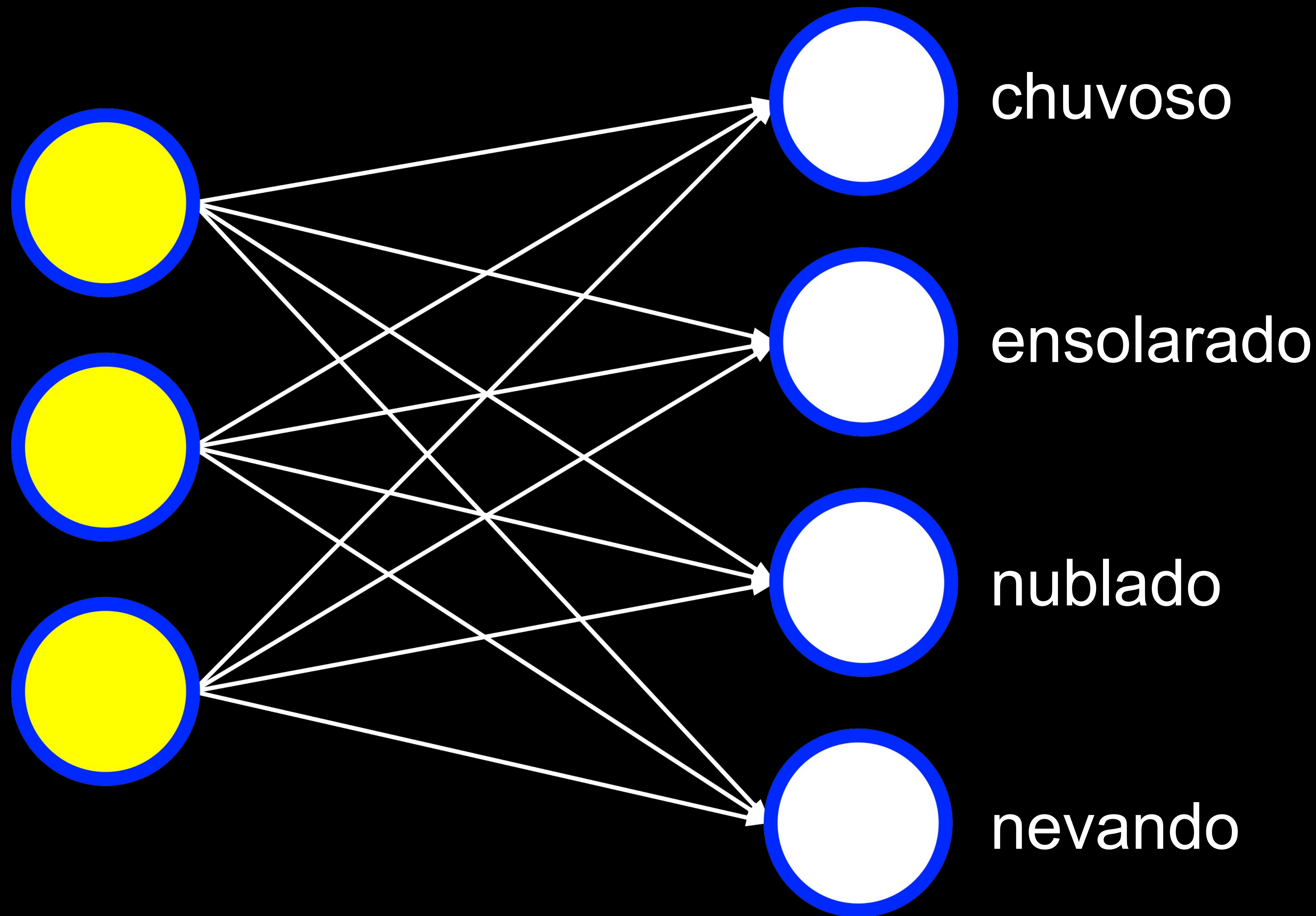
- Comece com pesos escolhidos aleatoriamente.
- Repita:
  - Calcule o gradiente com **um pequeno lote** a direção que reduz a perda.
  - Atualize os pesos de acordo com o gradiente.

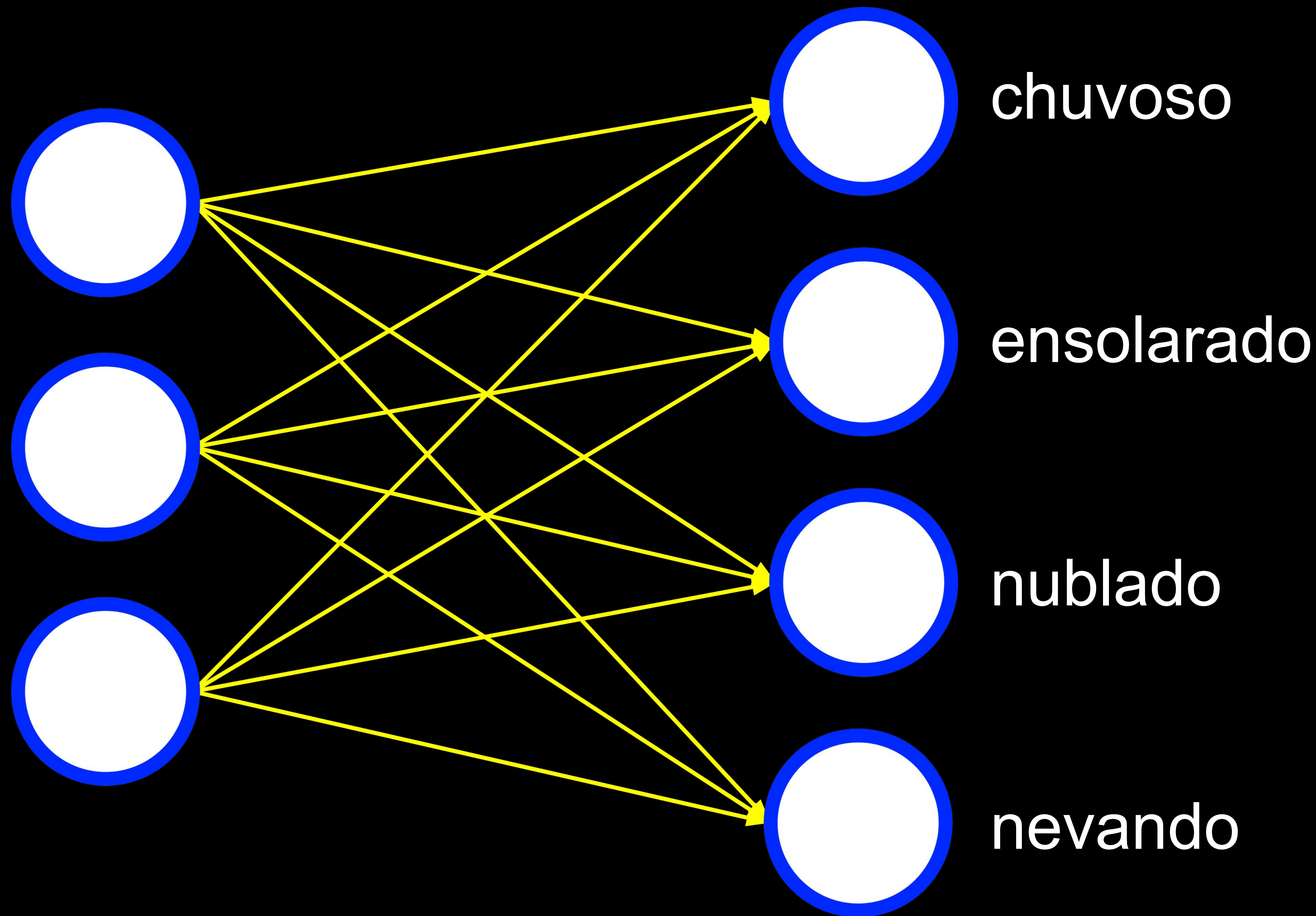


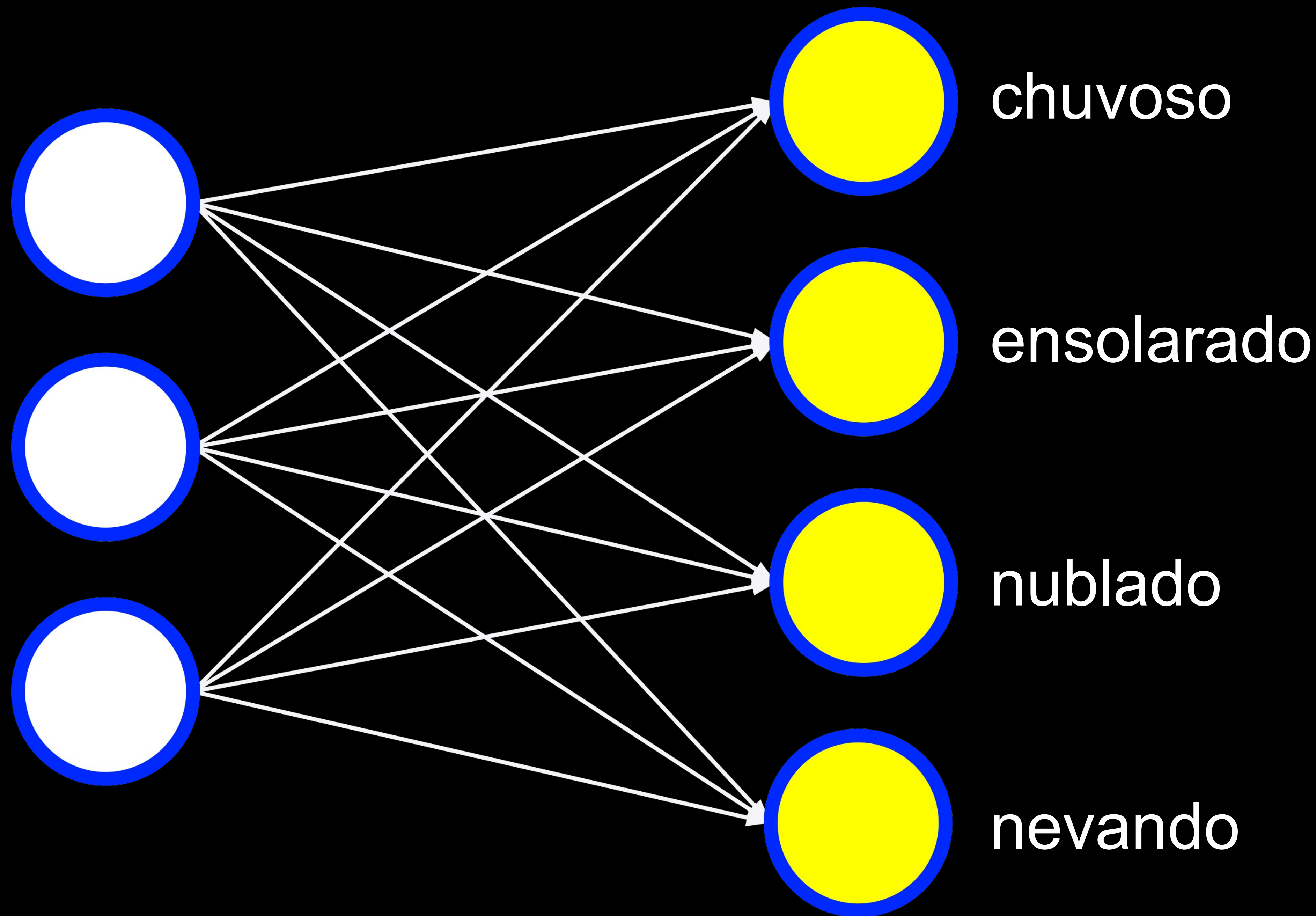


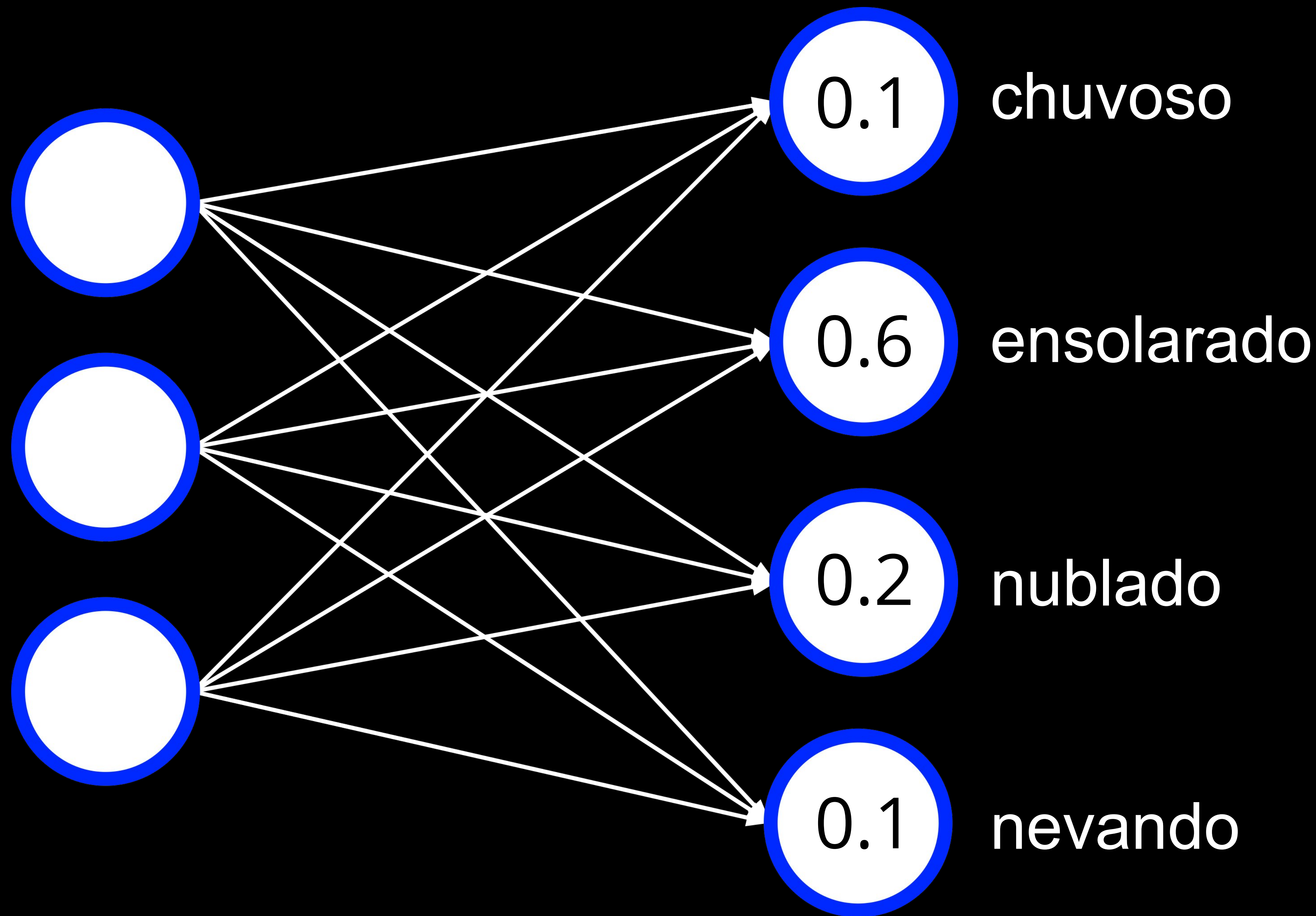


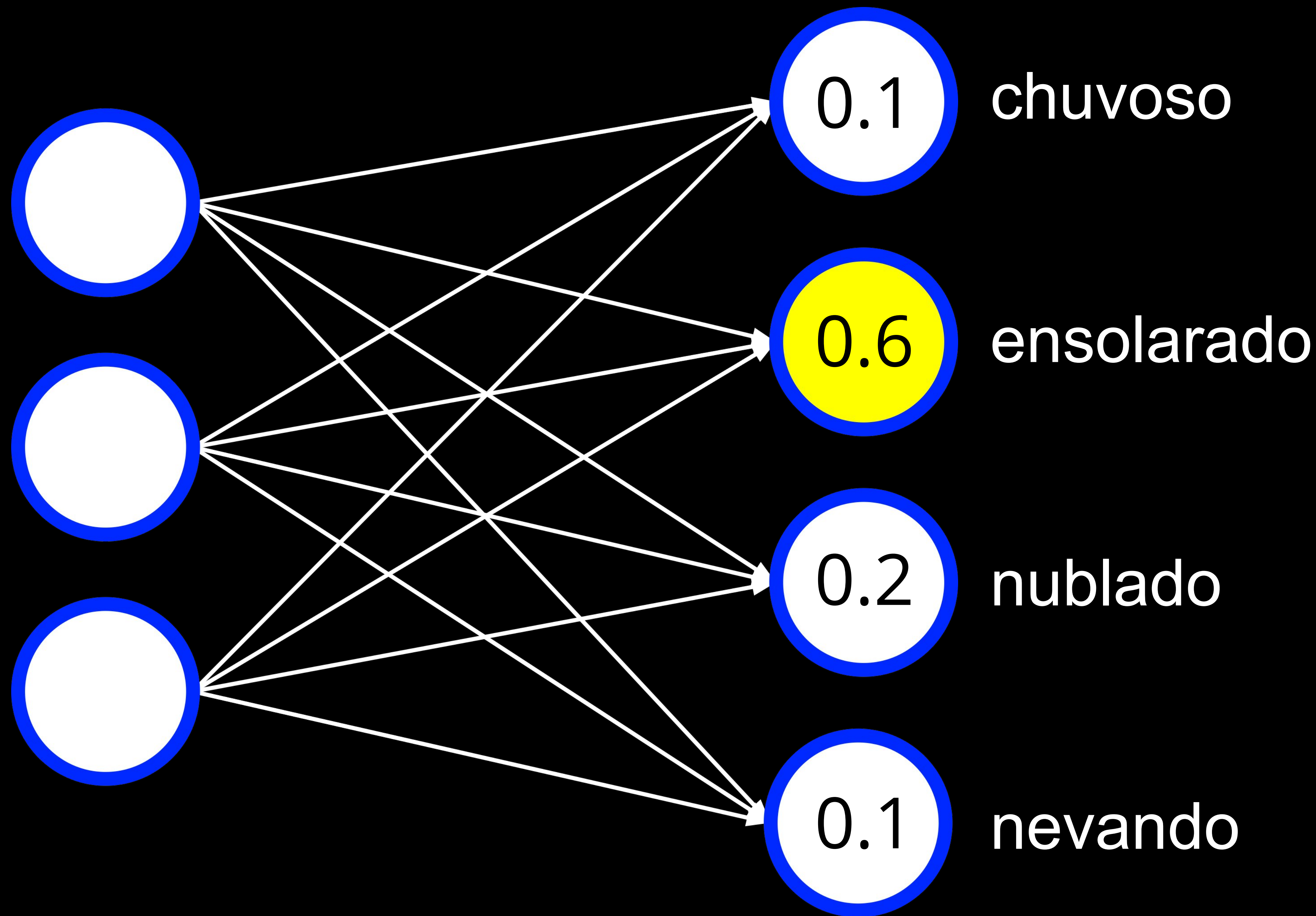


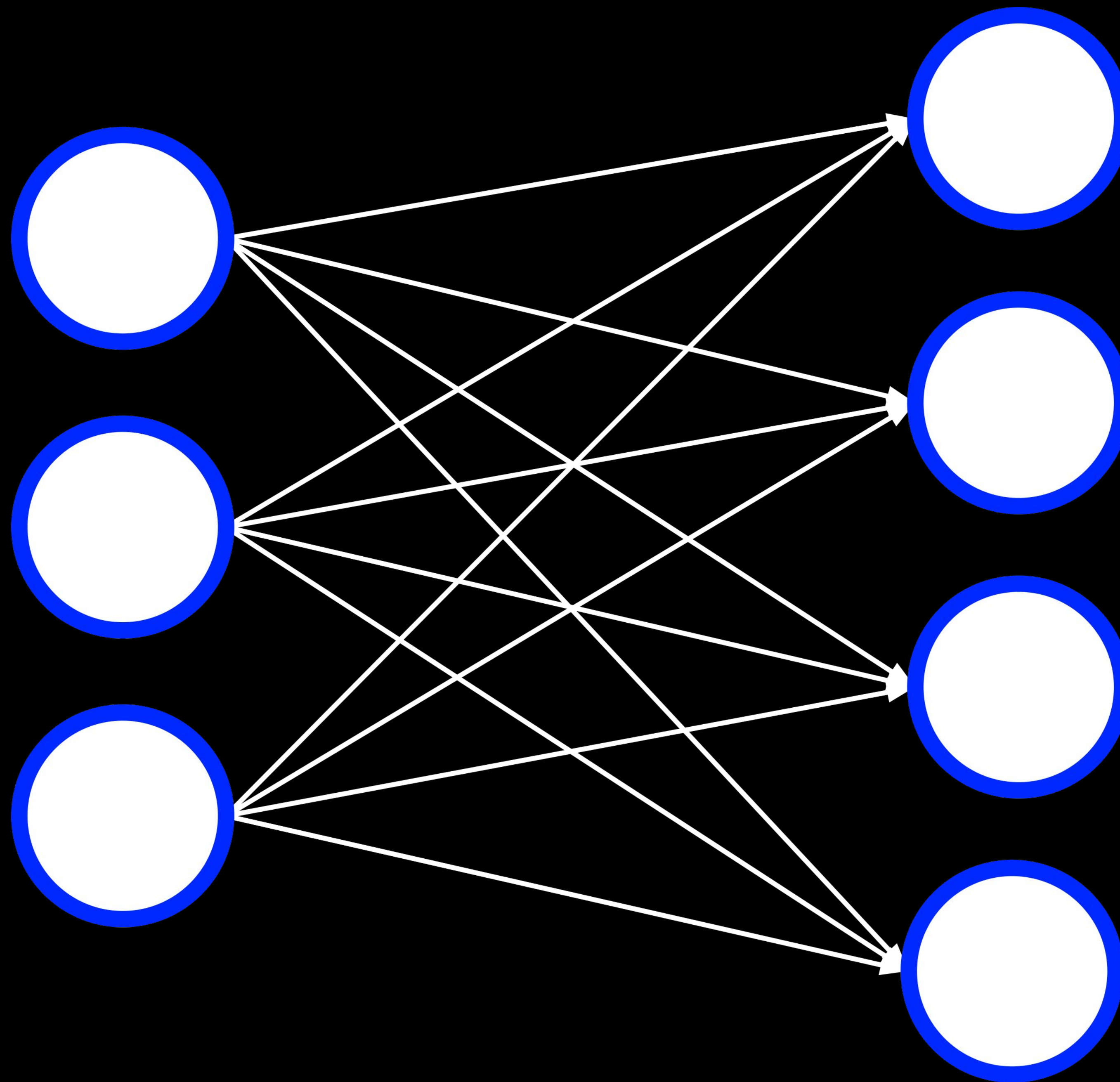


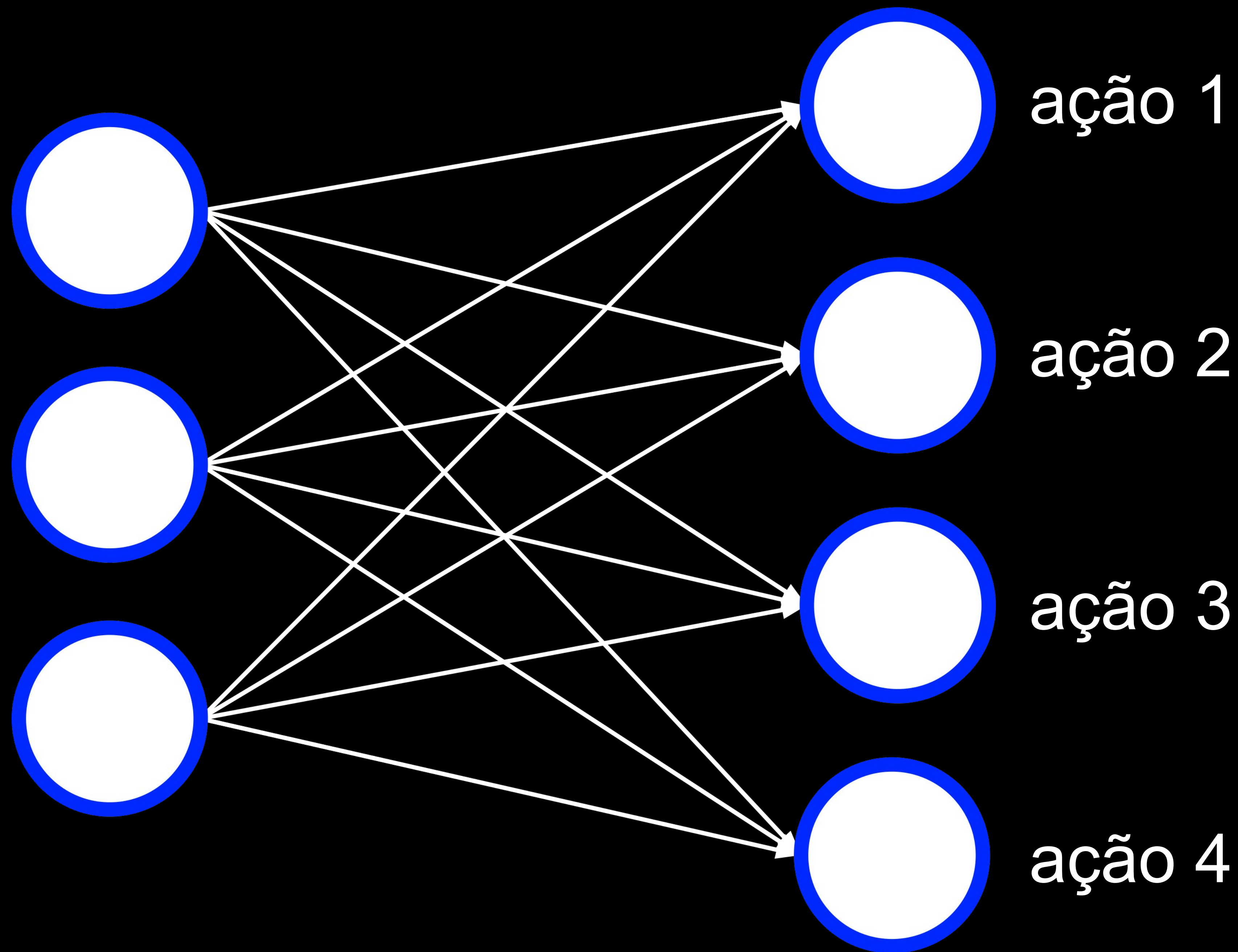


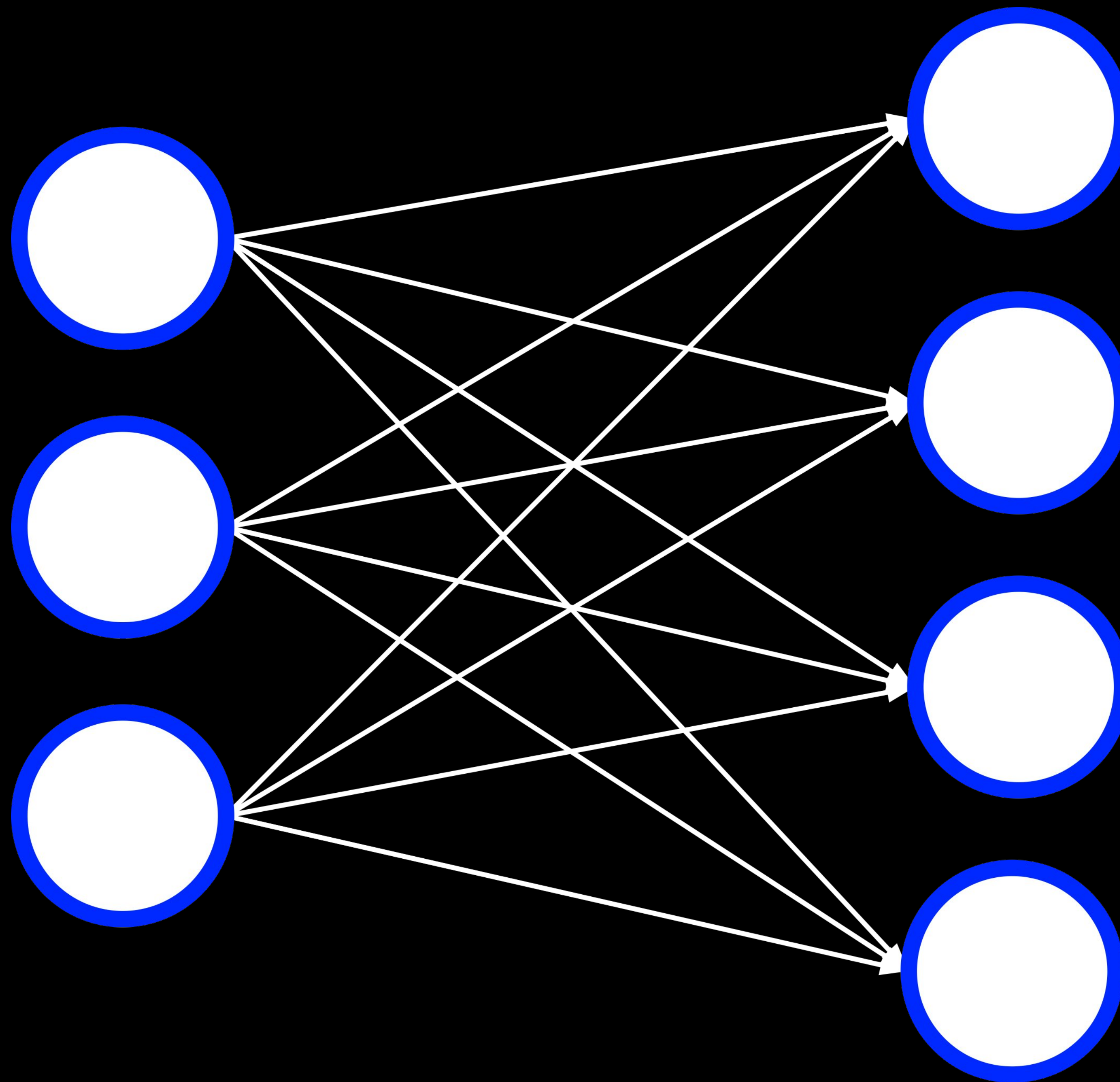


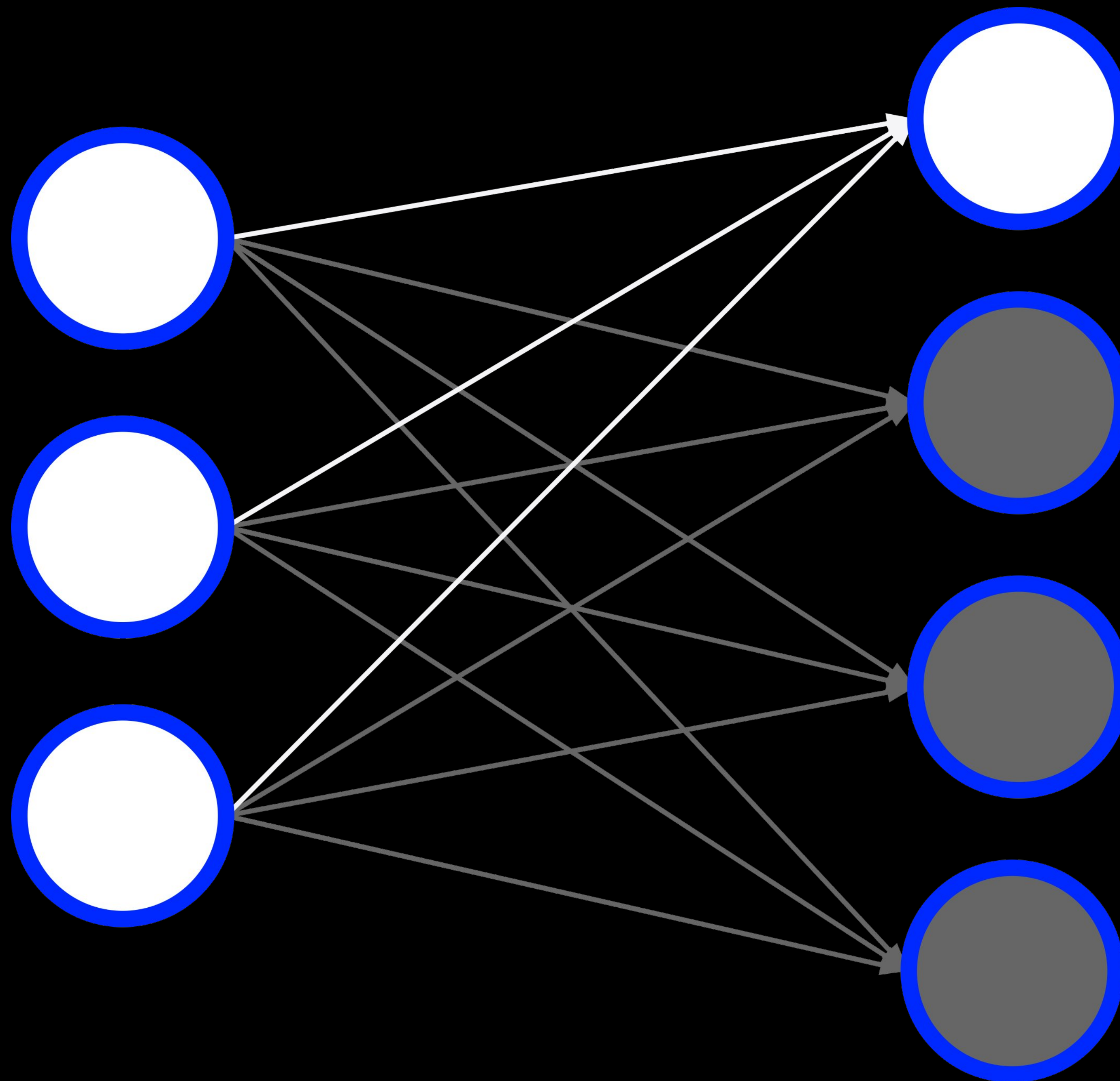


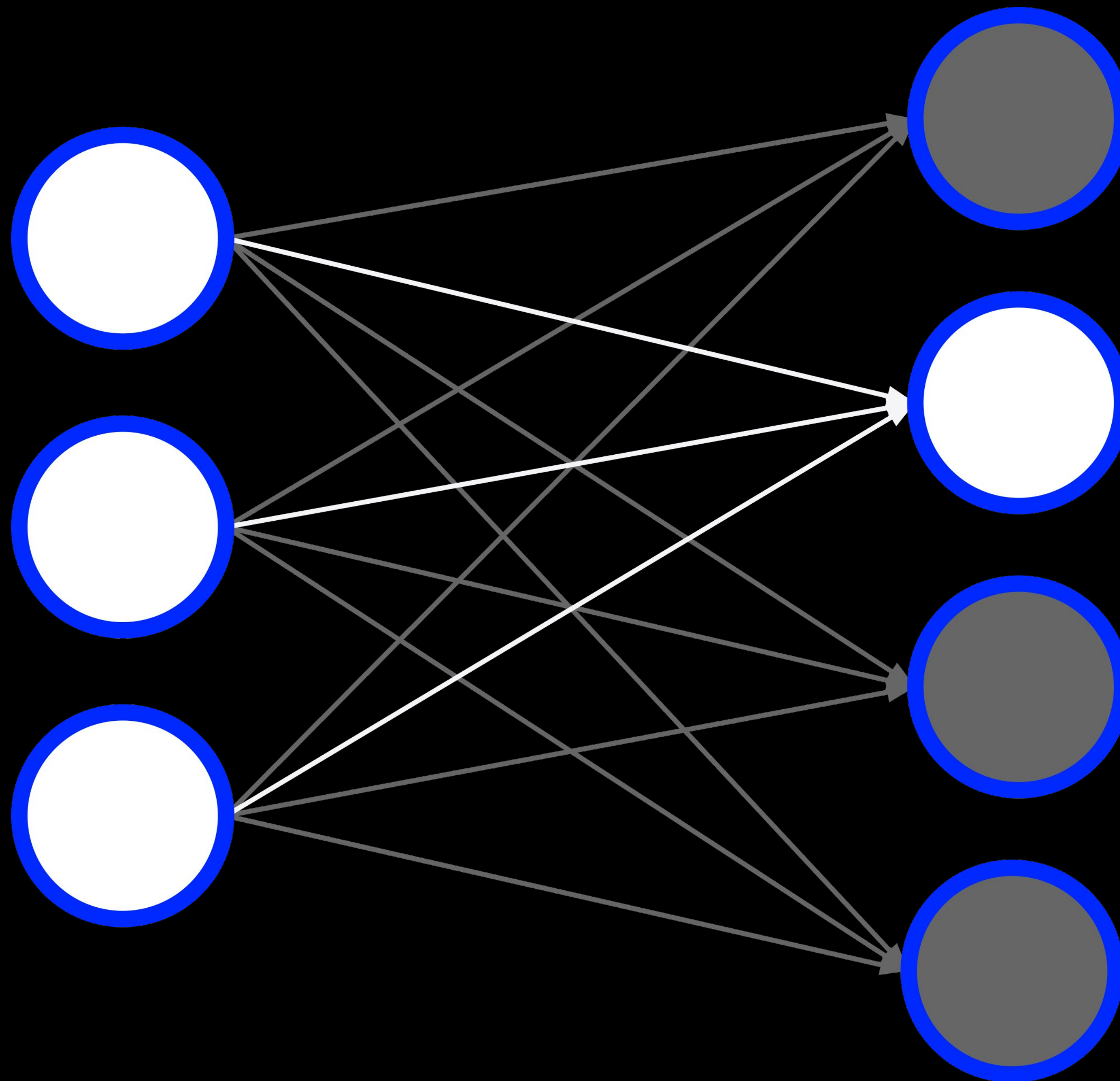


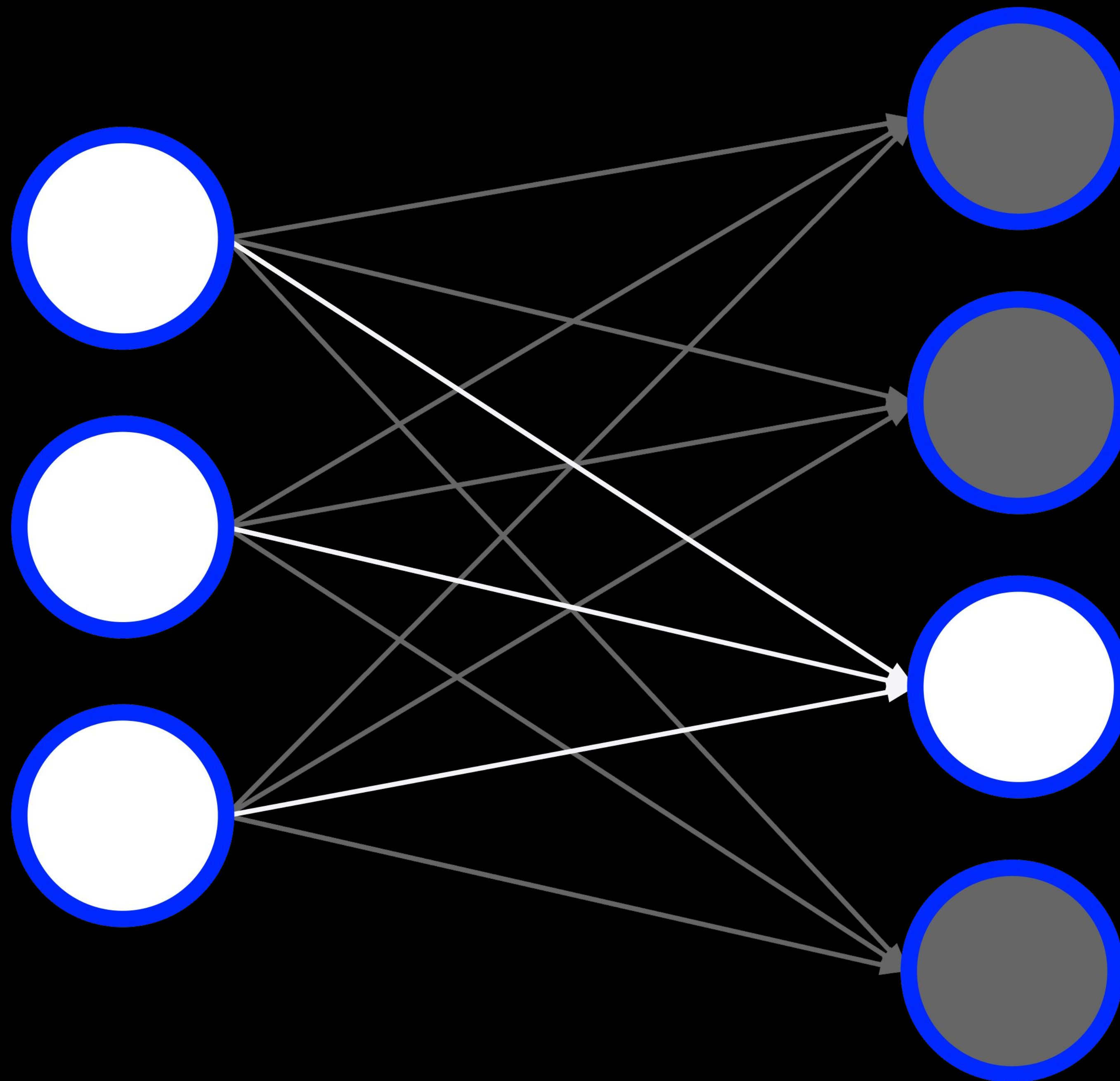


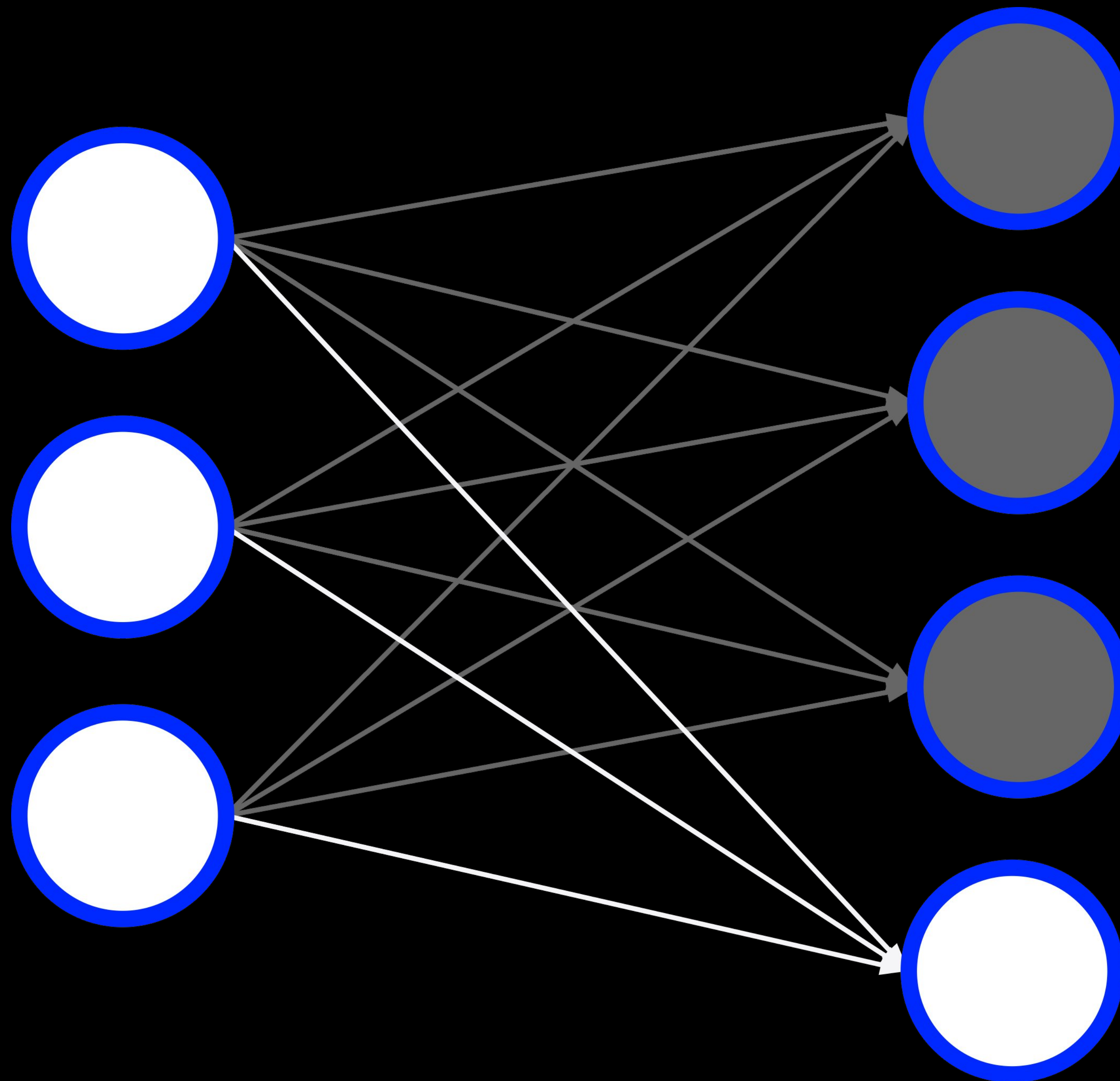


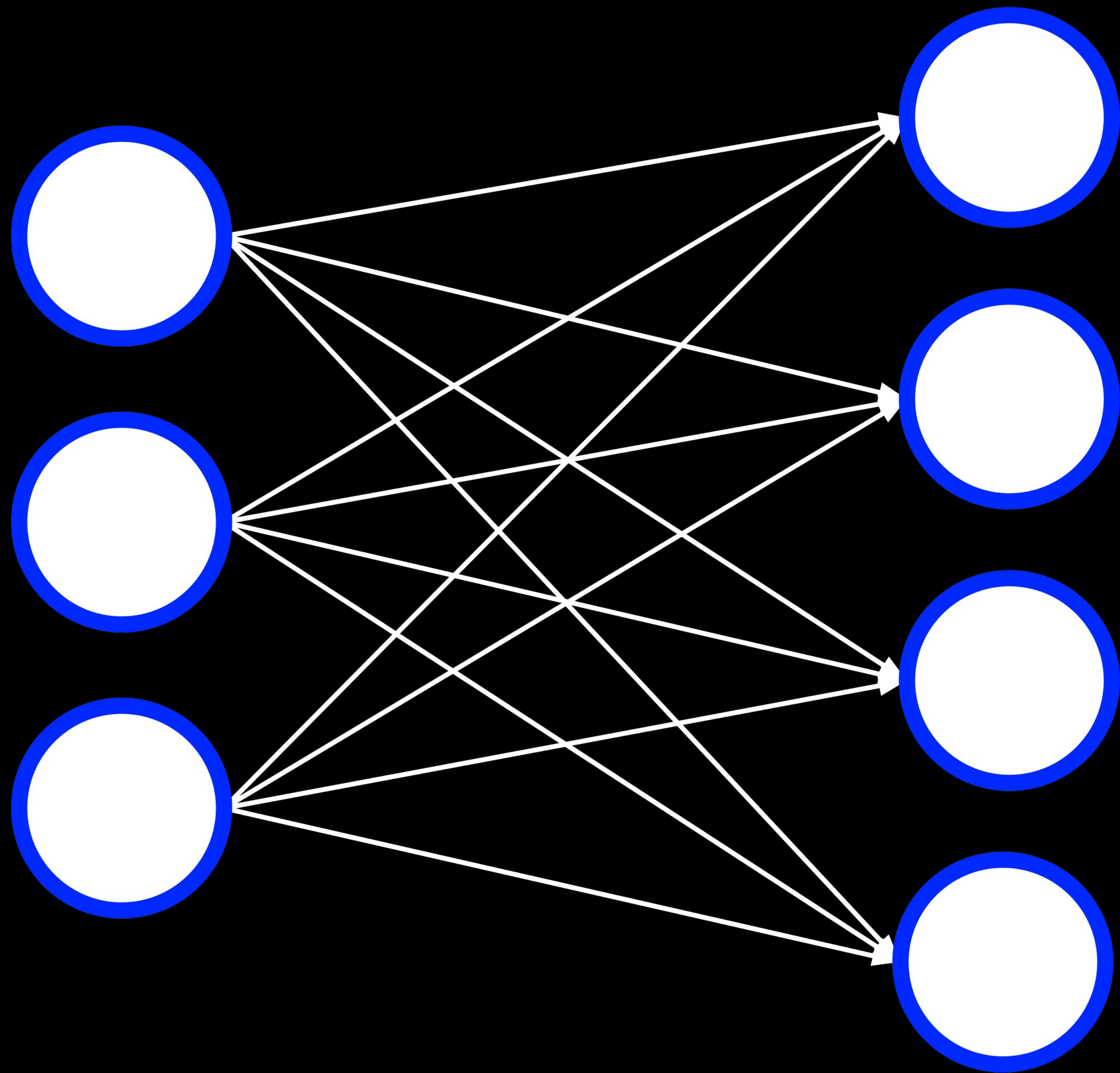


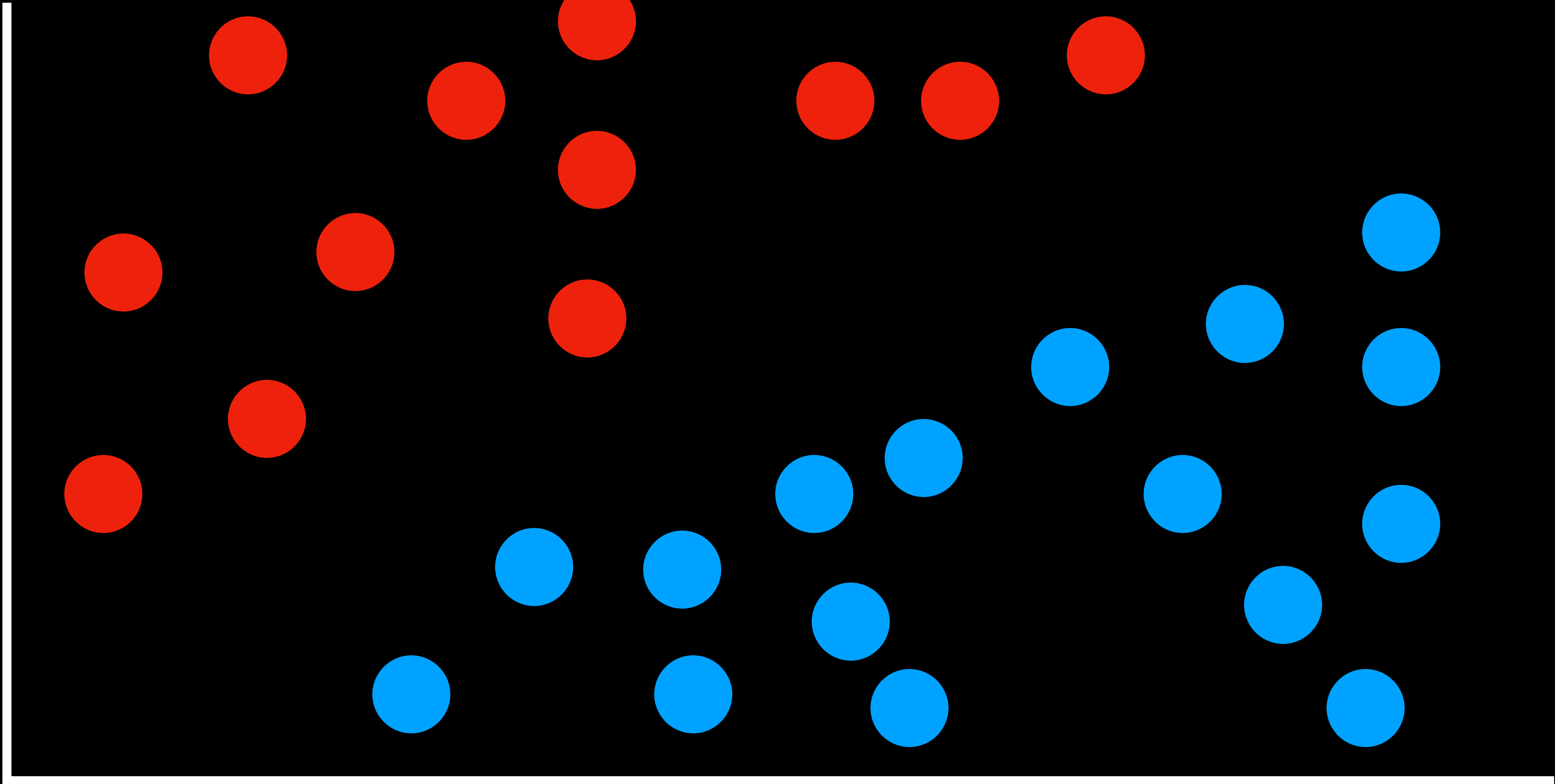


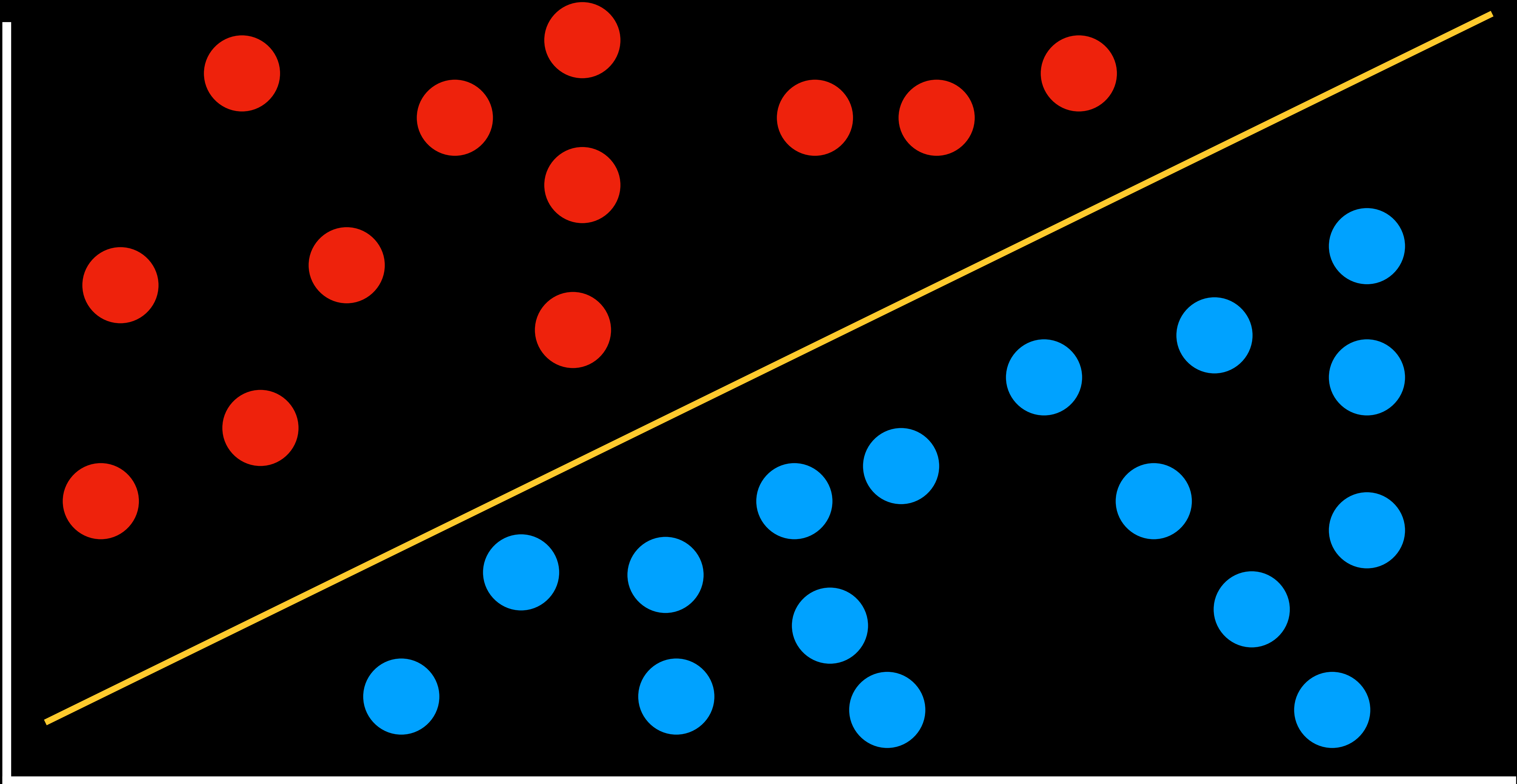


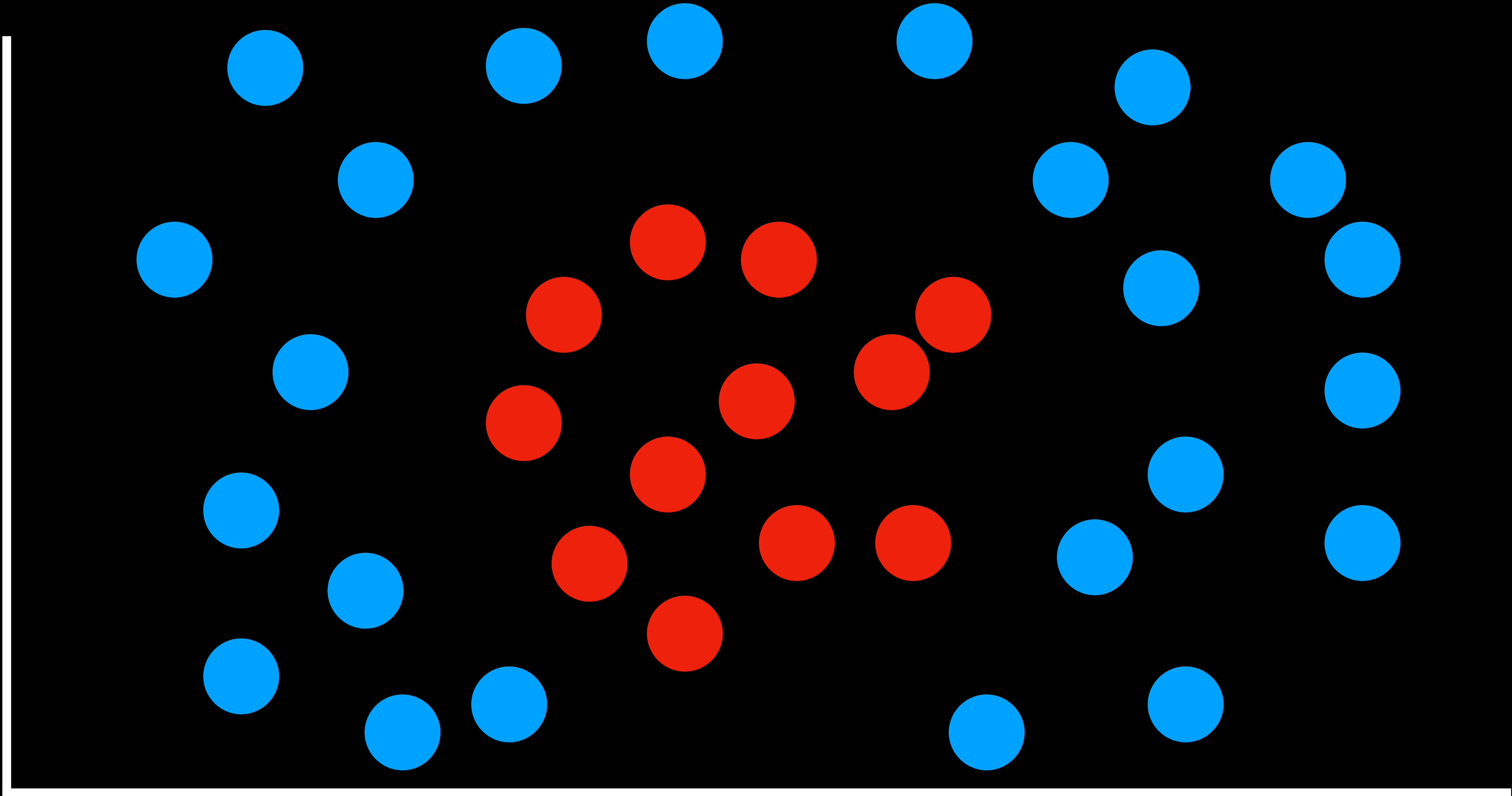










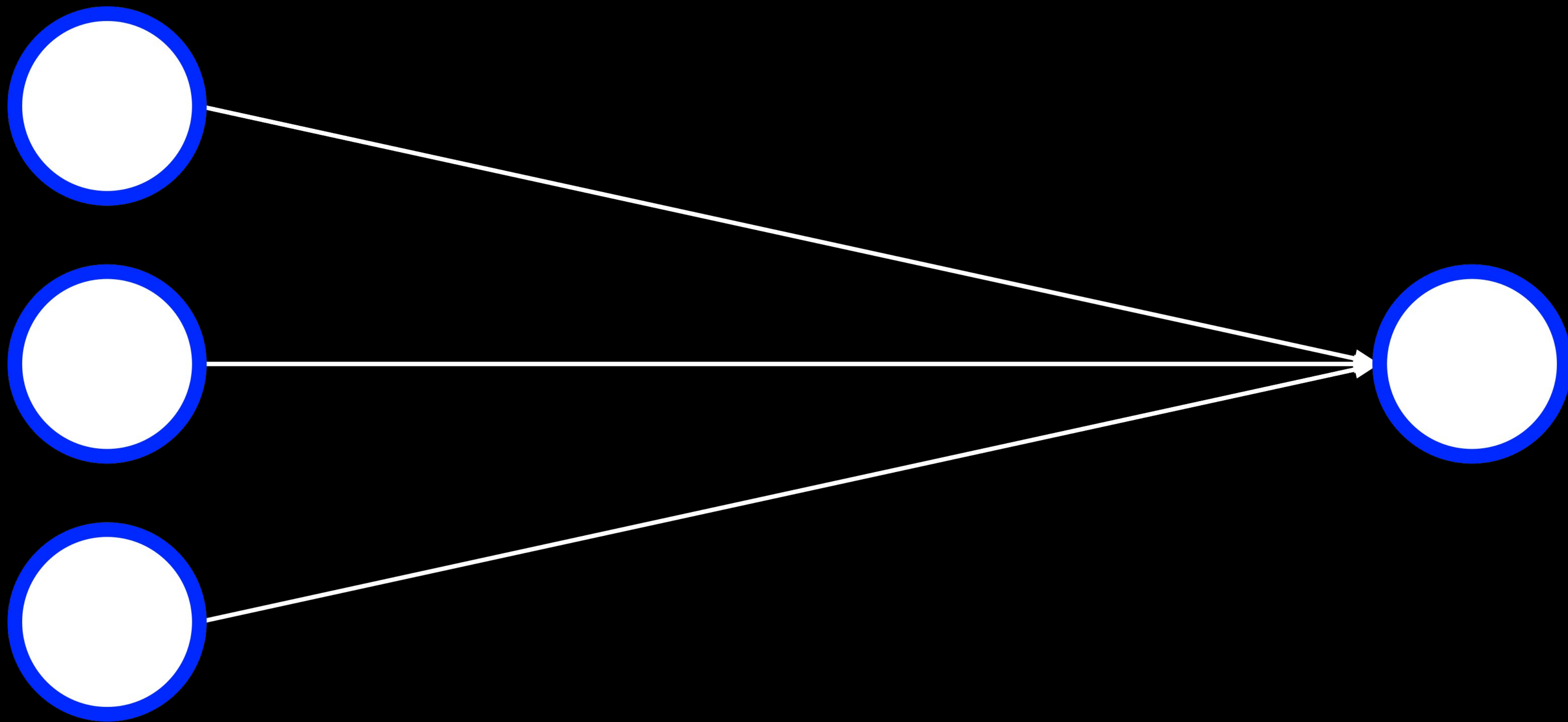


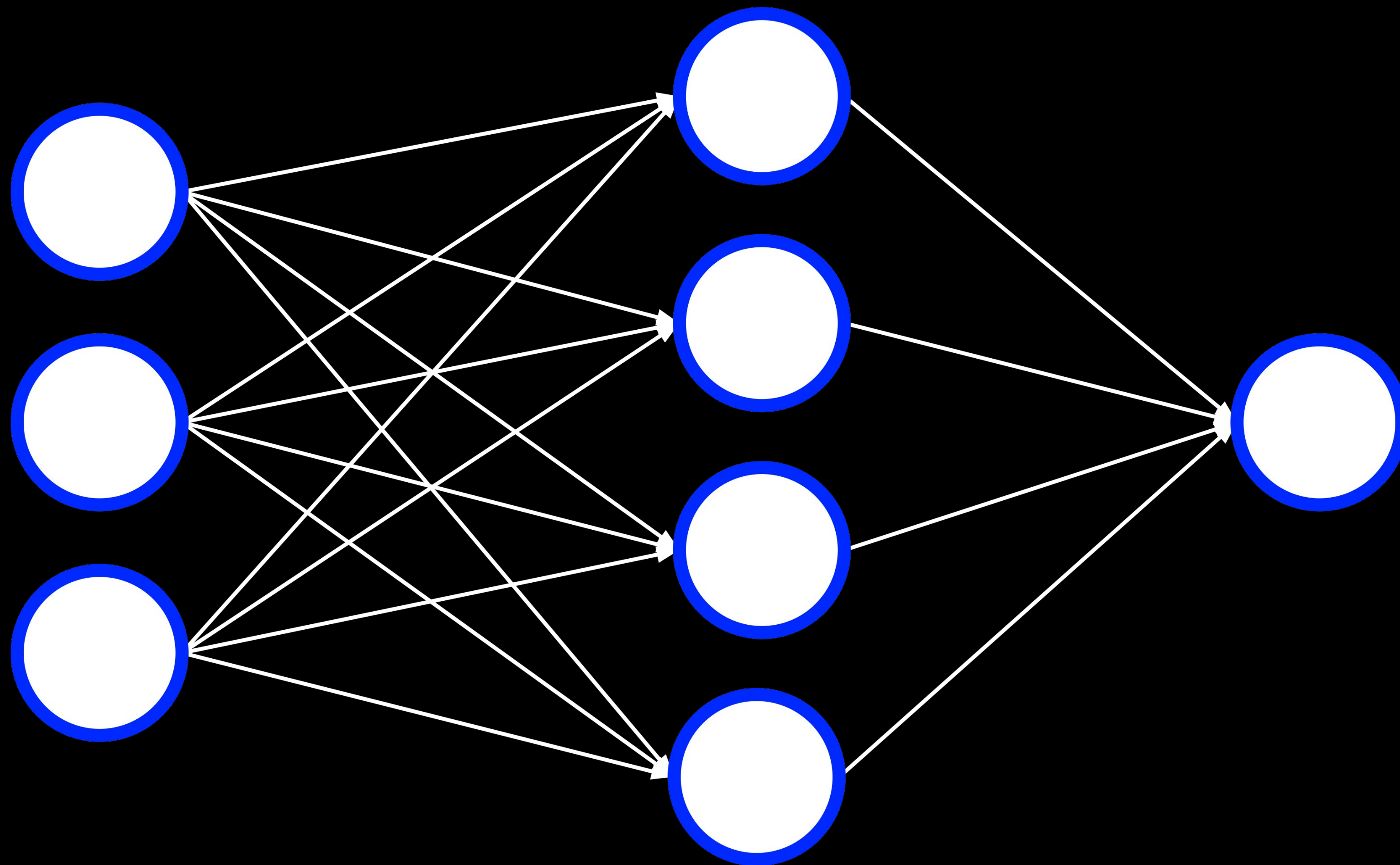
# Perceptron

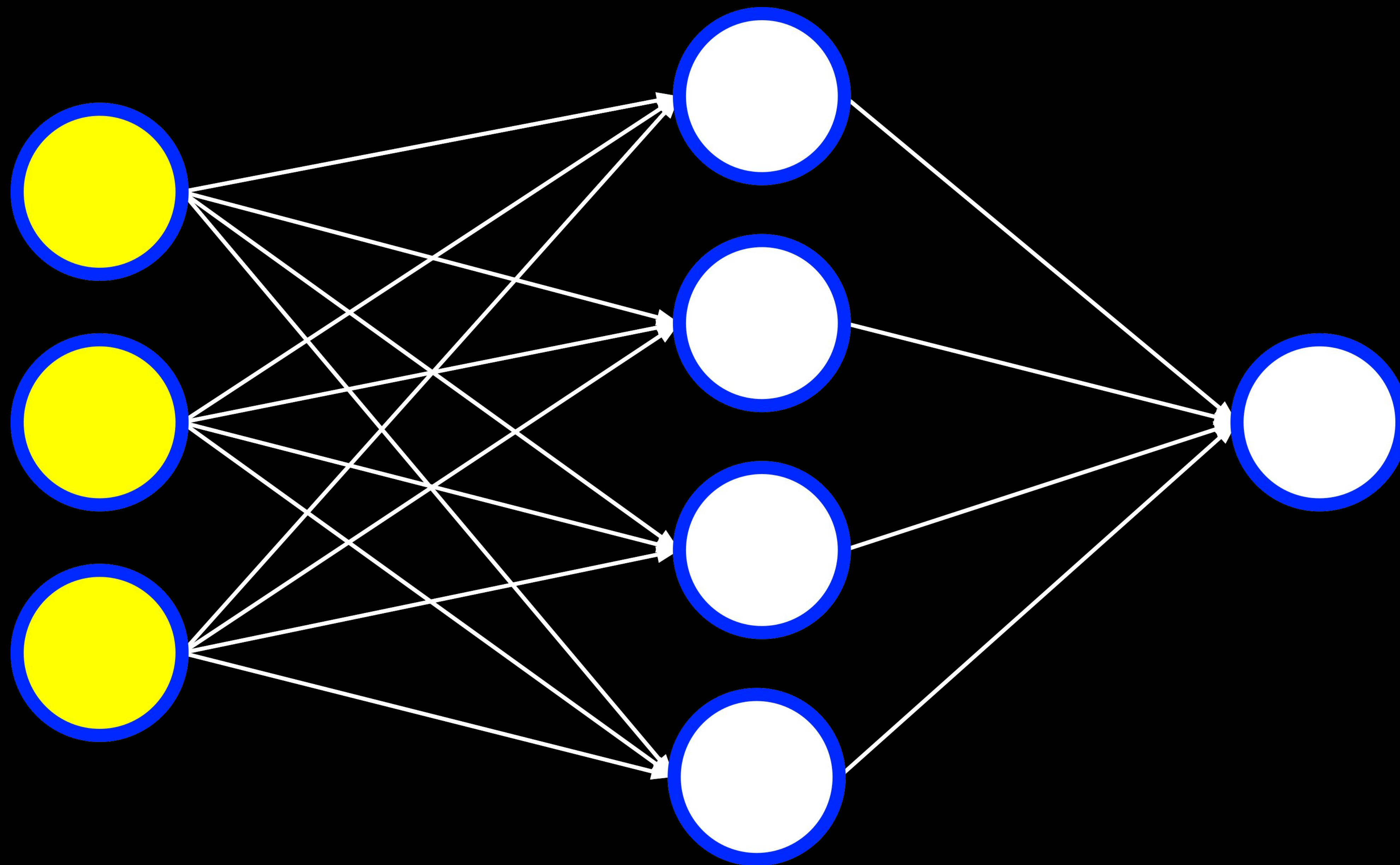
- Só consegue aprender uma fronteira de decisão linearmente separável.

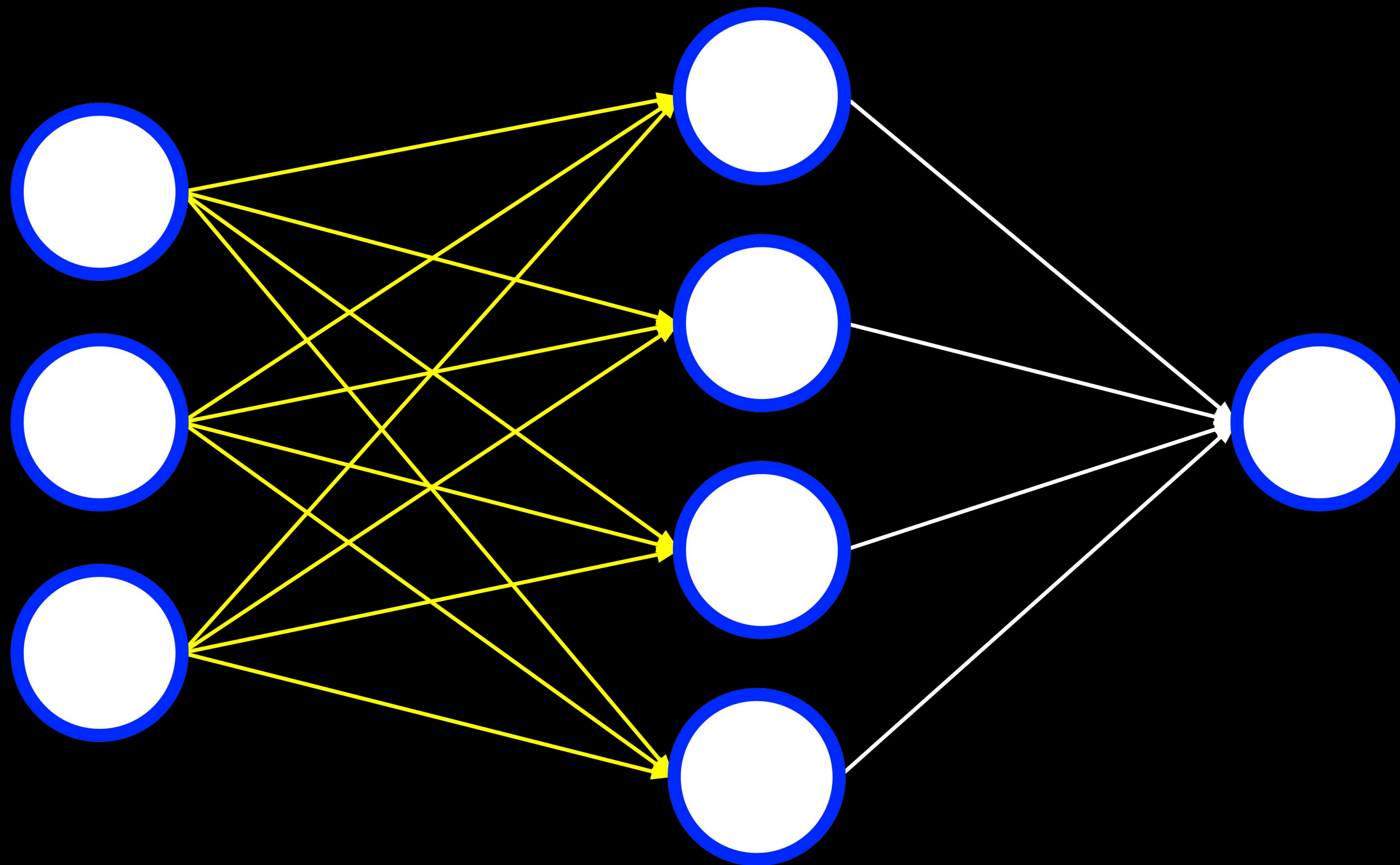
# rede neural multicamadas

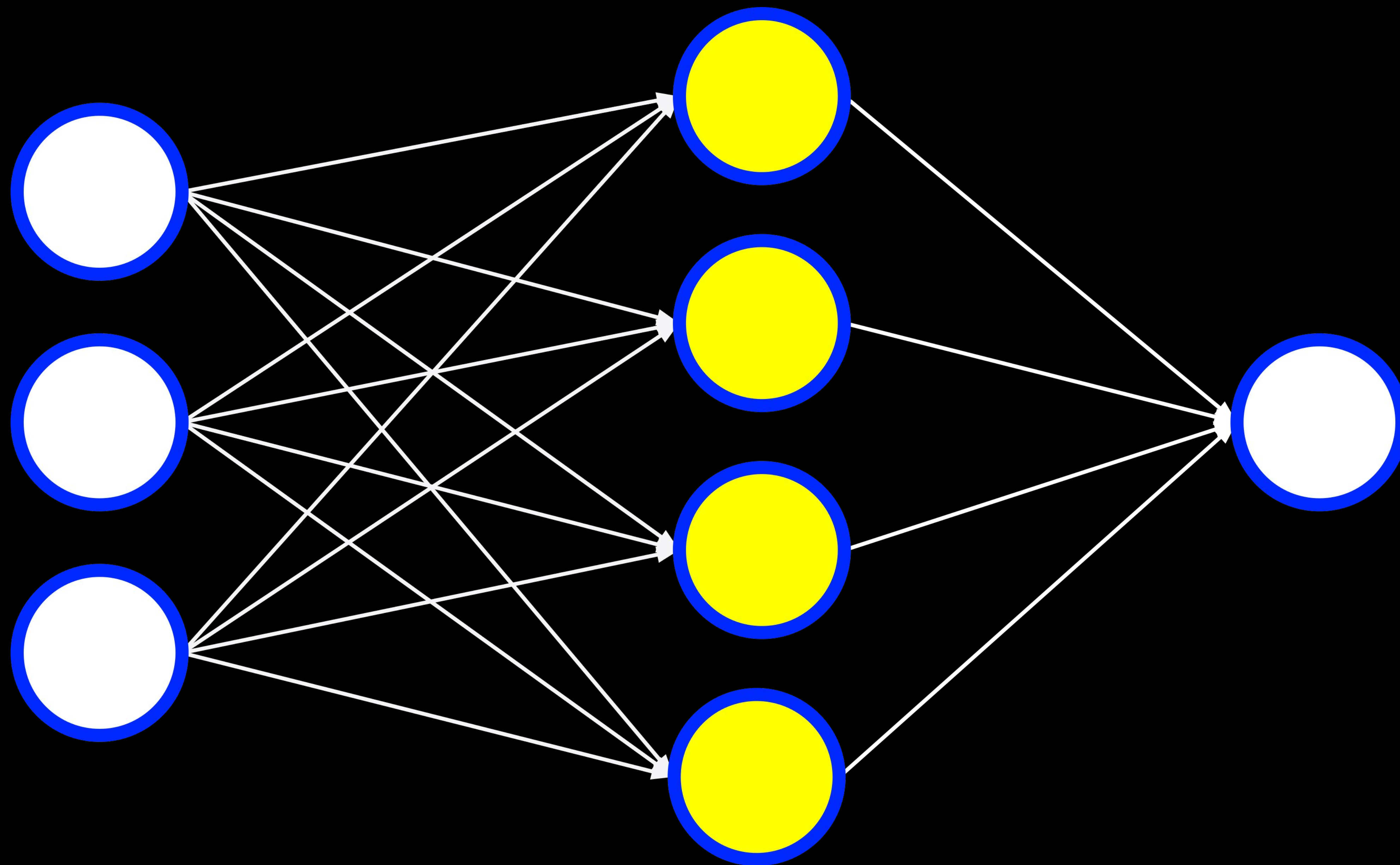
rede neural artificial com uma camada de entrada, uma camada de saída e pelo menos uma camada oculta

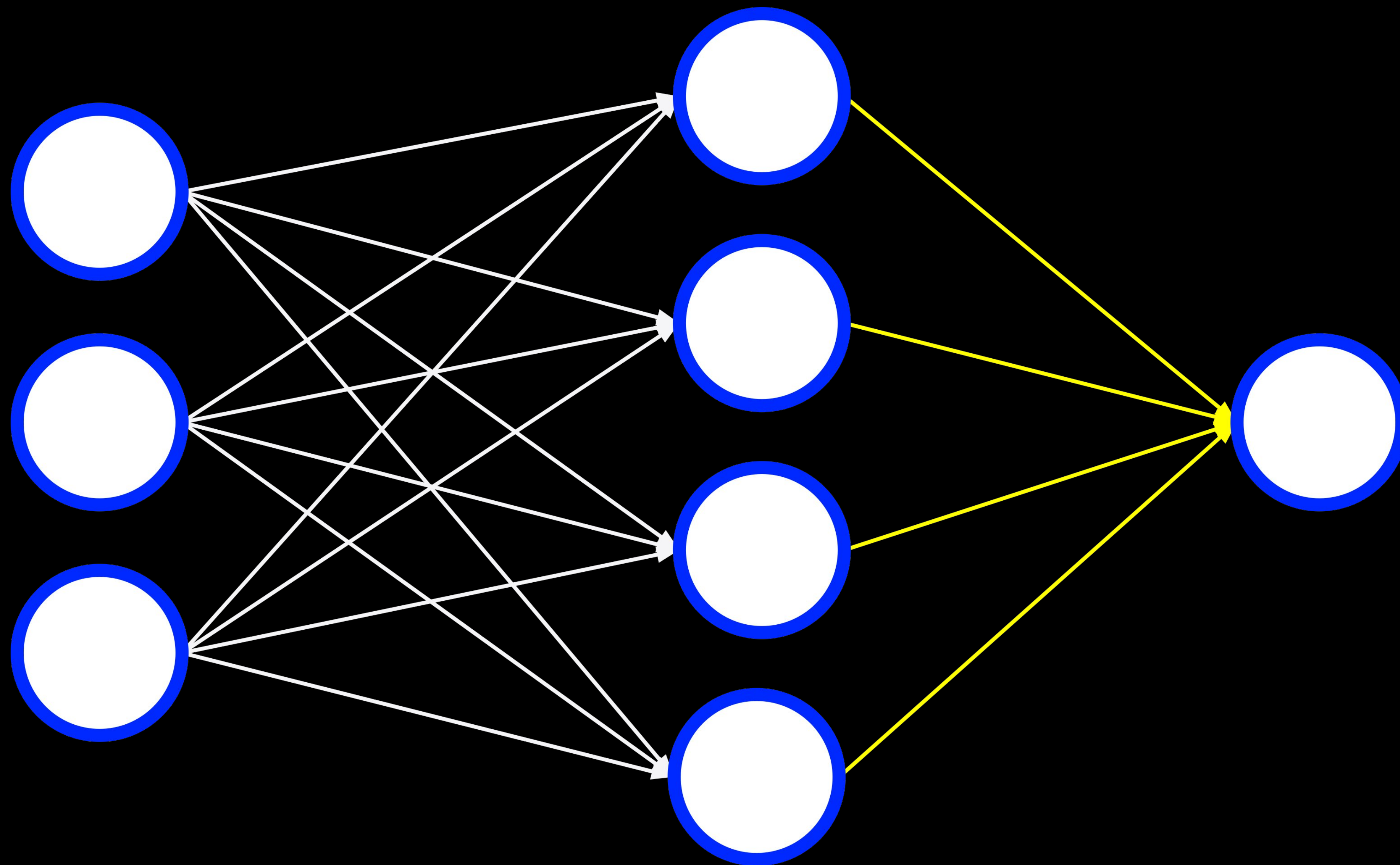


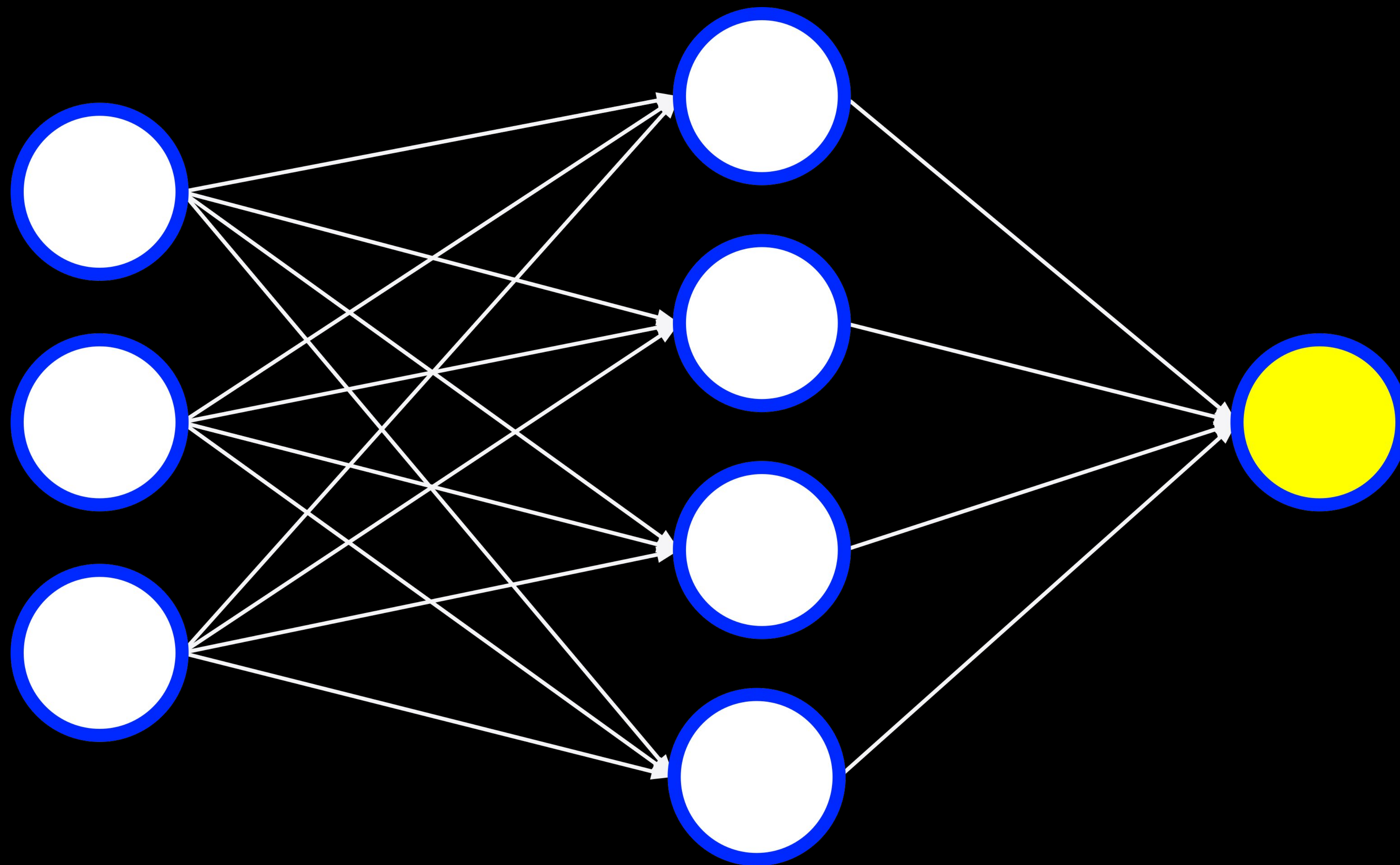


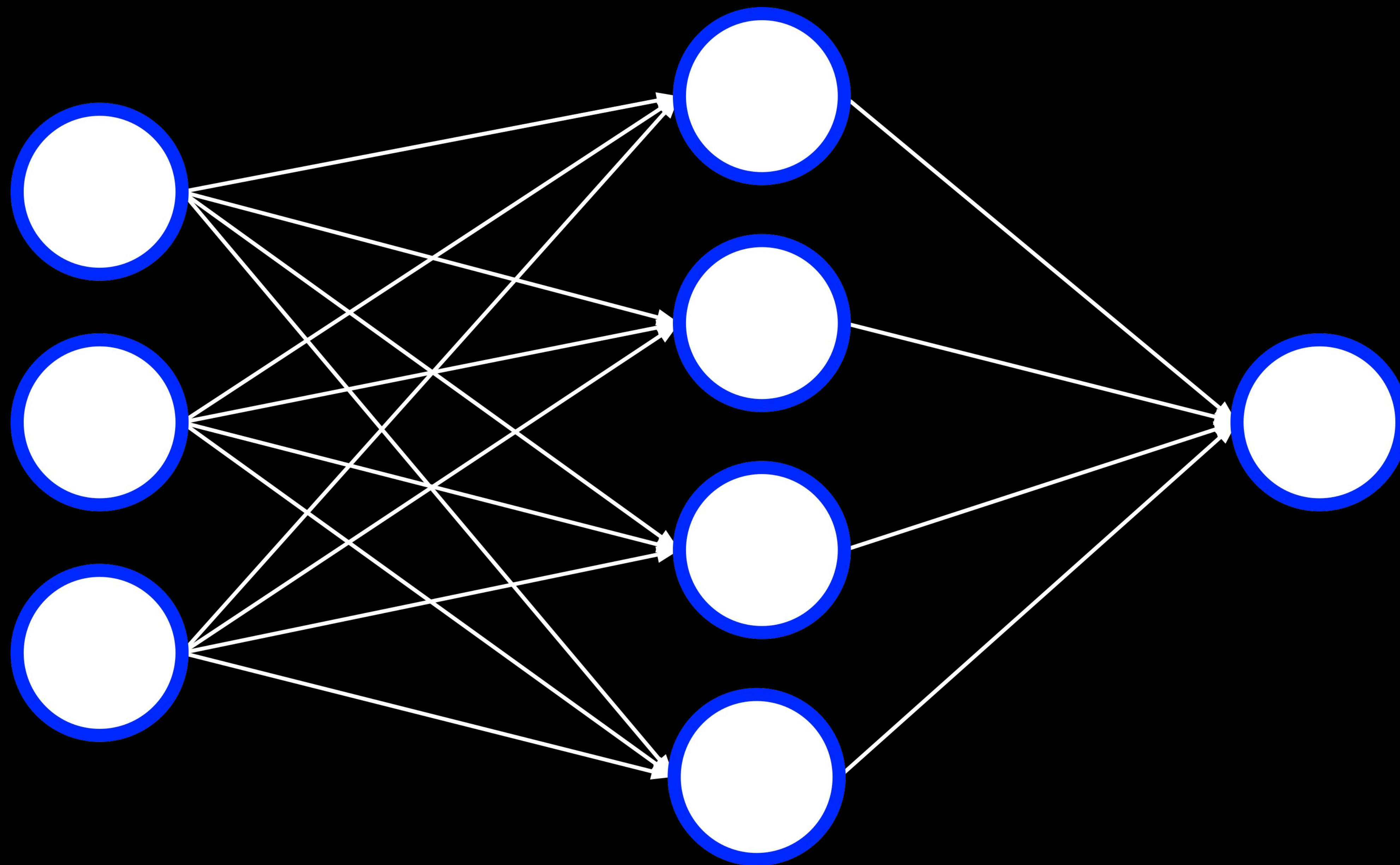












# Retropropagação (backpropagation)

algoritmo para treinar redes neurais com  
camadas ocultas

# Backpropagation

# Backpropagation

- Comece com pesos escolhidos aleatoriamente.

# Backpropagation

- Comece com pesos escolhidos aleatoriamente.
- Repita:

# Backpropagation

- Comece com pesos escolhidos aleatoriamente.
- Repita:
  - Calcule o erro da camada de saída.

# Backpropagation

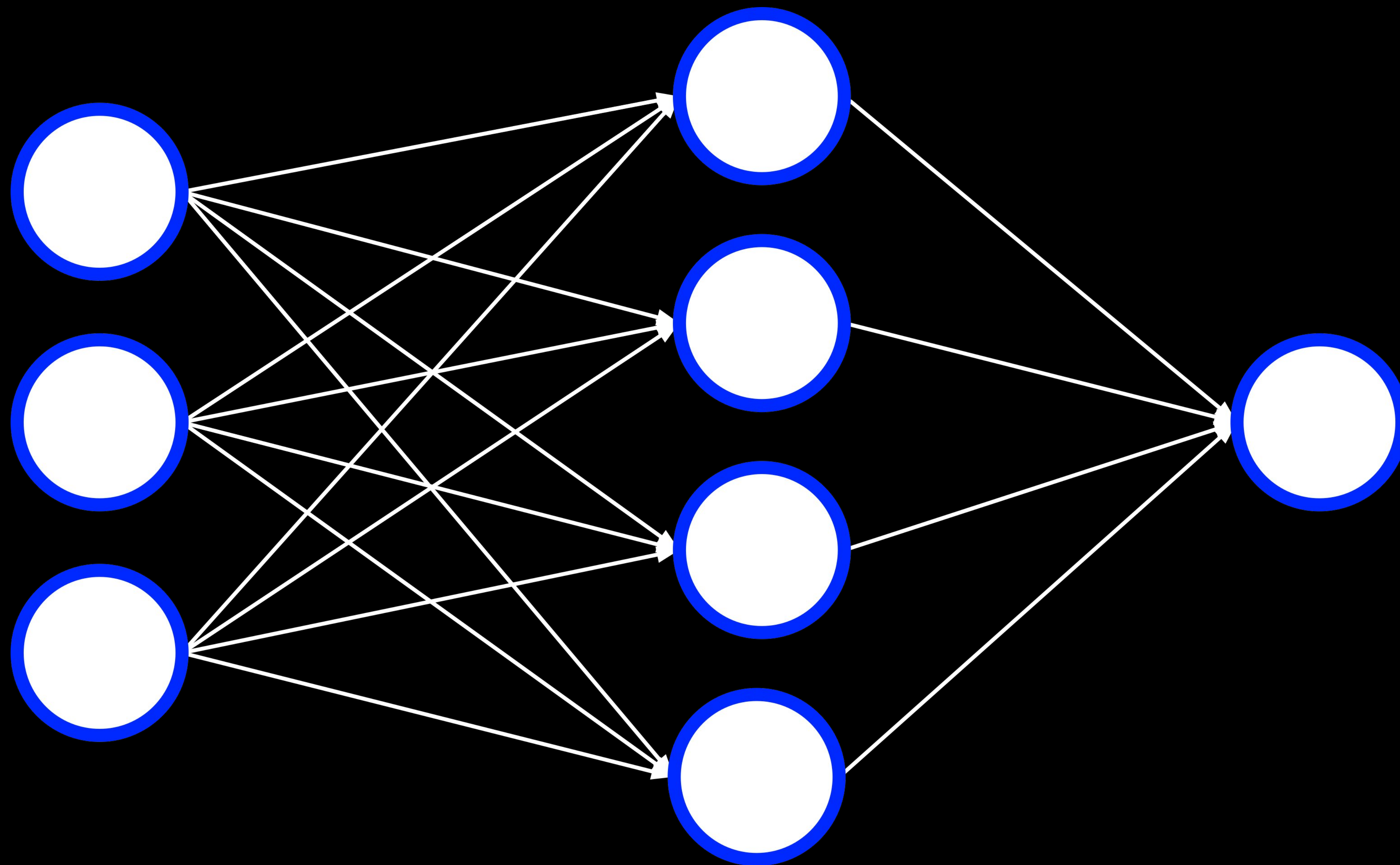
- Comece com pesos escolhidos aleatoriamente.
- Repita:
  - Calcule o erro da camada de saída.
  - Para cada camada, da saída em direção à primeira camada oculta:

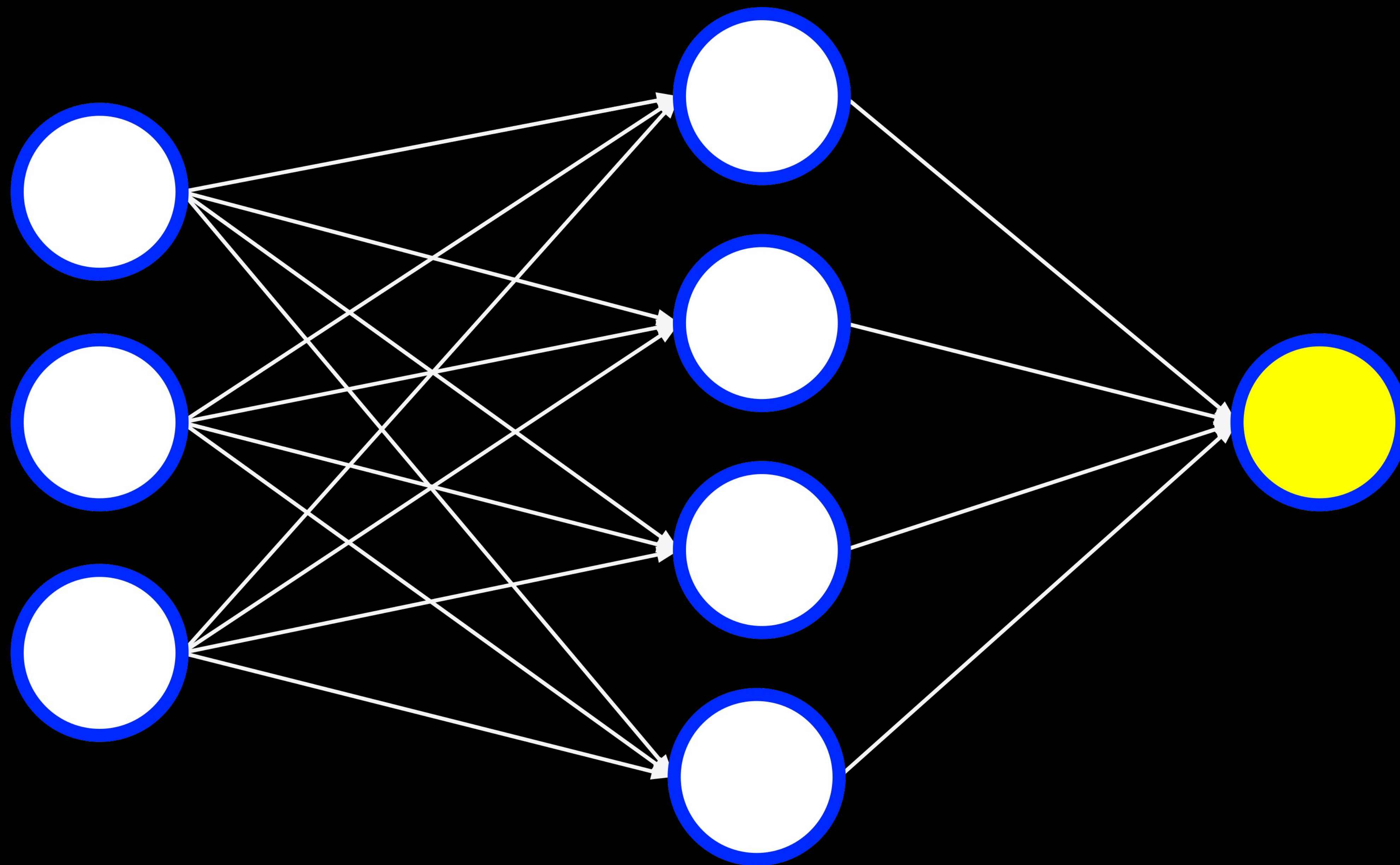
# Backpropagation

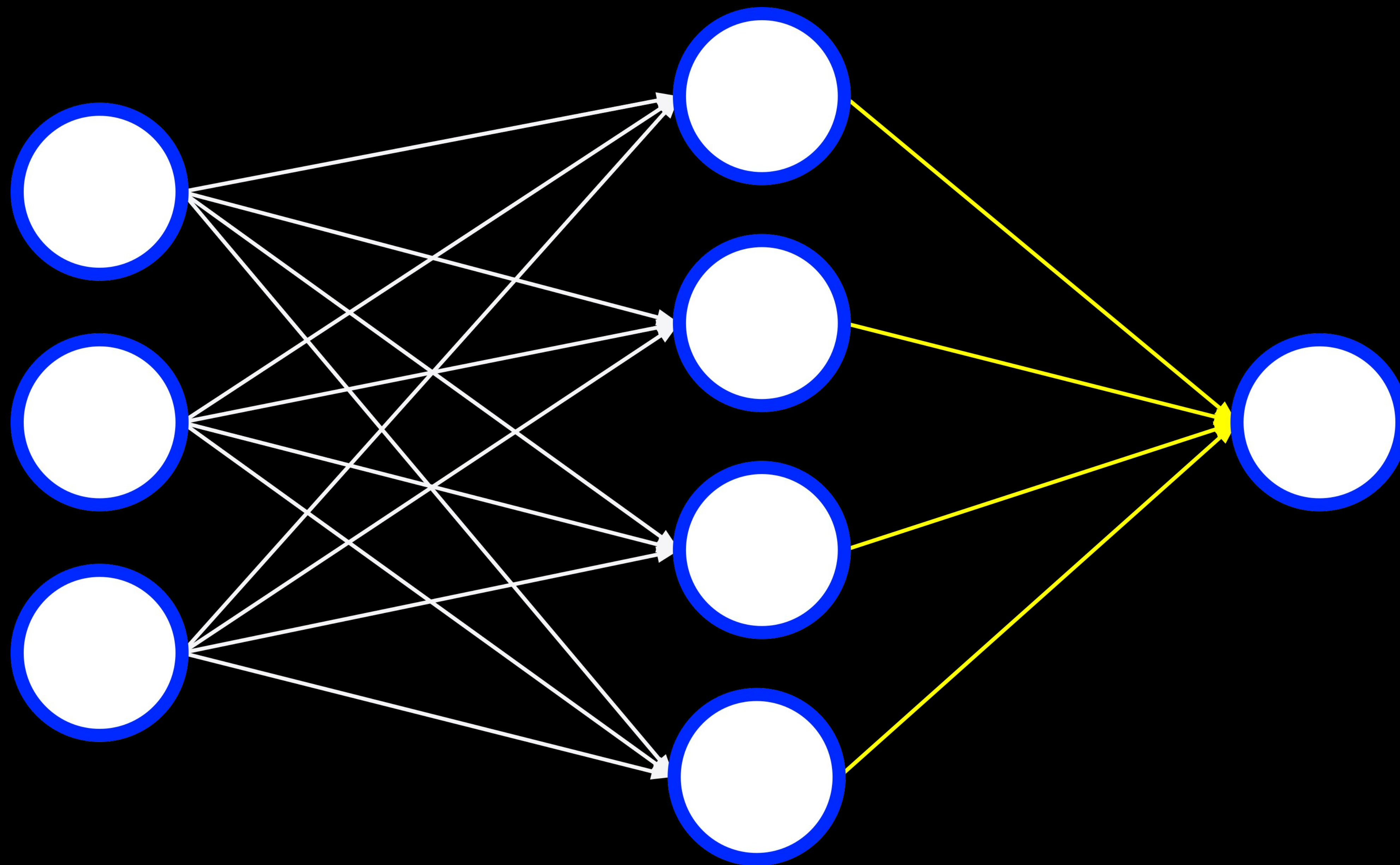
- Comece com pesos escolhidos aleatoriamente.
- Repita:
  - Calcule o erro da camada de saída.
  - Para cada camada, da saída em direção à primeira camada oculta:
    - Propague o erro uma camada para trás.

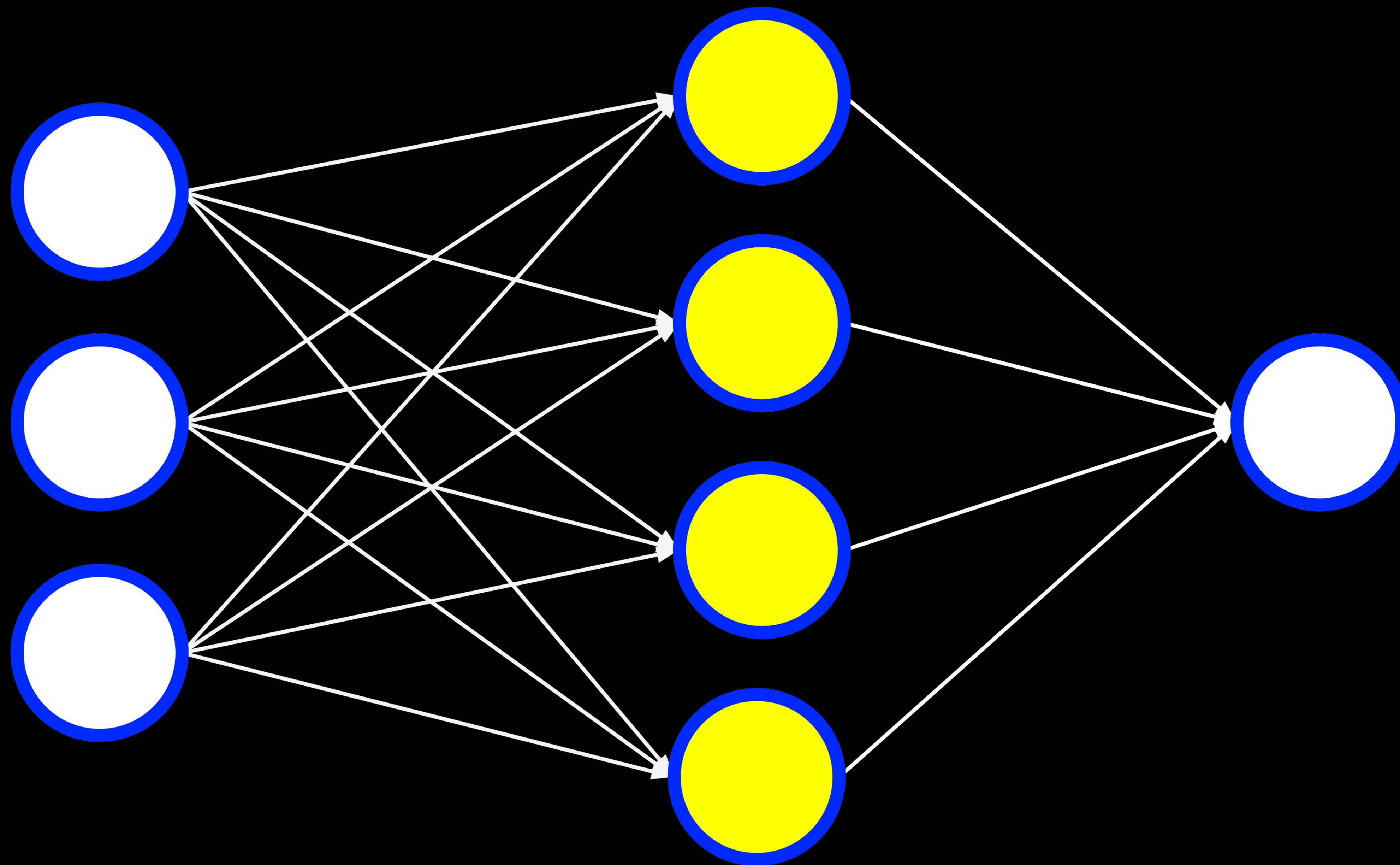
# Backpropagation

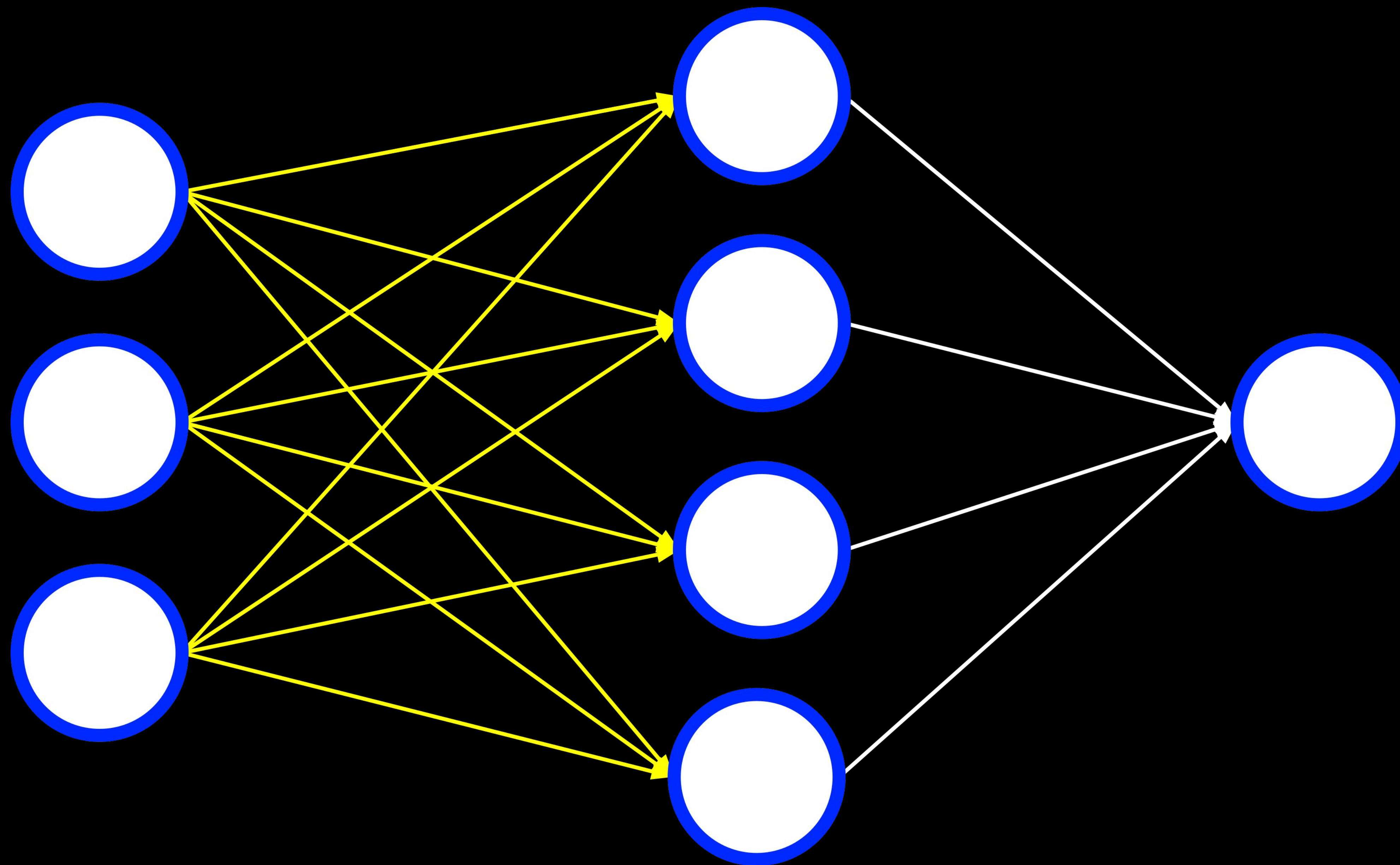
- Comece com pesos escolhidos aleatoriamente.
- Repita:
  - Calcule o erro da camada de saída.
  - Para cada camada, da saída em direção à primeira camada oculta:
    - Propague o erro uma camada para trás.
    - Atualize os pesos.

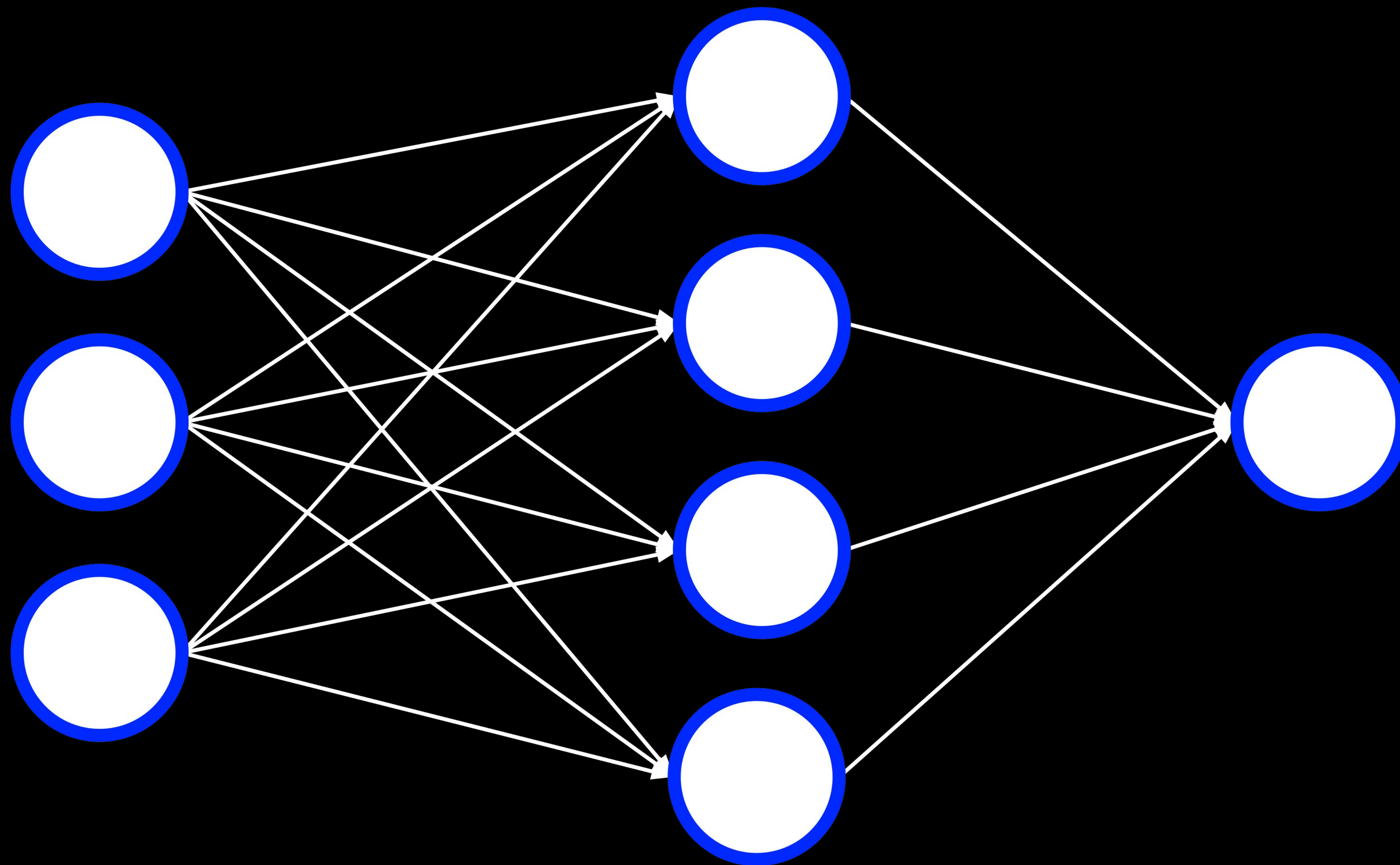


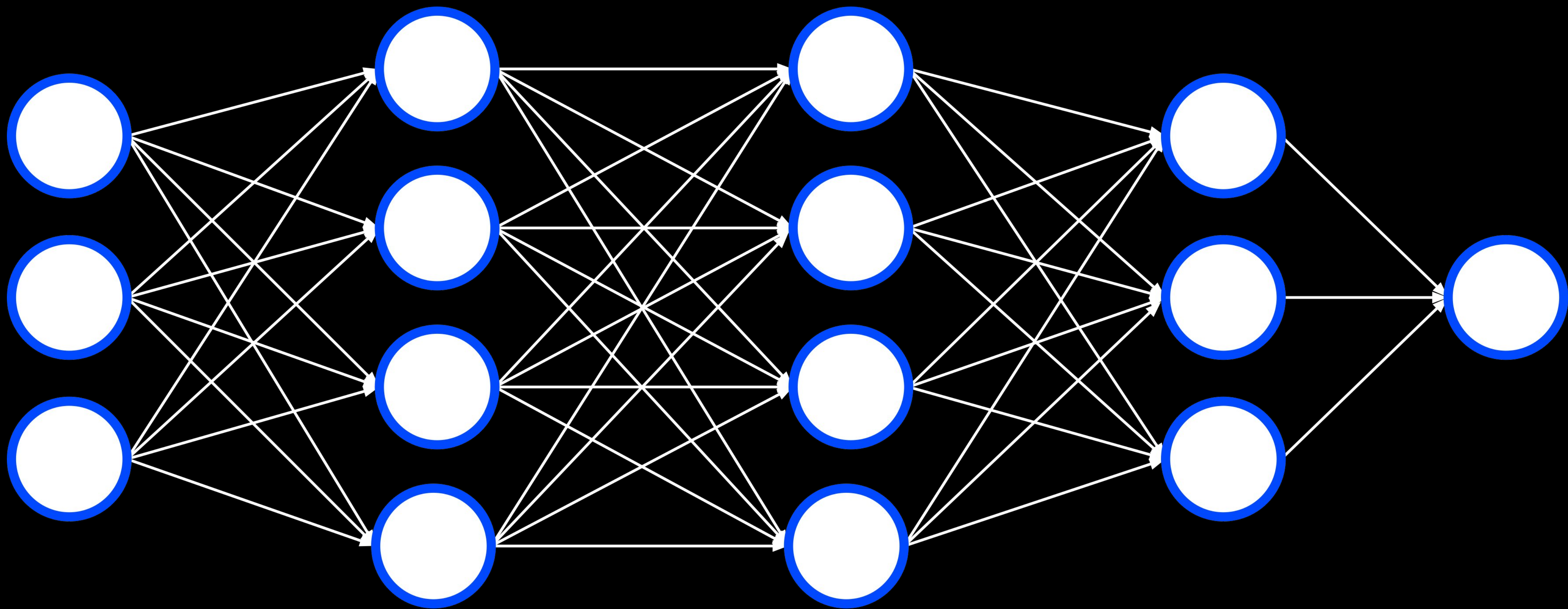






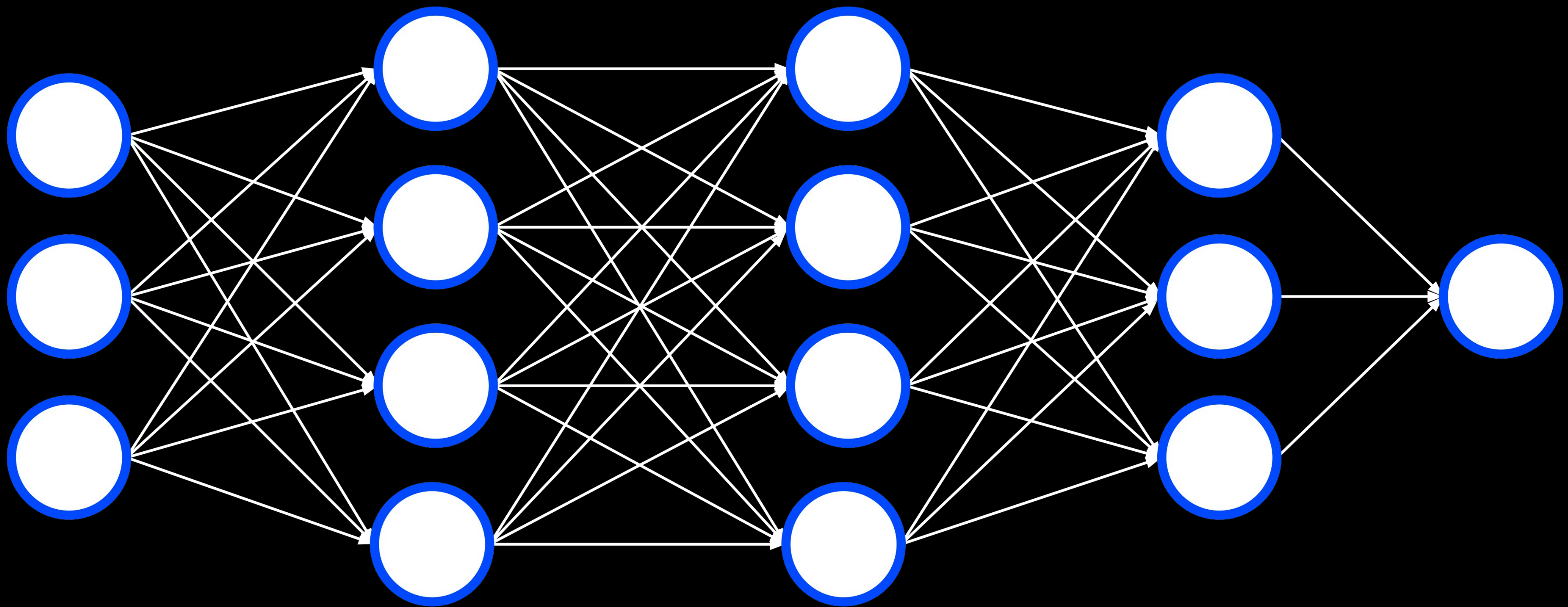






# redes neurais profundas

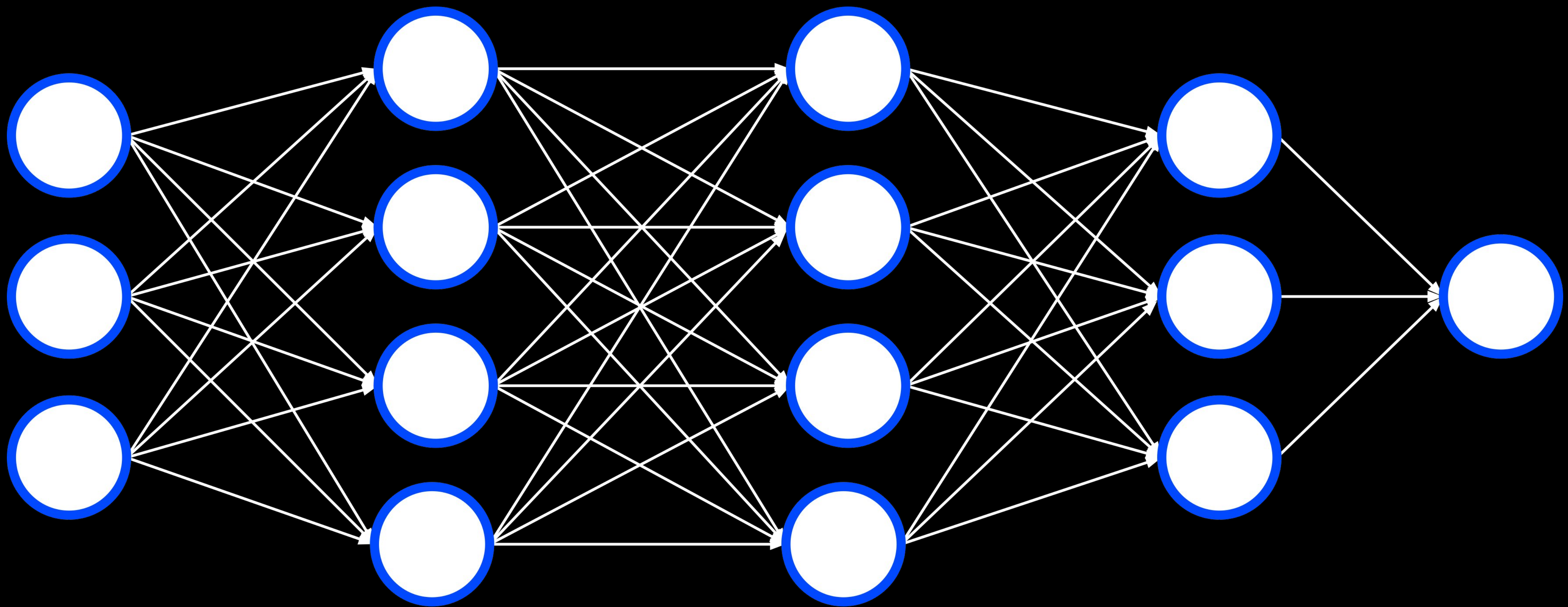
rede neural com várias camadas ocultas

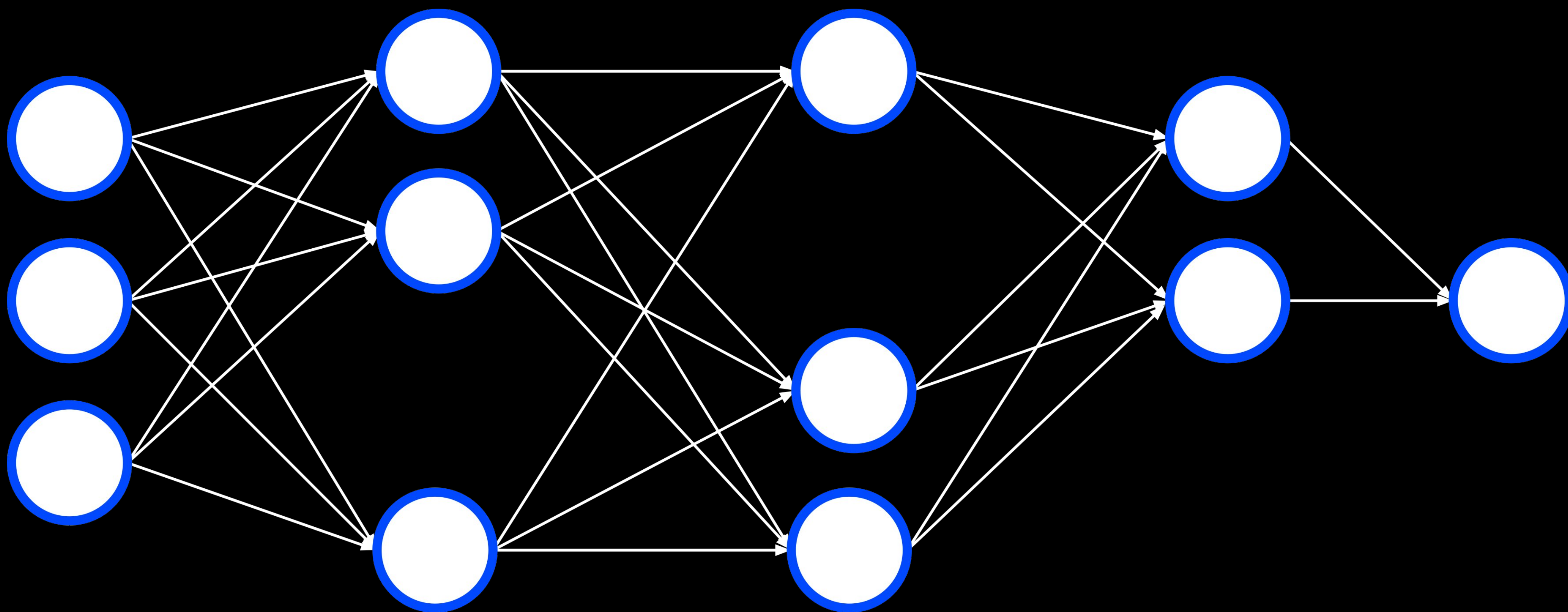


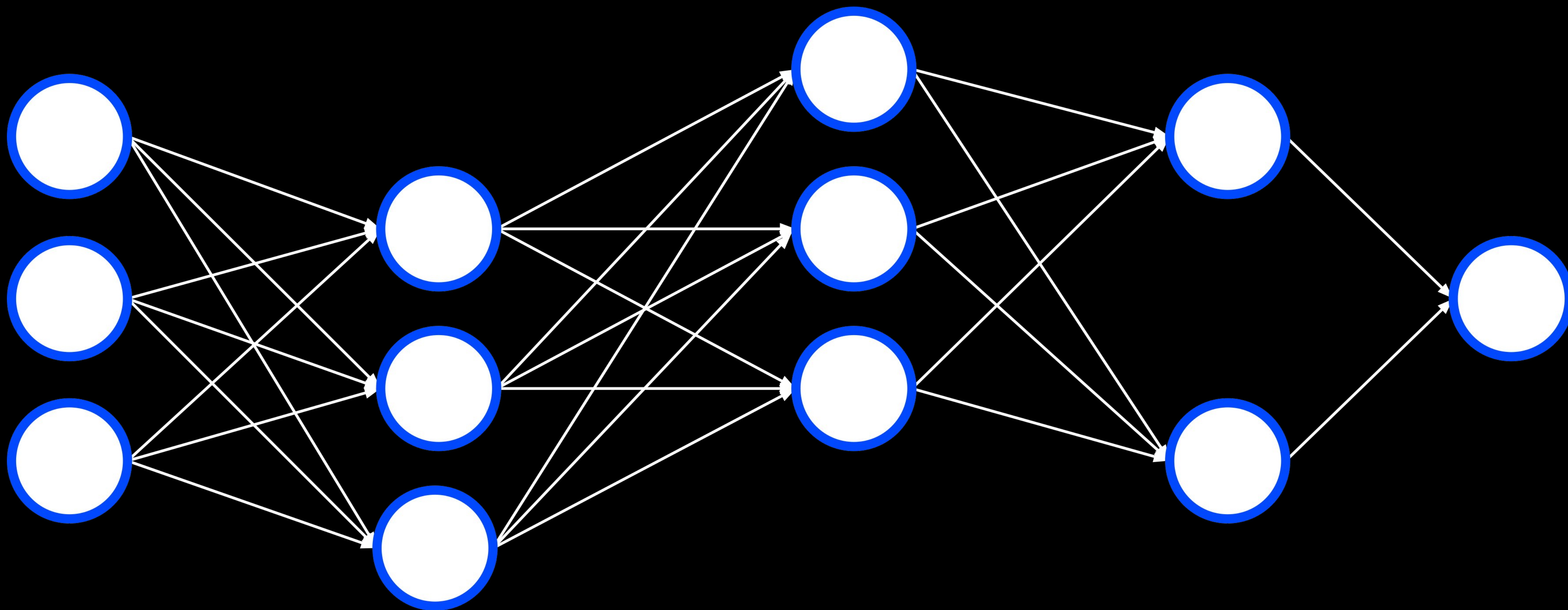
Sobreaajuste

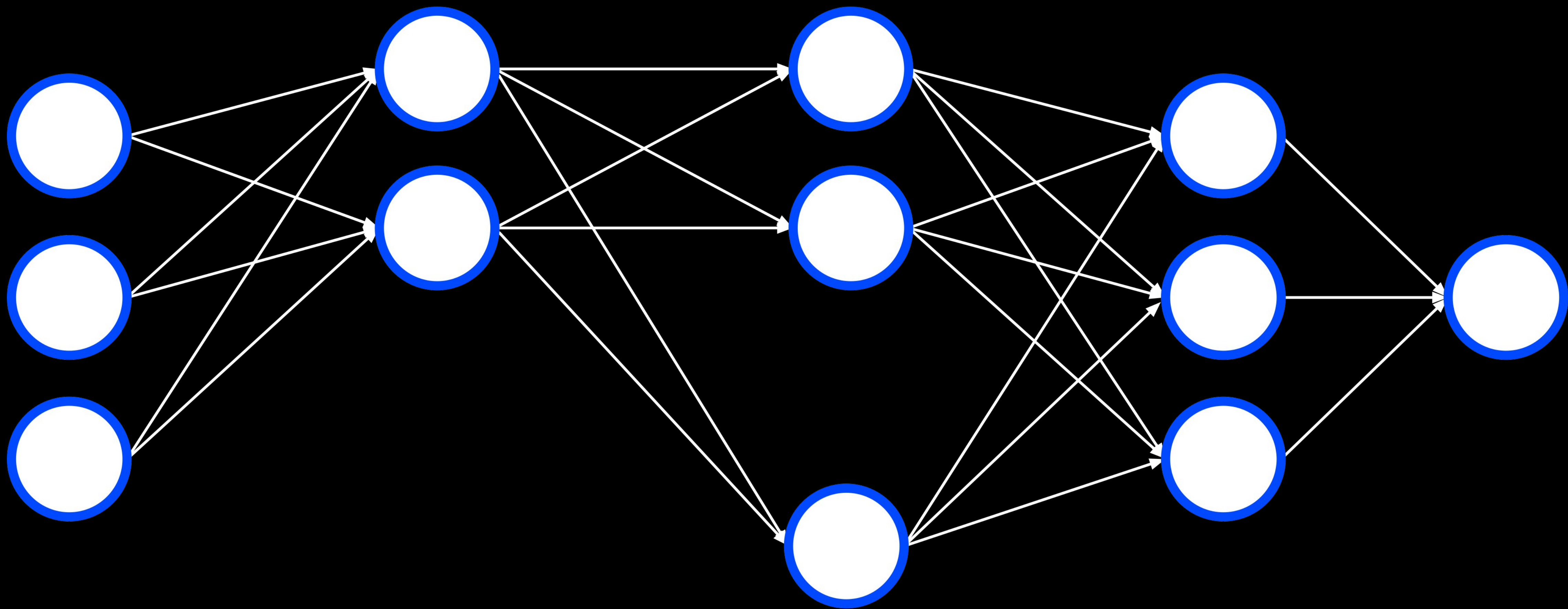
# dropout

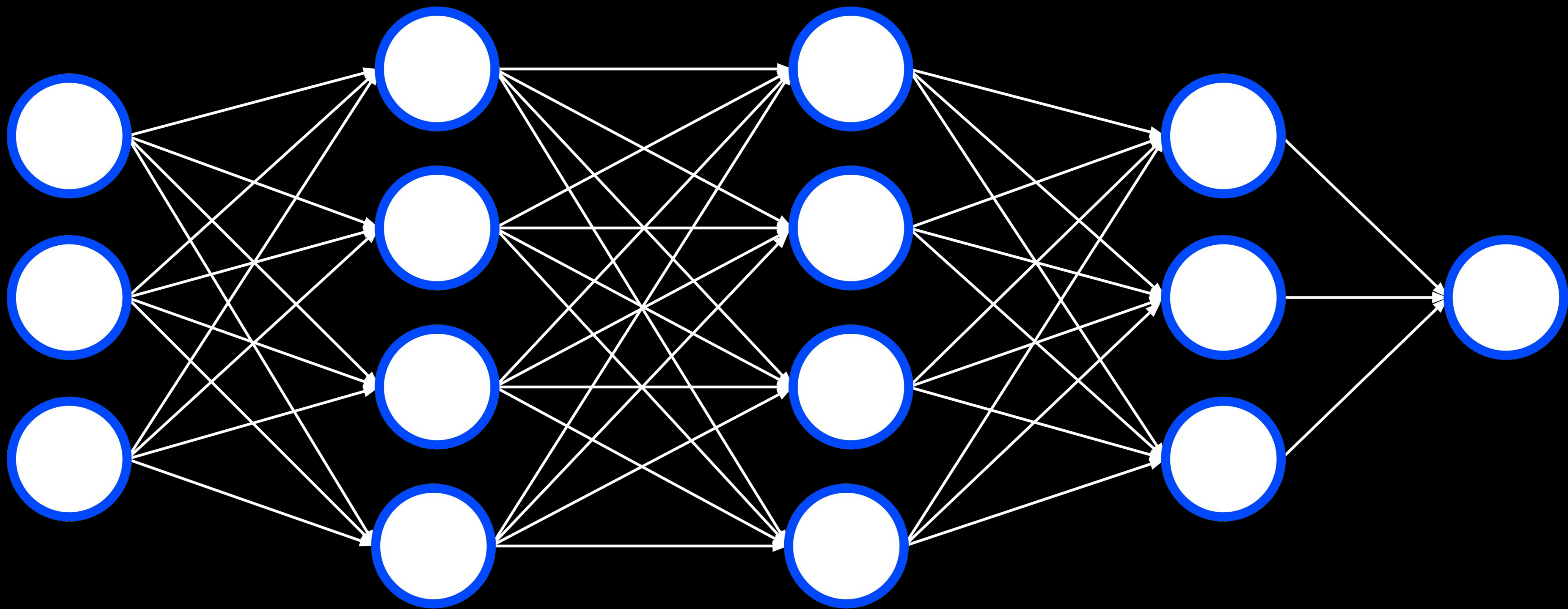
remoção temporária de unidades — escolhidas aleatoriamente — de uma rede neural para evitar dependência excessiva de algumas unidades











# PyTorch

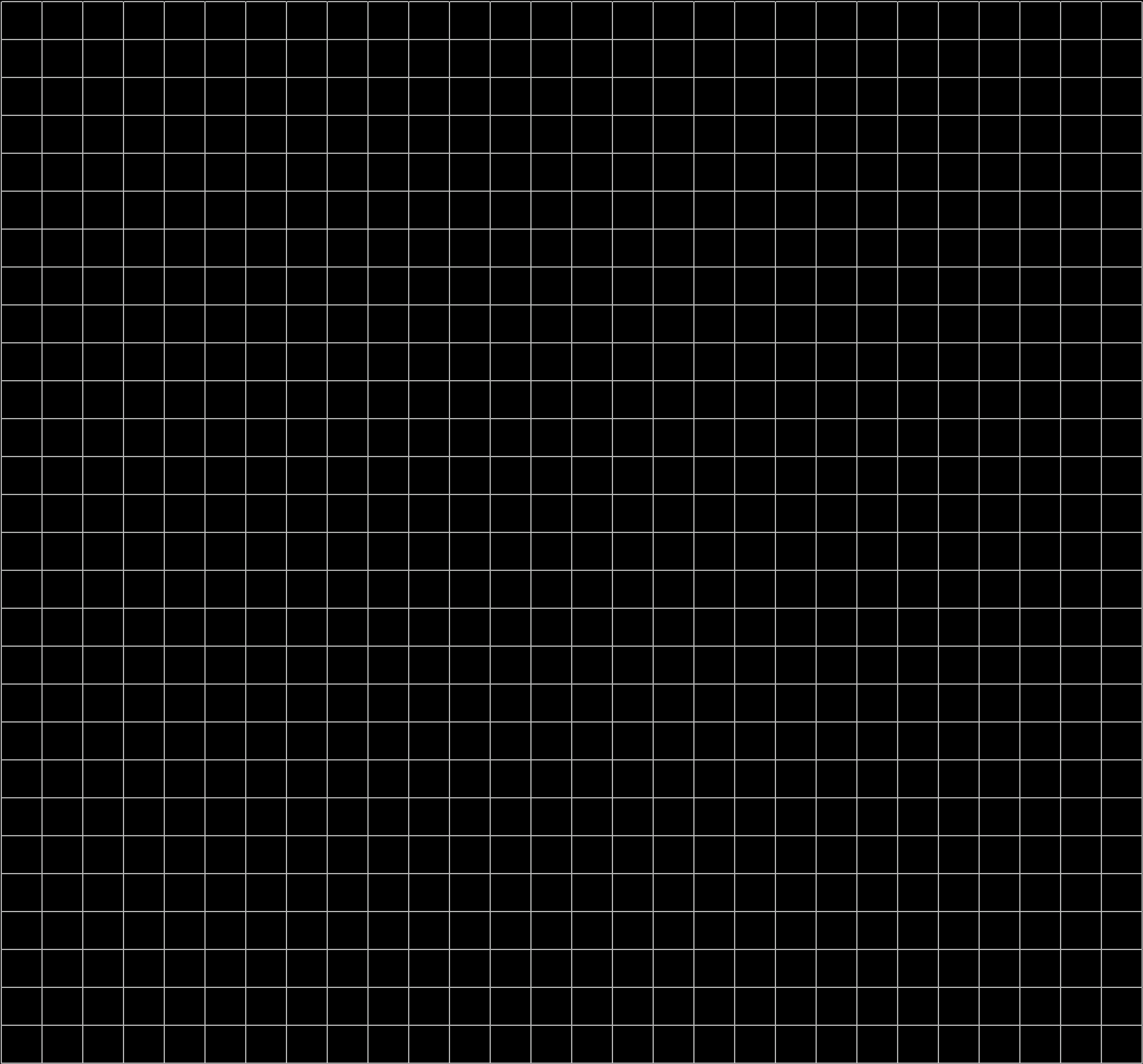
# TensorFlow

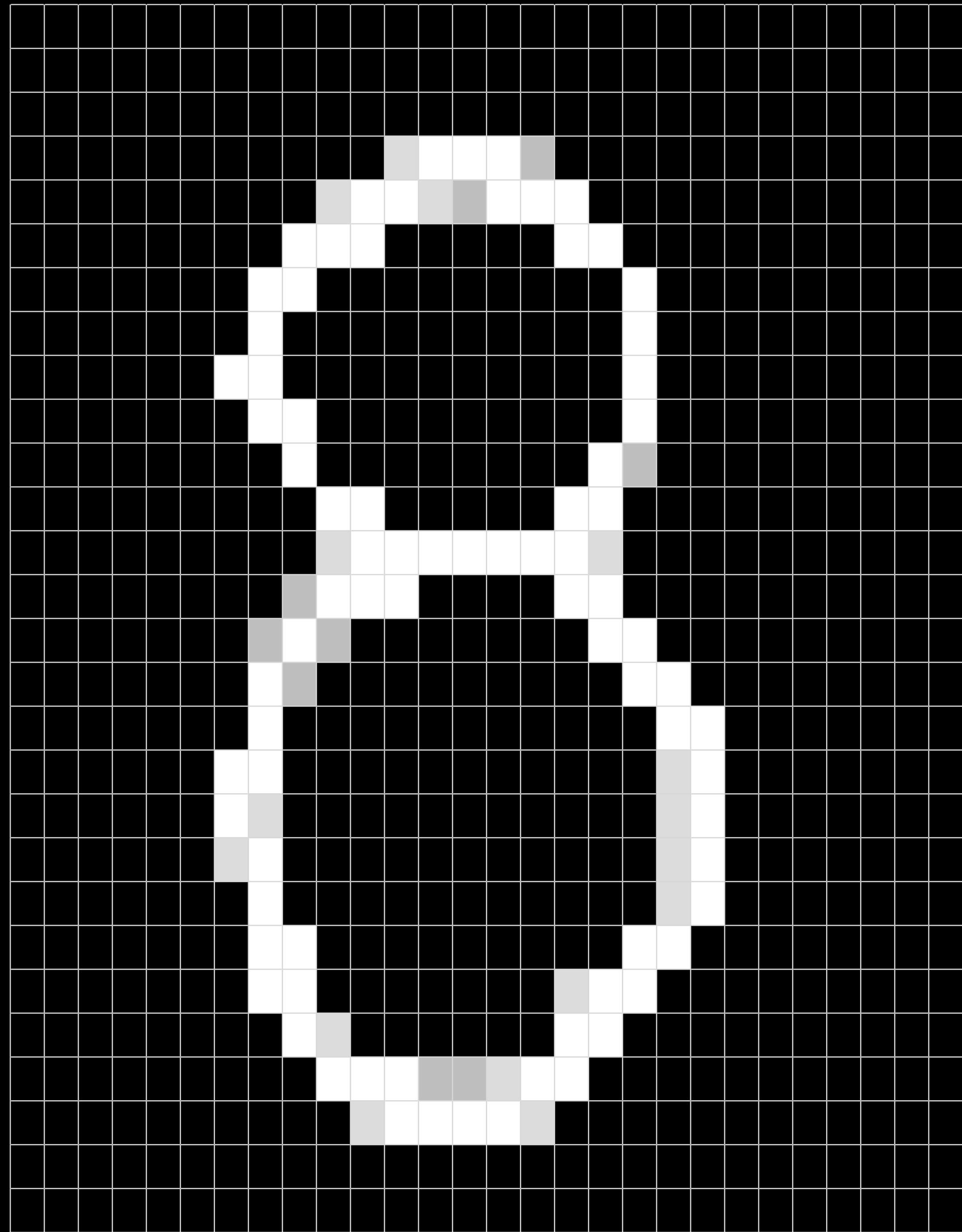
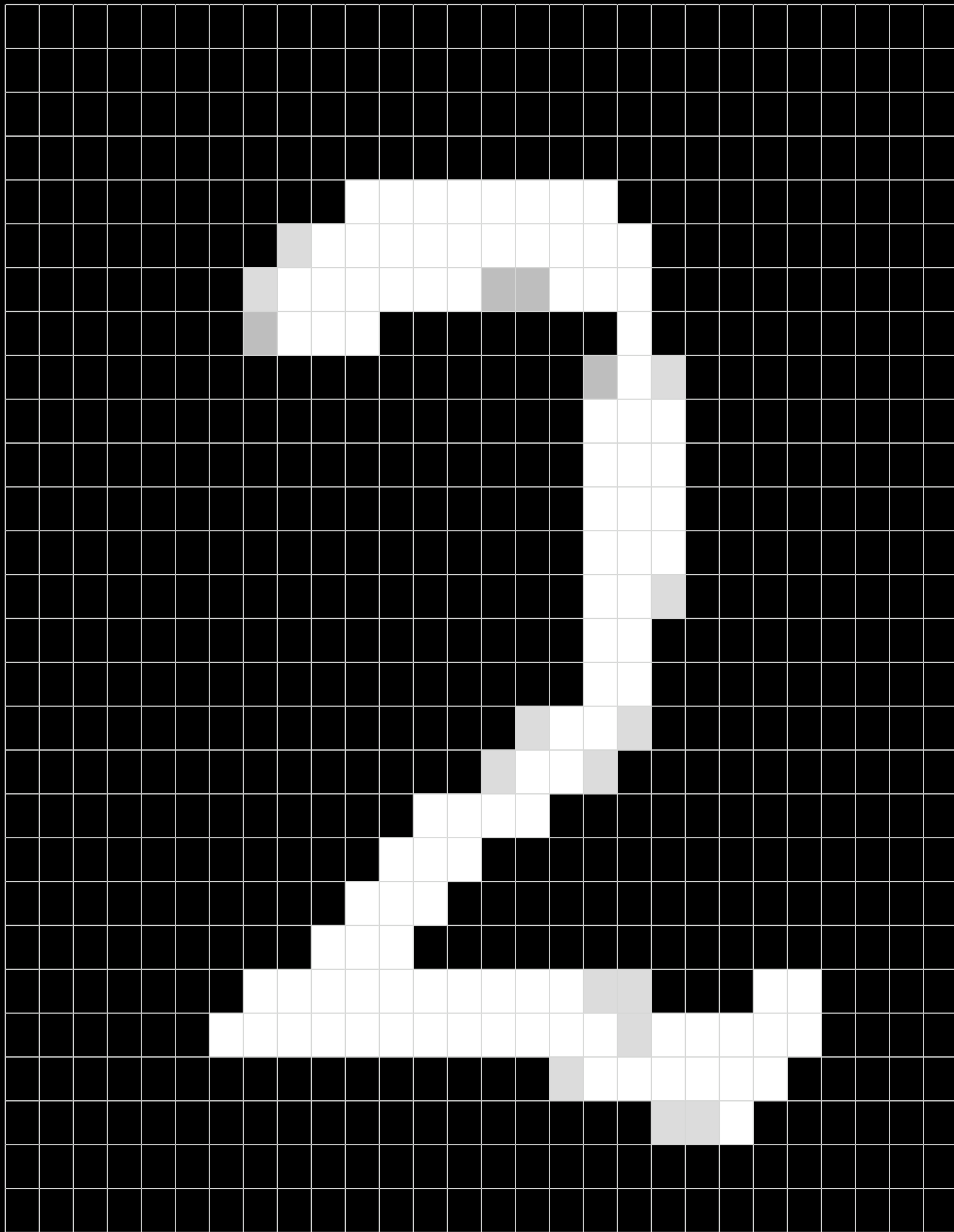


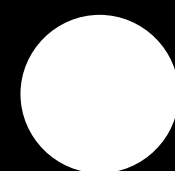
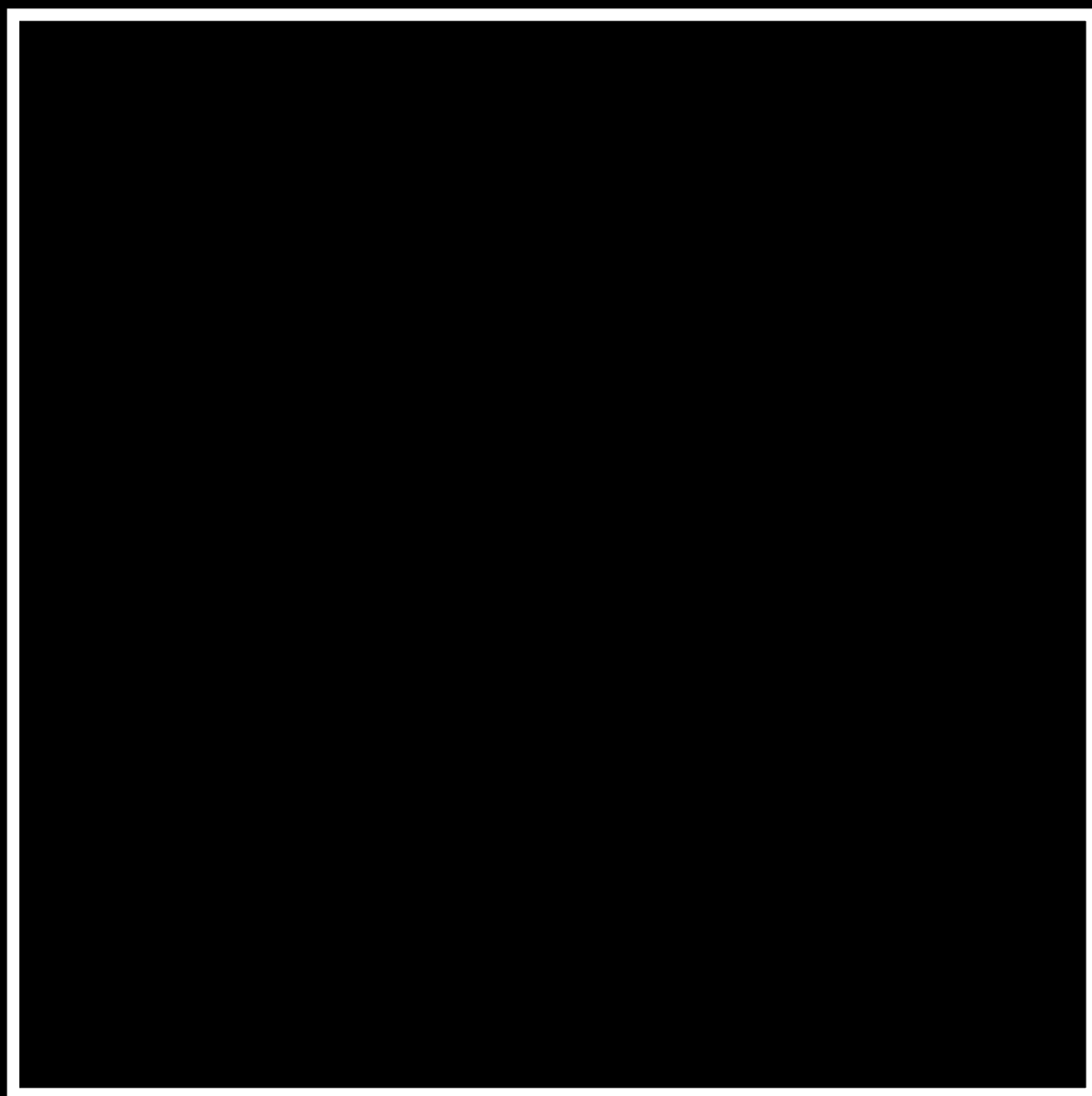
# visão computacional

métodos computacionais para analisar e  
compreender imagens digitais

[illegible]



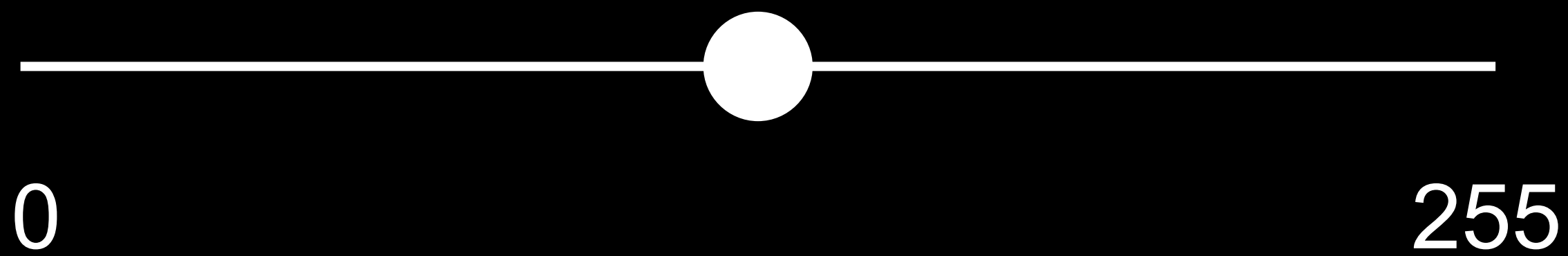
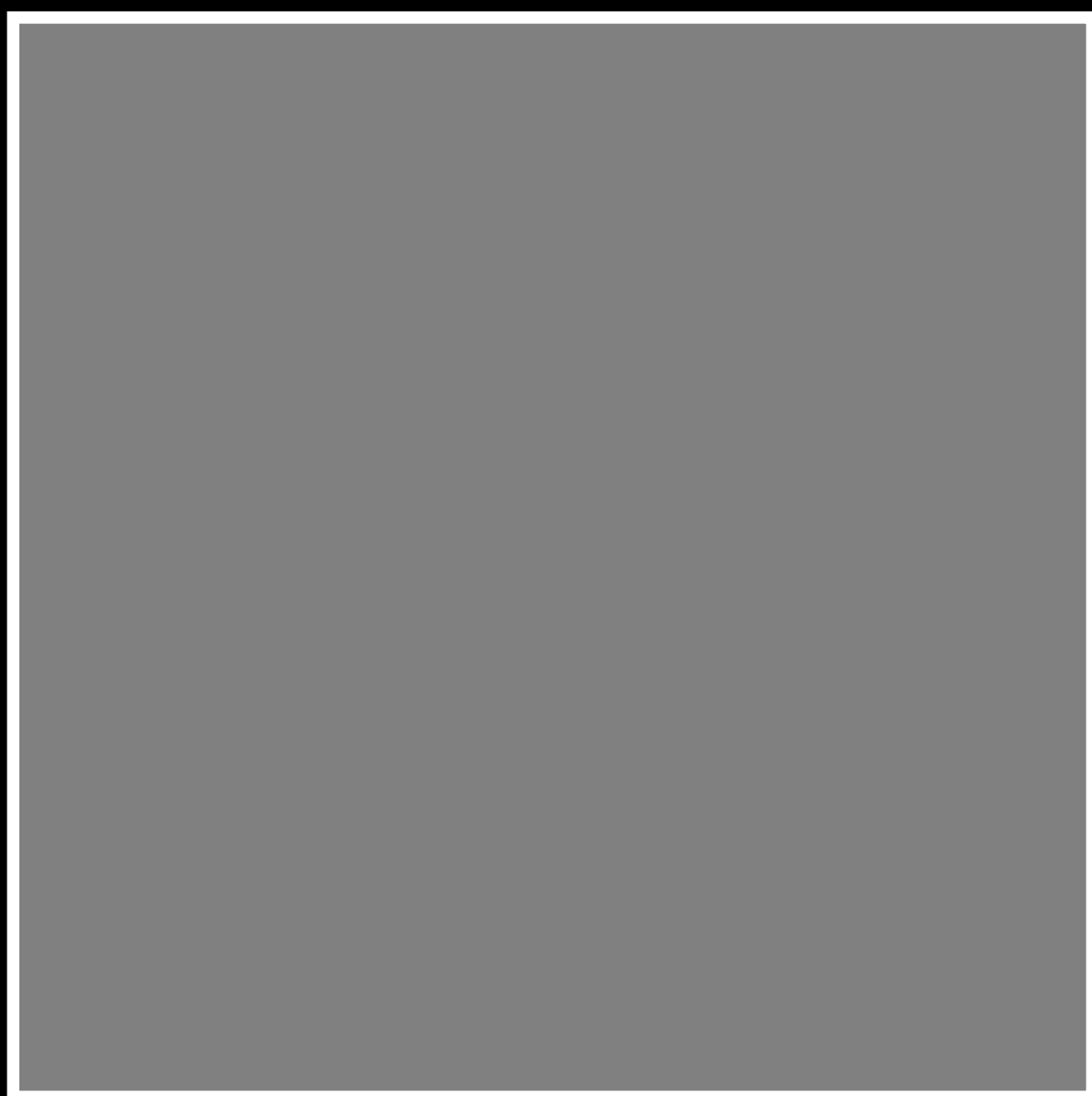


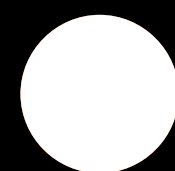
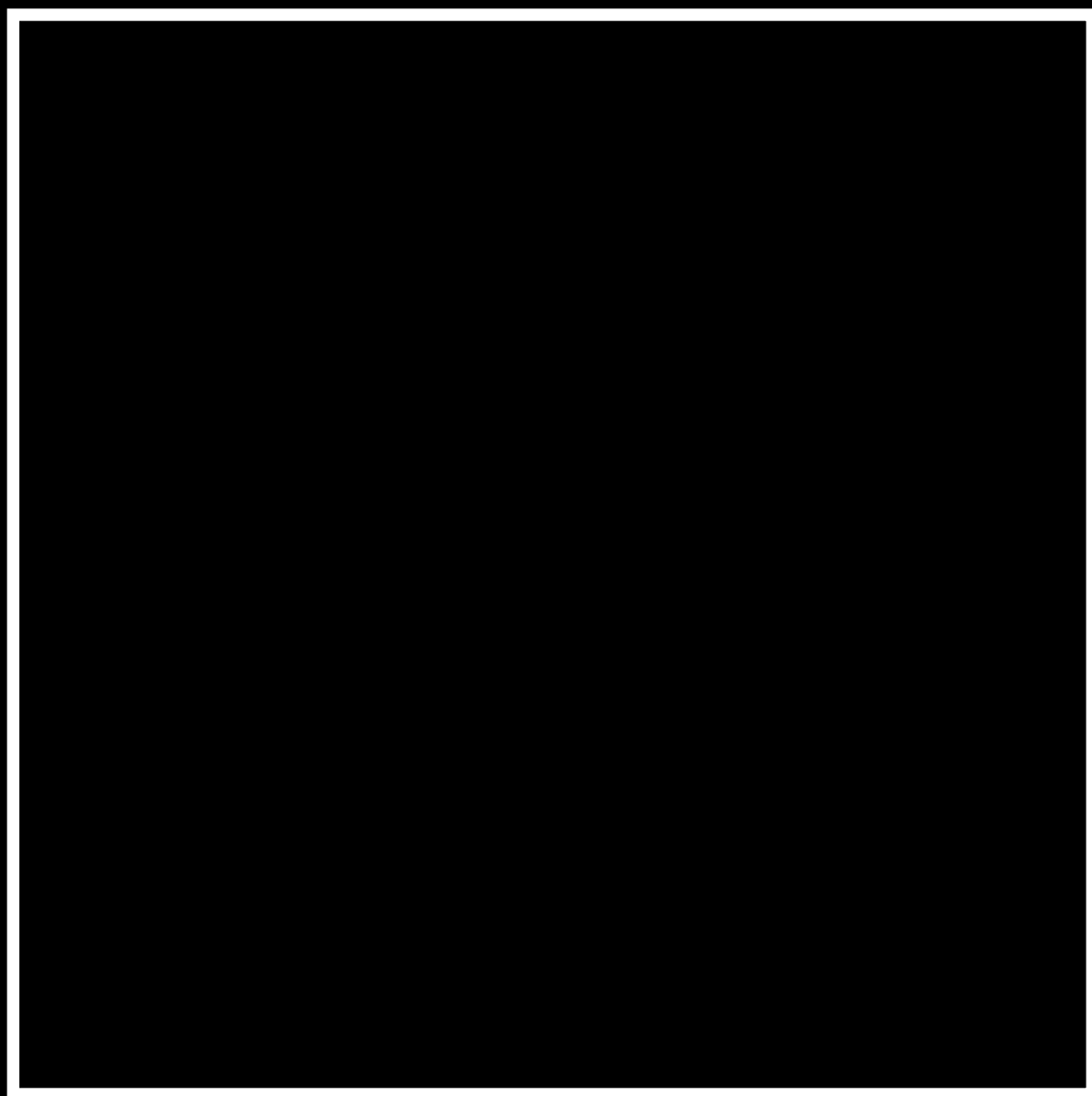


0

255

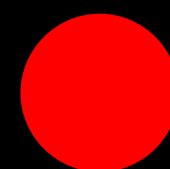
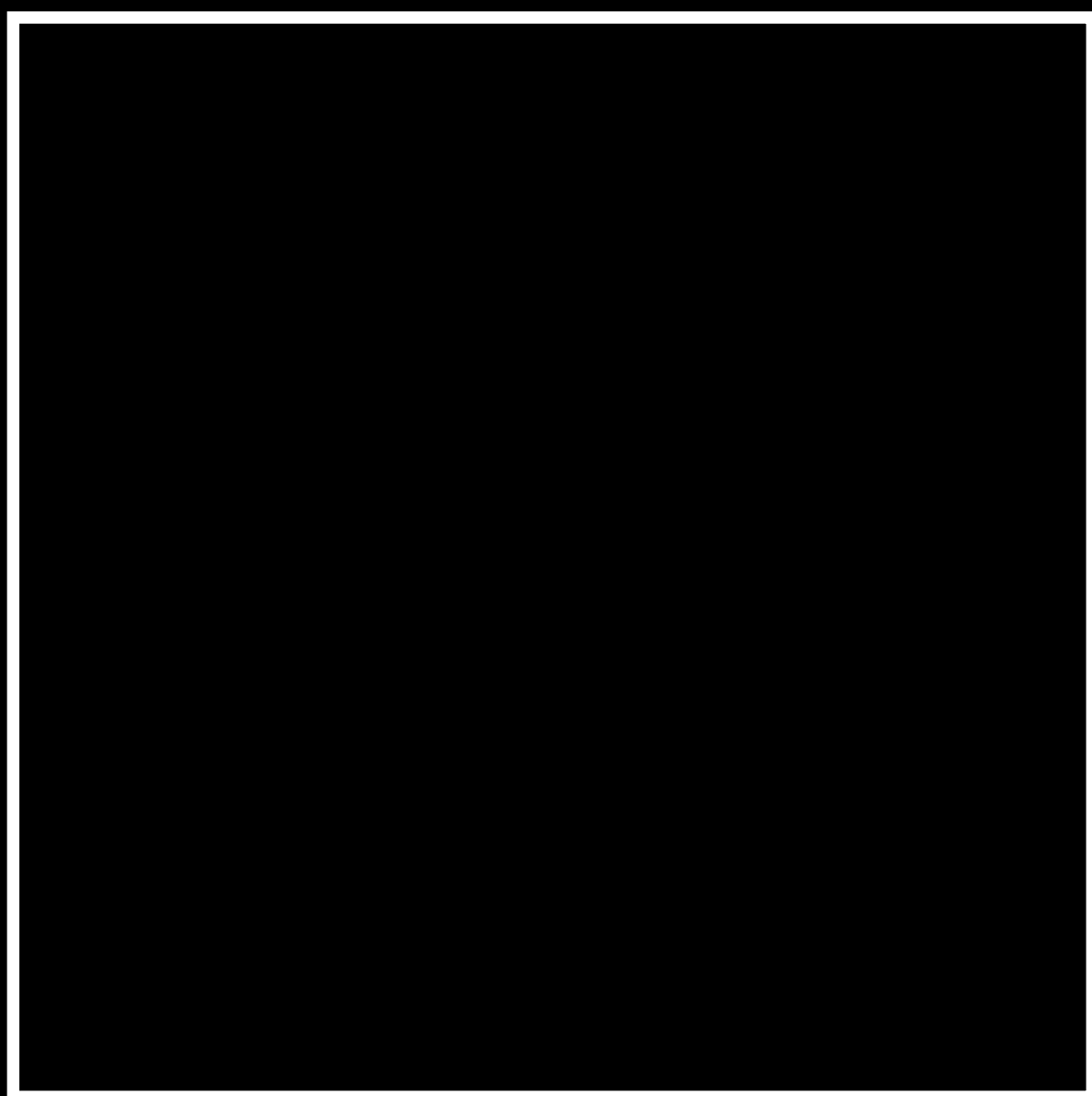






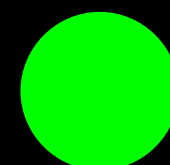
0

255



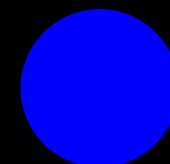
0

255



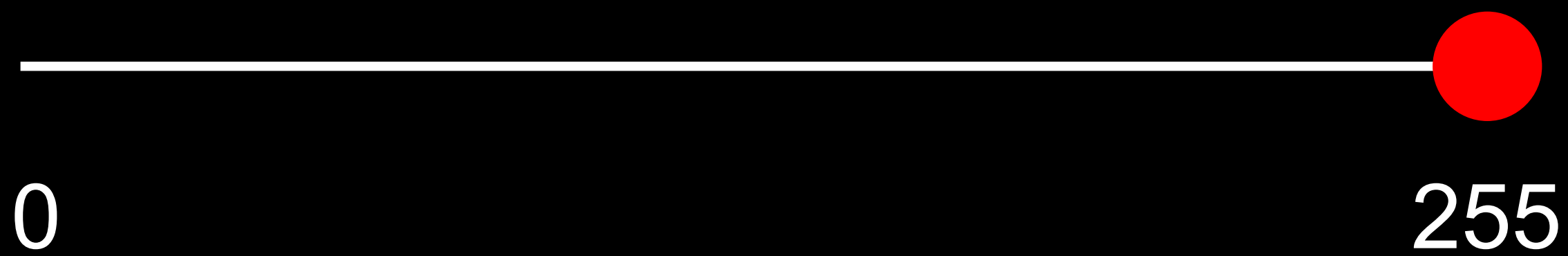
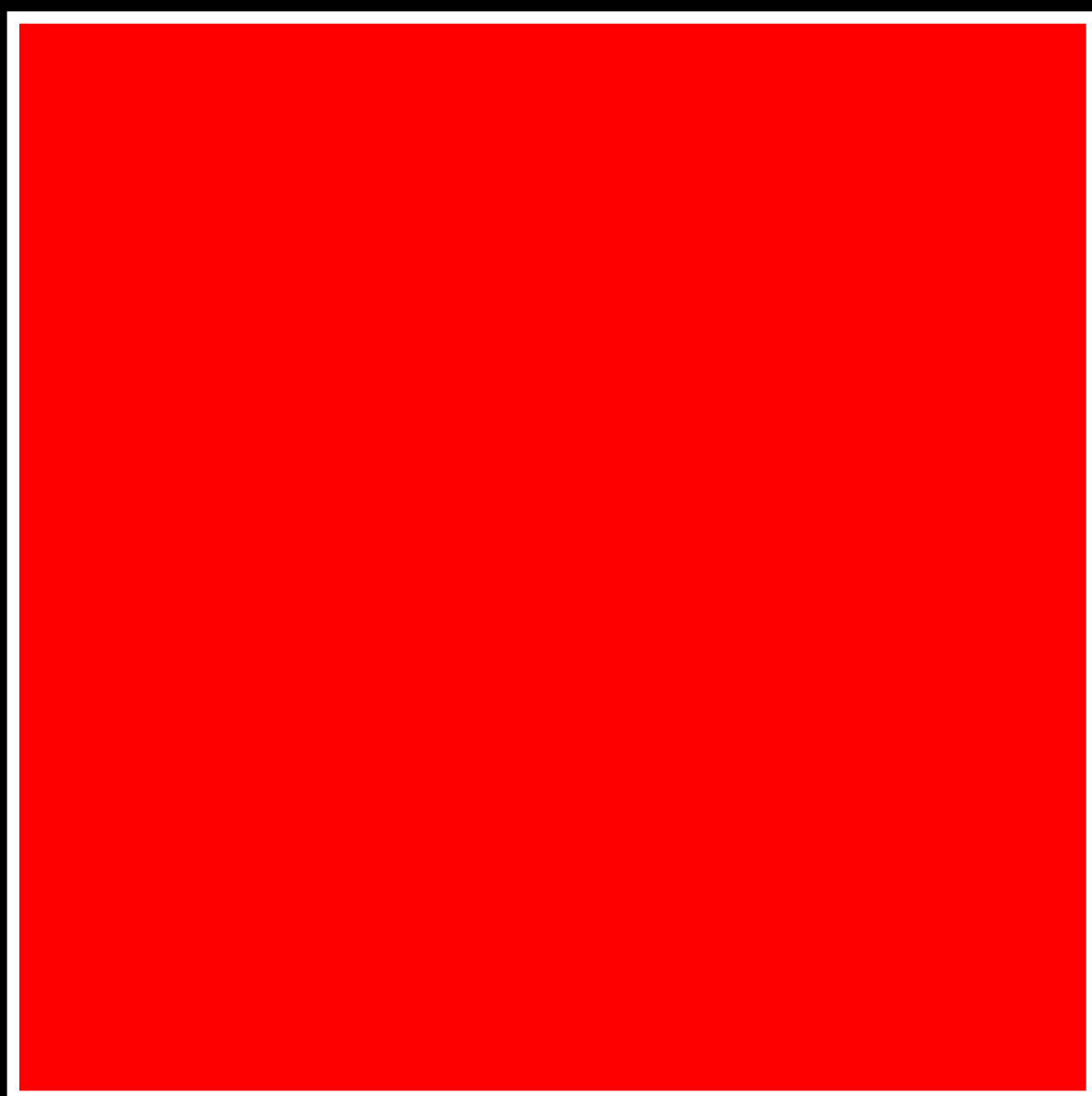
0

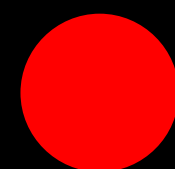
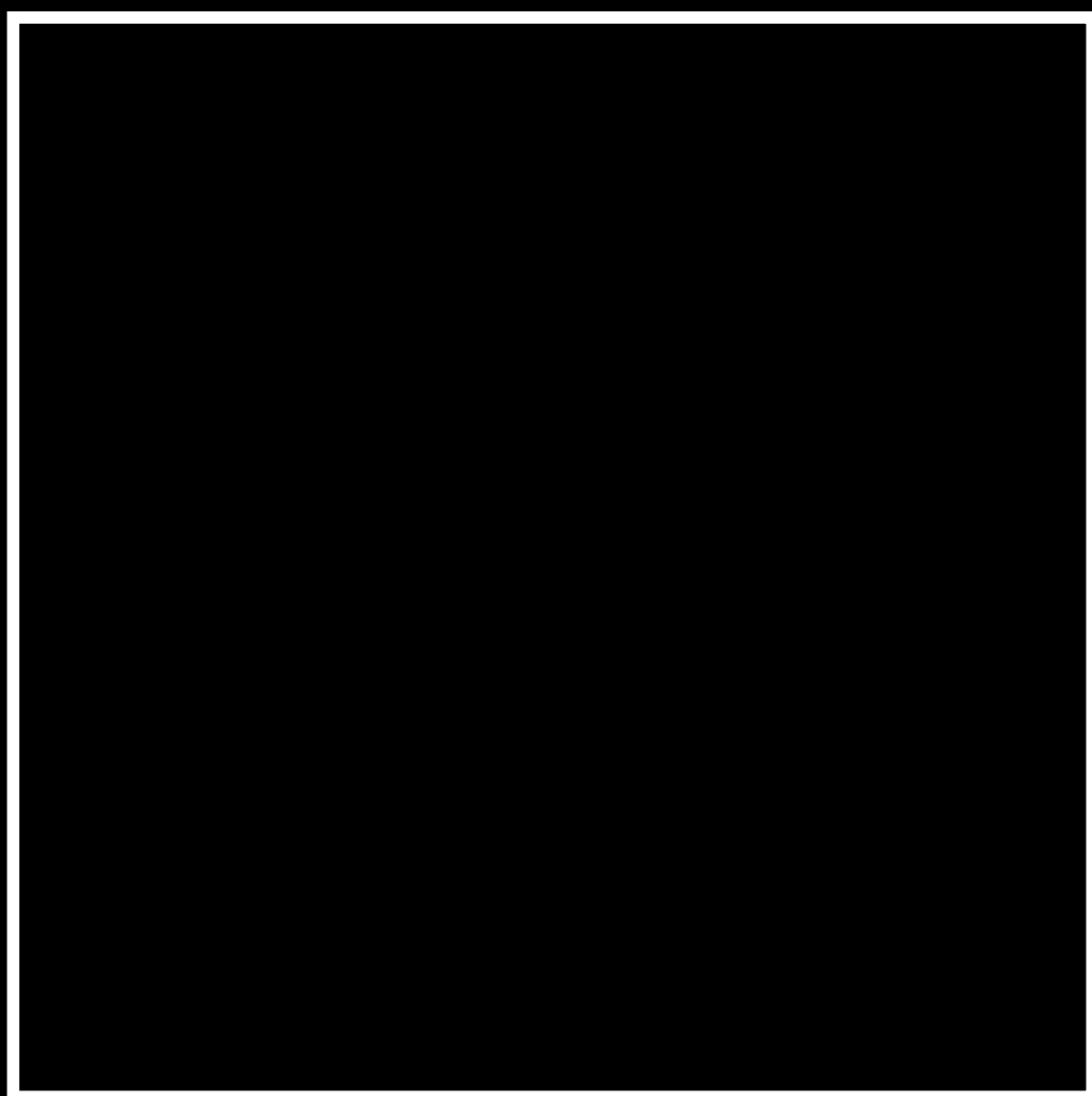
255



0

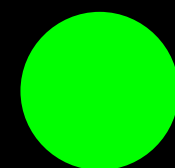
255





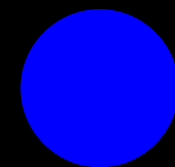
0

255



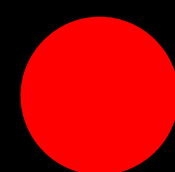
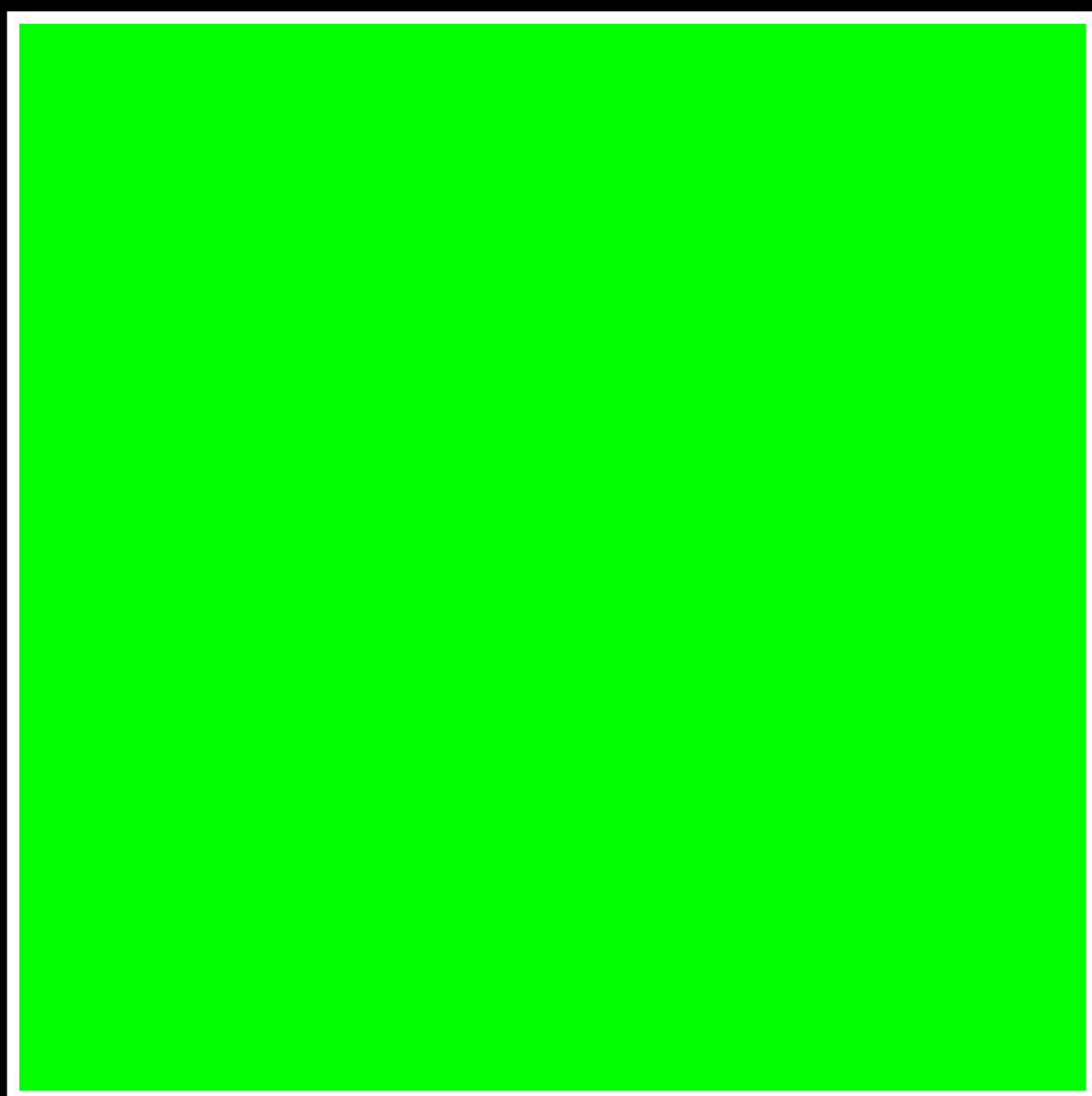
0

255



0

255



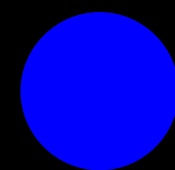
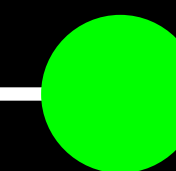
0

255



0

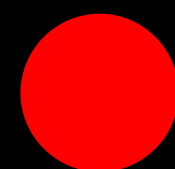
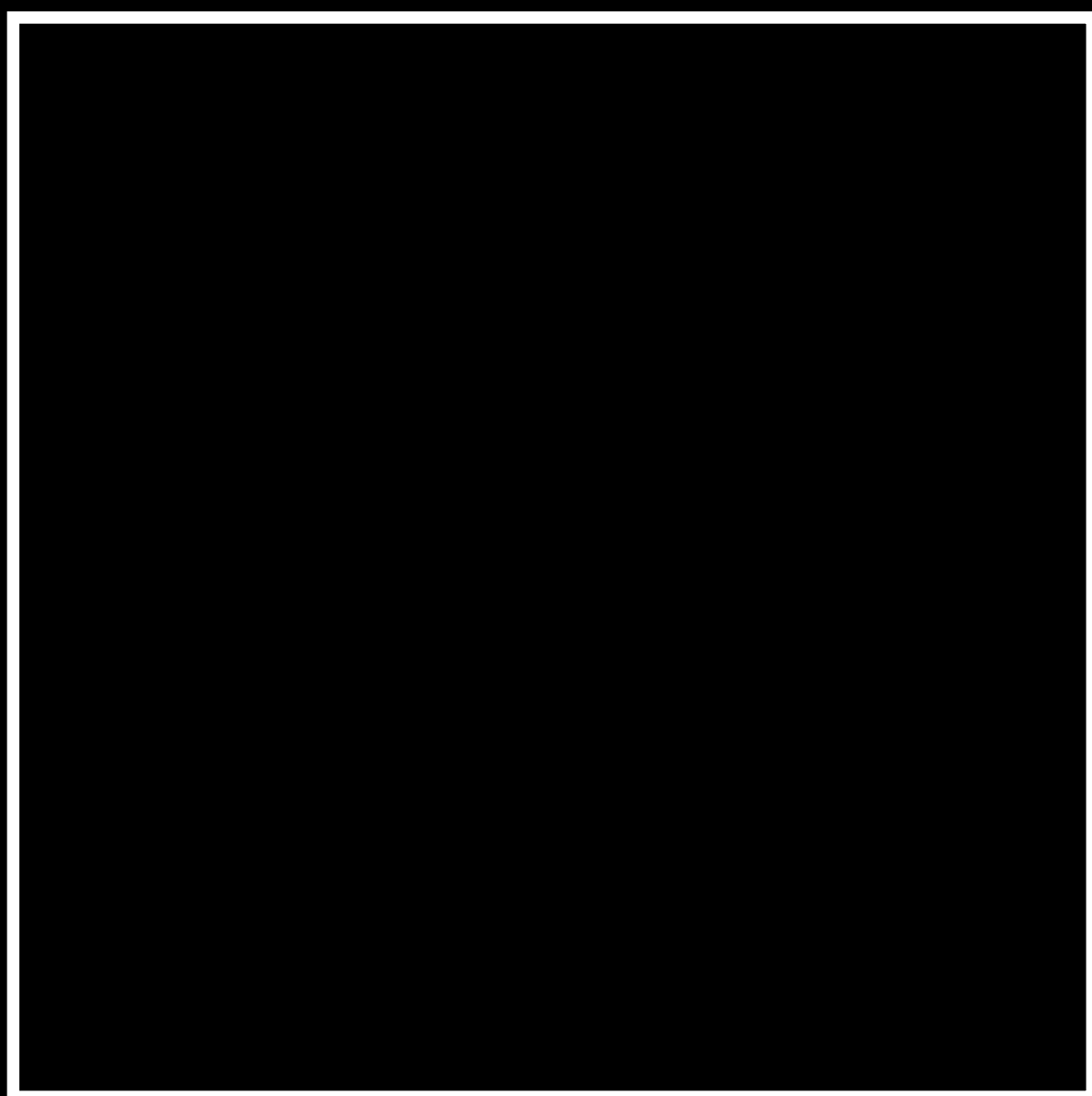
255



0

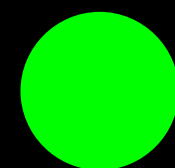
255





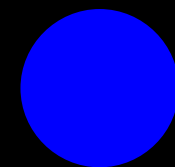
0

255



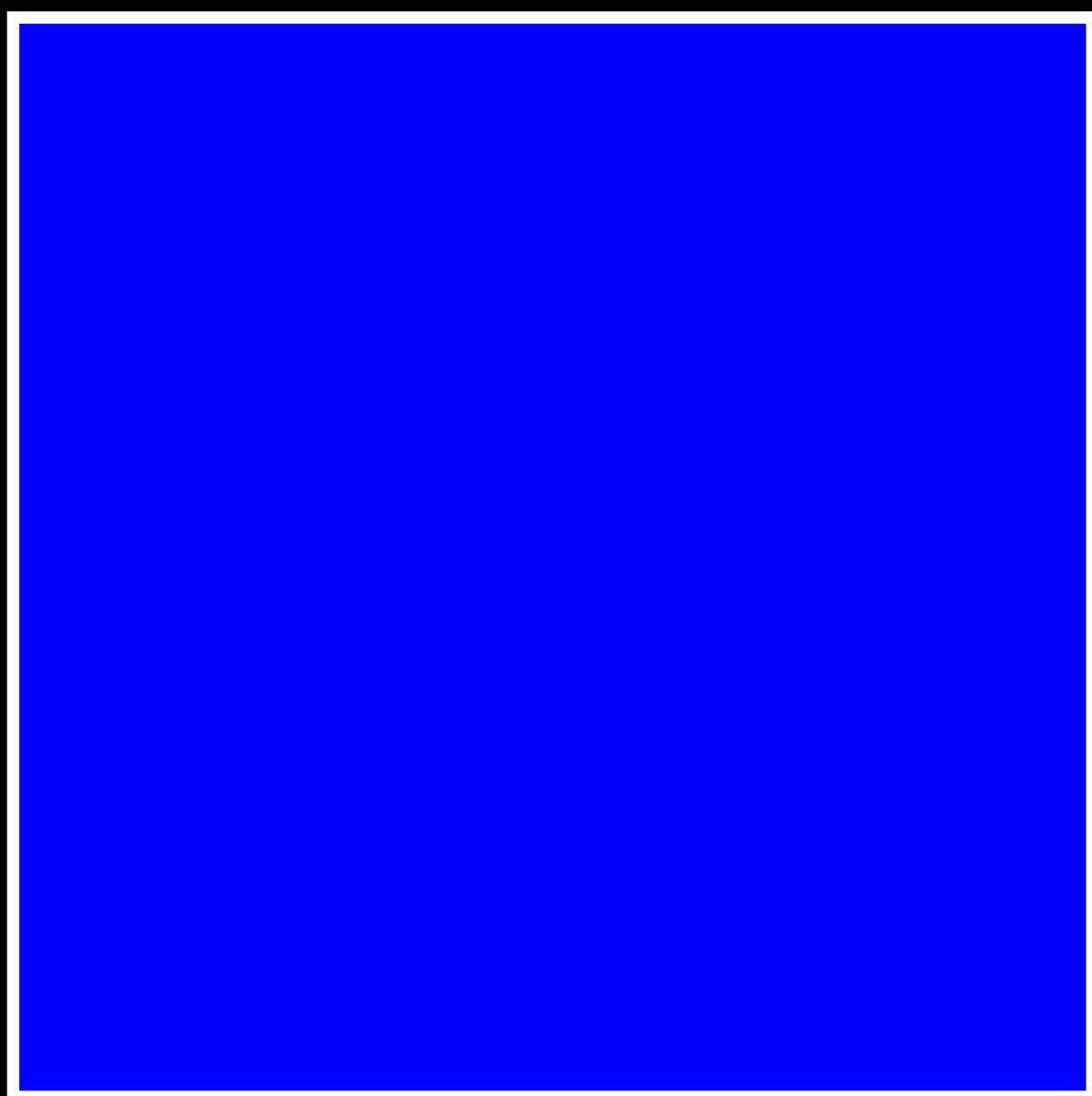
0

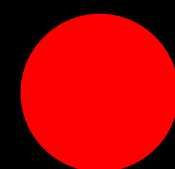
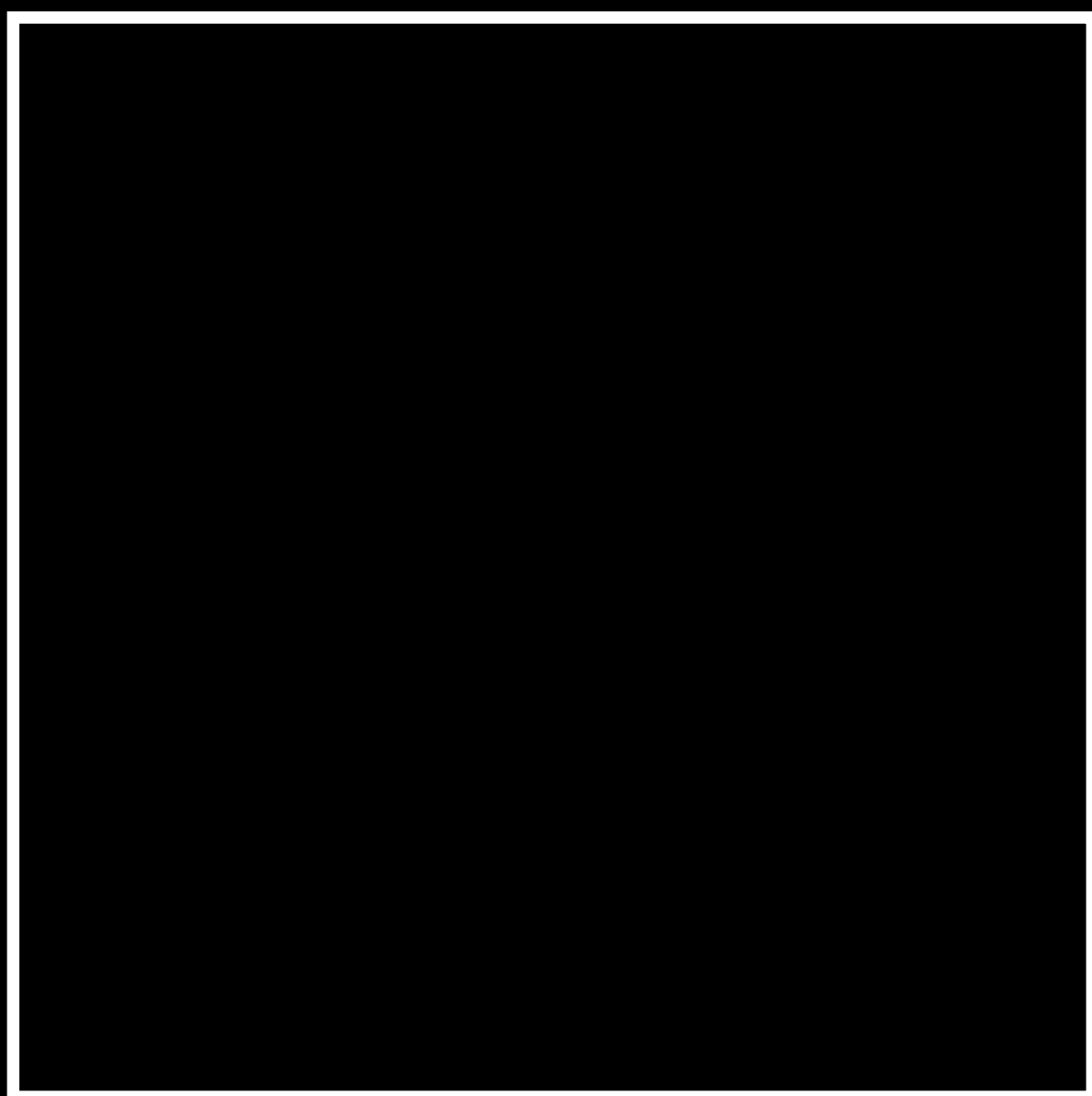
255



0

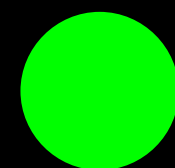
255





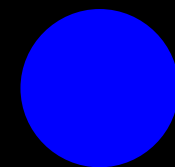
0

255



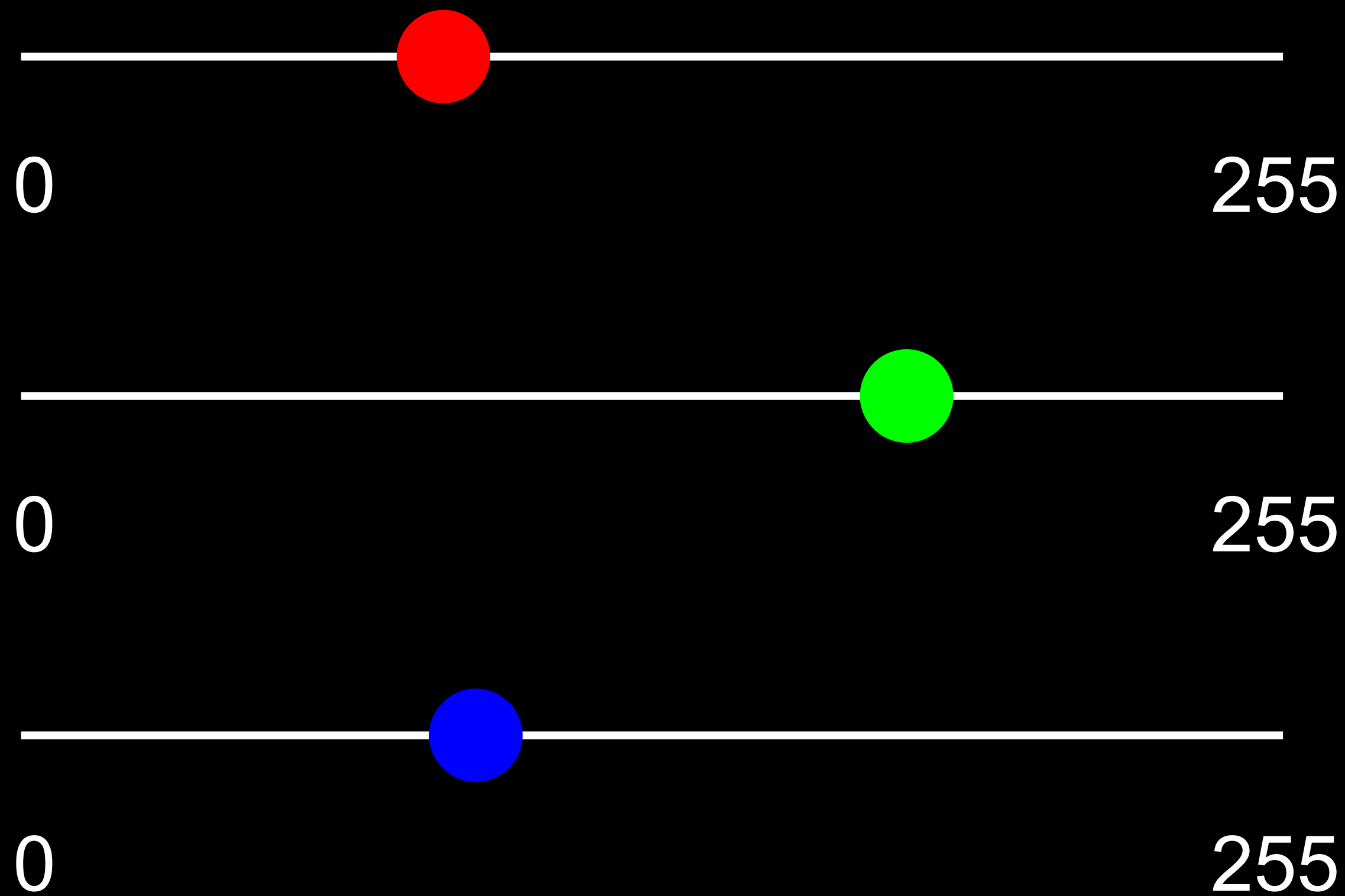
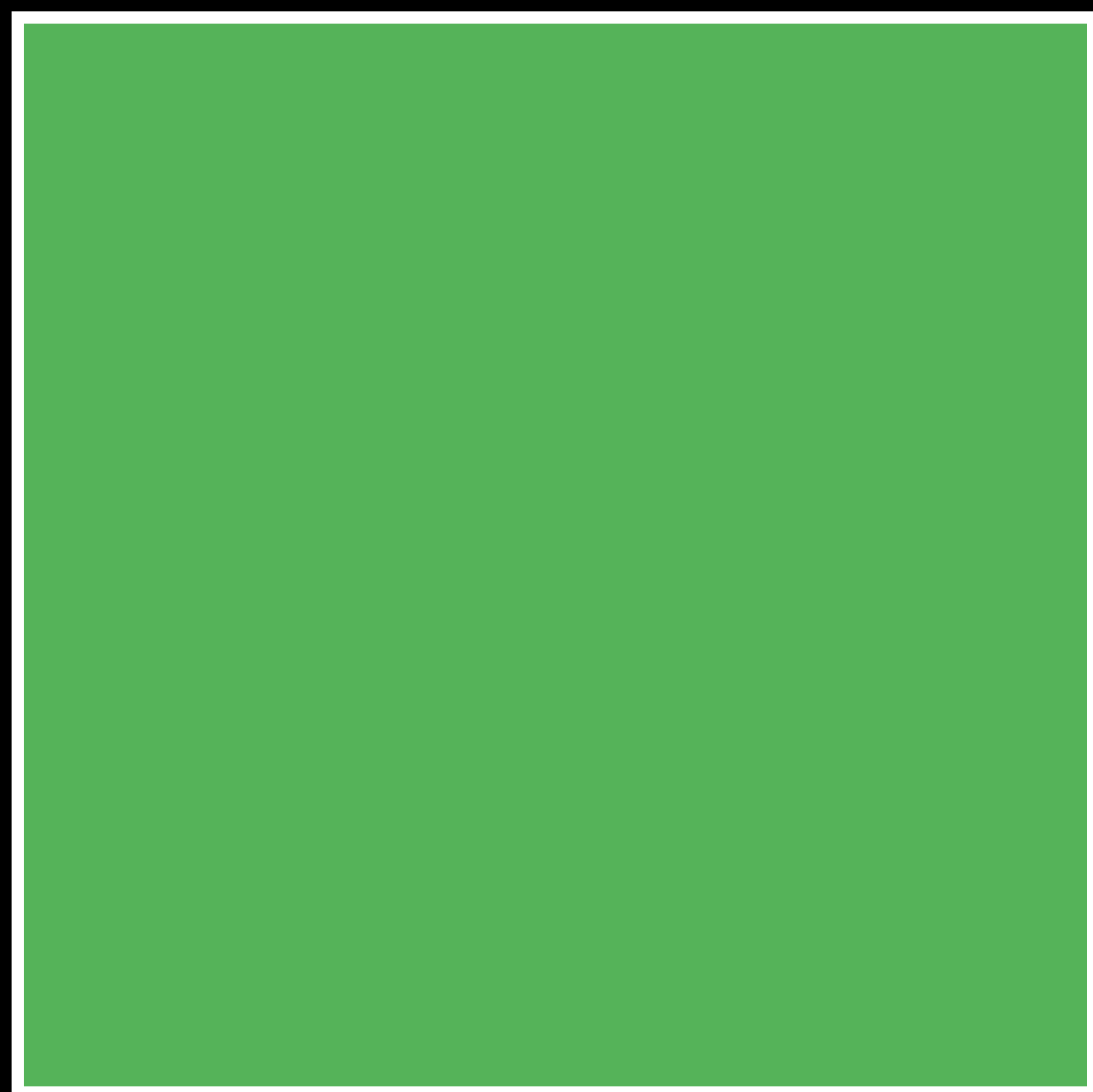
0

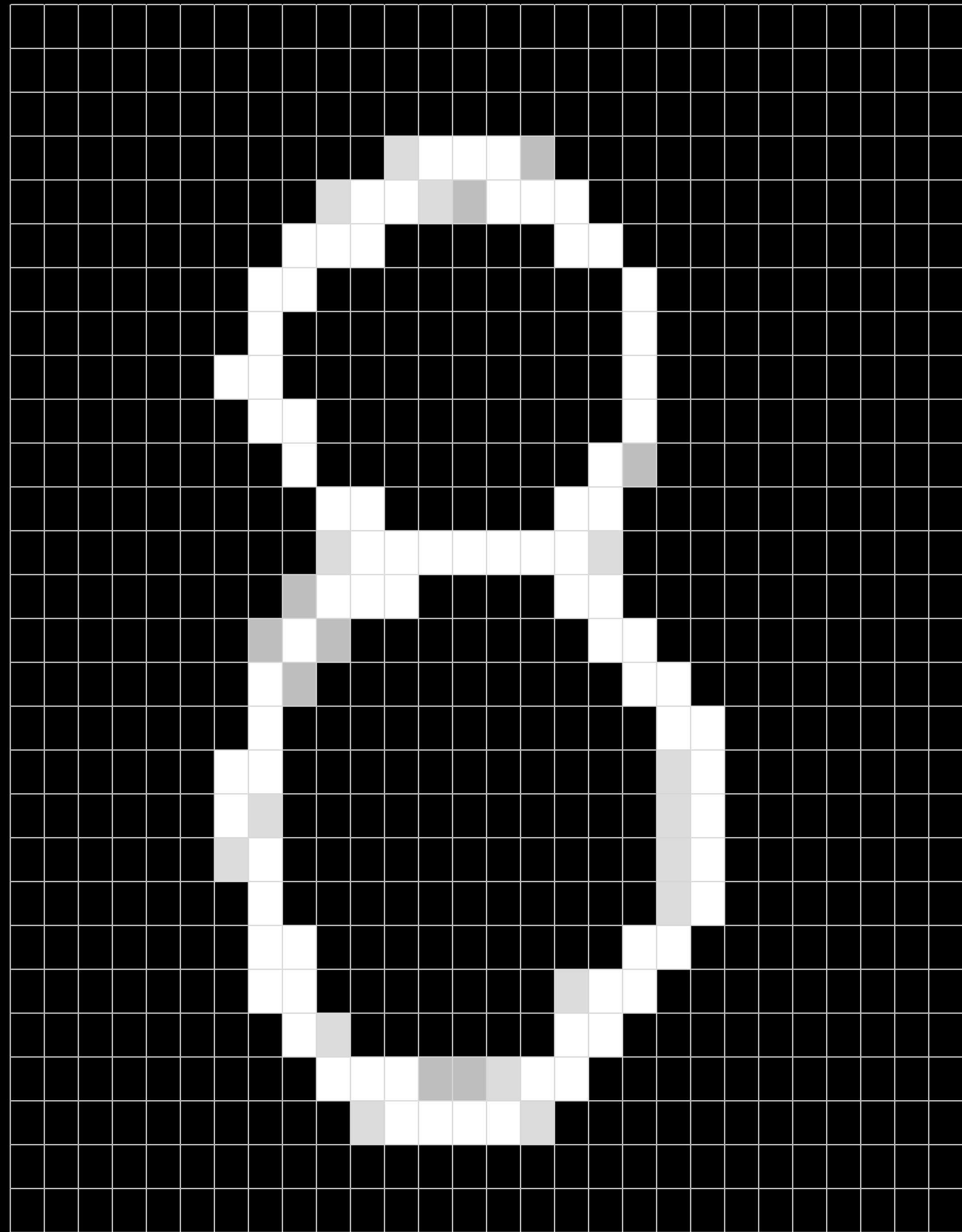
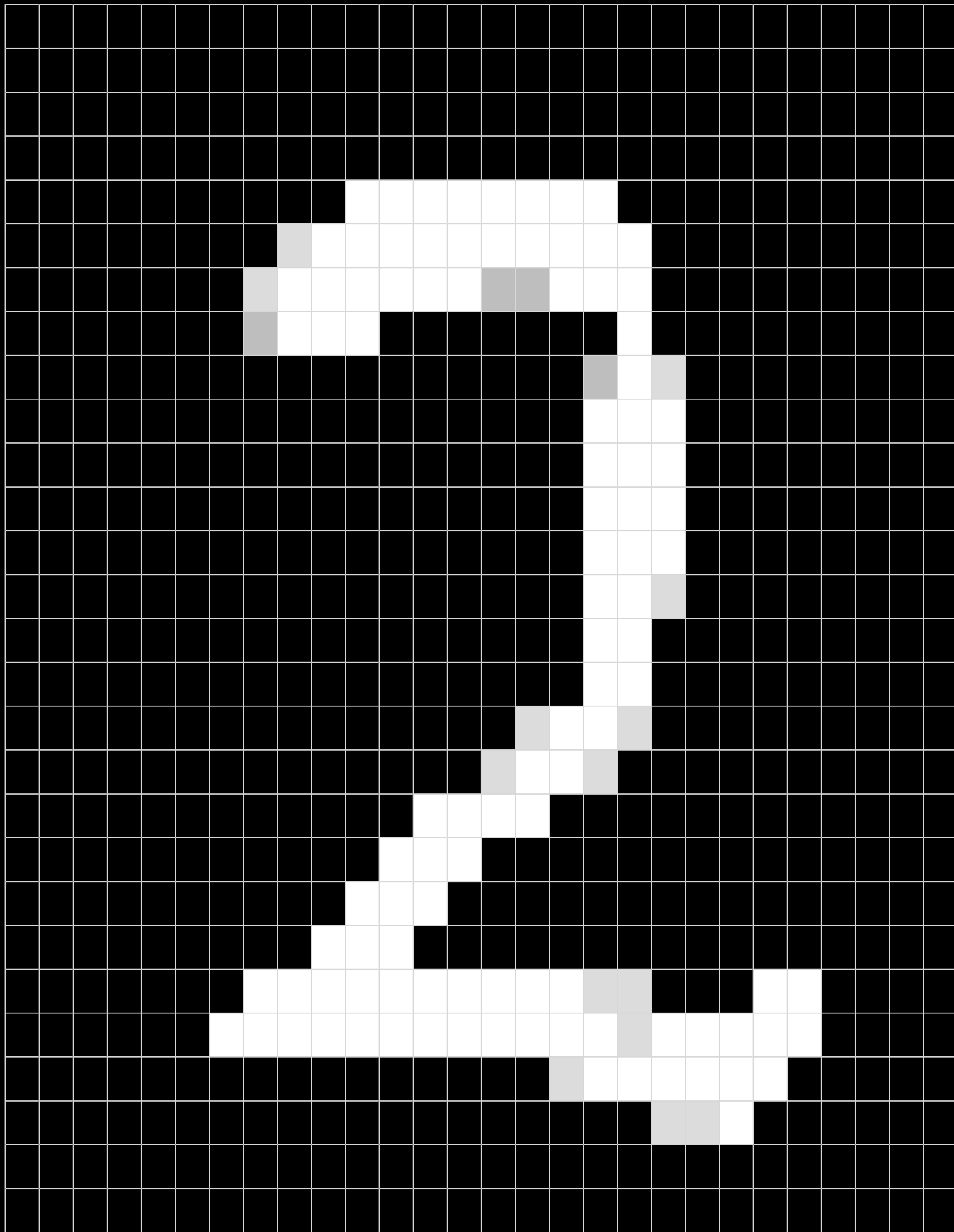
255

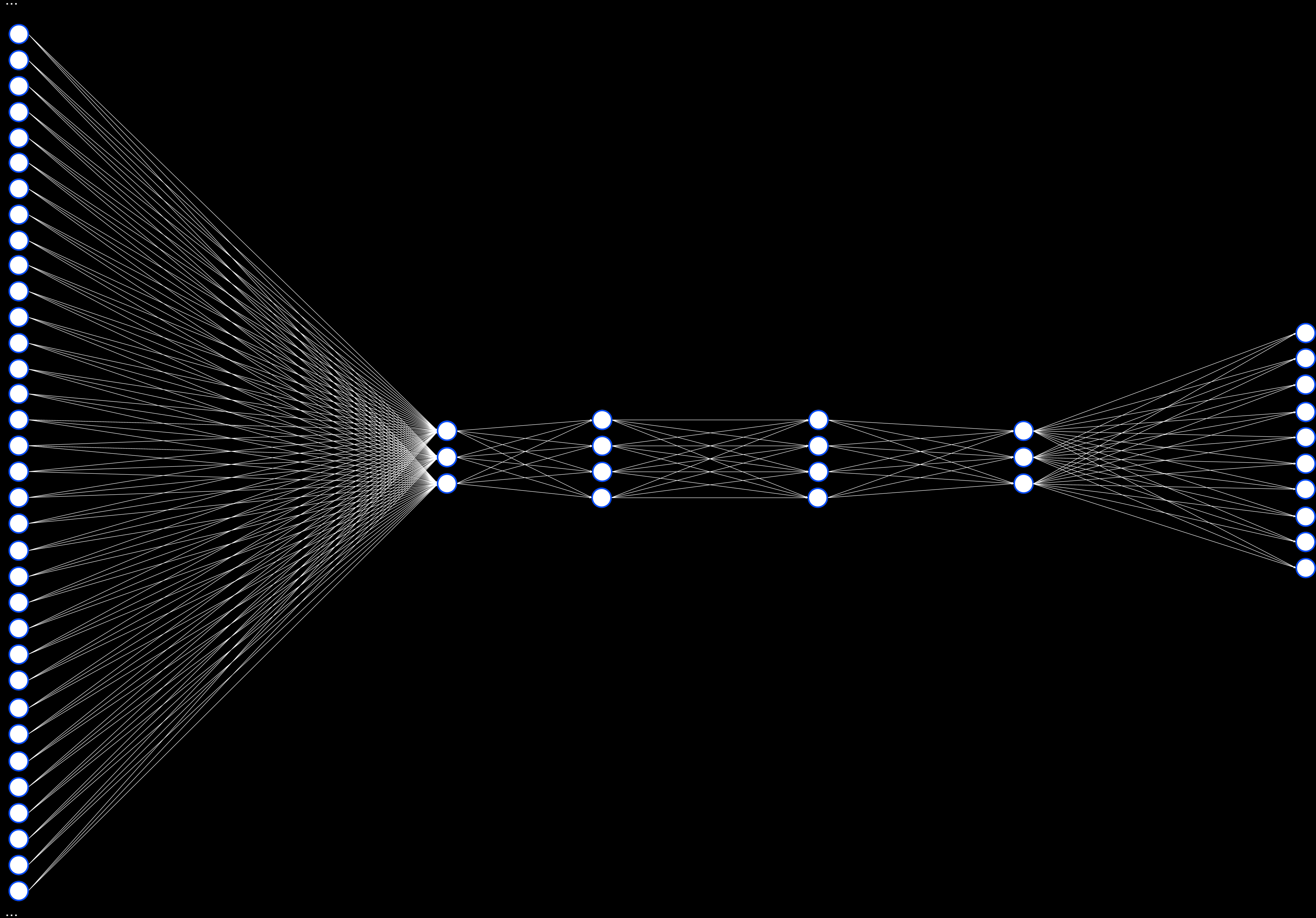


0

255







# convolução de imagem

aplicação de um filtro que soma cada valor de pixel aos vizinhos, ponderados por uma matriz convolucional (kernel)

|    |    |    |
|----|----|----|
| 0  | -1 | 0  |
| -1 | 5  | -1 |
| 0  | -1 | 0  |

|    |    |    |    |
|----|----|----|----|
| 10 | 20 | 30 | 40 |
| 10 | 20 | 30 | 40 |
| 20 | 30 | 40 | 50 |
| 20 | 30 | 40 | 50 |

|    |    |    |
|----|----|----|
| 0  | -1 | 0  |
| -1 | 5  | -1 |
| 0  | -1 | 0  |

|    |    |    |    |
|----|----|----|----|
| 10 | 20 | 30 | 40 |
| 10 | 20 | 30 | 40 |
| 20 | 30 | 40 | 50 |
| 20 | 30 | 40 | 50 |

|    |    |    |
|----|----|----|
| 0  | -1 | 0  |
| -1 | 5  | -1 |
| 0  | -1 | 0  |

|    |    |    |    |
|----|----|----|----|
| 10 | 20 | 30 | 40 |
| 10 | 20 | 30 | 40 |
| 20 | 30 | 40 | 50 |
| 20 | 30 | 40 | 50 |

|    |    |    |
|----|----|----|
| 0  | -1 | 0  |
| -1 | 5  | -1 |
| 0  | -1 | 0  |

|  |  |
|--|--|
|  |  |
|  |  |

|    |    |    |    |
|----|----|----|----|
| 10 | 20 | 30 | 40 |
| 10 | 20 | 30 | 40 |
| 20 | 30 | 40 | 50 |
| 20 | 30 | 40 | 50 |

|    |    |    |
|----|----|----|
| 0  | -1 | 0  |
| -1 | 5  | -1 |
| 0  | -1 | 0  |

|    |  |
|----|--|
| 10 |  |
|    |  |

|    |    |    |    |
|----|----|----|----|
| 10 | 20 | 30 | 40 |
| 10 | 20 | 30 | 40 |
| 20 | 30 | 40 | 50 |
| 20 | 30 | 40 | 50 |

|    |    |    |
|----|----|----|
| 0  | -1 | 0  |
| -1 | 5  | -1 |
| 0  | -1 | 0  |

|    |  |
|----|--|
| 10 |  |
|    |  |

|    |    |    |    |
|----|----|----|----|
| 10 | 20 | 30 | 40 |
| 10 | 20 | 30 | 40 |
| 20 | 30 | 40 | 50 |
| 20 | 30 | 40 | 50 |

|    |    |    |
|----|----|----|
| 0  | -1 | 0  |
| -1 | 5  | -1 |
| 0  | -1 | 0  |

|    |    |
|----|----|
| 10 | 20 |
|    |    |

|    |    |    |    |
|----|----|----|----|
| 10 | 20 | 30 | 40 |
| 10 | 20 | 30 | 40 |
| 20 | 30 | 40 | 50 |
| 20 | 30 | 40 | 50 |

|    |    |    |
|----|----|----|
| 0  | -1 | 0  |
| -1 | 5  | -1 |
| 0  | -1 | 0  |

|    |    |
|----|----|
| 10 | 20 |
|    |    |

| 10 | 20 | 30 | 40 |
|----|----|----|----|
| 10 | 20 | 30 | 40 |
| 20 | 30 | 40 | 50 |
| 20 | 30 | 40 | 50 |

|    |    |    |
|----|----|----|
| 0  | -1 | 0  |
| -1 | 5  | -1 |
| 0  | -1 | 0  |

|    |    |
|----|----|
| 10 | 20 |
| 40 |    |

|    |    |    |    |
|----|----|----|----|
| 10 | 20 | 30 | 40 |
| 10 | 20 | 30 | 40 |
| 20 | 30 | 40 | 50 |
| 20 | 30 | 40 | 50 |

|    |    |    |
|----|----|----|
| 0  | -1 | 0  |
| -1 | 5  | -1 |
| 0  | -1 | 0  |

|    |    |
|----|----|
| 10 | 20 |
| 40 |    |

|    |    |    |    |
|----|----|----|----|
| 10 | 20 | 30 | 40 |
| 10 | 20 | 30 | 40 |
| 20 | 30 | 40 | 50 |
| 20 | 30 | 40 | 50 |

|    |    |    |
|----|----|----|
| 0  | -1 | 0  |
| -1 | 5  | -1 |
| 0  | -1 | 0  |

|    |    |
|----|----|
| 10 | 20 |
| 40 | 50 |

|    |    |    |    |
|----|----|----|----|
| 10 | 20 | 30 | 40 |
| 10 | 20 | 30 | 40 |
| 20 | 30 | 40 | 50 |
| 20 | 30 | 40 | 50 |

|    |    |    |
|----|----|----|
| 0  | -1 | 0  |
| -1 | 5  | -1 |
| 0  | -1 | 0  |

|    |    |
|----|----|
| 10 | 20 |
| 40 | 50 |

|    |    |    |
|----|----|----|
| -1 | -1 | -1 |
| -1 | 8  | -1 |
| -1 | -1 | -1 |

|    |    |    |
|----|----|----|
| 20 | 20 | 20 |
| 20 | 20 | 20 |
| 20 | 20 | 20 |

|    |    |    |
|----|----|----|
| -1 | -1 | -1 |
| -1 | 8  | -1 |
| -1 | -1 | -1 |

|    |    |    |
|----|----|----|
| 20 | 20 | 20 |
| 20 | 20 | 20 |
| 20 | 20 | 20 |

|    |    |    |
|----|----|----|
| -1 | -1 | -1 |
| -1 | 8  | -1 |
| -1 | -1 | -1 |

$$\begin{aligned} & (20)(-1) + (20)(-1) + (20)(-1) \\ & + (20)(-1) + (20)(8) + (20)(-1) \\ & + (20)(-1) + (20)(-1) + (20)(-1) \end{aligned}$$

|    |    |    |
|----|----|----|
| 20 | 20 | 20 |
| 20 | 20 | 20 |
| 20 | 20 | 20 |

|    |    |    |
|----|----|----|
| -1 | -1 | -1 |
| -1 | 8  | -1 |
| -1 | -1 | -1 |

$$\begin{aligned}
 & (20)(-1) + (20)(-1) + (20)(-1) \\
 & + (20)(-1) + (20)(8) + (20)(-1) \\
 & + (20)(-1) + (20)(-1) + (20)(-1)
 \end{aligned}$$

0

|    |    |    |
|----|----|----|
| 20 | 20 | 20 |
| 50 | 50 | 50 |
| 50 | 50 | 50 |

|    |    |    |
|----|----|----|
| -1 | -1 | -1 |
| -1 | 8  | -1 |
| -1 | -1 | -1 |

|    |    |    |
|----|----|----|
| 20 | 20 | 20 |
| 50 | 50 | 50 |
| 50 | 50 | 50 |

|    |    |    |
|----|----|----|
| -1 | -1 | -1 |
| -1 | 8  | -1 |
| -1 | -1 | -1 |

$$\begin{aligned}
 &(20)(-1) + (20)(-1) + (20)(-1) \\
 &+ (50)(-1) + (50)(8) + (50)(-1) \\
 &+ (50)(-1) + (50)(-1) + (50)(-1)
 \end{aligned}$$

|    |    |    |
|----|----|----|
| 20 | 20 | 20 |
| 50 | 50 | 50 |
| 50 | 50 | 50 |

|    |    |    |
|----|----|----|
| -1 | -1 | -1 |
| -1 | 8  | -1 |
| -1 | -1 | -1 |

$$\begin{aligned}
 &(20)(-1) + (20)(-1) + (20)(-1) \\
 &+ (50)(-1) + (50)(8) + (50)(-1) \\
 &+ (50)(-1) + (50)(-1) + (50)(-1)
 \end{aligned}$$

90





# pooling

redução do tamanho de uma entrada por amostragem de suas regiões

# max-pooling

pooling pela escolha do maior valor em cada região

|    |    |     |     |
|----|----|-----|-----|
| 30 | 40 | 80  | 90  |
| 20 | 50 | 100 | 110 |
| 0  | 10 | 20  | 30  |
| 10 | 20 | 40  | 30  |

|    |    |     |     |
|----|----|-----|-----|
| 30 | 40 | 80  | 90  |
| 20 | 50 | 100 | 110 |
| 0  | 10 | 20  | 30  |
| 10 | 20 | 40  | 30  |

|  |  |
|--|--|
|  |  |
|  |  |

|    |    |     |     |
|----|----|-----|-----|
| 30 | 40 | 80  | 90  |
| 20 | 50 | 100 | 110 |
| 0  | 10 | 20  | 30  |
| 10 | 20 | 40  | 30  |

|  |  |
|--|--|
|  |  |
|  |  |

|    |    |     |     |
|----|----|-----|-----|
| 30 | 40 | 80  | 90  |
| 20 | 50 | 100 | 110 |
| 0  | 10 | 20  | 30  |
| 10 | 20 | 40  | 30  |

|    |  |
|----|--|
| 50 |  |
|    |  |

|    |    |     |     |
|----|----|-----|-----|
| 30 | 40 | 80  | 90  |
| 20 | 50 | 100 | 110 |
| 0  | 10 | 20  | 30  |
| 10 | 20 | 40  | 30  |

|    |  |
|----|--|
| 50 |  |
|    |  |

|    |    |     |     |
|----|----|-----|-----|
| 30 | 40 | 80  | 90  |
| 20 | 50 | 100 | 110 |
| 0  | 10 | 20  | 30  |
| 10 | 20 | 40  | 30  |

|    |     |
|----|-----|
| 50 | 110 |
|    |     |

|    |    |     |     |
|----|----|-----|-----|
| 30 | 40 | 80  | 90  |
| 20 | 50 | 100 | 110 |
| 0  | 10 | 20  | 30  |
| 10 | 20 | 40  | 30  |

|    |     |
|----|-----|
| 50 | 110 |
|    |     |

|    |    |     |     |
|----|----|-----|-----|
| 30 | 40 | 80  | 90  |
| 20 | 50 | 100 | 110 |
| 0  | 10 | 20  | 30  |
| 10 | 20 | 40  | 30  |

|    |     |
|----|-----|
| 50 | 110 |
| 20 |     |

|    |    |     |     |
|----|----|-----|-----|
| 30 | 40 | 80  | 90  |
| 20 | 50 | 100 | 110 |
| 0  | 10 | 20  | 30  |
| 10 | 20 | 40  | 30  |

|    |     |
|----|-----|
| 50 | 110 |
| 20 |     |

|    |    |     |     |
|----|----|-----|-----|
| 30 | 40 | 80  | 90  |
| 20 | 50 | 100 | 110 |
| 0  | 10 | 20  | 30  |
| 10 | 20 | 40  | 30  |

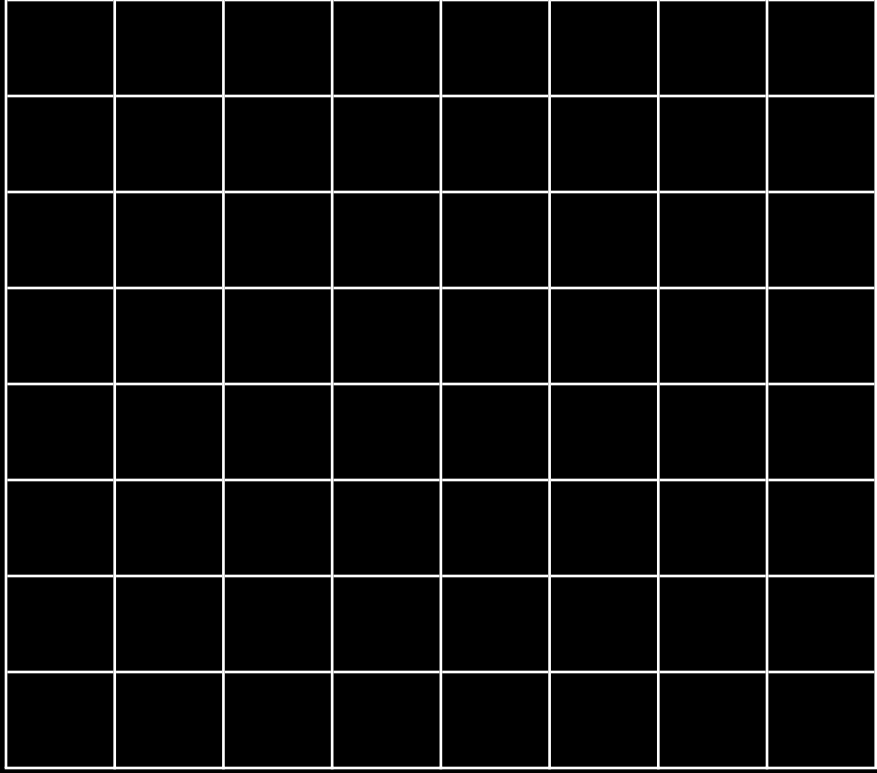
|    |     |
|----|-----|
| 50 | 110 |
| 20 | 40  |

|    |    |     |     |
|----|----|-----|-----|
| 30 | 40 | 80  | 90  |
| 20 | 50 | 100 | 110 |
| 0  | 10 | 20  | 30  |
| 10 | 20 | 40  | 30  |

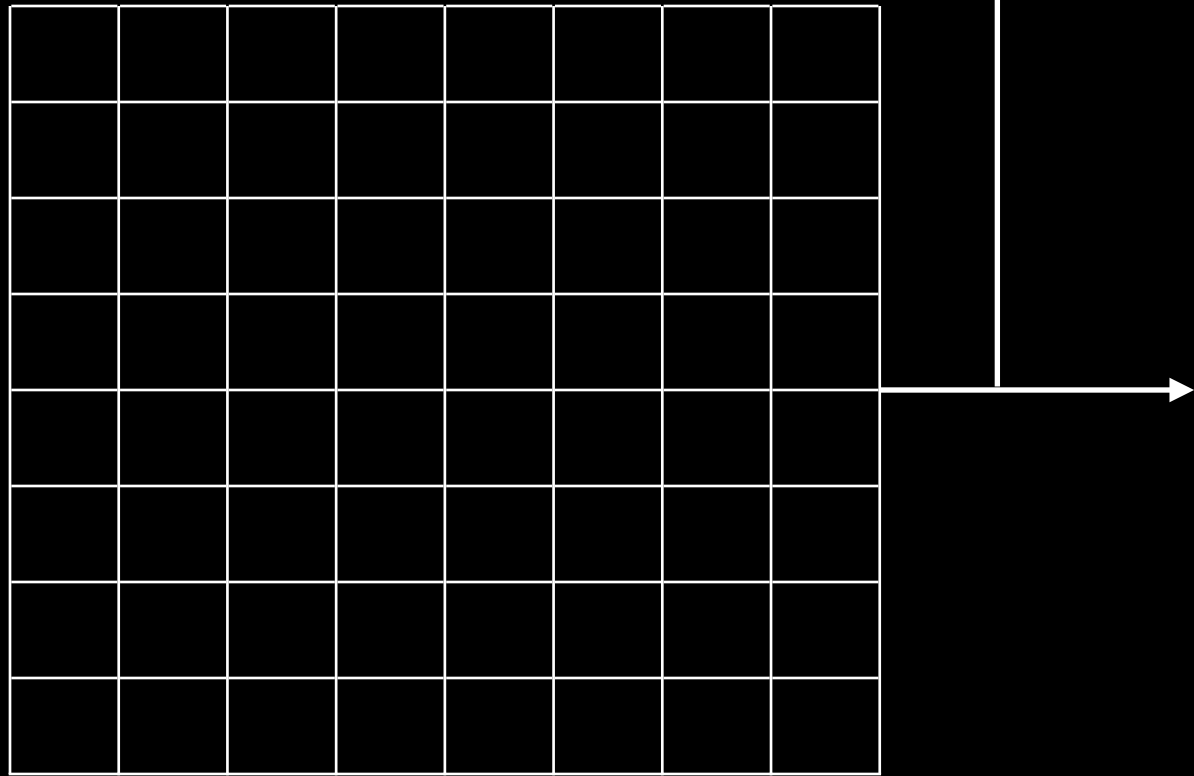
|    |     |
|----|-----|
| 50 | 110 |
| 20 | 40  |

# rede neural convolucional

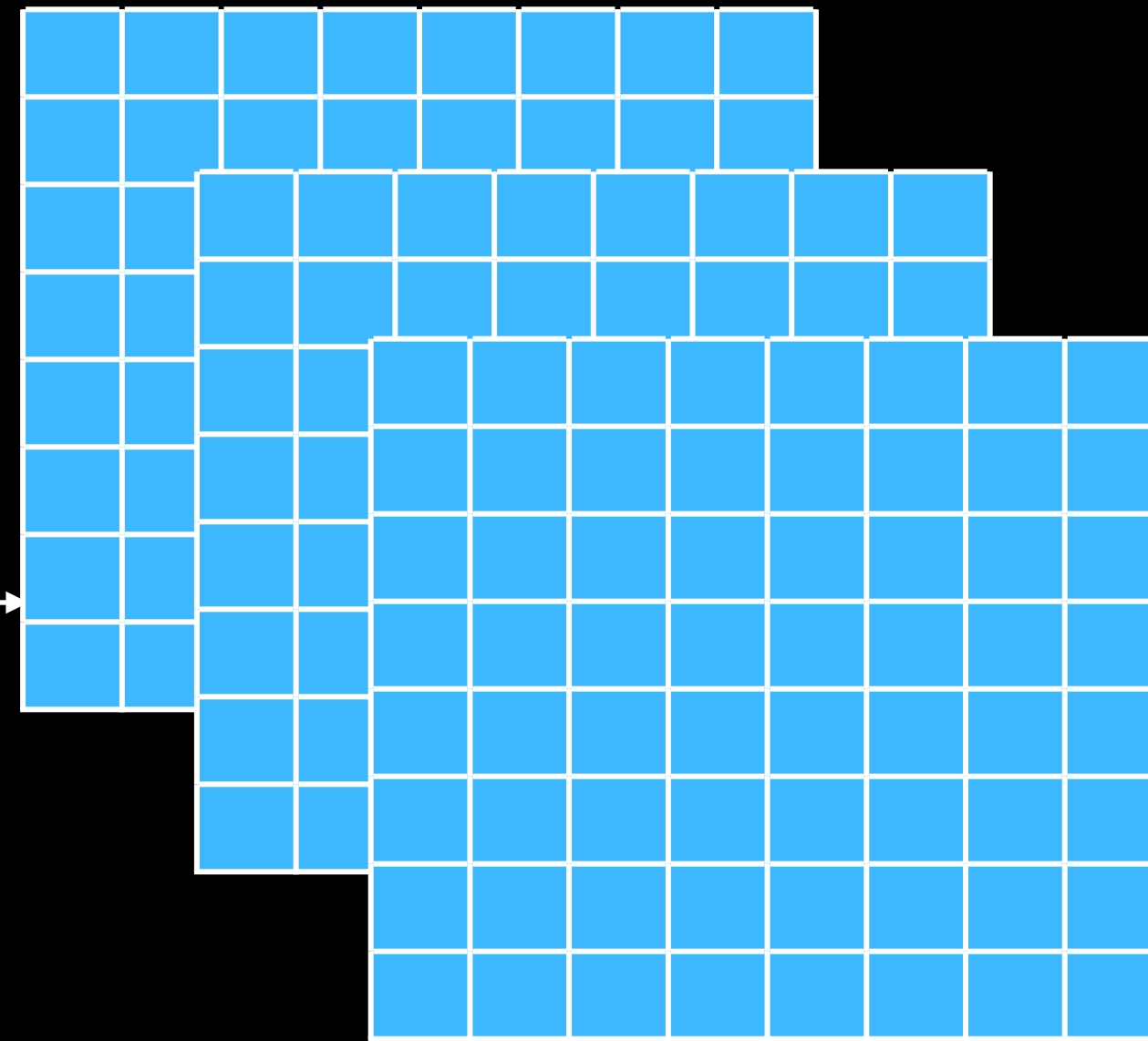
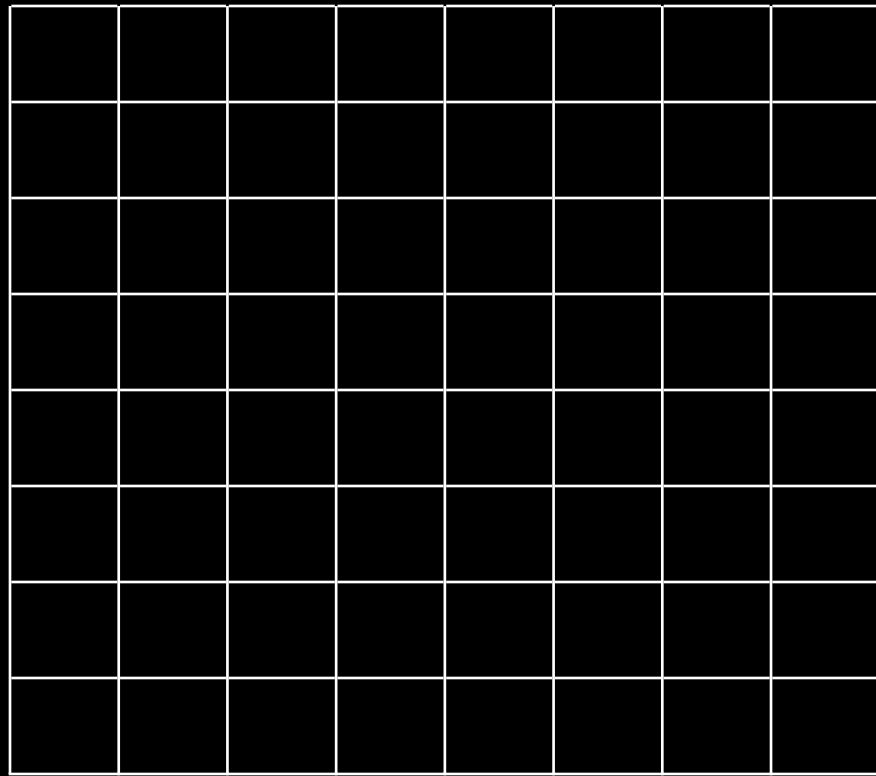
redes neurais que usam convolução, geralmente  
para analisar imagens

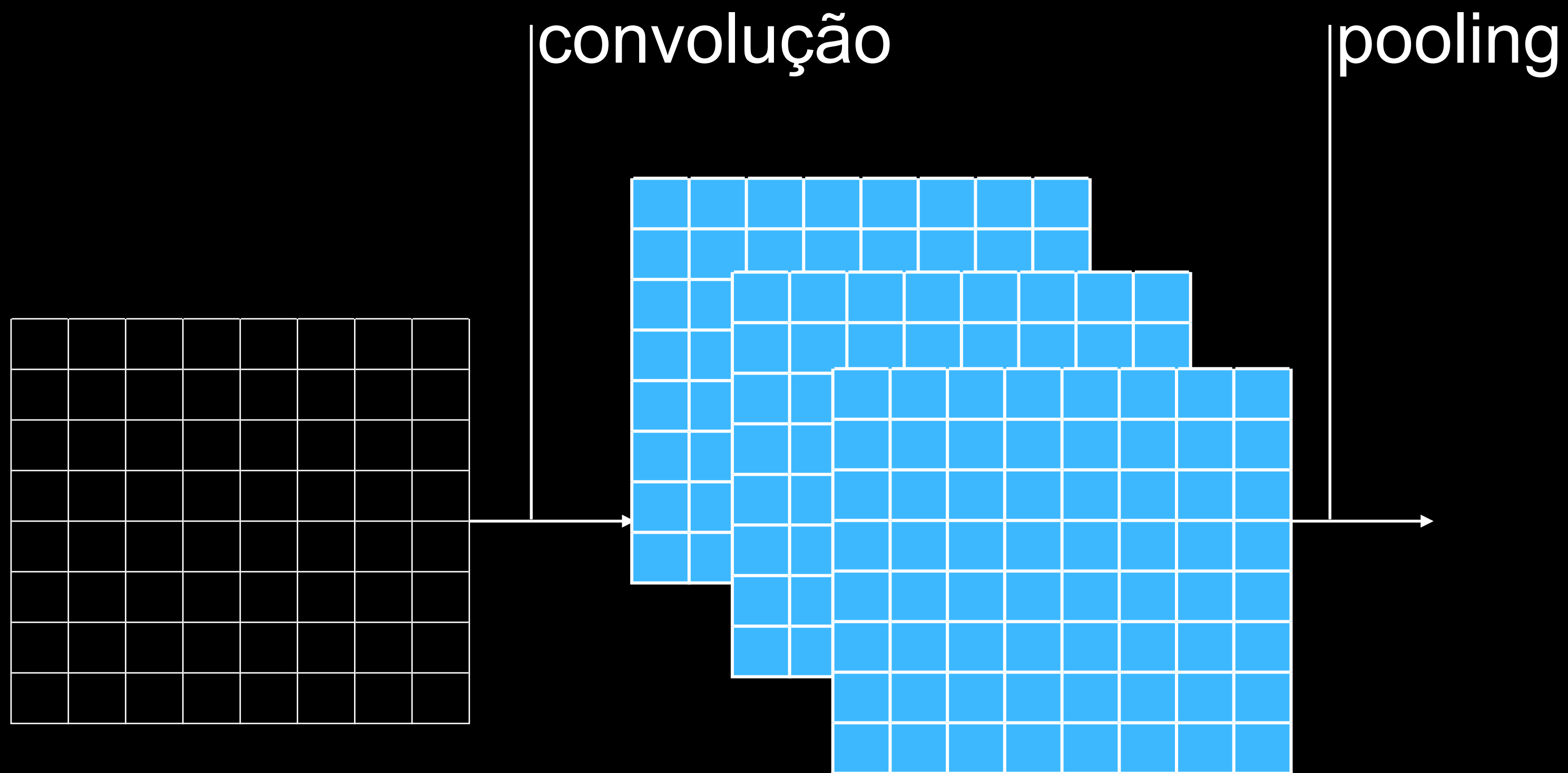


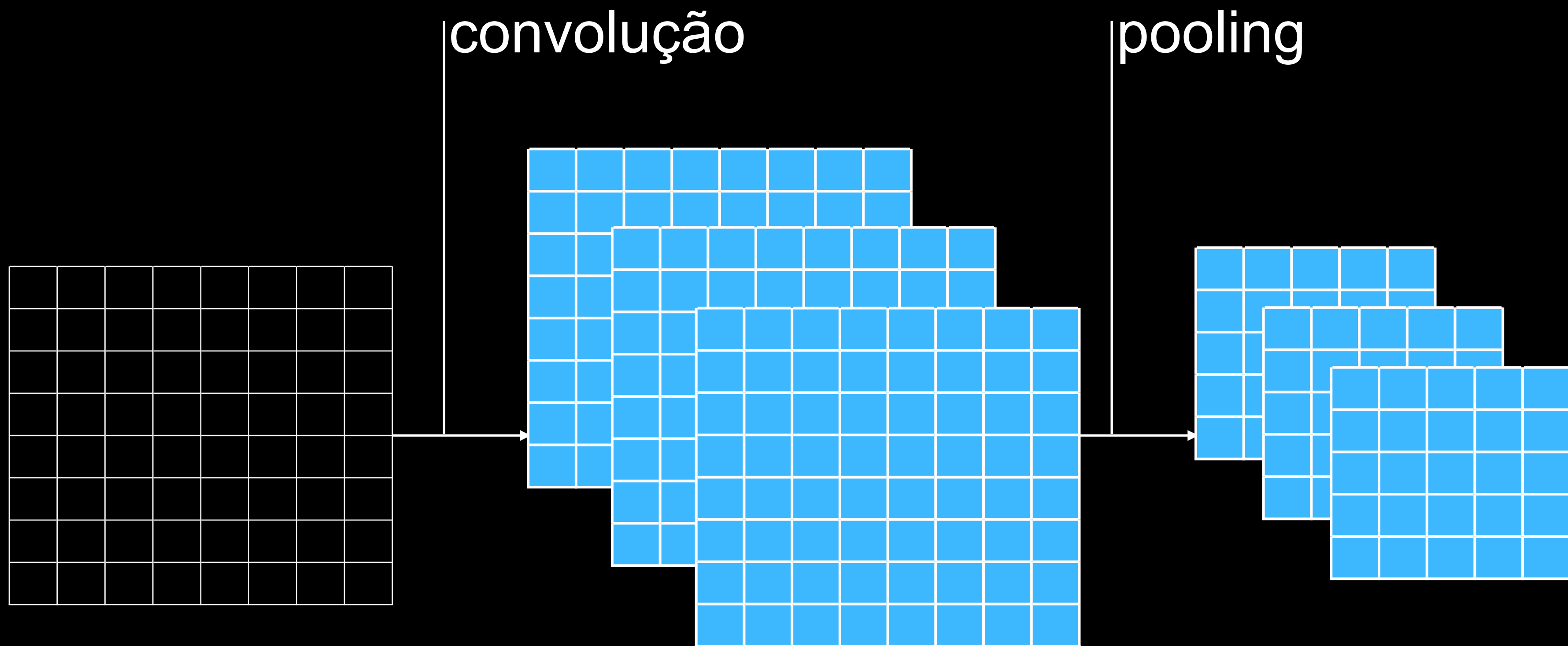
convolução

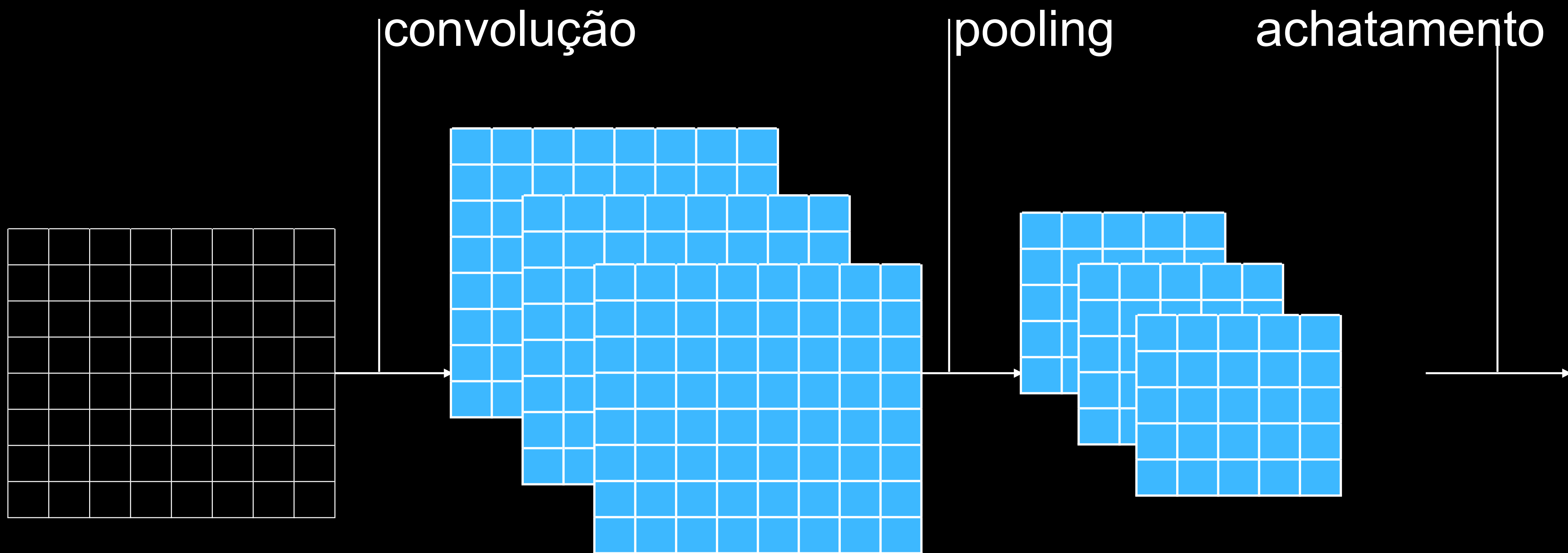


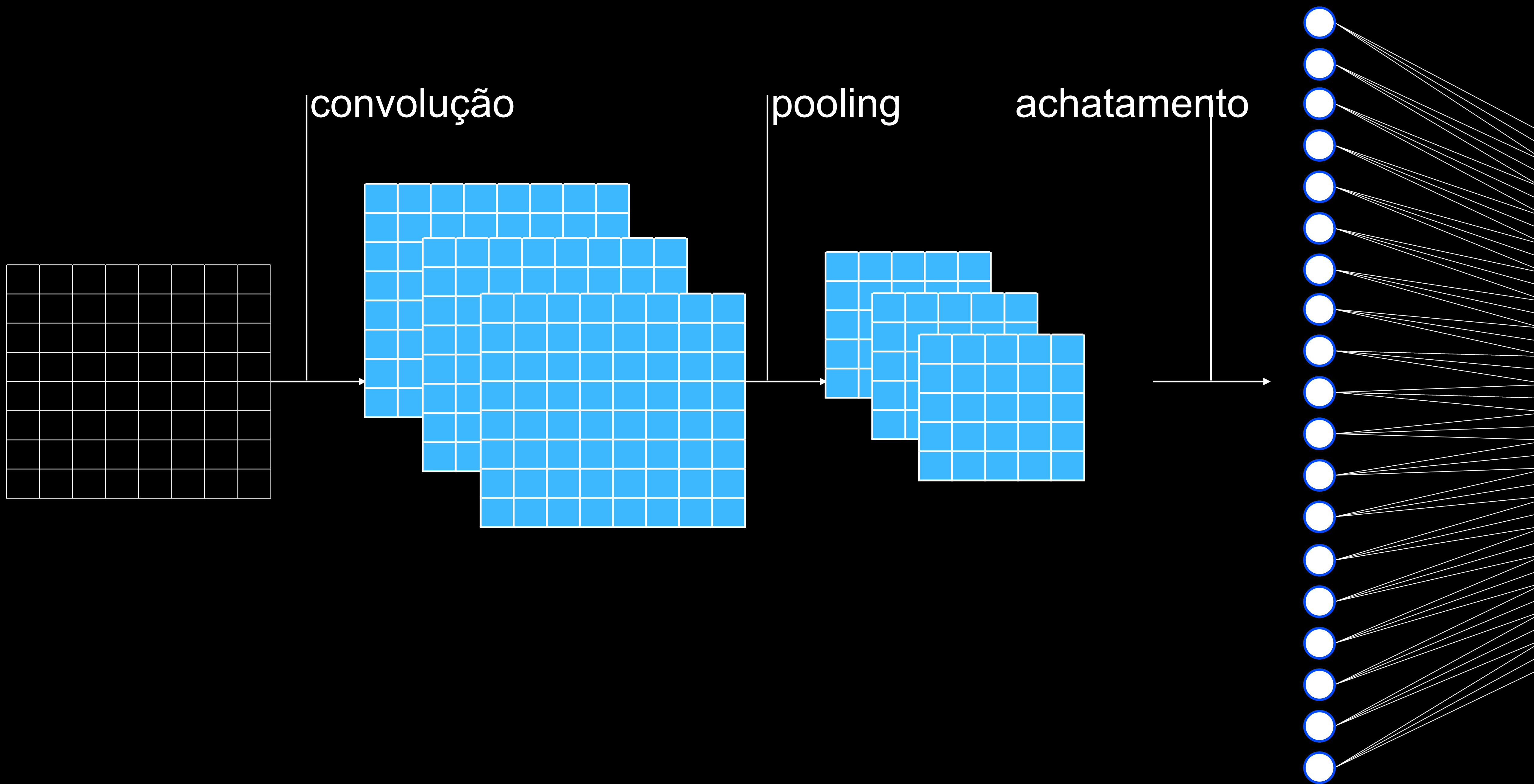
convolução

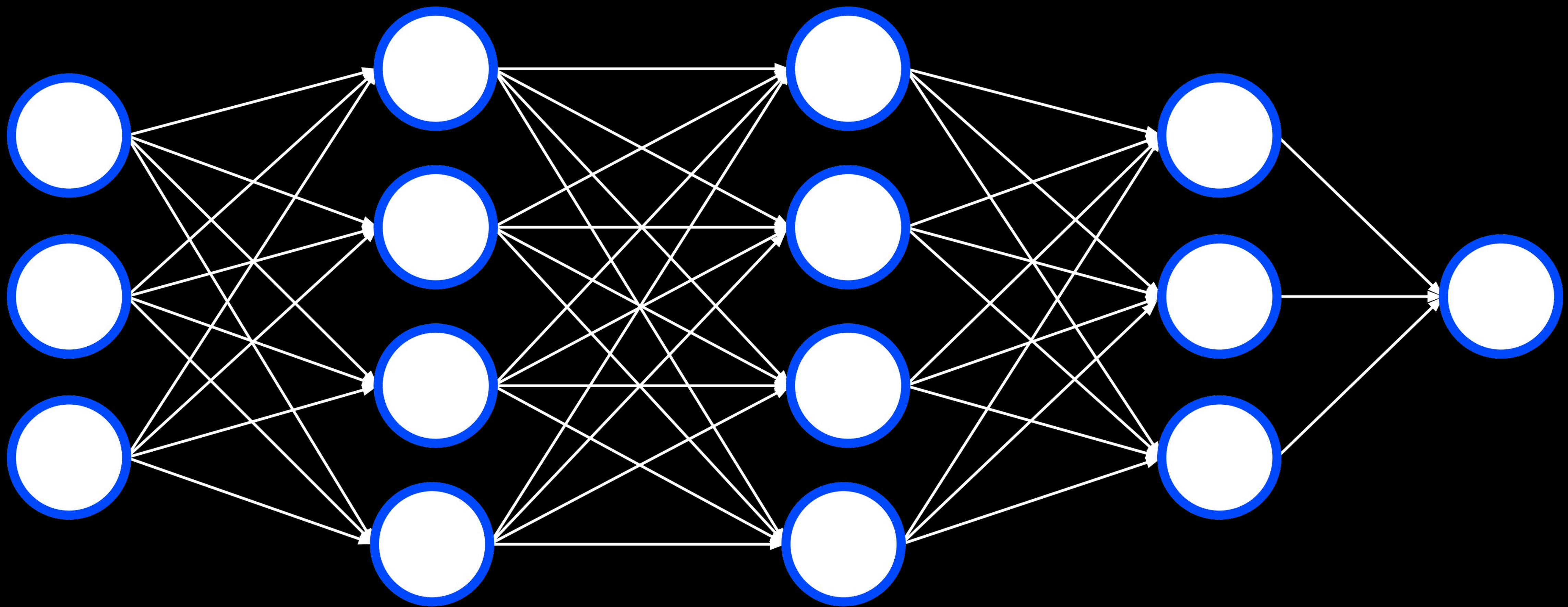


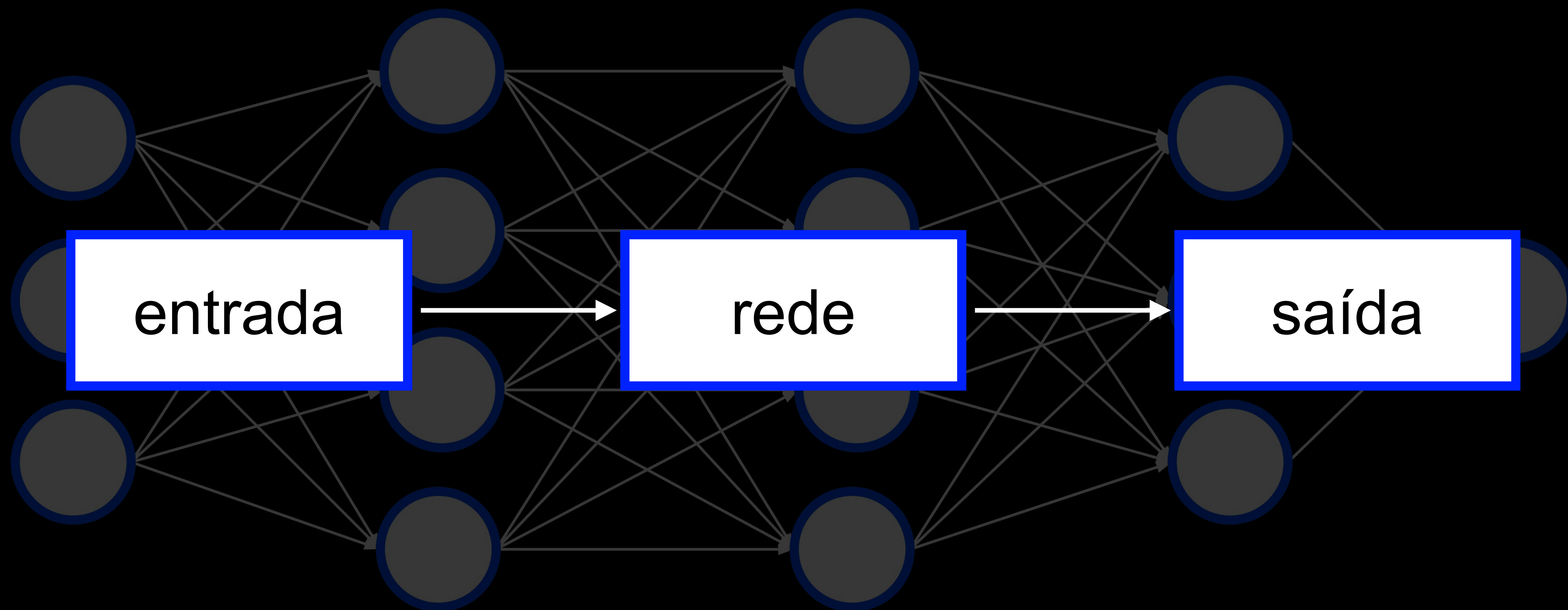












entrada



rede



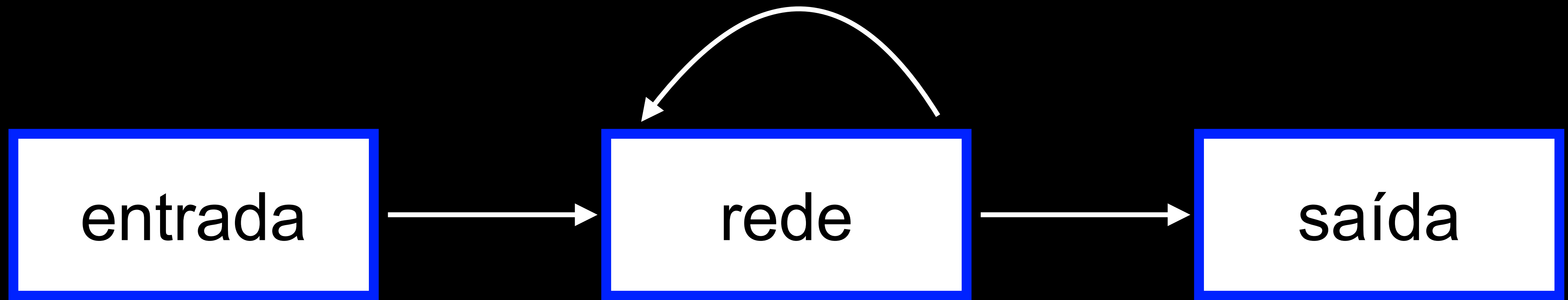
saída

# feed-forward neural network

rede neural com conexões em apenas uma  
direção

# recurrent neural network

rede neural cuja saída retorna como parte de  
suas próprias entradas





grupo de pessoas caminhando diante de um prédio



entrada

entrada



rede

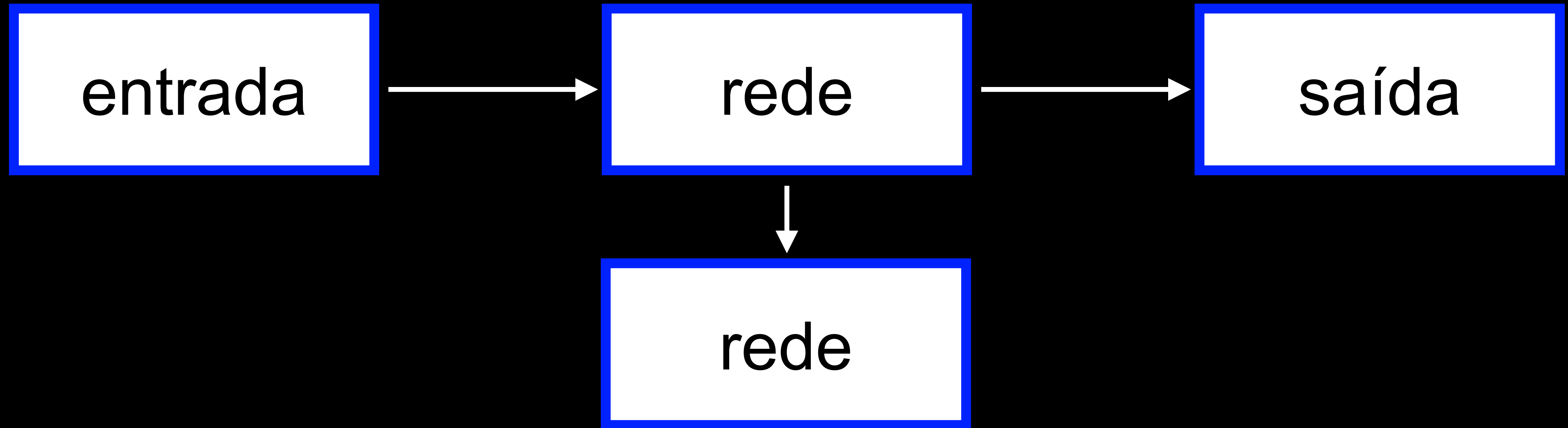
entrada

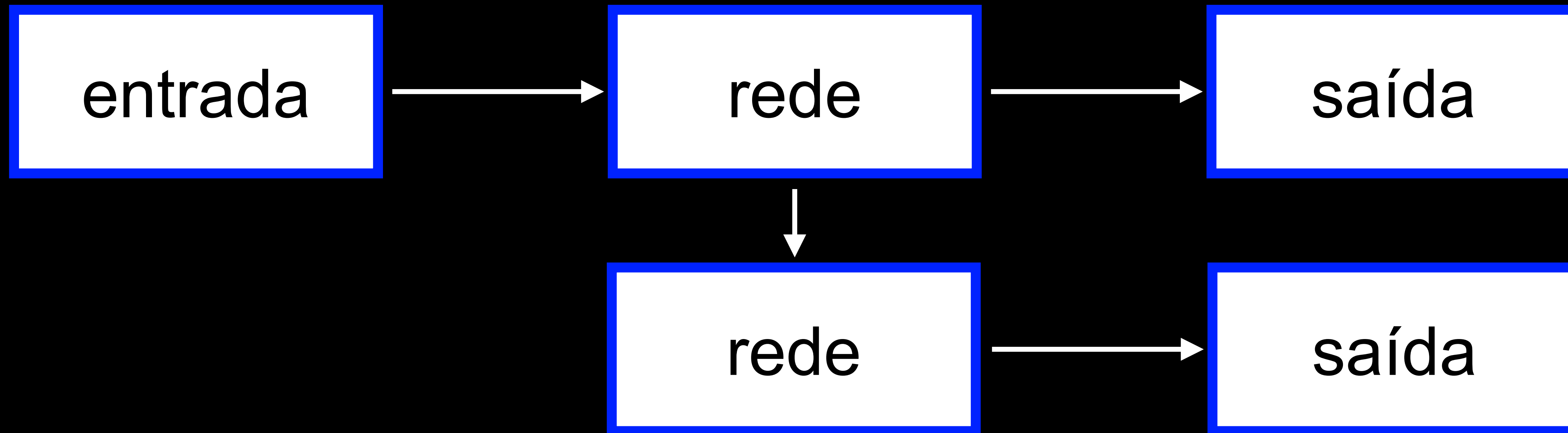


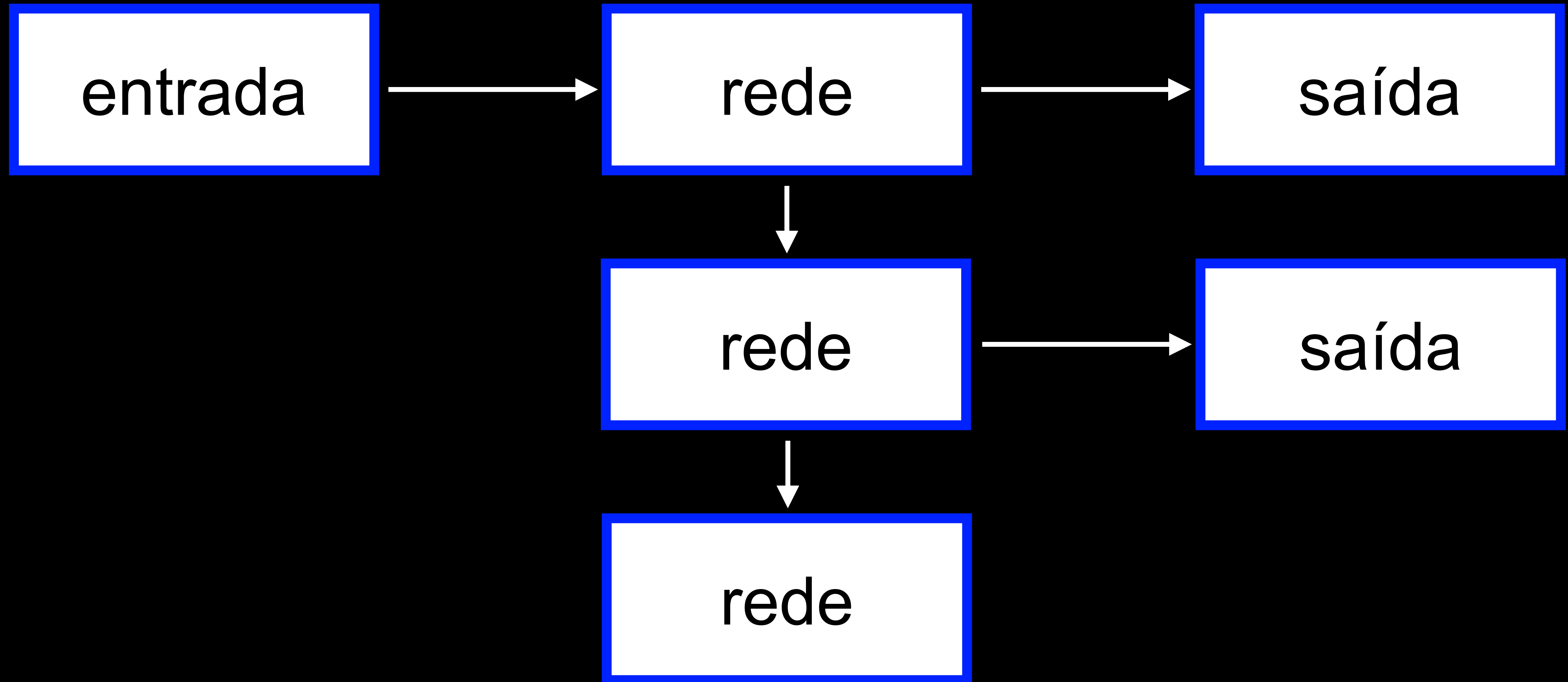
rede

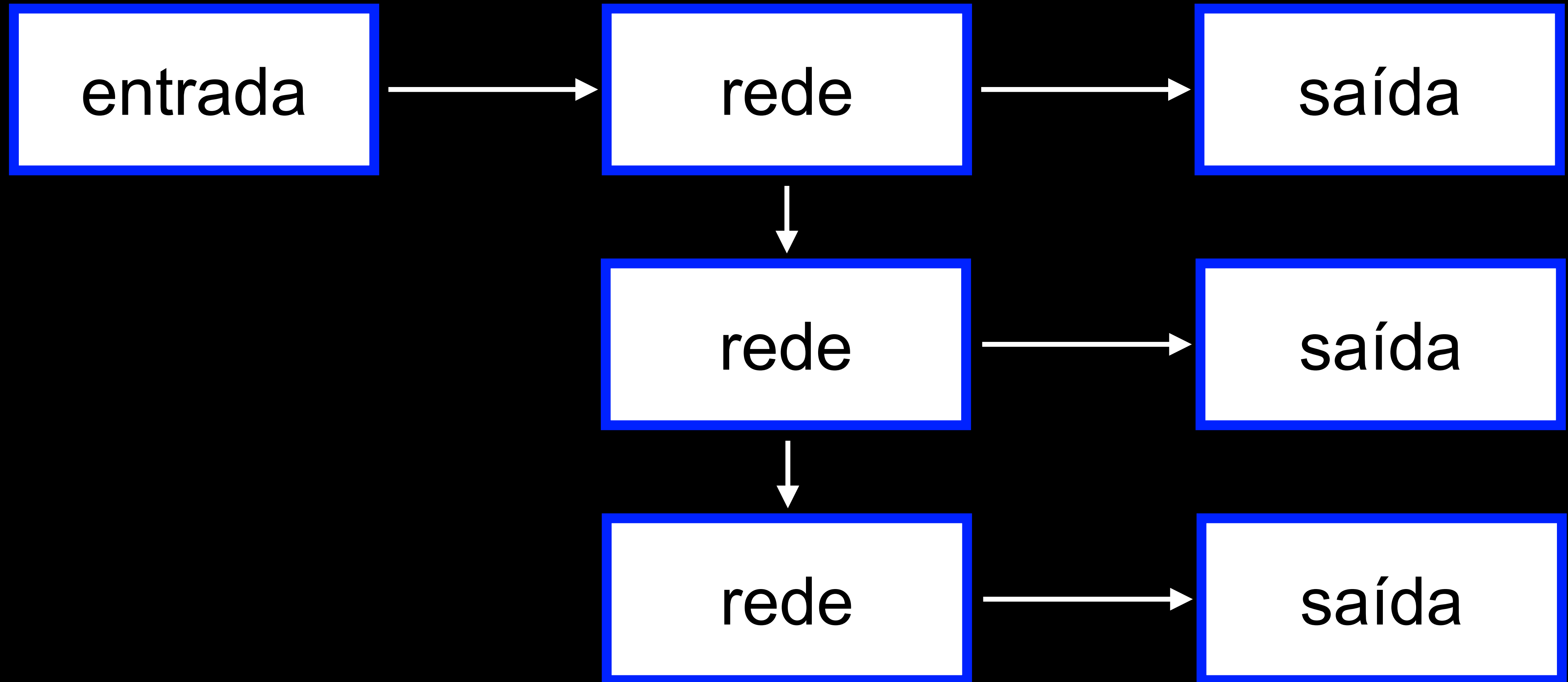


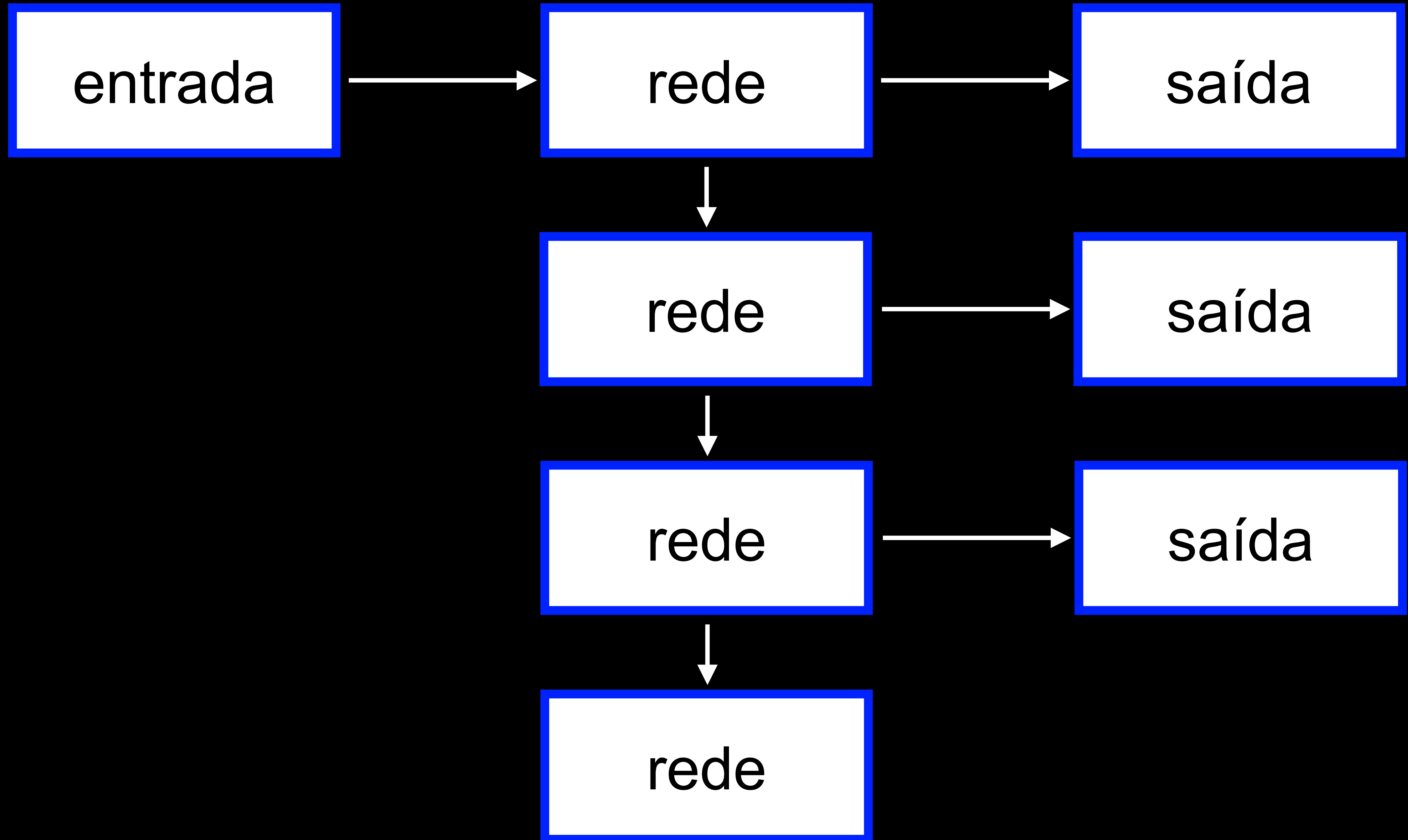
saída

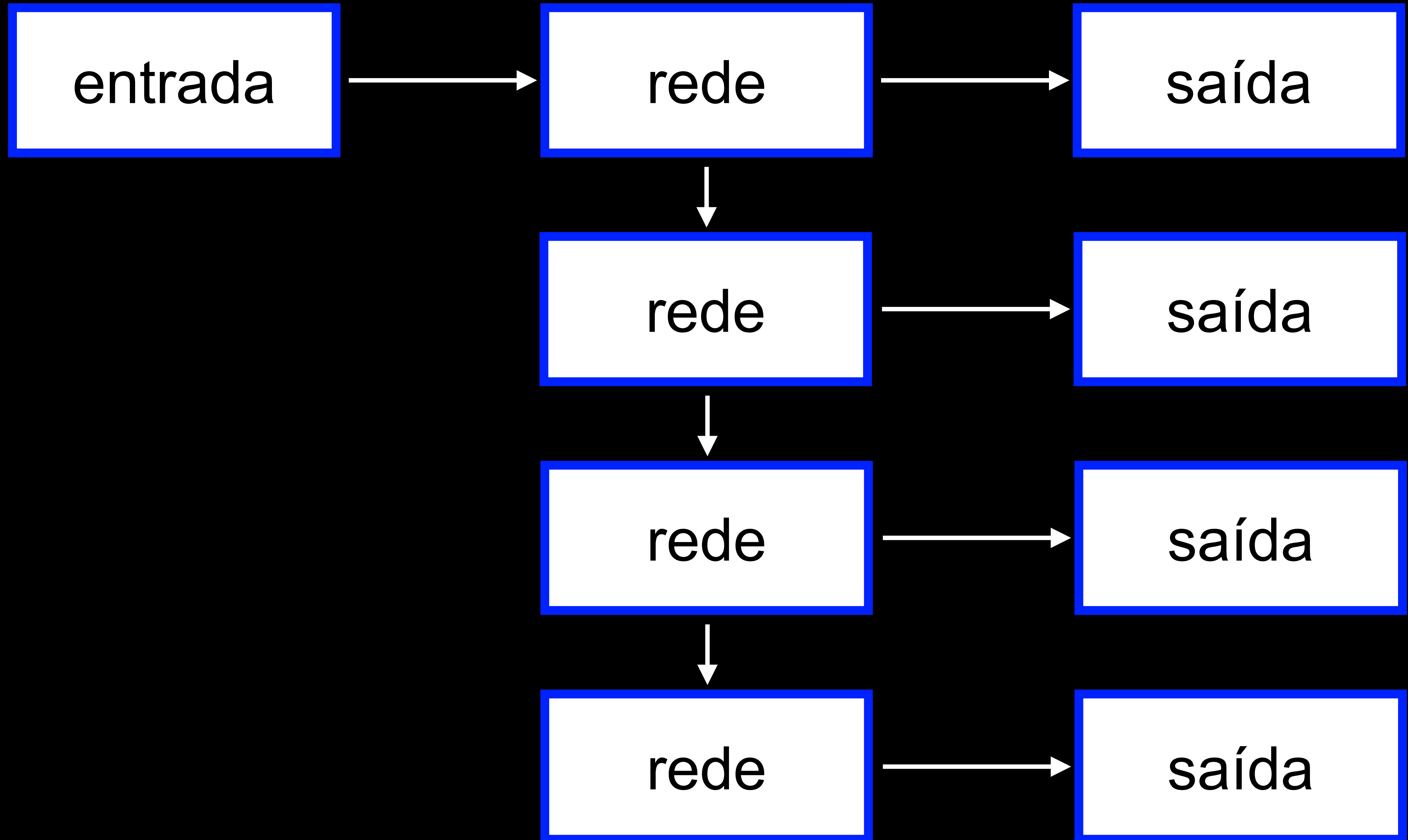




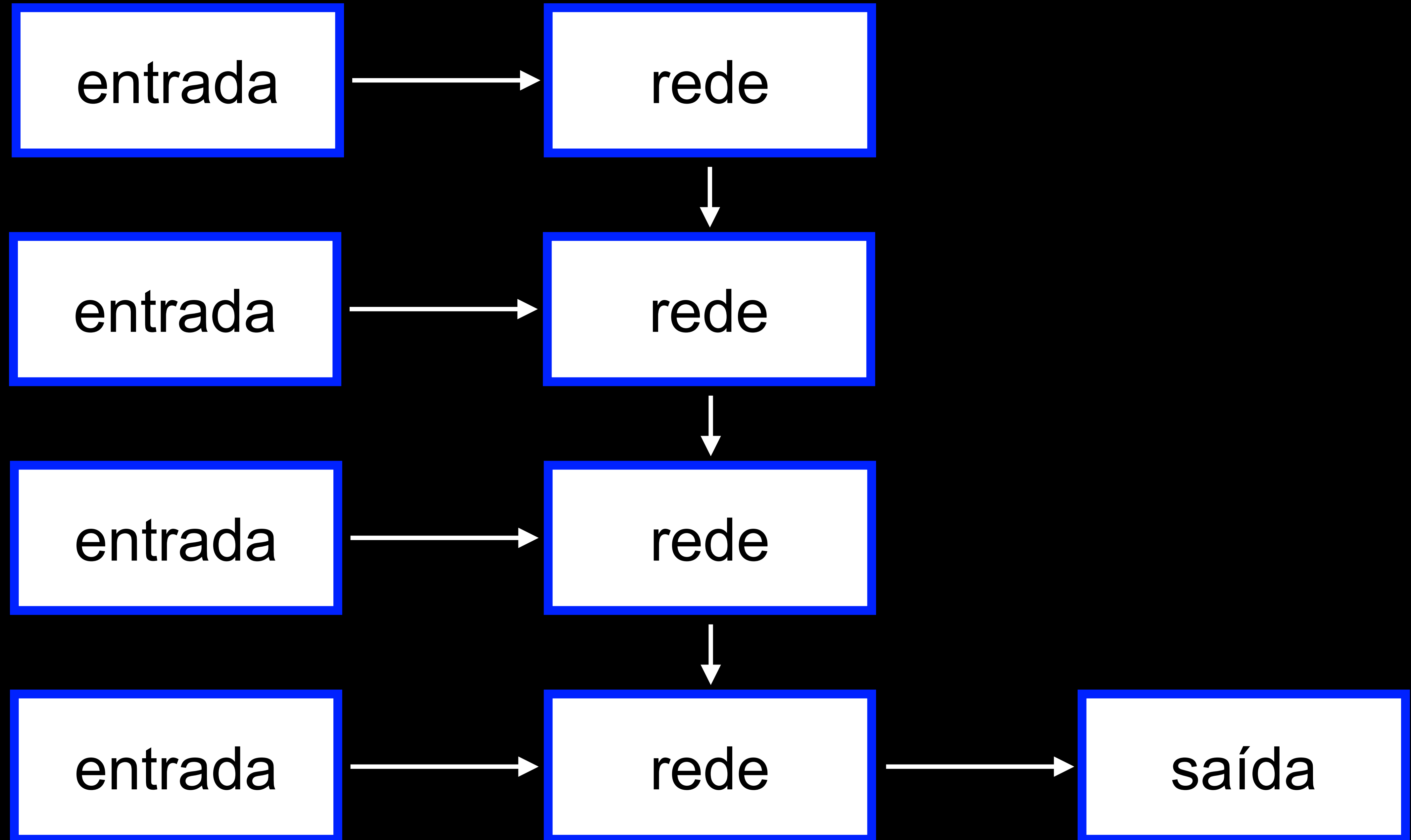














Ela está na biblioteca.



她在圖書館

